
Security Middleware Library - SMW

Release 5.2

NXP

Nov 21, 2025

CONTENTS:

1	Introduction	1
2	How to write a configuration file	2
2.1	Syntactic rules	2
2.2	Secure Subsystems definition	3
2.3	Security Operations definition	4
2.4	Example	6
3	Public APIs	8
3.1	SMW APIs	8
3.1.1	Library information APIs	8
3.1.1.1	Introduction	8
3.1.1.2	smw_get_version	8
3.1.2	Configuration APIs	8
3.1.2.1	Introduction	8
3.1.2.2	smw_config_subsystem_present	9
3.1.2.3	smw_config_subsystem_loaded	9
3.1.2.4	smw_config_check_digest	9
3.1.2.5	struct smw_key_info	10
3.1.2.6	smw_config_check_generate_key	11
3.1.2.7	smw_config_check_derive_key	11
3.1.2.8	struct smw_signature_info	12
3.1.2.9	smw_config_check_sign	12
3.1.2.10	smw_config_check_verify	13
3.1.2.11	struct smw_cipher_info	14
3.1.2.12	smw_config_check_cipher	14
3.1.2.13	struct smw_aead_info	15
3.1.2.14	smw_config_check_aead	16
3.1.2.15	struct smw_mac_info	16
3.1.2.16	smw_config_check_mac	17
3.1.2.17	smw_config_load	18
3.1.2.18	smw_config_unload	18
3.1.2.19	struct smw_asymmetric_encrypt_info	19
3.1.2.20	smw_config_check_asymmetric_encrypt	19
3.1.2.21	smw_config_check_asymmetric_decrypt	20
3.1.3	Cryptography APIs	21
3.1.3.1	struct smw_hash_args	21
3.1.3.2	struct smw_hash_init_args	21
3.1.3.3	struct smw_hash_update_args	22

3.1.3.4	struct smw_hash_final_args	23
3.1.3.5	struct smw_eddsa_params	23
3.1.3.6	struct smw_sign_verify_args	24
3.1.3.7	struct smw_sign_verify_init_args	25
3.1.3.8	struct smw_sign_verify_update_args	26
3.1.3.9	struct smw_sign_verify_final_args	26
3.1.3.10	struct smw_mac_args	27
3.1.3.11	struct smw_rng_args	28
3.1.3.12	struct smw_cipher_init_args	29
3.1.3.13	struct smw_cipher_data_args	30
3.1.3.14	struct smw_cipher_args	31
3.1.3.15	smw_hash	31
3.1.3.16	smw_hash_init	32
3.1.3.17	smw_hash_update	32
3.1.3.18	smw_hash_final	32
3.1.3.19	smw_sign	33
3.1.3.20	smw_sign_init	34
3.1.3.21	smw_sign_update	34
3.1.3.22	smw_sign_final	35
3.1.3.23	smw_verify	35
3.1.3.24	smw_verify_init	36
3.1.3.25	smw_verify_update	36
3.1.3.26	smw_verify_final	37
3.1.3.27	smw_rng	37
3.1.3.28	smw_cipher	37
3.1.3.29	smw_cipher_init	38
3.1.3.30	smw_cipher_update	39
3.1.3.31	smw_cipher_final	39
3.1.3.32	smw_mac	40
3.1.3.33	smw_mac_verify	40
3.1.3.34	Operation Context	41
3.1.3.35	Authentication Encryption/Decryption (AEAD)	43
3.1.3.36	Asymmetric Encryption/Decryption	52
3.1.4	Key Manager APIs	54
3.1.4.1	struct smw_keypair_gen	54
3.1.4.2	struct smw_keypair_rsa	54
3.1.4.3	struct smw_keypair_buffer	55
3.1.4.4	struct smw_key_attributes	56
3.1.4.5	struct smw_key_descriptor	57
3.1.4.6	struct smw_derived_key_descriptor	57
3.1.4.7	struct smw_generate_key_args	58
3.1.4.8	struct smw_derive_key_args	59
3.1.4.9	struct smw_kdf_tls12_args	60
3.1.4.10	struct smw_kdf_tls12_random_data	62
3.1.4.11	struct smw_kdf_tls12_session_hash	62
3.1.4.12	struct smw_kdf_tls12_master_secret_args	63
3.1.4.13	struct smw_kdf_tls12_key_expansion_args	64
3.1.4.14	struct smw_kdf_tls12_op_args	65
3.1.4.15	struct smw_kdf_tls13_args	66
3.1.4.16	struct smw_hkdf_extract_args	67
3.1.4.17	struct smw_hkdf_expand_args	67

3.1.4.18	struct smw_hkdf_args	68
3.1.4.19	struct smw_kdf_hkdf_args	68
3.1.4.20	struct smw_kdf_ecdh_args	70
3.1.4.21	struct smw_import_key_args	70
3.1.4.22	struct smw_export_key_args	71
3.1.4.23	struct smw_delete_key_args	72
3.1.4.24	struct smw_get_key_attributes_args	72
3.1.4.25	struct smw_commit_key_storage_args	73
3.1.4.26	struct smw_key_attestation_args	73
3.1.4.27	smw_generate_key	74
3.1.4.28	smw_derive_key	74
3.1.4.29	smw_import_key	75
3.1.4.30	smw_export_key	75
3.1.4.31	smw_delete_key	76
3.1.4.32	smw_get_key_buffers_lengths	76
3.1.4.33	smw_get_key_type_name	76
3.1.4.34	smw_get_security_size	77
3.1.4.35	smw_get_key_attributes	77
3.1.4.36	smw_commit_key_storage	78
3.1.4.37	smw_key_attestation	78
3.1.4.38	OEM Master Key agreement	78
3.1.5	Device management APIs	80
3.1.5.1	Introduction	80
3.1.5.2	struct smw_device_attestation_args	80
3.1.5.3	struct smw_device_uuid_args	81
3.1.5.4	struct smw_device_lifecycle_args	82
3.1.5.5	struct smw_device_reprovision_args	82
3.1.5.6	smw_device_attestation	83
3.1.5.7	smw_device_get_uuid	84
3.1.5.8	smw_device_set_lifecycle	84
3.1.5.9	smw_device_get_lifecycle	85
3.1.5.10	smw_device_reprovision_prepare	85
3.1.5.11	smw_device_reprovision	85
3.1.6	Storage APIs	86
3.1.6.1	Introduction	86
3.1.6.2	struct smw_data_attributes	86
3.1.6.3	struct smw_data_descriptor	87
3.1.6.4	struct smw_encryption_args	87
3.1.6.5	struct smw_sign_args	88
3.1.6.6	struct smw_store_data_args	89
3.1.6.7	struct smw_retrieve_data_args	89
3.1.6.8	struct smw_delete_data_args	90
3.1.6.9	struct smw_data_info_args	90
3.1.6.10	smw_store_data	91
3.1.6.11	smw_retrieve_data	92
3.1.6.12	smw_delete_data	92
3.1.6.13	smw_get_data_info	92
3.1.7	Object management APIs	93
3.1.7.1	struct smw_object_descriptor	93
3.1.7.2	struct smw_find_object_db_args	94
3.1.7.3	smw_update_object_db	94

3.1.7.4	smw_find_object_db	95
3.1.7.5	smw_find_object_db_init	95
3.1.7.6	smw_find_object_db_next	96
3.1.7.7	smw_find_object_db_final	96
3.1.8	TLS Helper APIs	96
3.1.8.1	struct smw_tls13_expand_label_args	96
3.1.8.2	macro SMW_TLS13_EXPANDED_LABEL_LENGTH	97
3.1.8.3	smw_tls13_expand_label	98
3.1.9	Return codes	98
3.1.9.1	enum smw_status_code	98
3.1.10	Attributes APIs	106
3.1.10.1	Introduction	106
3.1.10.2	typedef smw_attr_algo_t	106
3.1.10.3	typedef smw_attr_usage_t	108
3.1.10.4	typedef smw_attr_attributes_t	108
3.1.10.5	typedef smw_attr_storage_id_t	109
3.1.10.6	SMW_ATTR_xxx_OFFSET (smw_attr_algo_t)	109
3.1.10.7	SMW_ATTR_xxx_MASK (smw_attr_algo_t)	109
3.1.10.8	SMW_ATTR_LENGTH_MIN_FLAG	110
3.1.10.9	SMW_ATTR_SALT_xxx	110
3.1.10.10	SMW_ATTR_MAC_xxx	110
3.1.10.11	SMW_ATTR_TAG_xxx	110
3.1.10.12	SMW_ATTR_SIGN_PARAM_xxx	110
3.1.10.13	SMW_ATTR_SIGN_HASHED_FLAG	110
3.1.10.14	SMW_ATTR_ALGO_xxx	111
3.1.10.15	SMW_ATTR_MODE_xxx	111
3.1.10.16	SMW_ATTR_CURVE_xxx	112
3.1.10.17	SMW_ATTR_HASH_xxx	112
3.1.10.18	SMW_ATTR_CLASS_xxx	113
3.1.10.19	SMW_ATTR_USAGE_xxx	113
3.1.10.20	SMW_ATTR_xxx_OFFSET (smw_attr_attributes_t)	114
3.1.10.21	SMW_ATTR_xxx_MASK (smw_attr_attributes_t)	114
3.1.10.22	SMW_ATTR_PERSISTENCE_xxx	114
3.1.10.23	SMW_ATTR_LIFECYCLE_xxx	114
3.1.10.24	SMW_ATTR_RW_FLAGS_xxx	114
3.1.10.25	macro SMW_ATTR_SET_LENGTH	114
3.1.10.26	macro SMW_ATTR_SET_MIN_LENGTH	115
3.1.10.27	macro SMW_ATTR_IS_MIN_LENGTH	115
3.1.10.28	macro SMW_ATTR_GET_LENGTH	115
3.1.10.29	macro SMW_ATTR_SET_SALT_LENGTH	116
3.1.10.30	macro SMW_ATTR_SET_MAC_LENGTH	116
3.1.10.31	macro SMW_ATTR_SET_TAG_LENGTH	116
3.1.10.32	macro SMW_ATTR_SET_MIN_SALT_LENGTH	117
3.1.10.33	macro SMW_ATTR_SET_MIN_MAC_LENGTH	117
3.1.10.34	macro SMW_ATTR_SET_MIN_TAG_LENGTH	117
3.1.10.35	macro SMW_ATTR_IS_MIN_SALT_LENGTH	118
3.1.10.36	macro SMW_ATTR_IS_MIN_MAC_LENGTH	118
3.1.10.37	macro SMW_ATTR_IS_MIN_TAG_LENGTH	118
3.1.10.38	macro SMW_ATTR_GET_SALT_LENGTH	119
3.1.10.39	macro SMW_ATTR_GET_MAC_LENGTH	119
3.1.10.40	macro SMW_ATTR_GET_TAG_LENGTH	119

3.1.10.41	macro	SMW_ATTR_SET_MSG_HASHED	120
3.1.10.42	macro	SMW_ATTR_IS_MSG_HASHED	120
3.1.10.43	macro	SMW_ATTR_GET_SIGN_PARAM	120
3.1.10.44	macro	SMW_ATTR_SET_SIGN_PARAM	121
3.1.10.45	macro	SMW_ATTR_SET_SIGN_EDDSA_PREHASHED	121
3.1.10.46	macro	SMW_ATTR_IS_SIGN_EDDSA_PREHASHED	121
3.1.10.47	macro	SMW_ATTR_SET_SIGN_EDDSA_CONTEXT	122
3.1.10.48	macro	SMW_ATTR_IS_SIGN_EDDSA_CONTEXT	122
3.1.10.49	macro	SMW_ATTR_ALGO_DIGEST	122
3.1.10.50	macro	SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION	123
3.1.10.51	macro	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_ECDSA	123
3.1.10.52	macro	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_EDDSA	123
3.1.10.53	macro	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_DSA	124
3.1.10.54	macro	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_RSA	124
3.1.10.55	macro	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_NO_LABEL	125
3.1.10.56	macro	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_CLIENT	125
3.1.10.57	macro	SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_SERVER	125
3.1.10.58	macro	SMW_ATTR_ALGO_MAC	126
3.1.10.59	macro	SMW_ATTR_ALGO_MAC_HMAC	126
3.1.10.60	macro	SMW_ATTR_ALGO_AEAD	126
3.1.10.61	macro	SMW_ATTR_ALGO_KEY_DERIVATION_HKDF	127
3.1.10.62	macro	SMW_ATTR_ALGO_KEY_DERIVATION_DH	127
3.1.10.63	macro	SMW_ATTR_ALGO_KEY_DERIVATION_ECDH	127
3.1.10.64	macro	SMW_ATTR_ALGO_KEY_DERIVATION_EDDSA	128
3.1.10.65	macro	SMW_ATTR_ALGO_KEY_DERIVATION_TLS12	128
3.1.10.66	macro	SMW_ATTR_ALGO_KEY_DERIVATION_TLS13	128
3.1.10.67	macro	SMW_ATTR_ALGO_KEY_ATTESTATION_MAC	129
3.1.10.68	macro	SMW_ATTR_ALGO_KEY_ATTESTATION_ECDSA	129
3.1.10.69	macro	SMW_ATTR_ALGO_KEY_AGREEMENT	129
3.1.10.70	macro	SMW_ATTR_ALGO_ASYMMETRIC_ENCRYPTION_RSA	130
3.1.10.71	macro	SMW_ATTR_GET_ALGO	130
3.1.10.72	macro	SMW_ATTR_GET_MODE	130
3.1.10.73	macro	SMW_ATTR_GET_CURVE	131
3.1.10.74	macro	SMW_ATTR_GET_KDF	131
3.1.10.75	macro	SMW_ATTR_GET_HASH	131
3.1.10.76	macro	SMW_ATTR_GET_CLASS	132
3.1.10.77	macro	SMW_ATTR_SET_HASH	132
3.1.10.78	macro	SMW_ATTR_SET_MODE	132
3.1.10.79	macro	SMW_ATTR_USAGE_SET_CACHE	133
3.1.10.80	macro	SMW_ATTR_USAGE_SET_COPY	133
3.1.10.81	macro	SMW_ATTR_USAGE_SET_EXPORT	133
3.1.10.82	macro	SMW_ATTR_USAGE_SET_ENCRYPT	134
3.1.10.83	macro	SMW_ATTR_USAGE_SET_DECRYPT	134
3.1.10.84	macro	SMW_ATTR_USAGE_SET_SIGN_MESSAGE	134
3.1.10.85	macro	SMW_ATTR_USAGE_SET_VERIFY_MESSAGE	135
3.1.10.86	macro	SMW_ATTR_USAGE_SET_SIGN_HASH	135
3.1.10.87	macro	SMW_ATTR_USAGE_SET_VERIFY_HASH	135
3.1.10.88	macro	SMW_ATTR_USAGE_SET_DERIVE	136
3.1.10.89	macro	SMW_ATTR_USAGE_IS_CACHE	136
3.1.10.90	macro	SMW_ATTR_USAGE_IS_COPY	136
3.1.10.91	macro	SMW_ATTR_USAGE_IS_EXPORT	137

3.1.10.92	macro	SMW_ATTR_USAGE_IS_ENCRYPT	137
3.1.10.93	macro	SMW_ATTR_USAGE_IS_DECRYPT	137
3.1.10.94	macro	SMW_ATTR_USAGE_IS_SIGN_MESSAGE	138
3.1.10.95	macro	SMW_ATTR_USAGE_IS_VERIFY_MESSAGE	138
3.1.10.96	macro	SMW_ATTR_USAGE_IS_SIGN_HASH	138
3.1.10.97	macro	SMW_ATTR_USAGE_IS_VERIFY_HASH	139
3.1.10.98	macro	SMW_ATTR_USAGE_IS_DERIVE	139
3.1.10.99	macro	SMW_ATTR_SET_PERSISTENCE	139
3.1.10.100	macro	SMW_ATTR_GET_PERSISTENCE	140
3.1.10.101	macro	SMW_ATTR_SET_TRANSIENT	140
3.1.10.102	macro	SMW_ATTR_SET_PERSISTENT	140
3.1.10.103	macro	SMW_ATTR_SET_PERMANENT	141
3.1.10.104	macro	SMW_ATTR_IS_TRANSIENT	141
3.1.10.105	macro	SMW_ATTR_IS_PERSISTENT	141
3.1.10.106	macro	SMW_ATTR_IS_PERMANENT	142
3.1.10.107	macro	SMW_ATTR_SET_LC_CURRENT	142
3.1.10.108	macro	SMW_ATTR_SET_LC_OPEN	142
3.1.10.109	macro	SMW_ATTR_SET_LC_CLOSED	143
3.1.10.110	macro	SMW_ATTR_SET_LC_CLOSED_LOCKED	143
3.1.10.111	macro	SMW_ATTR_IS_LC_CURRENT	143
3.1.10.112	macro	SMW_ATTR_IS_LC_OPEN	144
3.1.10.113	macro	SMW_ATTR_IS_LC_CLOSED	144
3.1.10.114	macro	SMW_ATTR_IS_LC_CLOSED_LOCKED	144
3.1.10.115	macro	SMW_ATTR_SET_READ_ONLY	145
3.1.10.116	macro	SMW_ATTR_SET_READ_ONCE	145
3.1.10.117	macro	SMW_ATTR_IS_READ_ONLY	145
3.1.10.118	macro	SMW_ATTR_IS_READ_ONCE	146
3.1.11	Names APIs		146
3.1.11.1	typedef	smw_subsystem_t	146
3.1.11.2	typedef	smw_operation_t	146
3.1.11.3	typedef	smw_object_type_t	148
3.1.11.4	typedef	smw_key_type_t	148
3.1.11.5	typedef	smw_key_privacy_t	149
3.1.11.6	typedef	smw_hash_algo_t	149
3.1.11.7	typedef	smw_mac_algo_t	150
3.1.11.8	typedef	smw_cipher_mode_t	150
3.1.11.9	typedef	smw_aead_mode_t	150
3.1.11.10	typedef	smw_cipher_op_type_t	151
3.1.11.11	typedef	smw_aead_op_type_t	151
3.1.11.12	typedef	smw_key_format_t	151
3.1.11.13	typedef	smw_signature_algo_t	151
3.1.11.14	typedef	smw_signature_type_t	152
3.1.11.15	typedef	smw_tls12_kek_t	152
3.1.11.16	typedef	smw_tls12_enc_t	153
3.1.11.17	typedef	smw_tls12_op_t	153
3.1.11.18	typedef	smw_oem_master_key_op_t	154
3.1.11.19	typedef	smw_lifecycle_t	154
3.1.11.20	typedef	smw_kdf_t	154
3.1.11.21	typedef	smw_asymmetric_encryption_algo_t	155
3.1.11.22	typedef	smw_asymmetric_encryption_mode_t	155
3.1.12	Examples		155

3.1.12.1	Authentication Encryption/Decryption (AEAD)	155
3.2	PSA APIs	163
3.2.1	Cryptography APIs	163
3.2.1.1	Values	163
3.2.1.2	Sizes	211
3.2.1.3	Types	230
3.2.1.4	Structures	236
3.2.1.5	Functions	237
3.2.2	Initial Attestation APIs	339
3.2.2.1	Introduction	339
3.2.2.2	Reference	339
3.2.2.3	psa_initial_attest_get_token	340
3.2.2.4	psa_initial_attest_get_token_size	340
3.2.2.5	psa_attest_key	341
3.2.2.6	psa_attest_key_get_size	342
3.2.3	Storage APIs	343
3.2.3.1	Storage common APIs	343
3.2.3.2	Internal trusted storage APIs	344
3.2.3.3	Protected storage APIs	347
3.2.4	Return codes	354
3.2.4.1	Introduction	354
3.2.4.2	Reference	354
3.2.4.3	typedef psa_status_t	354
4	Subsystems Capabilities	359
4.1	ELE capabilities	359
4.1.1	Key manager	359
4.1.1.1	Key policy	360
4.1.2	Hash	363
4.1.3	Signature	364
4.1.3.1	Sign operation	365
4.1.3.2	Verify operation	365
4.1.4	Random	365
4.1.5	MAC	366
4.1.5.1	Compute MAC operation	366
4.1.5.2	Verify MAC operation	367
4.1.6	Cipher	368
4.1.6.1	Encrypt operation	368
4.1.6.2	Decrypt operation	368
4.1.7	AEAD	369
4.1.8	Asymmetric encryption and decryption	369
4.1.9	Device management	370
4.1.9.1	Device Attestation	370
4.1.9.2	Device lifecycle	370
4.1.10	Operation context	371
4.1.11	Data Storage manager	371
4.1.12	Key attestation	372
4.1.13	Storage Re-provisioning	373
4.1.14	Key Derivation	374
4.1.14.1	TLS 1.2 (TLS1-PRF)	374
4.1.14.2	TLS 1.3 (TLS13-KDF)	374

4.1.14.3	OEM Master key	375
4.1.14.4	Key Import	376
4.1.14.5	EdgeLock 2GO blob	377
4.1.14.6	EdgeLock Enclave blob	377
4.2	SECO capabilities	389
4.2.1	Key manager	389
4.2.1.1	Key policy	390
4.2.2	Hash	390
4.2.3	Signature	391
4.2.4	Random	391
4.2.5	MAC	391
4.2.6	Cipher	392
4.2.7	Operation context	392
4.2.8	Data Storage manager	392
4.2.9	AEAD	393
4.2.10	Key Derivation	393
4.3	TEE capabilities	394
4.3.1	Key manager	394
4.3.1.1	Key policy	394
4.3.2	Hash	395
4.3.3	Signature	397
4.3.4	MAC	398
4.3.5	Random	398
4.3.6	Cipher	399
4.3.7	Operation context	399
4.3.8	AEAD	399
4.3.9	Asymmetric encryption and decryption	400
4.3.10	Data Storage manager	400
4.3.11	Key Derivation	400
5	OSAL	402
5.1	SMW interface	402
5.1.1	Introduction	402
5.1.2	struct smw_osal_object	402
5.1.2.1	Definition	402
5.1.2.2	Members	402
5.1.2.3	Description	403
5.1.3	struct smw_ops	403
5.1.3.1	Definition	403
5.1.3.2	Members	403
5.1.3.3	Description	404
5.1.4	smw_init	404
5.1.4.1	Description	405
5.1.4.2	Return	405
5.1.5	smw_deinit	405
5.1.5.1	Description	405
5.1.5.2	Return	405
5.2	Linux example	405
5.2.1	Introduction	405
5.2.2	struct tee_info	406
5.2.2.1	Definition	406

5.2.2.2	Members	407
5.2.3	struct se_info	407
5.2.3.1	Definition	407
5.2.3.2	Members	407
5.2.4	union subsystem_info	407
5.2.4.1	Definition	407
5.2.4.2	Members	407
5.2.5	smw_osal_latest_subsystem_name	408
5.2.5.1	Description	408
5.2.5.2	Return	408
5.2.6	smw_osal_lib_init	408
5.2.6.1	Description	408
5.2.6.2	Return	408
5.2.7	smw_osal_set_subsystem_info	408
5.2.7.1	Description	408
5.2.7.2	Return	409
5.2.8	smw_osal_open_obj_db	409
5.2.8.1	Description	409
5.2.8.2	Return	409

INTRODUCTION

The Security Middleware library provides an application's generic API to perform Security operations supported by the Secure Subsystems present in the system.

This documentation is automatically generated from the source code.

It describes the public functions and the public structures used to perform the Security operations.

It also describes the return codes of these public functions.

HOW TO WRITE A CONFIGURATION FILE

This section describes how to write a configuration file. It gives the syntactic rules, the list of valid inputs and some examples.

2.1 Syntactic rules

The configuration format must respect following rules:

- Characters must be encoded in ASCII.
- Semicolon specifies end of line.
- Colon is a separator of multiples entries.
- Spaces and line separators are ignored.
- Decimal numbers are written using the US/UK format (i.e. separator is ‘.’).
- Negative numbers are preceded by ‘-’.
- String must not be quoted.
- Commented sections start with ‘/*’ and finish with ‘*/’. Comments are ignored.
- The first tag to define is the **VERSION** tag specifying the parser version compatibility.
- The tag **PSA_DEFAULT**, if present, must be after the tag **VERSION**. If present after the first occurrence of **[SECURE_SUBSYSTEM]**, it is ignored. The possible values are the Secure Subsystems names listed in [Table 2.1](#). Adding option **ALT** (“:ALT”) after the Secure Subsystem name allows the selection of another Secure Subsystem if the default one doesn’t support the requested Security Operation.
- There must be at least one occurrence of **[SECURE_SUBSYSTEM]**. This tag is the starter of a Secure Subsystem configuration.
- There must be only one block defining a Secure Subsystem.
- There must be only one **<string: name of subsystem>** per **[SECURE_SUBSYSTEM]**.
- The load/unload method **<string: load/unload method>** is optional. Only one occurrence is allowed if present.
- There must be at least one occurrence of **[SECURITY_OPERATION]** per **[SECURE_SUBSYSTEM]**.
- There must be one **<string: name of operation>** set in **[SECURITY_OPERATION]**. The possible values of string correspond to the external interfaces of each module as listed in [Table 2.3](#).

- There must be only one block defining a Security Operation for a given Secure Subsystem.
- The Security Operation can define its capabilities values (e.g. key types, hash algorithms...) using tags **<param#>_VALUES** (as listed in Table 2.4). Each value is a non-quoted string separated by a colon.
- The Security Operation can define its capabilities range using tags **<param#>_RANGE** (as listed in Table 2.5). Range values are integer defining minimum and/or maximum capability value.

Configuration file subsystem/operation definition pair represents the subsystem selection order when Secure Subsystem is not specified in the operation arguments.

Notes: Secure Subsystem is implicit when an operation uses a key identifier of a key already present in the Secure Subsystem key storage.

2.2 Secure Subsystems definition

```
[SECURE_SUBSYSTEM]
  <string: name of subsystem>;
  <string: load/unload method>;
  [SECURITY_OPERATION]
  ...
  [SECURITY_OPERATION]
  ...
```

The Secure Subsystems definition starts with the tag [SECURE_SUBSYSTEM] and is followed by its string name. The Table 2.1 below lists all Secure Subsystems supported by the Security Middleware library.

Following the Secure Subsystem capabilities, the configuration contains one or more operations associated to the Secure Subsystem as defined *Security Operations definition*.

Table 2.1: Secure Subsystems

Secure Subsystem string name	Description
SECO	Use the SECO protected secure mode on i.MX8 devices enabling a SECO Secure Enclave.
TEE	Use the Secure OS called OPTEE and running in ARM Trustzone Secure world.
ELE	Use the ELE (EdgeLock Enclave) protected secure mode on: • i.MX8ULP • i.MX9x

A different load and unload method can be specified for each Secure Subsystem through the <string: load/unload method> string following the subsystem's string name. The following Table 2.2 defines the possible string value of the load/unload method.

Table 2.2: Subsystem load/unload methods

Load/Unload string method	Description
AT_FIRST_CALL_LOAD	At first Secure Subsystem call, the Secure Subsystem is loaded. The Secure Subsystem is unloaded when the configuration is unloaded.
AT_CONTEXT_CREATION_DESTRUCTION	At Secure Subsystem context creation, the Secure Subsystem is loaded. It is unloaded when the Secure Subsystem operation is finish. In case multi-part operation, when the final step is performed or when the cancel context operation is called.

2.3 Security Operations definition

```
[SECURITY_OPERATION]
  <string: name of operation>;
  /* A combination of the lines below describes */
  /* the Secure Subsystem capabilities for this Security Operation. */
  <param1>_VALUES=<value1>:<value2>:<value3>;
  <param2>_RANGE=<integer: min>:<integer: max>;
  <param3>_RANGE=<integer: max>; /* threshold lower than */
  <param4>_RANGE=<integer: min>; /* threshold greater than */
```

The Security Operation starts with the tag [SECURITY_OPERATION] and is followed by its string name. The [Table 2.3](#) below lists all Security -Operations supported by the Security Middleware library.

Table 2.3: Security Operations

Security name	Operation string	Description
GENERATE_KEY		Generate a cryptographic key (private, keypair). Public key can be exported.
DERIVE_KEY		Derive a key from an existing cryptographic key.
IMPORT_KEY		Import cryptographic key (public, private, keypair).
EXPORT_KEY		Export cryptographic key. Private key exportation is function of the Secure Subsystem capabilities. Public key are always exportable.
DELETE_KEY		Delete a key.
HASH		Hash a message.
MAC		Message Authentication Code.
SIGN		Sign a message.
VERIFY		Verify the signature of a message.
CIPHER		Cipher encryption and decryption.
CIPHER_MULTI_PART		Cipher multi-part encryption and decryption.
AEAD		Authentication Encryption.
AEAD_MULTI_PART		Authentication Encryption multi-part.
ASYMM_ENCRYPT		Asymmetric Encryption.
ASYMM_DECRYPT		Asymmetric Decryption.
RNG		Generate a Random data number.
DEVICE_ATTESTATION		Device attestation certificate.
DEVICE_LIFECYCLE		Get and Set device lifecycle.
DEVICE_REPROVISION		Request the Secure Subsystem to reset the rollback protection to reprovision a new Non-volatile Secure Storage.
STORAGE_STORE		Store data in secure storage.
STORAGE_RETRIEVE		Retrieve data from secure storage.
STORAGE_DELETE		Delete data from secure storage.

Each Security Operations definition can specify capabilities using Values and Range tags definition as listed in the following tables.

Table 2.4: Security Operation values tag

Tag Values	Description
ALGO_VALUES	Define the security operation algorithms supported.
MODE_VALUES	Define the modes supported for a cryptographic security operation. e.g. Cipher modes(CBC, ECB etc.) for cipher operations Encryption padding schemes(PKCS1_1_5, OAEP, NO_PAD)
HASH_ALGO_VALUES	Define the Hash algorithms supported for a cryptographic security operation.
MAC_ALGO_VALUES	Define the MAC operation algorithms supported for a cryptographic security operation.
KEY_TYPE_VALUES	Define the Key types supported for a cryptographic security operation.
SIGN_TYPE_VALUES	Define the signature scheme supported for signature security operations (sign and verify).
OP_TYPE_VALUES	Define the type of operation when it has multiple possibilities. (ex: encryption vs decryption for cipher operation).
ENC_ALGO_VALUES	Define the asymmetric encryption algorithms supported for a cryptographic security operation.

Table 2.5: Security Operation range tags

Tag Range	Description
<KEY_TYPE>_SIZE_RANGE	Define the minimum and maximum key size bits of a key type listed by the KEY_TYPE_VALUES tag.
RNG_LENGTH_RANGE	Define the length range of a random number generated with the RNG operation.

Notice that all Values or Range are not useful for each operation. Refer to each operation to get the tags that could be defined and the corresponding value.

2.4 Example

On Linux the plaintext configuration may be a text file. This example defines the configuration supporting 2 Secure Subsystems: OPTEE and SECO.

PSA default Secure Subsystem is OPTEE. Secure Subsystem selection is enabled if OPTEE does not support the requested Security Operation.

OPTEE configuration:

- Subsystem is loaded/unloaded when configuration is loaded and unloaded, refer to *Secure Subsystems definition*.

- Cipher AES (ECB and CBC) and DES (ECB and CBC) operation. OPTEE is the default subsystem for this operation for the defined keys and modes.
- All keys defined by the Security Middleware can be generated using OPTEE Secure Subsystem.

SECO configuration:

- Subsystem is loaded/unloaded with the default method as defined in *Secure Subsystems definition*.
- Digest SHA256 operation.
- Generate 128 bits to 256 bits AES keys.
- Generate 56 bits DES keys.
- SECO is the default subsystem for this operation for the defined key capabilities.

```
/* Configuration file */
VERSION=1;
PSA_DEFAULT=TEE:ALT;
[SECURE_SUBSYSTEM]
    TEE;
    /* Load/unload method */
    AT_FIRST_CALL_LOAD;
    [SECURITY_OPERATION]
        CIPHER;
        /* Only AES and DES keys are supported */
        KEY_TYPE_VALUES=AES:DES;
        /* Only ECB and CBC modes are supported */
        MODE_VALUES=ECB:CBC;
    [SECURITY_OPERATION]
        GENERATE_KEY;
        /* No specific capabilities - all parameters are accepted */
[SECURE_SUBSYSTEM]
    SECO;
    /* No Load/unload method specified. Default is 1. */
    [SECURITY_OPERATION]
        HASH;
        HASH_ALGO_VALUES=SHA256;
    [SECURITY_OPERATION]
        GENERATE_KEY;
        /* Only AES and DES algorithms are supported */
        KEY_TYPE_VALUES=AES:DES;
        /* AES key size allowed is between 128 bits and 256 bits */
        AES_SIZE_RANGE=128:256;
        /* DES key size allowed is 56 bits */
        DES_SIZE_RANGE=56:56;
```

PUBLIC APIS

Two sets of APIs are exposed:

- The NXP's Security Middleware APIs called SMW APIs
- The ARM's Platform Security Architecture APIs called PSA APIs

Both APIs set are working independently without concurrence.

3.1 SMW APIs

3.1.1 Library information APIs

3.1.1.1 Introduction

Library user can get general library information using following APIs.

3.1.1.2 smw_get_version

enum *smw_status_code* **smw_get_version**(unsigned int *major, unsigned int *minor)

Get the library version.

Parameters

- **major** (unsigned int*) – Library major version.
- **minor** (unsigned int*) – Library minor version.

3.1.1.2.1 Return

See *enum smw_status_code*

- **SMW_STATUS_OK:**
Success
- **SMW_STATUS_INVALID_PARAM:**
Either major or minor parameter is NULL

3.1.2 Configuration APIs

3.1.2.1 Introduction

The configuration APIs allow user of the library to:

- Get information about the state of the Secure Subsystems.
- Get the capabilities of the library operations.

-
- Load/Unload library configuration.

3.1.2.2 smw_config_subsystem_present

enum *smw_status_code* **smw_config_subsystem_present**(*smw_subsystem_t* subsystem)

Check if the subsystem is present or not.

Parameters

- **subsystem** (*smw_subsystem_t*) – Name of the subsystem.

3.1.2.2.1 Return

See *enum smw_status_code*

- **SMW_STATUS_OK:**
subsystem is present
- **SMW_STATUS_INVALID_PARAM:**
subsystem is SMW_SUBSYSTEM_NAME_NONE
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
subsystem is not valid

3.1.2.3 smw_config_subsystem_loaded

enum *smw_status_code* **smw_config_subsystem_loaded**(*smw_subsystem_t* subsystem)

Return if the subsystem is loaded or not.

Parameters

- **subsystem** (*smw_subsystem_t*) – Name of the subsystem.

3.1.2.3.1 Return

See *enum smw_status_code*

- **SMW_STATUS_SUBSYSTEM_LOADED:**
subsystem is loaded
- **SMW_STATUS_SUBSYSTEM_NOT_LOADED:**
subsystem is not loaded
- **SMW_STATUS_INVALID_PARAM:**
subsystem is SMW_SUBSYSTEM_NAME_NONE
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
subsystem is not valid
- **SMW_STATUS_INVALID_LIBRARY_CONTEXT:**
Library context is not valid

3.1.2.4 smw_config_check_digest

enum *smw_status_code* **smw_config_check_digest**(*smw_subsystem_t* subsystem,
smw_hash_algo_t algo)

Check if a digest algo is supported

Parameters

- **subsystem** (*smw_subsystem_t*) – Name of the subsystem.
- **algo** (*smw_hash_algo_t*) – Digest algorithm name.

3.1.2.4.1 Description

Function checks if the digest algo is supported on the given subsystem. If subsystem is SMW_SUBSYSTEM_NAME_NONE, default subsystem digest capability is checked.

3.1.2.4.2 Return

See *enum smw_status_code*

- **SMW_STATUS_OK:**
algo is supported
- **SMW_STATUS_INVALID_PARAM:**
algo is SMW_HASH_ALGO_NAME_NONE
- **SMW_STATUS_UNKNOWN_ALGO_NAME:**
algo is not valid
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
algo is not supported
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
subsystem is not valid

3.1.2.5 struct smw_key_info

struct **smw_key_info**

Key information

3.1.2.5.1 Definition

```
struct smw_key_info {  
    smw_key_type_t key_type_name;  
    unsigned int security_size;  
    unsigned int security_size_min;  
    unsigned int security_size_max;  
}
```

3.1.2.5.2 Members

key_type_name

Key type name. See *typedef smw_key_type_t*

security_size

Key security size in bits

security_size_min

Key security size minimum in bits

security_size_max

Key security size maximum in bits

3.1.2.6 smw_config_check_generate_key

enum *smw_status_code* **smw_config_check_generate_key**(*smw_subsystem_t* subsystem, struct *smw_key_info* *info)

Check generate key type

Parameters

- **subsystem** (*smw_subsystem_t*) – Name of the subsystem.
- **info** (struct *smw_key_info**) – Key information.

3.1.2.6.1 Description

Function checks if the key type provided in the **info** structure is supported on the given subsystem.

If **info**'s security size field is equal 0, returns the security key range size in bits supported by the subsystem for the key type. Else checks if the security size is supported.

If **subsystem** is SMW_SUBSYSTEM_NAME_NONE, default subsystem key generation is checked.

3.1.2.6.2 Return

See enum *smw_status_code*

- **SMW_STATUS_OK:**
Key type is supported
- **SMW_STATUS_INVALID_PARAM:**
info is NULL or info->key_type_name is SMW_KEY_TYPE_NAME_NONE
- **SMW_STATUS_UNKNOWN_KEY_TYPE_NAME:**
info->key_type_name is not valid
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
Key type is not supported
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
subsystem is not valid

3.1.2.7 smw_config_check_derive_key

enum *smw_status_code* **smw_config_check_derive_key**(*smw_subsystem_t* subsystem, *smw_kdf_t* kdf)

Check if KDF kdf is supported.

Parameters

- **subsystem** (*smw_subsystem_t*) – Name of the subsystem.
- **kdf** (*smw_kdf_t*) – KDF name.

3.1.2.7.1 Description

Function checks if the KDF `kdf` is supported on the given subsystem.

If `subsystem` is `SMW_SUBSYSTEM_NAME_NONE`, function checks if the KDF is supported on the default subsystem.

3.1.2.7.2 Return

See `enum smw_status_code`

- **SMW_STATUS_OK:**
KDF is supported
- **SMW_STATUS_INVALID_PARAM:**
`kdf` is `SMW_KDF_NAME_NONE`
- **SMW_STATUS_UNKNOWN_KDF_NAME:**
`kdf` is not valid KDF
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
`kdf` is not supported
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
`subsystem` is not valid

3.1.2.8 struct smw_signature_info

struct **smw_signature_info**

Signature operation information

3.1.2.8.1 Definition

```
struct smw_signature_info {  
    smw_signature_algo_t algo_name;  
    smw_signature_type_t type_name;  
    smw_hash_algo_t hash_algo_name;  
}
```

3.1.2.8.2 Members

algo_name

Signature algo name. See `typedef smw_signature_algo_t`

type_name

Signature type name. See `typedef smw_signature_type_t`

hash_algo_name

Hash algorithm name. See `typedef smw_hash_algo_t`

3.1.2.9 smw_config_check_sign

enum `smw_status_code` **smw_config_check_sign**(`smw_subsystem_t` subsystem, struct `smw_signature_info` *info)

Check if signature generation operation is supported

Parameters

- **subsystem** (*smw_subsystem_t*) – Name of the subsystem.
- **info** (struct *smw_signature_info**) – Signature information.

3.1.2.9.1 Description

info hash algorithm name and signature type name fields are optional.

If set, function checks if the hash algorithm is supported on the given **subsystem** for the signature generation operation. If set, function checks if the signature type is supported on the given **subsystem** for the signature generation operation.

If **subsystem** is **SMW_SUBSYSTEM_NAME_NONE**, default subsystem signature generation capability is checked.

3.1.2.9.2 Return

See **enum smw_status_code**

- **SMW_STATUS_OK:**
Signature operation is supported
- **SMW_STATUS_INVALID_PARAM:**
info is NULL or **info->algo_name** is **SMW_SIGNATURE_ALGO_NAME_NONE**
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
Signature operation is not supported
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
subsystem is not valid

3.1.2.10 smw_config_check_verify

enum smw_status_code smw_config_check_verify(*smw_subsystem_t* subsystem, struct *smw_signature_info* *info)

Check if signature verification operation is supported

Parameters

- **subsystem** (*smw_subsystem_t*) – Name of the subsystem.
- **info** (struct *smw_signature_info**) – Signature information.

3.1.2.10.1 Description

info hash algorithm name and signature type name fields are optional.

If set, function checks if the hash algorithm is supported on the given **subsystem** for the signature verification operation. If set, function checks if the signature type is supported on the given **subsystem** for the signature verification operation.

If **subsystem** is **SMW_SUBSYSTEM_NAME_NONE**, default subsystem signature verification capability is checked.

3.1.2.10.2 Return

See `enum smw_status_code`

- **SMW_STATUS_OK:**
Verify operation is supported
- **SMW_STATUS_INVALID_PARAM:**
info is NULL or info->algo_name is SMW_SIGNATURE_ALGO_NAME_NONE
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
Verify operation is not supported

3.1.2.11 struct smw_cipher_info

struct **smw_cipher_info**

Cipher operation information

3.1.2.11.1 Definition

```
struct smw_cipher_info {  
    bool multipart;  
    smw_key_type_t key_type_name;  
    smw_cipher_mode_t mode_name;  
    smw_cipher_op_type_t op_type_name;  
}
```

3.1.2.11.2 Members

multipart

True if it's a cipher multi-part operation

key_type_name

Key type name. See `typedef smw_key_type_t`

mode_name

Operation mode name. See `typedef smw_cipher_mode_t`

op_type_name

Operation type name. See `typedef smw_cipher_op_type_t`

3.1.2.12 smw_config_check_cipher

enum `smw_status_code` **smw_config_check_cipher**(`smw_subsystem_t` subsystem, struct `smw_cipher_info` *info)

Check if cipher operation is supported

Parameters

- **subsystem** (`smw_subsystem_t`) – Name of the subsystem.
- **info** (struct `smw_cipher_info*`) – Cipher information.

3.1.2.12.1 Description

Function checks if all fields provided in the `info` structure are supported on the given subsystem for a cipher one-shot or multi-part operation.

If subsystem is `SMW_SUBSYSTEM_NAME_NONE`, default subsystem cipher capability is checked.

3.1.2.12.2 Return

See `enum smw_status_code`

- **SMW_STATUS_OK:**
Cipher operation is supported
- **SMW_STATUS_INVALID_PARAM:**
info is NULL or info->key_type_name is `SMW_KEY_TYPE_NAME_NONE` or info->mode_name is `SMW_CIPHER_MODE_NAME_NONE` or info->op_type_name is `SMW_CIPHER_OP_TYPE_NAME_NONE`
- **SMW_STATUS_UNKNOWN_KEY_TYPE_NAME:**
info->key_type_name is not valid
- **SMW_STATUS_UNKNOWN_MODE_NAME:**
info->mode is not valid
- **SMW_STATUS_UNKNOWN_OP_TYPE_NAME:**
info->op_type is not valid
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
Cipher operation is not supported
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
subsystem is not valid

3.1.2.13 struct smw_aead_info

struct **smw_aead_info**

AEAD operation information

3.1.2.13.1 Definition

```
struct smw_aead_info {  
    bool multipart;  
    smw_key_type_t key_type_name;  
    smw_aead_mode_t mode_name;  
    smw_aead_op_type_t op_type_name;  
}
```

3.1.2.13.2 Members

multipart

True if it's a AEAD multi-part operation

key_type_name

Key type name. See `typedef smw_key_type_t`

mode_name

Operation mode name. See *typedef smw_aead_mode_t*

op_type_name

Operation type name. See *typedef smw_aead_op_type_t*

3.1.2.14 smw_config_check_aead

enum *smw_status_code* **smw_config_check_aead**(*smw_subsystem_t* subsystem, struct *smw_aead_info* *info)

Check if AEAD operation is supported

Parameters

- **subsystem** (*smw_subsystem_t*) – Name of the subsystem.
- **info** (struct *smw_aead_info**) – AEAD information.

3.1.2.14.1 Description

Function checks if all fields provided in the *info* structure are supported on the given *subsystem* for a AEAD one-shot or multi-part operation.

If *subsystem* is *SMW_SUBSYSTEM_NAME_NONE*, default subsystem AEAD capability is checked.

3.1.2.14.2 Return

See *enum smw_status_code*

- **SMW_STATUS_OK:**
AEAD operation is supported
- **SMW_STATUS_INVALID_PARAM:**
info is NULL or *info->key_type_name* is *SMW_KEY_TYPE_NAME_NONE* or *info->mode_name* is *SMW_AEAD_MODE_NAME_NONE* or *info->op_type_name* is *SMW_AEAD_OP_TYPE_NAME_NONE*
- **SMW_STATUS_UNKNOWN_KEY_TYPE_NAME:**
info->key_type_name is not valid
- **SMW_STATUS_UNKNOWN_MODE_NAME:**
info->mode is not valid
- **SMW_STATUS_UNKNOWN_OP_TYPE_NAME:**
info->op_type is not valid
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
AEAD operation is not supported
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
subsystem is not valid

3.1.2.15 struct smw_mac_info

struct **smw_mac_info**

MAC operation information

3.1.2.15.1 Definition

```
struct smw_mac_info {  
    smw_key_type_t key_type_name;  
    smw_mac_algo_t mac_algo_name;  
    smw_hash_algo_t hash_algo_name;  
}
```

3.1.2.15.2 Members

key_type_name

Key type name. See `typedef smw_key_type_t`

mac_algo_name

MAC algorithm name. See `typedef smw_mac_algo_t`

hash_algo_name

Hash algorithm name. See `typedef smw_hash_algo_t`

3.1.2.16 smw_config_check_mac

enum `smw_status_code` **smw_config_check_mac**(`smw_subsystem_t` subsystem, struct `smw_mac_info` *info)

Check if MAC operation is supported

Parameters

- **subsystem** (`smw_subsystem_t`) – Name of the subsystem.
- **info** (struct `smw_mac_info`*) – MAC information.

3.1.2.16.1 Description

Function checks if all fields provided in the `info` structure are supported on the given `subsystem` for MAC operation. If set, function checks if the hash algorithm is supported on the given `subsystem` for the signature generation operation. If set, function checks if the MAC algorithm is supported on the given `subsystem` for the signature generation operation.

If `subsystem` is `SMW_SUBSYSTEM_NAME_NONE`, default subsystem MAC capability is checked.

3.1.2.16.2 Return

See `enum smw_status_code`

- **SMW_STATUS_OK:**
MAC operation is supported
- **SMW_STATUS_INVALID_PARAM:**
info is NULL or info->key_type_name is `SMW_KEY_TYPE_NAME_NONE`
- **SMW_STATUS_UNKNOWN_KEY_TYPE_NAME:**
info->key_type_name is not valid
- **SMW_STATUS_UNKNOWN_ALGO_NAME:**
info->mac_algo or info->hash_algo is not valid

-
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
MAC operation is not supported
 - **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
subsystem is not valid

3.1.2.17 smw_config_load

enum *smw_status_code* **smw_config_load**(char *buffer, unsigned int size, unsigned int *offset)

Load a configuration.

Parameters

- **buffer** (char*) – pointer to the plaintext configuration.
- **size** (unsigned int) – size of the plaintext configuration.
- **offset** (unsigned int*) – current offset in plaintext configuration.

3.1.2.17.1 Description

This function loads a configuration. The plaintext configuration is parsed and the content is stored in the Configuration database. If the parsing of plaintext configuration fails, **offset** points to the number of characters that have been correctly parsed. The beginning of the remaining plaintext which cannot be parsed is printed out.

3.1.2.17.2 Return

SMW_STATUS_OK - Configuration load is successful
SMW_STATUS_INVALID_LIBRARY_CONTEXT - Library context is not valid
SMW_STATUS_INVALID_BUFFER - buffer is NULL or size is 0
SMW_STATUS_CONFIG_ALREADY_LOADED - A configuration is already loaded
error code otherwise

3.1.2.18 smw_config_unload

enum *smw_status_code* **smw_config_unload**(void)

Unload the current configuration.

Parameters

- **void** – no arguments

3.1.2.18.1 Description

This function unloads the current configuration. It frees all memory dynamically allocated by SMW.

3.1.2.18.2 Return

SMW_STATUS_OK - Configuration unload is successful
SMW_STATUS_INVALID_LIBRARY_CONTEXT - Library context is not valid
SMW_STATUS_NO_CONFIG_LOADED - No configuration is loaded

3.1.2.19 struct smw_asymmetric_encrypt_info

struct **smw_asymmetric_encrypt_info**

Asymmetric encryption/decryption operation information

3.1.2.19.1 Definition

```
struct smw_asymmetric_encrypt_info {  
    smw_asymmetric_encryption_algo_t algo_name;  
    smw_asymmetric_encryption_mode_t mode_name;  
    smw_hash_algo_t hash_algo_name;  
}
```

3.1.2.19.2 Members

algo_name

Encryption/decryption algorithm name. See *typedef smw_asymmetric_encryption_algo_t*

mode_name

Encryption/decryption mode (padding scheme) name. See *typedef smw_asymmetric_encryption_mode_t*

hash_algo_name

Hash algorithm name. See *typedef smw_hash_algo_t*

3.1.2.20 smw_config_check_asymmetric_encrypt

enum *smw_status_code* **smw_config_check_asymmetric_encrypt**(*smw_subsystem_t* subsystem,
struct
smw_asymmetric_encrypt_info
*info)

Check if asymmetric encryption operation is supported

Parameters

- **subsystem** (*smw_subsystem_t*) – Name of the subsystem.
- **info** (struct *smw_asymmetric_encrypt_info**) – Asymmetric encryption/decryption operation information.

3.1.2.20.1 Description

`info.hash_algo_name` and `info.mode_name` fields are optional. `info.algo_name` is mandatory filled and the function checks if the algorithm is supported on the given `subsystem` for the asymmetric encryption operation. If `info.hash_algo_name` is set, function checks if the hash algorithm is supported on the given `subsystem` for the asymmetric encryption operation. If `info.mode_name` is set, function checks if the encryption mode is supported on the given `subsystem` for the asymmetric encryption operation.

If `subsystem` is `SMW_SUBSYSTEM_NAME_NONE`, default subsystem asymmetric encryption capability is checked.

3.1.2.20.2 Return

See `enum smw_status_code`

- **SMW_STATUS_OK:**
Encryption operation is supported
- **SMW_STATUS_INVALID_PARAM:**
info is NULL or info->algo_name is SMW_ASYMMETRIC_ENCRYPTION_ALGO_NAME_NONE
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
Encryption operation is not supported
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
subsystem is not valid

3.1.2.21 smw_config_check_asymmetric_decrypt

`enum smw_status_code smw_config_check_asymmetric_decrypt(smw_subsystem_t subsystem, struct smw_asymmetric_encrypt_info *info)`

Check if asymmetric decryption operation is supported

Parameters

- **subsystem** (`smw_subsystem_t`) – Name of the subsystem.
- **info** (struct `smw_asymmetric_encrypt_info*`) – Asymmetric encryption/decryption operation information.

3.1.2.21.1 Description

info.hash_algo_name and info.mode_name fields are optional. info.algo_name is mandatory filled and the function checks if the algorithm is supported on the given subsystem for the asymmetric decryption operation. If info.hash_algo_name is set, function checks if the hash algorithm is supported on the given subsystem for the asymmetric decryption operation. If info.mode_name is set, function checks if the decryption mode is supported on the given subsystem for the asymmetric decryption operation.

If subsystem is SMW_SUBSYSTEM_NAME_NONE, default subsystem asymmetric decryption capability is checked.

3.1.2.21.2 Return

See `enum smw_status_code`

- **SMW_STATUS_OK:**
Decryption operation is supported
- **SMW_STATUS_INVALID_PARAM:**
info is NULL or info->algo_name is SMW_ASYMMETRIC_ENCRYPTION_ALGO_NAME_NONE
- **SMW_STATUS_OPERATION_NOT_CONFIGURED:**
Decryption operation is not supported
- **SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME:**
subsystem is not valid

3.1.3 Cryptography APIs

3.1.3.1 struct smw_hash_args

struct **smw_hash_args**

Hash arguments

3.1.3.1.1 Definition

```
struct smw_hash_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    smw_hash_algo_t algo_name;  
    unsigned char *input;  
    unsigned int input_length;  
    unsigned char *output;  
    unsigned int output_length;  
}
```

3.1.3.1.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

algo_name

Algorithm name. See *typedef smw_hash_algo_t*

input

Location of the stream to be hashed

input_length

Length of the stream to be hashed

output

Location where the digest has to be written

output_length

Length of the digest

3.1.3.1.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is `SMW_SUBSYSTEM_NAME_NONE`, the default configured Secure Subsystem is used.

3.1.3.2 struct smw_hash_init_args

struct **smw_hash_init_args**

Hash multi-part initialization arguments

3.1.3.2.1 Definition

```
struct smw_hash_init_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    smw_hash_algo_t algo_name;  
    unsigned char *input;  
    unsigned int input_length;  
    struct smw_op_context *context;  
}
```

3.1.3.2.2 Members

version

Version of this structure (must be set to 1)

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

algo_name

Algorithm name. See *typedef smw_hash_algo_t*

input

Location of the stream to be hashed

input_length

Length of the stream to be hashed

context

Pointer to an opaque operation context structure

3.1.3.2.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is `SMW_SUBSYSTEM_NAME_NONE`, the default configured Secure Subsystem is used.

`version` must be set to 1 to support the `subsystem_name` field.

3.1.3.3 struct smw_hash_update_args

struct **smw_hash_update_args**

Hash multi-part update arguments

3.1.3.3.1 Definition

```
struct smw_hash_update_args {  
    unsigned char version;  
    struct smw_op_context *context;  
    unsigned char *input;  
    unsigned int input_length;  
}
```

3.1.3.3.2 Members

version

Version of this structure

context

Pointer to an opaque operation context structure

input

Location of the stream to be hashed

input_length

Length of the stream to be hashed

3.1.3.4 struct smw_hash_final_args

struct **smw_hash_final_args**

Hash multi-part final arguments

3.1.3.4.1 Definition

```
struct smw_hash_final_args {  
    unsigned char version;  
    struct smw_op_context *context;  
    unsigned char *input;  
    unsigned int input_length;  
    unsigned char *output;  
    unsigned int output_length;  
}
```

3.1.3.4.2 Members

version

Version of this structure

context

Pointer to an opaque operation context structure

input

Location of the stream to be hashed

input_length

Length of the stream to be hashed

output

Location where the digest has to be written

output_length

Length of the digest

3.1.3.5 struct smw_eddsa_params

struct **smw_eddsa_params**

eddsa parameters

3.1.3.5.1 Definition

```
struct smw_eddsa_params {  
    unsigned char *context;  
    unsigned int context_length;  
}
```

3.1.3.5.2 Members

context

Location of the context

context_length

Length of the context

3.1.3.6 struct smw_sign_verify_args

struct **smw_sign_verify_args**

Sign or verify arguments

3.1.3.6.1 Definition

```
struct smw_sign_verify_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor *key_descriptor;  
    smw_attr_algo_t sign_algo;  
    unsigned char *message;  
    unsigned int message_length;  
    unsigned char *signature;  
    unsigned int signature_length;  
    union {  
        struct smw_eddsa_params *eddsa_params;  
    } ;  
}
```

3.1.3.6.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

key_descriptor

Pointer to a Key descriptor object. See *struct smw_key_descriptor*

sign_algo

Signature algorithm and attributes. See *typedef smw_attr_algo_t*

message

Location of the message

message_length

Length of the message

signature

Location of the signature

signature_length

Length of the signature

{unnamed_union}

anonymous

eddsa_params

Pointer to eddsa parameters

3.1.3.6.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is `SMW_SUBSYSTEM_NAME_NONE`, the default configured Secure Subsystem is used.

3.1.3.7 struct smw_sign_verify_init_args

struct **smw_sign_verify_init_args**

Sign or verify multi-part initialization arguments

3.1.3.7.1 Definition

```
struct smw_sign_verify_init_args {
    unsigned char version;
    smw_subsystem_t subsystem_name;
    struct smw_key_descriptor *key_descriptor;
    smw_attr_algo_t sign_algo;
    unsigned char *message;
    unsigned int message_length;
    union {
        struct smw_eddsa_params *eddsa_params;
    } ;
    struct smw_op_context *context;
}
```

3.1.3.7.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

key_descriptor

Pointer to a Key descriptor object. See *struct smw_key_descriptor*

sign_algo

Signature algorithm and attributes. See *typedef smw_attr_algo_t*

message

Location of the message

message_length

Length of the message

{unnamed_union}

anonymous

eddsa_params

Pointer to edwards signature parameters

context

Pointer to an opaque operation context structure

3.1.3.7.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is `SMW_SUBSYSTEM_NAME_NONE`, the default configured Secure Subsystem is used.

3.1.3.8 struct smw_sign_verify_update_args

struct **smw_sign_verify_update_args**

Sign or verify multi-part update arguments

3.1.3.8.1 Definition

```
struct smw_sign_verify_update_args {  
    unsigned char version;  
    struct smw_op_context *context;  
    unsigned char *message;  
    unsigned int message_length;  
}
```

3.1.3.8.2 Members

version

Version of this structure

context

Pointer to an opaque operation context structure

message

Location of the message

message_length

Length of the message

3.1.3.9 struct smw_sign_verify_final_args

struct **smw_sign_verify_final_args**

Sign or verify multi-part final arguments

3.1.3.9.1 Definition

```
struct smw_sign_verify_final_args {  
    unsigned char version;  
    struct smw_op_context *context;  
    unsigned char *message;  
    unsigned int message_length;  
    unsigned char *signature;  
    unsigned int signature_length;  
}
```

3.1.3.9.2 Members

version

Version of this structure

context

Pointer to an opaque operation context structure

message

Location of the message

message_length

Length of the message

signature

Location of the signature

signature_length

Length of the signature

3.1.3.10 struct smw_mac_args

struct **smw_mac_args**

MAC arguments

3.1.3.10.1 Definition

```
struct smw_mac_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor *key_descriptor;  
    smw_mac_algo_t algo_name;  
    smw_hash_algo_t hash_name;  
    unsigned char *input;  
    unsigned int input_length;  
    unsigned char *mac;  
    unsigned int mac_length;  
}
```

3.1.3.10.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

key_descriptor

Pointer to a Key descriptor object. See *struct smw_key_descriptor*

algo_name

MAC algorithm name. See *typedef smw_mac_algo_t*

hash_name

Hash algorithm name. See *typedef smw_hash_algo_t*

input

Location of the message to be authenticated

input_length

Length of the message

mac

Location where the MAC has to be written or compared

mac_length

Length of the MAC

3.1.3.10.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is `SMW_SUBSYSTEM_NAME_NONE`, the default configured Secure Subsystem is used or the Secure Subsystem handling the key specified.

3.1.3.11 struct smw_rng_args

struct smw_rng_args

Random number generator arguments

3.1.3.11.1 Definition

```
struct smw_rng_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    unsigned char *output;  
    unsigned int output_length;  
}
```

3.1.3.11.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

output

Location where the random number has to be written

output_length

Length of the random number

3.1.3.11.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is `SMW_SUBSYSTEM_NAME_NONE`, the default configured Secure Subsystem is used.

3.1.3.12 struct `smw_cipher_init_args`

struct `smw_cipher_init_args`

Cipher multi-part initialization arguments

3.1.3.12.1 Definition

```
struct smw_cipher_init_args {
    unsigned char version;
    smw_subsystem_t subsystem_name;
    struct smw_key_descriptor **keys_desc;
    unsigned int nb_keys;
    smw_cipher_mode_t mode_name;
    smw_cipher_op_type_t op_type_name;
    unsigned char *iv;
    unsigned int iv_length;
    struct smw_op_context *context;
}
```

3.1.3.12.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

keys_desc

Pointer to an array of pointers to key descriptors. See *struct smw_key_descriptor*

nb_keys

Number of entries of `keys_desc`

mode_name

Cipher mode name. See *typedef smw_cipher_mode_t*.

op_type_name

Cipher operation type name. See *typedef smw_cipher_op_type_t*

iv

Pointer to initialization vector

iv_length

iv length in bytes

context

Pointer to an opaque operation context structure

3.1.3.12.3 Description

Switch mode, iv is optional and represents:

- Initialization Vector (CBC, CTS)
- Initial Counter Value (CTR)
- Tweak Value (XTS)

3.1.3.13 struct smw_cipher_data_args

struct **smw_cipher_data_args**

Cipher data arguments

3.1.3.13.1 Definition

```
struct smw_cipher_data_args {  
    unsigned char version;  
    struct smw_op_context *context;  
    unsigned char *input;  
    unsigned int input_length;  
    unsigned char *output;  
    unsigned int output_length;  
}
```

3.1.3.13.2 Members

version

Version of this structure

context

Pointer to an opaque operation context structure

input

Input data buffer

input_length

input length in bytes

output

Output data buffer

output_length

output length in bytes

3.1.3.13.3 Description

In case of final operation, input and input_length are optional.

3.1.3.14 struct smw_cipher_args

struct **smw_cipher_args**

Cipher one-shot arguments

3.1.3.14.1 Definition

```
struct smw_cipher_args {  
    struct smw_cipher_init_args init;  
    struct smw_cipher_data_args data;  
}
```

3.1.3.14.2 Members

init

Initialization arguments. See *struct smw_cipher_init_args*

data

Data arguments. See *struct smw_cipher_data_args*

3.1.3.14.3 Description

Field context present in *init* and *data* is ignored.

3.1.3.15 smw_hash

enum *smw_status_code* **smw_hash**(struct *smw_hash_args* *args)

Compute hash.

Parameters

- **args** (struct *smw_hash_args**) – Pointer to the structure that contains the Hash arguments.

3.1.3.15.1 Description

This function computes a hash.

To query the required digest buffer length, set *args->output* to NULL. The function will then set the required digest buffer length in *args->output_length* and return *SMW_STATUS_OK*.

On operation completion, the *args->output_length* is updated to the correct value when

- Digest buffer length is bigger than expected. In this case, operation succeeds.
- Digest buffer length is shorter than expected. In this case, operation fails and returns *SMW_STATUS_OUTPUT_TOO_SHORT*.

3.1.3.15.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.16 smw_hash_init

enum *smw_status_code* **smw_hash_init**(struct *smw_hash_init_args* *args)

Hash multi-part initialization

Parameters

- **args** (struct *smw_hash_init_args**) – Pointer to the structure that contains the hash multi-part initialization arguments.

3.1.3.16.1 Description

This function executes a hash multi-part initialization. The operation context must be allocated using *smw_allocate_context()* API prior to invoking this API.

If the returned error code is SMW_STATUS_OK or SMW_STATUS_INVALID_PARAM, the operation is not terminated and the context remains valid.

3.1.3.16.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.17 smw_hash_update

enum *smw_status_code* **smw_hash_update**(struct *smw_hash_update_args* *args)

Hash multi-part update

Parameters

- **args** (struct *smw_hash_update_args**) – Pointer to the structure that contains the hash multi-part update arguments.

3.1.3.17.1 Description

This function executes a hash multi-part update operation.

The context used must be initialized by the hash multi-part initialization.

If the returned error code is SMW_STATUS_OK, SMW_STATUS_INVALID_PARAM or SMW_STATUS_VERSION_NOT_SUPPORTED the operation is not terminated and the context remains valid.

3.1.3.17.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.18 smw_hash_final

enum *smw_status_code* **smw_hash_final**(struct *smw_hash_final_args* *args)

Hash multi-part final

Parameters

- **args** (struct *smw_hash_final_args**) – Pointer to the structure that contains the hash multi-part final arguments.

3.1.3.18.1 Description

This function executes a hash multi-part final operation.

The context used must be initialized by the hash multi-part initialization.

Input data field of **args** can be a NULL pointer if no additional data are used.

Output data field of **args** can be a NULL pointer to get the required output buffer length. If this feature succeed, returned error code is **SMW_STATUS_OK** and the operation is not terminated (context remains valid) unless required output buffer length is 0.

Output length **args** field is updated to the correct value when:

- Output length is bigger than expected. In this case operation succeeded.
- Output length is shorter than expected. In this case operation failed and returned **SMW_STATUS_OUTPUT_TOO_SHORT**.

If the returned error code is **SMW_STATUS_INVALID_PARAM**, **SMW_STATUS_VERSION_NOT_SUPPORTED** or **SMW_STATUS_OUTPUT_TOO_SHORT** the operation is not terminated and the context remains valid.

3.1.3.18.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.19 smw_sign

enum *smw_status_code* **smw_sign**(struct *smw_sign_verify_args* *args)

Generate a signature.

Parameters

- **args** (struct *smw_sign_verify_args**) – Pointer to the structure that contains the Sign arguments.

3.1.3.19.1 Description

This function generates a signature. When **TLS_MAC_FINISH** attribute is set, the key type must be **TLS_MASTER**.

To query the required signature buffer length, set **args->signature** to NULL. The function will then set the required signature buffer length in **args->signature_length** and return **SMW_STATUS_OK**.

On operation completion, the **args->signature_length** is updated to the correct value when

- Signature buffer length is bigger than expected. In this case, operation succeeds.
- Signature buffer length is shorter than expected. In this case, operation fails and returns **SMW_STATUS_OUTPUT_TOO_SHORT**.

3.1.3.19.2 Return

enum *smw_status_code*

- Common return codes
- Specific return codes - Signature

3.1.3.20 smw_sign_init

enum *smw_status_code* **smw_sign_init**(struct *smw_sign_verify_init_args* *args)

Signature generation multi-part initialization

Parameters

- **args** (struct *smw_sign_verify_init_args**) – Pointer to the structure that contains the signature multi-part initialization arguments.

3.1.3.20.1 Description

This function executes a signature generation multi-part initialization. The operation context must be allocated using *smw_allocate_context()* API prior to invoking this API.

If the returned error code is SMW_STATUS_OK or SMW_STATUS_INVALID_PARAM, the operation is not terminated and the context remains valid.

3.1.3.20.2 Return

See enum *smw_status_code*

- Common return codes

3.1.3.21 smw_sign_update

enum *smw_status_code* **smw_sign_update**(struct *smw_sign_verify_update_args* *args)

Signature generation multi-part update

Parameters

- **args** (struct *smw_sign_verify_update_args**) – Pointer to the structure that contains the signature multi-part update arguments.

3.1.3.21.1 Description

This function executes a signature generation multi-part update operation.

The context used must be initialized by the signature generation multi-part initialization.

If the returned error code is SMW_STATUS_OK, SMW_STATUS_INVALID_PARAM or SMW_STATUS_VERSION_NOT_SUPPORTED the operation is not terminated and the context remains valid.

3.1.3.21.2 Return

See enum *smw_status_code*

- Common return codes

3.1.3.22 smw_sign_final

enum *smw_status_code* **smw_sign_final**(struct *smw_sign_verify_final_args* *args)

Signature generation multi-part final

Parameters

- **args** (struct *smw_sign_verify_final_args**) – Pointer to the structure that contains the signature multi-part final arguments.

3.1.3.22.1 Description

This function executes a signature generation multi-part final operation.

The context used must be initialized by the signature generation multi-part initialization.

Input message field of args can be a NULL pointer if no additional data are used.

Output signature field of args can be a NULL pointer to get the required output buffer length. If this feature succeed, returned error code is SMW_STATUS_OK and the operation is not terminated (context remains valid) unless required output signature length is 0.

Output signature length args field is updated to the correct value when:

- Length is bigger than expected. In this case operation succeeded.
- Length is shorter than expected. In this case operation failed and returned SMW_STATUS_OUTPUT_TOO_SHORT.

If the returned error code is SMW_STATUS_INVALID_PARAM, SMW_STATUS_VERSION_NOT_SUPPORTED or SMW_STATUS_OUTPUT_TOO_SHORT the operation is not terminated and the context remains valid.

3.1.3.22.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.23 smw_verify

enum *smw_status_code* **smw_verify**(struct *smw_sign_verify_args* *args)

Verify a signature.

Parameters

- **args** (struct *smw_sign_verify_args**) – Pointer to the structure that contains the Verify arguments.

3.1.3.23.1 Description

This function verifies a signature.

3.1.3.23.2 Return

See *enum smw_status_code*

- Common return codes

-
- Specific return codes - Signature

3.1.3.24 smw_verify_init

enum *smw_status_code* **smw_verify_init**(struct *smw_sign_verify_init_args* *args)

Signature verification multi-part initialization

Parameters

- **args** (struct *smw_sign_verify_init_args**) – Pointer to the structure that contains the signature multi-part initialization arguments.

3.1.3.24.1 Description

This function executes a signature verification multi-part initialization. The operation context must be allocated using *smw_allocate_context()* API prior to invoking this API.

If the returned error code is SMW_STATUS_OK or SMW_STATUS_INVALID_PARAM, the operation is not terminated and the context remains valid.

3.1.3.24.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.25 smw_verify_update

enum *smw_status_code* **smw_verify_update**(struct *smw_sign_verify_update_args* *args)

Signature verification multi-part update

Parameters

- **args** (struct *smw_sign_verify_update_args**) – Pointer to the structure that contains the signature multi-part update arguments.

3.1.3.25.1 Description

This function executes a signature verification multi-part update operation.

The context used must be initialized by the signature verification multi-part initialization.

If the returned error code is SMW_STATUS_OK, SMW_STATUS_INVALID_PARAM or SMW_STATUS_VERSION_NOT_SUPPORTED the operation is not terminated and the context remains valid.

3.1.3.25.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.26 smw_verify_final

enum *smw_status_code* **smw_verify_final**(struct *smw_sign_verify_final_args* *args)

Signature verification multi-part final

Parameters

- **args** (struct *smw_sign_verify_final_args**) – Pointer to the structure that contains the signature multi-part final arguments.

3.1.3.26.1 Description

This function executes a signature verification multi-part final operation.

The context used must be initialized by the signature generation multi-part initialization.

Input message field of args can be a NULL pointer if no additional data are used.

Input signature field of args must be set with a correct length to perform the signature verification of message given in all multi-part steps.

If the returned error code is SMW_STATUS_INVALID_PARAM, SMW_STATUS_VERSION_NOT_SUPPORTED the operation is not terminated and the context remains valid.

3.1.3.26.2 Return

See enum *smw_status_code*

- Common return codes

3.1.3.27 smw_rng

enum *smw_status_code* **smw_rng**(struct *smw_rng_args* *args)

Compute a random number.

Parameters

- **args** (struct *smw_rng_args**) – Pointer to the structure that contains the RNG arguments.

3.1.3.27.1 Description

This function computes a random number.

3.1.3.27.2 Return

See enum *smw_status_code*

- Common return codes

3.1.3.28 smw_cipher

enum *smw_status_code* **smw_cipher**(struct *smw_cipher_args* *args)

Cipher one-shot

Parameters

-
- **args** (struct *smw_cipher_args**) – Pointer to the structure that contains the cipher arguments.

3.1.3.28.1 Description

This function executes a cipher encryption or decryption.

Output data field of **args** can be a NULL pointer to get the required output buffer length. If this feature succeed, returned error code is *SMW_STATUS_OK*.

Output length **args** field is updated to the correct value when:

- Output length is bigger than expected. In this case operation succeeded.
- Output length is shorter than expected. In this case operation failed and returned *SMW_STATUS_OUTPUT_TOO_SHORT*.

Keys used can be defined as buffer and as key ID. All key types must be identical and must be linked to the same subsystem. If at least one key ID is set, subsystem name field of **args** is optional. If set it must be coherent with the key ID.

3.1.3.28.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.29 smw_cipher_init

enum smw_status_code **smw_cipher_init**(struct *smw_cipher_init_args* *args)

Cipher multi-part initialization

Parameters

- **args** (struct *smw_cipher_init_args**) – Pointer to the structure that contains the cipher multi-part initialization arguments.

3.1.3.29.1 Description

This function executes a cipher multi-part encryption or decryption initialization. The operation context must be allocated using *smw_allocate_context()* API prior to invoking this API.

If the returned error code is *SMW_STATUS_OK* or *SMW_STATUS_INVALID_PARAM*, the operation is not terminated and the context remains valid.

Keys used can be defined as buffer and as key ID. All key types must be identical and must be linked to the same subsystem. If at least one key ID is set, subsystem name field of **args** is optional. If set it must be coherent with the key ID.

3.1.3.29.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.30 smw_cipher_update

enum *smw_status_code* **smw_cipher_update**(struct *smw_cipher_data_args* *args)

Cipher multi-part update

Parameters

- **args** (struct *smw_cipher_data_args**) – Pointer to the structure that contains the cipher multi-part update arguments.

3.1.3.30.1 Description

This function executes a cipher multi-part encryption or decryption update operation.

The context used must be initialized by the cipher multi-part initialization.

Output data field of **args** can be a NULL pointer to get the required output buffer length. If this feature succeed, returned error code is SMW_STATUS_OK.

Output length **args** field is updated to the correct value when:

- Output length is bigger than expected. In this case operation succeeded.
- Output length is shorter than expected. In this case operation failed and returned SMW_STATUS_OUTPUT_TOO_SHORT.

If the returned error code is SMW_STATUS_OK, SMW_STATUS_INVALID_PARAM, SMW_STATUS_VERSION_NOT_SUPPORTED or SMW_STATUS_OUTPUT_TOO_SHORT the operation is not terminated and the context remains valid.

3.1.3.30.2 Return

See enum *smw_status_code*

- Common return codes

3.1.3.31 smw_cipher_final

enum *smw_status_code* **smw_cipher_final**(struct *smw_cipher_data_args* *args)

Cipher multi-part final

Parameters

- **args** (struct *smw_cipher_data_args**) – Pointer to the structure that contains the cipher multi-part final arguments.

3.1.3.31.1 Description

This function executes a cipher multi-part encryption or decryption final operation.

The context used must be initialized by the cipher multi-part initialization.

Input data field of **args** can be a NULL pointer if no additional data are used.

Output data field of **args** can be a NULL pointer to get the required output buffer length. If this feature succeed, returned error code is SMW_STATUS_OK and the operation is no terminated (context remains valid) unless required output buffer length is 0.

Output length **args** field is updated to the correct value when:

- Output length is bigger than expected. In this case operation succeeded.
- Output length is shorter than expected. In this case operation failed and returned `SMW_STATUS_OUTPUT_TOO_SHORT`.

If the returned error code is `SMW_STATUS_INVALID_PARAM`, `SMW_STATUS_VERSION_NOT_SUPPORTED` or `SMW_STATUS_OUTPUT_TOO_SHORT` the operation is not terminated and the context remains valid.

3.1.3.31.2 Return

See `enum smw_status_code`

- Common return codes

3.1.3.32 smw_mac

`enum smw_status_code smw_mac(struct smw_mac_args *args)`

Compute a MAC.

Parameters

- **args** (struct `smw_mac_args*`) – Pointer to the structure that contains the MAC arguments.

3.1.3.32.1 Description

This function computes a Message Authentication Code.

To query the required MAC buffer length, set `args->mac` to `NULL`. The function will then set the required MAC buffer length in `args->mac_length` and return `SMW_STATUS_OK`.

On operation completion, the `args->mac_length` is updated to the correct value when

- MAC buffer length is bigger than expected. In this case, operation succeeds.
- MAC buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

3.1.3.32.2 Return

See `enum smw_status_code`

- Common return codes

3.1.3.33 smw_mac_verify

`enum smw_status_code smw_mac_verify(struct smw_mac_args *args)`

Compute and verify a MAC.

Parameters

- **args** (struct `smw_mac_args*`) – Pointer to the structure that contains the MAC arguments.

3.1.3.33.1 Description

This function computes then verifies a Message Authentication Code.

3.1.3.33.2 Return

See `enum smw_status_code`

- Common return codes

3.1.3.34 Operation Context

3.1.3.34.1 struct smw_context_args

struct **smw_context_args**

SMW cryptographic operation context arguments

3.1.3.34.1.1 Definition

```
struct smw_context_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_op_context *context;  
}
```

3.1.3.34.1.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See `typedef smw_subsystem_t` (**deprecated **)

context

Pointer to an opaque operation context structure

3.1.3.34.1.3 Description

The `context` parameter allocated by SMW should not be modified by the application.

This opaque structure is dynamically allocated by the SMW library upon invoking the function `smw_allocate_context()`. It is deallocated when any of the following conditions are met:

- Upon successful completion of the associated multi-part final operation.
- In the event of critical failure during the associated operation.
- When `smw_cancel_operation()` function is invoked.

Warning: `subsystem_name` is deprecated and no more used. Subsystem is selected when initializing the cryptographic operation. If not specified during the operation initialization, the default configured subsystem will be used.

3.1.3.34.2 struct smw_copy_context_args

struct **smw_copy_context_args**

SMW cryptographic copy operation context args

3.1.3.34.2.1 Definition

```
struct smw_copy_context_args {  
    unsigned char version;  
    struct smw_op_context *src_context;  
    struct smw_op_context *dst_context;  
}
```

3.1.3.34.2.2 Members

version

Version of this structure

src_context

Pointer to source opaque operation context structure

dst_context

Pointer to destination opaque operation context structure

3.1.3.34.3 smw_allocate_context

enum *smw_status_code* **smw_allocate_context**(struct *smw_context_args* *args)

Allocate SMW operation context

Parameters

- **args** (struct *smw_context_args**) – Pointer to operation context arguments structure.

3.1.3.34.3.1 Description

This function allocates an operation context for a new cryptographic operation.

This function is intended for use in the multi-part cryptographic operation and it serves as the first step in such operations. This function does not need to be called for one-shot cryptographic operation.

3.1.3.34.3.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.34.4 smw_cancel_operation

enum *smw_status_code* **smw_cancel_operation**(struct *smw_context_args* *args)

Cancel on-going cryptographic multi-part operation

Parameters

- **args** (struct *smw_context_args**) – Pointer to an operation context arguments structure.

3.1.3.34.4.1 Description

This function cancels the on-going cryptographic multi-part operation and releases all the memory allocated to SMW operation context. Additionally, this function can be used to release the SMW operation context even if there are no on-going multi-part operations associated with it.

Upon successful completion, args->context field is set to NULL.

3.1.3.34.4.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.34.5 smw_copy_context

enum *smw_status_code* **smw_copy_context**(struct *smw_copy_context_args* *args)

Copy an operation context

Parameters

- **args** (struct *smw_copy_context_args**) – Pointer to copy operation context arguments structure.

3.1.3.34.5.1 Description

This function copies the last state of the source operation context to the destination operation context. Destination context must be allocated using *smw_allocate_context()* function prior to invoking this function.

3.1.3.34.5.2 Return

See *enum smw_status_code*

- Common return codes

3.1.3.35 Authentication Encryption/Decryption (AEAD)

3.1.3.35.1 struct smw_aead_init_args

struct **smw_aead_init_args**

AEAD initialization arguments

3.1.3.35.1.1 Definition

```
struct smw_aead_init_args {
    unsigned char version;
    smw_subsystem_t subsystem_name;
    struct smw_key_descriptor *key_desc;
    smw_aead_mode_t mode_name;
```

(continues on next page)

```

smw_aead_op_type_t op_type_name;
unsigned char *user_iv;
unsigned int user_iv_length;
unsigned int iv_length;
unsigned int aad_length;
unsigned int tag_length;
unsigned int plaintext_length;
struct smw_op_context *context;
}

```

3.1.3.35.1.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See [typedef smw_subsystem_t](#)

key_desc

Pointer to a key descriptor object. See [struct smw_key_descriptor](#)

mode_name

AEAD mode name. See [typedef smw_aead_mode_t](#)

op_type_name

AEAD operation name. See [typedef smw_aead_op_type_t](#)

user_iv

Pointer to user initialization vector

user_iv_length

User IV buffer length in bytes

iv_length

Requested IV buffer length in bytes

aad_length

Additional authentication data length in bytes

tag_length

Tag buffer length in bytes

plaintext_length

Length in bytes of the data to encrypt

context

Pointer to an opaque operation context structure

3.1.3.35.1.3 Description

If subsystem offers the capability to generate partial or complete IV, the user can set the input `init->user_iv_length` to 0 (requesting full generated IV) or set `init->iv_length` to the requested IV size (requesting partial generated IV). Otherwise, if `init->user_iv_length` is set to the maximum IV size supported by the subsystem, `init->iv_length` is not taking into consideration. Please refer to the AEAD subsystems capabilities.

3.1.3.35.2 struct smw_aead_data_args

struct **smw_aead_data_args**

AEAD data arguments

3.1.3.35.2.1 Definition

```
struct smw_aead_data_args {  
    unsigned char version;  
    struct smw_op_context *context;  
    unsigned char *input;  
    unsigned int input_length;  
    unsigned char *output;  
    unsigned int output_length;  
}
```

3.1.3.35.2.2 Members

version

Version of this structure

context

Pointer to an opaque operation context structure

input

Pointer to input data buffer to be encrypted or decrypted

input_length

Input data buffer length in bytes

output

Pointer to output buffer

output_length

Output buffer length in bytes

3.1.3.35.3 struct smw_aead_aad_args

struct **smw_aead_aad_args**

Authentication Encryption AAD arguments

3.1.3.35.3.1 Definition

```
struct smw_aead_aad_args {  
    unsigned char version;  
    unsigned char *data;  
    unsigned int data_length;  
    struct smw_op_context *context;  
}
```

3.1.3.35.3.2 Members

version

Version of this structure

data

Pointer to additional authentication data

data_length

AAD length in bytes

context

Pointer to an opaque operation context structure

3.1.3.35.4 struct smw_aead_final_args

struct **smw_aead_final_args**

AEAD final arguments

3.1.3.35.4.1 Definition

```
struct smw_aead_final_args {  
    unsigned char version;  
    struct smw_aead_data_args *data;  
    smw_aead_op_type_t op_type_name;  
    unsigned char *tag;  
    unsigned int tag_length;  
    unsigned int output_iv_length;  
    unsigned char *output_iv;  
}
```

3.1.3.35.4.2 Members

version

Version of this structure

data

Pointer to AEAD data arguments. See *struct smw_aead_data_args*

op_type_name

AEAD operation name. See *typedef smw_aead_op_type_t*

tag

Pointer to tag buffer

tag_length

Tag buffer length in bytes

output_iv_length

Length of output IV buffer

output_iv

Pointer to output IV buffer

3.1.3.35.4.3 Description

`output_iv` buffer should be allocated by the caller application for encryption operation. Upon successful encryption operation, `output_iv` will contain the IV used by subsystem during operation.

Fields `output_iv_length` and `output_iv` are ignored for decryption operation.

3.1.3.35.5 struct `smw_aead_args`

struct **`smw_aead_args`**

AEAD one-shot arguments

3.1.3.35.5.1 Definition

```
struct smw_aead_args {  
    unsigned char version;  
    struct smw_aead_aad_args *aad;  
    struct smw_aead_init_args *init;  
    struct smw_aead_final_args *final;  
}
```

3.1.3.35.5.2 Members

version

Version of this structure

aad

Pointer to AAD arguments. See [struct `smw\_aead\_aad\_args`](#)

init

Pointer to initialization arguments. See [struct `smw\_aead\_init\_args`](#)

final

Pointer to final arguments. See [struct `smw\_aead\_final\_args`](#)

3.1.3.35.5.3 Description

Field context present in `init`, `aad` and `final` is ignored. Field `operation_name` present in `final` is ignored.

3.1.3.35.6 `smw_aead`

enum `smw_status_code` **`smw_aead`**(struct `smw_aead_args` *args)

One-shot AEAD operation.

Parameters

- **args** (struct `smw_aead_args`*) – Pointer to the structure that contains the AEAD one-shot arguments.

3.1.3.35.6.1 Description

This function executes one-shot AEAD encryption or decryption operation.

- One-shot AEAD encryption operation:
 - This function encrypts a message and computes the tag.
 - If the `args->final->tag` field is set, the computed tag will be stored in the dedicated `args->final->tag` field.
 - If the `args->final->tag` field is not set, `args->final->data->output` field will contain the ciphertext followed by the tag.
- One-shot AEAD decryption operation:
 - This function authenticates and decrypts the ciphertext.
 - If the `args->final->tag` field is not set, the `args->final->data->input` should contain the ciphertext followed by the tag.
 - If the computed tag does not match the supplied tag, the operation will be terminated.

If `args->final->data->output` is a NULL pointer, then the function updates `args->final->data->output_length` field and returns error code `SMW_STATUS_OK`. Additionally, if the operation is encryption, the tag length `args->final->tag_length` and output IV length `args->final->output_iv_length` are updated with generated tag value length and IV length, respectively.

On operation completion, the `args->final->data->output_length` is updated to the correct value when

- Output length is bigger than expected. In this case, operation succeeds.
- Output length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.
- In the above mentioned two scenarios, if the operation is encryption, the function also updates the required tag buffer length `args->final->tag_length` and output IV length `args->final->output_iv_length`.

If `args->final->data->output` is not a NULL pointer, then

- For an encryption operation, if the `args->final->tag` field is not set, `args->final->data->output` should be sufficiently large to accommodate both the ciphertext and tag.
- For an encryption operation, if the `args->final->tag` field is set, `args->final->data->output` should be sufficiently large to accommodate the ciphertext.
- For decryption operation, `args->final->data->output` should be sufficiently large to accommodate the plaintext.

If the IV is generated fully or partially by the subsystem, `args->final->output_iv` will hold the IV generated by subsystem. If the IV is supplied fully by the user, `args->final->output_iv` will hold IV supplied by the user.

3.1.3.35.6.2 Return

See `enum smw_status_code`

- Common return codes

3.1.3.35.7 smw_aead_init

`enum smw_status_code smw_aead_init(struct smw_aead_init_args *args)`

AEAD multi-part initialization.

Parameters

- **args** (struct `smw_aead_init_args*`) – Pointer to the structure that contains the AEAD initialization arguments.

3.1.3.35.7.1 Description

This function initializes AEAD multi-part encryption or decryption operation.

The operation context must be allocated using `smw_allocate_context()` API prior to invoking this API.

If the returned error code is `SMW_STATUS_OK` or `SMW_STATUS_INVALID_PARAM`, the operation is not terminated and the context remains valid.

Key used can be defined either as a buffer or as a key ID.

3.1.3.35.7.2 Return

See `enum smw_status_code`

- Common return codes

3.1.3.35.8 smw_aead_update_aad

`enum smw_status_code smw_aead_update_aad(struct smw_aead_aad_args *args)`

Update AEAD additional authenticated data.

Parameters

- **args** (struct `smw_aead_aad_args*`) – Pointer to the structure that contains the AEAD additional data arguments.

3.1.3.35.8.1 Description

This function can be called multiple time while the update data (to encrypt or to decrypt) step is not called.

The context used must be initialized by the AEAD multi-part initialization.

If the returned error code is `SMW_STATUS_OK`, `SMW_STATUS_INVALID_PARAM` or `SMW_STATUS_VERSION_NOT_SUPPORTED`, the operation is not terminated, and the context remains valid.

3.1.3.35.8.2 Return

See [enum smw_status_code](#)

- Common return codes

3.1.3.35.9 smw_aead_update

enum [smw_status_code](#) **smw_aead_update**(struct [smw_aead_data_args](#) *args)

AEAD multi-part update operation

Parameters

- **args** (struct [smw_aead_data_args](#)*) – Pointer to the structure that contains the AEAD multi-part data arguments.

3.1.3.35.9.1 Description

This function executes a AEAD multi-part encryption or decryption update operation.

The context used must be initialized by the AEAD multi-part initialization.

The args->output can be a NULL pointer to get the required output buffer length. If this feature succeeds, returned error code is SMW_STATUS_OK.

The args->output_length field is updated to the correct value when:

- Output length is bigger than expected. In this case operation succeeded.
- Output length is shorter than expected. In this case operation failed and returned SMW_STATUS_OUTPUT_TOO_SHORT.

If the returned error code is SMW_STATUS_OK, SMW_STATUS_INVALID_PARAM, SMW_STATUS_VERSION_NOT_SUPPORTED or SMW_STATUS_OUTPUT_TOO_SHORT the operation is not terminated and the context remains valid.

3.1.3.35.9.2 Return

See [enum smw_status_code](#)

- Common return codes

3.1.3.35.10 smw_aead_final

enum [smw_status_code](#) **smw_aead_final**(struct [smw_aead_final_args](#) *args)

AEAD multi-part encryption/decryption final operation

Parameters

- **args** (struct [smw_aead_final_args](#)*) – Pointer to the structure that contains the AEAD multi-part final arguments.

3.1.3.35.10.1 Description

This function completes the active AEAD multi-part encryption/decryption operation.

The context used must be initialized by the AEAD multi-part initialization.

-
- AEAD Encryption final operation:
 - This function finishes encrypting a message in an active multi-part AEAD operation and computes the tag.
 - If the `args->tag` field is set, the computed tag will be stored in the dedicated `args->tag` field.
 - If the `args->tag` field is not set, `args->data->output` field will point to the ciphertext followed by the tag.
 - AEAD Decryption final operation:
 - This function finishes authenticating and decrypting a message in an active multi-part AEAD operation.
 - If the computed tag does not match the supplied tag, the operation will be terminated. The returned error code is `SMW_STATUS_SIGNATURE_INVALID`.
 - If the `args->tag` is not supplied in the tag field, the `args->data->input` field should be sufficiently large to accommodate both the ciphertext and the tag.

If `args->data->output` is a NULL pointer, then the function updates `args->data->output_length` field and returns error code `SMW_STATUS_OK`. In this case, if the operation is encryption, the function also updates the required tag buffer length `args->tag_length`.

The `args->data->output_length` field is updated to the correct value when:

- Output length is bigger than expected. In this case, operation succeeds.
- Output length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT` error code.
- In the above mentioned two scenarios, if the operation is encryption, the function also updates the required tag buffer length `args->tag_length`.

If `args->data->output` field is not a NULL pointer, then

- For an encryption operation, if the `args->tag` field is not set, `args->data->output` should be sufficiently large to accommodate both the ciphertext and tag.
- For an encryption operation, if the `args->tag` field is set, `args->data->output` should be sufficiently large to accommodate the ciphertext.
- For decryption operation, `args->data->output` should be sufficiently large to accommodate the plaintext.

If the IV is generated fully or partially by the subsystem, `args->output_iv` will hold the IV generated by subsystem. If the IV is supplied fully by the user, `args->output_iv` will hold IV supplied by the user.

If the returned error code is `SMW_STATUS_INVALID_PARAM`, `SMW_STATUS_VERSION_NOT_SUPPORTED` or `SMW_STATUS_OUTPUT_TOO_SHORT` the operation is not terminated and the context remains valid.

3.1.3.35.10.2 Return

See `enum smw_status_code`

- Common return codes

3.1.3.36 Asymmetric Encryption/Decryption

3.1.3.36.1 struct smw_asymmetric_encryption_args

struct **smw_asymmetric_encryption_args**

Asymmetric encryption and decryption arguments structure

3.1.3.36.1.1 Definition

```
struct smw_asymmetric_encryption_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor *key_descriptor;  
    smw_attr_algo_t algo;  
    unsigned char *input;  
    unsigned int input_length;  
    unsigned char *output;  
    unsigned int output_length;  
    unsigned char *salt;  
    unsigned int salt_length;  
}
```

3.1.3.36.1.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See `typedef smw_subsystem_t`

key_descriptor

Pointer to a Key descriptor object. See `struct smw_key_descriptor`

algo

Encryption/decryption algorithm and attributes. See `typedef smw_attr_algo_t`

input

Pointer to input data buffer

input_length

Input data buffer length in bytes

output

Pointer to output buffer

output_length

Output buffer length in bytes

salt

Pointer to salt or label buffer

salt_length

Salt buffer length in bytes

3.1.3.36.1.3 Description

algo is defined by set of parameters, including encryption algorithm, mode (padding schemes), hashing algorithm used in the padding scheme and operation class. salt and salt_length are optional arguments. These parameters are ignored if not supported by the encryption modes.

3.1.3.36.2 smw_asymmetric_encrypt

enum *smw_status_code* **smw_asymmetric_encrypt**(struct *smw_asymmetric_encryption_args* *args)

Encrypt a message

Parameters

- **args** (struct *smw_asymmetric_encryption_args**) – Pointer to asymmetric encryption and decryption arguments structure.

3.1.3.36.2.1 Description

This function encrypts a message using public key ID or key buffer.

This function updates args->output_length field to required output buffer length if args->output is a NULL pointer and returns error code SMW_STATUS_OK.

args->output_length field is updated to the correct value in the following cases

- If the provided output length is bigger than required, the operation succeeds.
- If the provided output length is shorter than required, the operation fails and returns SMW_STATUS_OUTPUT_TOO_SHORT.

3.1.3.36.2.2 Return

See enum *smw_status_code*

- Common return codes

3.1.3.36.3 smw_asymmetric_decrypt

enum *smw_status_code* **smw_asymmetric_decrypt**(struct *smw_asymmetric_encryption_args* *args)

Decrypt a message

Parameters

- **args** (struct *smw_asymmetric_encryption_args**) – Pointer to asymmetric encryption and decryption arguments structure.

3.1.3.36.3.1 Description

This function decrypts a message using private key ID or key buffer.

args->output_length field is updated to the correct value in the following cases

- If the provided output length is bigger than required, the operation succeeds.

-
- If the provided output length is shorter than required, the operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

3.1.3.36.3.2 Return

See `enum smw_status_code`

- Common return codes

3.1.4 Key Manager APIs

3.1.4.1 struct smw_keypair_gen

struct **smw_keypair_gen**

Generic Keypair object

3.1.4.1.1 Definition

```
struct smw_keypair_gen {  
    unsigned char *public_data;  
    unsigned int public_length;  
    unsigned char *private_data;  
    unsigned int private_length;  
}
```

3.1.4.1.2 Members

public_data

Pointer to the public key

public_length

Length of public_data in bytes

private_data

Pointer to the private key

private_length

Length of private_data in bytes

3.1.4.1.3 Description

Asymmetric Keypair common structure definition. It's the basis of the other asymmetric keypair structure.

3.1.4.2 struct smw_keypair_rsa

struct **smw_keypair_rsa**

RSA Keypair object

3.1.4.2.1 Definition

```
struct smw_keypair_rsa {  
    unsigned char *public_data;  
    unsigned int public_length;  
    unsigned char *private_data;  
    unsigned int private_length;  
    unsigned char *modulus;  
    unsigned int modulus_length;  
    unsigned char *public_exponent;  
    unsigned int public_exponent_length;  
}
```

3.1.4.2.2 Members

public_data

Pointer to the RSA public exponent

public_length

Length of public_data in bytes

private_data

Pointer to the RSA private exponent

private_length

Length of private_data in bytes

modulus

Pointer to the RSA modulus

modulus_length

Length of modulus in bytes

public_exponent

Pointer to the input RSA public exponent

public_exponent_length

Length of public_exponent in bytes

3.1.4.2.3 Description

First fields are common to the struct smw_keypair_gen and must be kept common. Input parameters public_exponent and public_exponent_length are only used for key generation. For other operations, they are ignored, and public_data and public_length must be set instead.

3.1.4.3 struct smw_keypair_buffer

struct **smw_keypair_buffer**

Keypair buffer

3.1.4.3.1 Definition

```
struct smw_keypair_buffer {
    smw_key_format_t format_name;
    union {
        struct smw_keypair_gen gen;
        struct smw_keypair_rsa rsa;
    } ;
}
```

3.1.4.3.2 Members

format_name

Defines the encoding format of all buffers. See *typedef smw_key_format_t*

{unnamed_union}

anonymous

gen

Generic keypair object definition. See *struct smw_keypair_gen*

rsa

RSA keypair object definition. See *struct smw_keypair_rsa*

3.1.4.3.3 Description

By default if format name is not specified, there will be no encoding (equivalent to “HEX”)

3.1.4.4 struct smw_key_attributes

struct **smw_key_attributes**

Key attributes

3.1.4.4.1 Definition

```
struct smw_key_attributes {
    smw_attr_algo_t permitted_algo;
    smw_attr_usage_t usage_flags;
    smw_attr_storage_id_t storage_id;
    smw_attr_attributes_t attributes;
}
```

3.1.4.4.2 Members

permitted_algo

Permitted algorithm. See *typedef smw_attr_algo_t*

usage_flags

Permitted usage flags. See *typedef smw_attr_usage_t*

storage_id

Storage identifier. See *typedef smw_attr_storage_id_t*

attributes

Attributes. See *typedef smw_attr_attributes_t*

3.1.4.5 struct smw_key_descriptor

struct **smw_key_descriptor**

Key descriptor

3.1.4.5.1 Definition

```
struct smw_key_descriptor {  
    smw_key_type_t type_name;  
    unsigned int security_size;  
    unsigned int id;  
    struct smw_key_attributes attributes;  
    struct smw_keypair_buffer *buffer;  
}
```

3.1.4.5.2 Members

type_name

Key type name. See *typedef smw_key_type_t*

security_size

Security size in bits

id

Key identifier

attributes

Key attributes. see *struct smw_key_attributes*

buffer

Key pair buffer. See *struct smw_keypair_buffer*

3.1.4.5.3 Description

attributes field is not used by all APIs. It's documented in API's argument when this field is used.

3.1.4.6 struct smw_derived_key_descriptor

struct **smw_derived_key_descriptor**

Derived key descriptor structure

3.1.4.6.1 Definition

```
struct smw_derived_key_descriptor {  
    smw_key_type_t type_name;  
    unsigned int security_size;  
    unsigned int id;  
    struct smw_key_attributes attributes;  
    smw_key_format_t format_name;  
}
```

(continues on next page)

```

unsigned char *shared_secret;
unsigned int shared_secret_len;
}

```

3.1.4.6.2 Members

type_name

Key type name. See *typedef smw_key_type_t*

security_size

Security size in bits

id

Key identifier

attributes

Key attributes. see *struct smw_key_attributes*

format_name

Defines the encoding format of shared secret buffer See *typedef smw_key_format_t*

shared_secret

Shared secret buffer

shared_secret_len

shared_secret length in bytes

3.1.4.7 struct smw_generate_key_args

struct **smw_generate_key_args**

Key generation arguments

3.1.4.7.1 Definition

```

struct smw_generate_key_args {
    unsigned char version;
    smw_subsystem_t subsystem_name;
    struct smw_key_descriptor *key_descriptor;
}

```

3.1.4.7.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

key_descriptor

Pointer to a Key descriptor object. See *struct smw_key_descriptor*

3.1.4.7.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is NULL, the default Secure Subsystem configured for this Security Operation is used. The `key_descriptor` fields `type_name` and `security_size` must be given as input to know the type of key to generate. The `key_descriptor` field `buffer` is optional. Only the public key will be returned if the corresponding pointer and size are set. The `key_descriptor` field `id`, if set by the caller (other than 0) will be the created key identifier on operation success. Else the API will returned a new key identifier if `id` is set as 0.

The `key_descriptor.attributes` is used to define the key attributes of the generated key.

3.1.4.8 struct smw_derive_key_args

struct **smw_derive_key_args**

Key derivation arguments

3.1.4.8.1 Definition

```
struct smw_derive_key_args {
    unsigned char version;
    smw_subsystem_t subsystem_name;
    smw_kdf_t kdf_name;
    void *kdf_arguments;
    bool store_derived_key;
    struct smw_key_descriptor *key_descriptor_base;
    struct smw_derived_key_descriptor *key_descriptor_derived;
}
```

3.1.4.8.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

kdf_name

Key derivation function name. See *typedef smw_kdf_t*

kdf_arguments

Key derivation function arguments

store_derived_key

If true, store the derived key.

key_descriptor_base

Pointer to a Key base descriptor. See *struct smw_key_descriptor*

key_descriptor_derived

Pointer to the Key derived descriptor structure. See *struct smw_derived_key_descriptor*

3.1.4.8.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is set to `SMW_SUBSYSTEM_NAME_NONE`, the default Secure Subsystem configured for this Security Operation is used.

A new key is derived from a given key base (`@key_descriptor_base`) using the key derivation function `kdf_name`. If the key derivation function requires more arguments, the `kdf_arguments` refers to the associated key derivation function arguments, else this pointer is not used and can be `NULL`.

Upon successful completion of the key derivation operation, if `store_derived_key` is set to true, new key ID is set in the `key_descriptor_derived->id` and shared secret data is exported if `key_descriptor_derived->shared_secret` and `key_descriptor_derived->shared_secret_len` are set. Refer to the subsystem capabilities for more details.

The `key_descriptor_derived.attributes` is used to define the key attributes of the derived key.

3.1.4.9 struct smw_kdf_tls12_args

struct **smw_kdf_tls12_args**

Key derivation function TLS 1.2 arguments

3.1.4.9.1 Definition

```
struct smw_kdf_tls12_args {
    smw_tls12_keas_t key_exchange_name;
    smw_tls12_enc_t encryption_name;
    smw_hash_algo_t prf_name;
    bool ext_master_key;
    unsigned char *kdf_input;
    unsigned int kdf_input_length;
    unsigned int master_sec_key_id;
    unsigned int client_w_enc_key_id;
    unsigned int server_w_enc_key_id;
    unsigned int client_w_mac_key_id;
    unsigned int server_w_mac_key_id;
    unsigned char *client_w_iv;
    unsigned int client_w_iv_length;
    unsigned char *server_w_iv;
    unsigned int server_w_iv_length;
}
```

3.1.4.9.2 Members

key_exchange_name

Name of the key exchange algorithm. See *typedef smw_tls12_keas_t*

encryption_name

Name of the encryption algorithm. See *typedef smw_tls12_enc_t*

prf_name

Name of the Pseudo-Random Function (PRF). See *typedef smw_hash_algo_t*

ext_master_key

If true, generates an extended master secret key

kdf_input

Key derivation input data used to generate the master secret key

kdf_input_length

Length in bytes of the kdf_input buffer

master_sec_key_id

Generated master key identifier

client_w_enc_key_id

Generated client write encryption key identifier

server_w_enc_key_id

Generated server write encryption key identifier

client_w_mac_key_id

Generated client write MAC key identifier (see note 1)

server_w_mac_key_id

Generated server write MAC key identifier (see note 1)

client_w_iv

Pointer to the Client IV buffer (see note 2)

client_w_iv_length

Length of client_w_iv in bytes (see note 2)

server_w_iv

Pointer to the Server IV buffer (see note 2)

server_w_iv_length

Length of server_w_iv in bytes (see note 2)

3.1.4.9.3 Description

This structure defines the additional arguments needed for the TLS 1.2 Key derivation (&smw_derive_key_args->kdf_name = *TLS12_KEY_EXCHANGE*).

Note 1: Client/Server write MAC key are not generated with AES GCM cipher encryption.

Note 2: Client/Server write IVs are generated only in case of Authentication Encryption with Additional Data Cipher mode (like AES CCM or GCM).

The key derivation *smw_derive_key_args->key_descriptor_derived* is filled only if the *key_exchange_name* request for an ephemeral public key. Following *smw_derive_key_args->key_descriptor_derived* fields are filled:

- *id*: set to 0
- *type_name*: Set the key type name
- *security_size*: Size in bits of the derived key
- *shared_secret*: Shared secret buffer
- *shared_secret_len*: Shared secret buffer length

WARNING

This structure is deprecated and will be removed in a future library release. Please use the new `smw_kdf_tls12_op_args` API instead.

3.1.4.10 struct `smw_kdf_tls12_random_data`

struct `smw_kdf_tls12_random_data`

TLS 1.2 random data

3.1.4.10.1 Definition

```
struct smw_kdf_tls12_random_data {  
    unsigned char version;  
    unsigned char *client_random;  
    unsigned int client_random_length;  
    unsigned char *server_random;  
    unsigned int server_random_length;  
}
```

3.1.4.10.2 Members

`version`

[in] Version of this structure

`client_random`

[in] Client generated random data buffer

`client_random_length`

[in] `client_random` length in bytes

`server_random`

[in] Server generated random data buffer

`server_random_length`

[in] `server_random` length in bytes

3.1.4.11 struct `smw_kdf_tls12_session_hash`

struct `smw_kdf_tls12_session_hash`

TLS 1.2 session hash

3.1.4.11.1 Definition

```
struct smw_kdf_tls12_session_hash {  
    unsigned char version;  
    unsigned char *hash;  
    unsigned int hash_length;  
}
```

3.1.4.11.2 Members

version

[in] Version of this structure

hash

[in] Hash of the session data

hash_length

[in] hash length in bytes

3.1.4.12 struct smw_kdf_tls12_master_secret_args

struct **smw_kdf_tls12_master_secret_args**

TLS 1.2 master secret arguments

3.1.4.12.1 Definition

```
struct smw_kdf_tls12_master_secret_args {  
    unsigned char version;  
    smw_tls12_kec_t key_exchange_name;  
    bool ext_master_key;  
    unsigned char *peer_public_buffer;  
    unsigned int peer_public_buffer_length;  
    union {  
        struct smw_kdf_tls12_random_data *random_data;  
        struct smw_kdf_tls12_session_hash *session_hash;  
    } ;  
}
```

3.1.4.12.2 Members

version

[in] Version of this structure

key_exchange_name

[in] Name of the key exchange algorithm See *typedef smw_tls12_kec_t*

ext_master_key

[in] If true, generates an extended master secret key

peer_public_buffer

[in] Peer public key used for ECDH(E)

peer_public_buffer_length

[in] peer_public_buffer length in bytes

{unnamed_union}

anonymous

random_data

[in] The session random data, to be used when ext_master_key is false

session_hash

[in] The session hash, to be used when ext_master_key is true

3.1.4.12.3 Description

This structure defines the additional arguments needed for the TLS 1.2 Master Secret. (&smw_derive_key_args->kdf_name = *TLS12_OP_KEY_EXCHANGE*).

When *ext_master_key* is true (Extended Master Secret - TLS1.2 extension, RFC 7627), the *session_hash* should be set appropriately. Otherwise, *random_data* should be filled in.

3.1.4.13 struct smw_kdf_tls12_key_expansion_args

struct **smw_kdf_tls12_key_expansion_args**

TLS 1.2 key expansion arguments

3.1.4.13.1 Definition

```
struct smw_kdf_tls12_key_expansion_args {  
    unsigned char version;  
    smw_tls12_enc_t encryption_name;  
    struct smw_kdf_tls12_random_data *random_data;  
    unsigned int client_w_enc_key_id;  
    unsigned int server_w_enc_key_id;  
    unsigned int client_w_mac_key_id;  
    unsigned int server_w_mac_key_id;  
    unsigned char *client_w_iv;  
    unsigned int client_w_iv_length;  
    unsigned char *server_w_iv;  
    unsigned int server_w_iv_length;  
}
```

3.1.4.13.2 Members

version

[in] Version of this structure

encryption_name

[in] Name of the encryption algorithm See *typedef smw_tls12_enc_t*

random_data

[in] The session random data

client_w_enc_key_id

[out] Generated client write encryption key identifier

server_w_enc_key_id

[out] Generated server write encryption key identifier

client_w_mac_key_id

[out] Generated client write MAC key identifier (see note 1)

server_w_mac_key_id

[out] Generated server write MAC key identifier (see note 1)

client_w_iv

[in/out] Pointer to the Client IV buffer (see note 2)

client_w_iv_length

[in/out] client_w_iv length in bytes (see note 2)

server_w_iv

[in/out] Pointer to the Server IV buffer (see note 2)

server_w_iv_length

[in/out] server_w_iv length in bytes (see note 2)

3.1.4.13.3 Description

This structure defines the additional arguments needed for the TLS 1.2 Key Expansion (&smw_derive_key_args->kdf_name = *TLS12_OP_KEY_EXCHANGE*).

Note 1: Client/Server write MAC key are not generated with AEAD cipher encryption (CCM, GCM, CHACHA20_POLY1305).

Note 2: Client/Server write IVs are generated only in case of AEAD cipher modes (CCM, GCM, CHACHA20_POLY1305).

3.1.4.14 struct smw_kdf_tls12_op_args

struct **smw_kdf_tls12_op_args**

TLS 1.2 “operation-based” arguments

3.1.4.14.1 Definition

```
struct smw_kdf_tls12_op_args {
    unsigned char version;
    smw_hash_algo_t prf_name;
    smw_tls12_op_t op_name;
    struct smw_op_context *context;
    union {
        struct smw_kdf_tls12_master_secret_args master_secret;
        struct smw_kdf_tls12_key_expansion_args key_expansion;
    };
}
```

3.1.4.14.2 Members

version

[in] Version of this structure

prf_name

[in] Name of the Pseudo-Random Function (PRF) See *typedef smw_hash_algo_t*

op_name

[in] Name of the operation to execute See *typedef smw_tls12_op_t*

context

[in] Pointer to an opaque operation context structure See *struct smw_op_context*

{unnamed_union}

anonymous

master_secret

[in] The TLS1.2 Master Secret parameters

key_expansion

[in] The TLS1.2 Key Expansion parameters

3.1.4.14.3 Description

The context passed through this structure must be a valid context which is the result of the `smw_allocate_context` function. Subsystems may allocate data internally and associate it with the context. The same context needs to be passed to the master secret and key expansion operations.

Upon completion of the operations (with either success or error), the context is not released and remains valid. Calling `smw_cancel_operation` will release it and any associated data.

3.1.4.15 struct smw_kdf_tls13_args

struct **smw_kdf_tls13_args**

TLS1.3 KDF arguments structure

3.1.4.15.1 Definition

```
struct smw_kdf_tls13_args {  
    unsigned char version;  
    struct smw_key_descriptor *psk;  
    smw_hash_algo_t prf_name;  
    unsigned char *peer_public_buffer;  
    unsigned int peer_public_buffer_length;  
    unsigned char *expanded_label;  
    unsigned int expanded_label_length;  
}
```

3.1.4.15.2 Members

version

[in] Version of this structure.

psk

[in, optional] The Pre-Shared Key to use for Early Secret derivation. See [*struct smw_key_descriptor*](#)

prf_name

[in] Name of the Pseudo-Random Function (PRF). See [*typedef smw_hash_algo_t*](#)

peer_public_buffer

[in] Peer public buffer

peer_public_buffer_length

[in] `peer_public_buffer` length in bytes

expanded_label

[in] The expanded label to use for the TLS1.3 “Derived-Secret” function.

expanded_label_length

[in] `expanded_label` length in bytes

3.1.4.15.3 Description

The `expand_label` must be a buffer that contains the output of TLS1.3's "HKDF-Expand_label". `smw_tls13_expand_label()` is a helper function you may use to compute the expanded label, from the input label and context data (the context is usually the Transcript Hash).

The value of the input label controls which actual secrets get derived, e.g.:

- "ext binder" -> `binder_key`
- "c hs traffic" -> `client_handshake_traffic_secret`
- "c ap traffic" -> `client_application_traffic_secret_0`

Please refer to RFC 8446, section 7.1, to see the possible values for the label.

3.1.4.16 struct `smw_hkdf_extract_args`

struct **`smw_hkdf_extract_args`**

HKDF extract step arguments structure

3.1.4.16.1 Definition

```
struct smw_hkdf_extract_args {  
    unsigned char *salt;  
    unsigned int salt_len;  
    unsigned char *peer_public_buffer;  
    unsigned int peer_public_buffer_len;  
}
```

3.1.4.16.2 Members

`salt`

[in] Salt buffer

`salt_len`

[in] salt length in bytes

`peer_public_buffer`

[in] Peer public buffer

`peer_public_buffer_len`

[in] `peer_public_buffer` in bytes

3.1.4.16.3 Description

`salt` and `salt_len` are optional parameters.

`peer_public_buffer` should be a valid public key and should be in hex format.

3.1.4.17 struct `smw_hkdf_expand_args`

struct **`smw_hkdf_expand_args`**

HKDF expand arguments structure

3.1.4.17.1 Definition

```
struct smw_hkdf_expand_args {  
    unsigned char *info;  
    unsigned int info_len;  
}
```

3.1.4.17.2 Members

info

[in] Context and application specific information

info_len

[in] info length in bytes

3.1.4.17.3 Description

info and info_len are optional parameters.

3.1.4.18 struct smw_hkdf_args

struct **smw_hkdf_args**

HKDF full arguments structure

3.1.4.18.1 Definition

```
struct smw_hkdf_args {  
    struct smw_hkdf_extract_args extract_args;  
    struct smw_hkdf_expand_args expand_args;  
}
```

3.1.4.18.2 Members

extract_args

[in] HKDF extract arguments structure

expand_args

[in] HKDF expand arguments structure

3.1.4.19 struct smw_kdf_hkdf_args

struct **smw_kdf_hkdf_args**

HMAC-based Key derivation function arguments

3.1.4.19.1 Definition

```
struct smw_kdf_hkdf_args {  
    smw_hash_algo_t hash_algo;  
    union {  
        struct smw_hkdf_args hkdf_args;  
        struct smw_hkdf_extract_args hkdf_extract_args;  
    };  
}
```

(continues on next page)

```

    struct smw_hkdf_expand_args hkdf_expand_args;
} ;
}

```

3.1.4.19.2 Members

hash_algo

[in] Hash algorithm name. See *typedef smw_hash_algo_t*

{unnamed_union}

anonymous

hkdf_args

[in/out] HKDF full arguments. See *struct smw_hkdf_args*

hkdf_extract_args

[in/out] HKDF extract step arguments. See *struct hkdf_extract_args*

hkdf_expand_args

[in] HKDF expand step arguments. See *struct hkdf_expand_args*

3.1.4.19.3 Description

If user requests to store the derived key, user must provide *smw_derive_key_args.key_descriptor_derived.type_name*, *smw_derive_key_args.key_descriptor_derived.shared_secret*, *smw_derive_key_args.key_attributes.usage_flags*, *smw_derive_key_args.key_attributes.attributes* and, if required by the subsystem, the *smw_derive_key_args.key_attributes.permitted_algo*.

Key derivation using HKDF can be performed either in two dedicated steps (extract and expand) or combined into a single step, but only if the subsystem supports this capability.

- Step #1: Extract

- *smw_derive_key_args.store_derived_key* is ignored. Upon successful completion of this step, PRK buffer is exported if *smw_derive_key_args.key_descriptor_derived.shared_secret* and *hkdf_extract_args.prk_len* are set or PRK ID *smw_derive_key_args.key_descriptor_derived.id* is set.
- PRK is temporarily stored in subsystem key storage. It's deleted when the key derivation operation is completed.
- If base key is a plaintext buffer, the base key type should be set to *SMW_KEY_TYPE_NAME_RAW*.

- Step #2: Expand

- If PRK ID *smw_derive_key_args.key_descriptor_base.id* is set, PRK buffer *smw_derive_key_args.key_descriptor_base.buffer->gen.public_data* is ignored. If PRK buffer is exported, the base key type should be set to *SMW_KEY_TYPE_NAME_RAW*.
- Upon successful completion of the key derivation operation, derived key descriptor structure *smw_derive_key_args.key_descriptor_derived* is updated. The new key ID is set and shared secret data is exported if *shared_secret* and *shared_secret_len* are set in the derived key descriptor structure *smw_derive_key_args.key_descriptor_derived*.

- Full HKDF (step 1 and step 2 combined)
 - Upon successful completion of the key derivation operation, derived key descriptor structure `smw_derive_key_args.key_descriptor_derived` is updated. The new derived key ID is set, and shared secret data is exported if `shared_secret` and `shared_secret_len` are set in the derived key descriptor structure `smw_derive_key_args.key_descriptor_derived`.
 - If base key is a plaintext buffer, the base key type should be set to `SMW_KEY_TYPE_NAME_RAW`.

3.1.4.20 struct smw_kdf_ecdh_args

struct **smw_kdf_ecdh_args**

Key derivation function ECDH arguments

3.1.4.20.1 Definition

```
struct smw_kdf_ecdh_args {  
    unsigned char *peer_public_buffer;  
    unsigned int peer_public_buffer_length;  
}
```

3.1.4.20.2 Members

peer_public_buffer

Key derivation input data used to generate the shared secret key

peer_public_buffer_length

Length in bytes of the `peer_public_buffer` buffer

3.1.4.20.3 Description

This structure defines the additional arguments needed for the ECDH Key derivation (&`smw_derive_key_args`->`kdf_name` = *ECDH*).

Upon successful completion of the key derivation operation, derived key descriptor structure `smw_derive_key_args.key_descriptor_derived` is updated. The new derived key ID is set, and shared secret data is exported if `shared_secret` and `shared_secret_len` are set in the derived key descriptor structure `smw_derive_key_args.key_descriptor_derived`.

3.1.4.21 struct smw_import_key_args

struct **smw_import_key_args**

Key import arguments

3.1.4.21.1 Definition

```
struct smw_import_key_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor *key_descriptor;  
}
```

3.1.4.21.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

key_descriptor

Pointer to a Key descriptor object. See *struct smw_key_descriptor*

3.1.4.21.3 Description

subsystem_name designates the Secure Subsystem to be used. If this field is NULL, the default Secure Subsystem configured for this Security Operation is used. The key_descriptor fields type_name and security_size must be given as input to define the type of key to import. The key_descriptor field buffer is mandatory. A public key, a private key or a key pair is imported if the corresponding pointer and size is set. The key_descriptor field id, if set by the caller (other than 0) will be the created key identifier on operation success. Else the API will returned a new key identifier if id is set as 0. The buffer field format_name is optional. The default value is "HEX".

The key_descriptor.attributes is used to define the key attributes of the imported key.

3.1.4.22 struct smw_export_key_args

struct smw_export_key_args

Key export arguments

3.1.4.22.1 Definition

```
struct smw_export_key_args {  
    unsigned char version;  
    struct smw_key_descriptor *key_descriptor;  
}
```

3.1.4.22.2 Members

version

Version of this structure

key_descriptor

Pointer to a Key descriptor object. See *struct smw_key_descriptor*

3.1.4.22.3 Description

The key_descriptor fields id must be given as input. The key_descriptor field buffer is mandatory. The public key buffer must be set in order to export the public Key, in case of asymmetric Keys. The private key buffer must be set in order to export the private Key, only if the Secure Subsystem supports it. In that case, the private Key may be encrypted, not plaintext. The user can use *smw_get_key_buffers_lengths()* to set correct lengths for the public/private key buffer(s).

3.1.4.23 struct smw_delete_key_args

struct **smw_delete_key_args**

Key deletion arguments

3.1.4.23.1 Definition

```
struct smw_delete_key_args {  
    unsigned char version;  
    struct smw_key_descriptor *key_descriptor;  
}
```

3.1.4.23.2 Members

version

Version of this structure (must be equal 1).

key_descriptor

Pointer to a Key descriptor object. See *struct smw_key_descriptor*

3.1.4.23.3 Description

The key_descriptor fields id must be given as input. The key_descriptor fields buffer is ignored.

3.1.4.24 struct smw_get_key_attributes_args

struct **smw_get_key_attributes_args**

Get key attributes arguments

3.1.4.24.1 Definition

```
struct smw_get_key_attributes_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_key_descriptor *key_descriptor;  
    smw_key_privacy_t key_privacy_name;  
}
```

3.1.4.24.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

key_descriptor

Pointer to a Key descriptor object. See *struct smw_key_descriptor*

key_privacy_name

Key privacy name

3.1.4.24.3 Description

The `key_descriptor` fields `id` must be given as input. The `key_descriptor` fields `buffer` is ignored. The `key_descriptor` fields `type_name` and `security_size` are output.

The `key_descriptor.attributes` is used to get the key attributes.

3.1.4.25 struct `smw_commit_key_storage_args`

struct **smw_commit_key_storage_args**

Commit non-volatile key storage arguments

3.1.4.25.1 Definition

```
struct smw_commit_key_storage_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
}
```

3.1.4.25.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

3.1.4.26 struct `smw_key_attestation_args`

struct **smw_key_attestation_args**

Device attestation arguments

3.1.4.26.1 Definition

```
struct smw_key_attestation_args {  
    unsigned char version;  
    struct smw_key_descriptor *key_descriptor;  
    struct smw_key_descriptor *attest_key_descriptor;  
    smw_attr_algo_t sign_algo;  
    unsigned char *challenge;  
    unsigned int challenge_length;  
    unsigned char *certificate;  
    unsigned int certificate_length;  
}
```

3.1.4.26.2 Members

version

Version of this structure

key_descriptor

Pointer to a Key descriptor object of the key to be attested. See *struct smw_key_descriptor*

attest_key_descriptor

Pointer to a Key descriptor object of the attestation key. See *struct smw_key_descriptor*

sign_algo

Signature algorithm and attributes. See *typedef smw_attr_algo_t*

challenge

Caller unique ephemeral value (e.g. nonce)

challenge_length

Length (in bytes) of the challenge value

certificate

Device attestation certificate.

certificate_length

Length (in bytes) of the certificate.

3.1.4.26.3 Description

challenge length depends on the key (refer to the subsystem capabilities). If the length is bigger than expected, it will be cut to keep only the maximum size. If the length is shorter, the challenge value will be completed with 0's.

3.1.4.27 smw_generate_key

enum *smw_status_code* **smw_generate_key**(struct *smw_generate_key_args* *args)

Generate a Key.

Parameters

- **args** (struct *smw_generate_key_args**) – Pointer to the structure that contains the Key generation arguments.

3.1.4.27.1 Description

This function generates a Key.

3.1.4.27.2 Return

See *enum smw_status_code*

- Common return codes

3.1.4.28 smw_derive_key

enum *smw_status_code* **smw_derive_key**(struct *smw_derive_key_args* *args)

Derive a Key.

Parameters

- **args** (struct *smw_derive_key_args**) – Pointer to the structure that contains the Key derivation arguments.

3.1.4.28.1 Description

This function derives a Key.

If the shared secret length `args->key_descriptor_derived->shared_secret_len` is shorter than expected, this function returns status code `SMW_STATUS_OUTPUT_TOO_SHORT` and updates the shared secret length to the correct value.

3.1.4.28.2 Return

See `enum smw_status_code`

- Common return codes

3.1.4.29 smw_import_key

`enum smw_status_code smw_import_key(struct smw_import_key_args *args)`

Import a Key.

Parameters

- **args** (struct `smw_import_key_args*`) – Pointer to the structure that contains the Key import arguments.

3.1.4.29.1 Description

This function imports a Key into the storage managed by the Secure Subsystem. The key must be plain text.

3.1.4.29.2 Return

See `enum smw_status_code`

- Common return codes

3.1.4.30 smw_export_key

`enum smw_status_code smw_export_key(struct smw_export_key_args *args)`

Export a Key.

Parameters

- **args** (struct `smw_export_key_args*`) – Pointer to the structure that contains the Key export arguments.

3.1.4.30.1 Description

This function exports a Key.

3.1.4.30.2 Return

See `enum smw_status_code`

- Common return codes

3.1.4.31 smw_delete_key

enum *smw_status_code* **smw_delete_key**(struct *smw_delete_key_args* *args)

Delete a Key.

Parameters

- **args** (struct *smw_delete_key_args**) – Pointer to the structure that contains the Key deletion arguments.

3.1.4.31.1 Description

This function deletes a Key.

3.1.4.31.2 Return

See *enum smw_status_code*

- Common return codes

3.1.4.32 smw_get_key_buffers_lengths

enum *smw_status_code* **smw_get_key_buffers_lengths**(struct *smw_key_descriptor* *descriptor)

Gets Key buffers lengths.

Parameters

- **descriptor** (struct *smw_key_descriptor**) – Pointer to the Key descriptor.

3.1.4.32.1 This function calculates either

- The subsystem key buffers' lengths of the given descriptor field id. Only the exportable buffer lengths are returned.
- The standard key buffers's lengths of the given descriptor fields type_name, security_size.

The descriptor field buffer is mandatory. The buffer field format_name is optional. The buffer fields public_length, modulus and private_length are updated.

3.1.4.32.2 Return

See *enum smw_status_code*

- Common return codes

3.1.4.33 smw_get_key_type_name

enum *smw_status_code* **smw_get_key_type_name**(struct *smw_key_descriptor* *descriptor)

Gets the Key type name.

Parameters

- **descriptor** (struct *smw_key_descriptor**) – Pointer to the Key descriptor.

3.1.4.33.1 Description

This function gets the Key type name given the Key ID. The descriptor field `id` must be given as input. The descriptor fields `type_name` is updated.

3.1.4.33.2 Return

See `enum smw_status_code`

- Common return codes

3.1.4.34 smw_get_security_size

`enum smw_status_code smw_get_security_size(struct smw_key_descriptor *descriptor)`

Gets the Security size.

Parameters

- **descriptor** (struct `smw_key_descriptor*`) – Pointer to the Key descriptor.

3.1.4.34.1 Description

This function gets the Security size given the Key ID. The descriptor field `id` must be given as input. The descriptor fields `security_size` is updated.

3.1.4.34.2 Return

See `enum smw_status_code`

- Common return codes

3.1.4.35 smw_get_key_attributes

`enum smw_status_code smw_get_key_attributes(struct smw_get_key_attributes_args *args)`

Get the key attributes.

Parameters

- **args** (struct `smw_get_key_attributes_args*`) – Pointer to the structure that contains the Key attributes arguments.

3.1.4.35.1 Description

This function gets the Key attributes retrieved for the subsystem owning the given key identifier. If some key attributes are not supported, the output values are empty.

The `args.subsystem_name` field returned is the subsystem name that owns the key.

3.1.4.35.2 Return

See `enum smw_status_code`

- Common return codes

3.1.4.36 smw_commit_key_storage

enum *smw_status_code* **smw_commit_key_storage**(struct *smw_commit_key_storage_args* *args)

Commit the active non-volatile key storage

Parameters

- **args** (struct *smw_commit_key_storage_args**) – Pointer to the structure that contains the commit storage arguments.

3.1.4.36.1 Description

This function ensures that the non-volatile key storage opened by the subsystem is pushed in physical memory and associated anti-rollback protection counter is incremented.

3.1.4.36.2 Return

See *enum smw_status_code*

- Common return codes

3.1.4.37 smw_key_attestation

enum *smw_status_code* **smw_key_attestation**(struct *smw_key_attestation_args* *args)

Get the key attestation certificate.

Parameters

- **args** (struct *smw_key_attestation_args**) – Pointer to the structure that contains the key attestation arguments.

3.1.4.37.1 Description

Reads the key attestation certificate.

Certificate length of args field is updated to the correct value when:

- Length is bigger than expected. In this case operation succeeded.
- Length is shorter than expected. In this case operation failed and returned SMW_STATUS_OUTPUT_TOO_SHORT.
- Certificate buffer is set the NULL. In this case operation returned SMW_STATUS_OK

3.1.4.37.2 Return

See *enum smw_status_code*

- Common return codes

3.1.4.38 OEM Master Key agreement

3.1.4.38.1 struct smw_kdf_oem_master_key_args

struct **smw_kdf_oem_master_key_args**

OEM Master key derivation arguments

3.1.4.38.1.1 Definition

```
struct smw_kdf_oem_master_key_args {  
    unsigned char version;  
    smw_oem_master_key_op_t op;  
    bool use_peer_key_digest_kdf;  
    bool use_oem_srkh_kdf;  
    unsigned char *peer_public_buffer;  
    unsigned int peer_public_buffer_length;  
    unsigned char *info;  
    unsigned int info_len;  
    unsigned char *payload;  
    unsigned int payload_length;  
}
```

3.1.4.38.1.2 Members

version

[in] Version of this structure.

op

[in] OEM Master key derivation operation. See [typedef smw_oem_master_key_op_t](#)

use_peer_key_digest_kdf

[in] True if the peer key digest value is used as salt to derive the OEM Master key.

use_oem_srkh_kdf

[in] True if the OEM SRKH is used as salt to derive OEM wrapping and signing keys.

peer_public_buffer

[in] Key derivation input data used to generate the shared secret key

peer_public_buffer_length

[in] Length in bytes of the peer_public_buffer buffer

info

[in] [optional] Context and application specific information

info_len

[in] info length in bytes

payload

[in/out] Depends on the operation type defined by op

payload_length

[in/out] Length of the payload buffer.

3.1.4.38.1.3 Description

OEM Master key derivation operation is not supported on all subsystems, refer to the Subsystems Capabilities.

This key is used to import keys in the Secure Storage using a blob transporting the key in a secure manner. The blob is a TLV encoded message in which the key is encrypted and blob is signed. The encryption of the key uses a key derived from the OEM Master key and the blob signature uses a key derived from the OEM Master key. More details are available in the Subsystems Capabilities.

3.1.4.38.1.4 This structure contains the information required to

- Either prepare the payload data that must be signed.
- Or derive the OEM Master key that will be used to import a key in the Secure Enclave.

This arguments structure is the `kdf_arguments` of the `struct smw_derive_key_args` where `kdf_name` is set to `SMW_KDF_NAME_OEM_MASTER_KEY`

All `struct smw_derive_key_args` fields may not be used for this operation and if set may be overwritten with hardcoded value as described in the subsystem capabilities.

The supported operations (@op) are:

- Prepare the OEM Master key payload to sign: The payload is an output, it's built based on the user parameters given for the operation. This payload must be signed to attest the derive operation. It's an optional functionality proposed to facilitate the user process. Other NXP tool, like `SPSDK` can be used to build and sign the payload. Calling this operation with `@payload = NULL` will return the expected length of the payload buffer in the `payload_length`.
- Derive the OEM Master key: The payload is an input. It must be a payload signed.

3.1.5 Device management APIs

3.1.5.1 Introduction

The device APIs allow user of the library to:

- Get information about the device.
- Change the device lifecycle.

3.1.5.2 struct smw_device_attestation_args

struct `smw_device_attestation_args`

Device attestation arguments

3.1.5.2.1 Definition

```
struct smw_device_attestation_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    unsigned char *challenge;  
    unsigned int challenge_length;  
    unsigned char *certificate;  
    unsigned int certificate_length;  
}
```

3.1.5.2.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See `typedef smw_subsystem_t`

challenge

Caller unique ephemeral value (e.g. nonce)

challenge_length

Length (in bytes) of the challenge value

certificate

Device attestation certificate.

certificate_length

Length (in bytes) of the certificate.

3.1.5.2.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is NULL, the default configured Secure Subsystem is used.

challenge length depends of the device (refer to the subsystem capabilities). If the length is bigger than expected, it will be cut to keep only the device maximum size. If the length is shorter, the challenge value will be completed with 0's.

3.1.5.3 struct smw_device_uuid_args

struct **smw_device_uuid_args**

Device UUID arguments

3.1.5.3.1 Definition

```
struct smw_device_uuid_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    unsigned char *certificate;  
    unsigned int certificate_length;  
    unsigned char *uuid;  
    unsigned int uuid_length;  
}
```

3.1.5.3.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

certificate

Device attestation certificate.

certificate_length

Length (in bytes) of the certificate.

uuid

Device UUID buffer

uuid_length

Length (in bytes) of the uuid

3.1.5.3.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is NULL, the default configured Secure Subsystem is used.

Two methods are allowed to get the Device UUID.

- **Method #1**
Extract the device UUID from the device certificate. The Device Certificate (@certificate) is previously read using the `smw_device_attestation()` API.
- **Method #2**
Read the device UUID without providing the Device Certificate. The field `certificate` must be set to NULL.

3.1.5.4 struct smw_device_lifecycle_args

struct **smw_device_lifecycle_args**

Device lifecycle arguments

3.1.5.4.1 Definition

```
struct smw_device_lifecycle_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    smw_lifecycle_t lifecycle_name;  
}
```

3.1.5.4.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See `typedef smw_subsystem_t`

lifecycle_name

Device lifecycle name. See `typedef smw_lifecycle_t`

3.1.5.4.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is NULL, the default configured Secure Subsystem is used.

3.1.5.5 struct smw_device_reprovision_args

struct **smw_device_reprovision_args**

Device storage reprovisioning arguments

3.1.5.5.1 Definition

```
struct smw_device_reprovision_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    unsigned char *data;  
    unsigned int data_length;  
}
```

3.1.5.5.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

data

Data message to prepare or to send to the device

data_length

Length of the data buffer

3.1.5.5.3 Description

`subsystem_name` designates the Secure Subsystem to be used. If this field is NULL, the default configured Secure Subsystem is used.

3.1.5.6 smw_device_attestation

enum *smw_status_code* **smw_device_attestation**(struct *smw_device_attestation_args* *args)

Get the device attestation certificate.

Parameters

- **args** (struct *smw_device_attestation_args**) – Pointer to the structure that contains the device attestation arguments.

3.1.5.6.1 Description

Reads the device attestation certificate.

Certificate length args field is updated to the correct value when:

- Length is bigger than expected. In this case operation succeeded.
- Length is shorter than expected. In this case operation failed and returned `SMW_STATUS_OUTPUT_TOO_SHORT`.
- Certificate buffer is set the NULL. In this case operation returned `SMW_STATUS_OK`

3.1.5.6.2 Return

See [enum smw_status_code](#)

- Common return codes

3.1.5.7 smw_device_get_uuid

enum [smw_status_code](#) **smw_device_get_uuid**(struct [smw_device_uuid_args](#) *args)

Get the device UUID.

Parameters

- **args** (struct [smw_device_uuid_args](#)*) – Pointer to the structure that contains the device UUID arguments.

3.1.5.7.1 Description

Extracts device UUID from the device certificate or reads the device UUID without device certificate.

Device UUID buffer is in big endian format.

UUID length args field is updated to the correct value when:

- Length is bigger than expected. In this case operation succeeded.
- Length is shorter than expected. In this case operation failed and returned SMW_STATUS_OUTPUT_TOO_SHORT.
- UUID buffer is set the NULL. In this case operation returned SMW_STATUS_OK

3.1.5.7.2 Return

See [enum smw_status_code](#)

- Common return codes

3.1.5.8 smw_device_set_lifecycle

enum [smw_status_code](#) **smw_device_set_lifecycle**(struct [smw_device_lifecycle_args](#) *args)

Set the device to given lifecycle.

Parameters

- **args** (struct [smw_device_lifecycle_args](#)*) – Pointer to the structure that contains the device lifecycle arguments.

3.1.5.8.1 Description

Forward the device lifecycle to the given value. The device must be reset to propagate the new lifecycle.

Caution: Forwarding device lifecycle is not reversible.

3.1.5.8.2 Return

See [enum smw_status_code](#)

- Common return codes

3.1.5.9 smw_device_get_lifecycle

enum *smw_status_code* **smw_device_get_lifecycle**(struct *smw_device_lifecycle_args* *args)

Get the device active lifecycle.

Parameters

- **args** (struct *smw_device_lifecycle_args**) – Pointer to the structure that contains the device lifecycle arguments.

3.1.5.9.1 Return

See *enum smw_status_code*

- Common return codes

3.1.5.10 smw_device_reprovision_prepare

enum *smw_status_code* **smw_device_reprovision_prepare**(struct *smw_device_reprovision_args* *args)

Fill the device reprovisioning message

Parameters

- **args** (struct *smw_device_reprovision_args**) – Pointer to the structure that contains the reprovisioning arguments

3.1.5.10.1 Description

This function is used to fill the reprovisioning message. The field `data_length` of `args` is updated to the correct value when:

- Length is bigger than expected. In this case operation succeeded.
- Length is shorter than expected. In this case operation failed and returned `SMW_STATUS_OUTPUT_TOO_SHORT`.
- Data buffer is set the `NULL`. In this case operation returned `SMW_STATUS_OK`

3.1.5.10.2 Return

See *enum smw_status_code*

- Common return codes

3.1.5.11 smw_device_reprovision

enum *smw_status_code* **smw_device_reprovision**(struct *smw_device_reprovision_args* *args)

Request device storage reprovisioning

Parameters

- **args** (struct *smw_device_reprovision_args**) – Pointer to the structure that contains the reprovisioning arguments

3.1.5.11.1 Description

This function is used to request the subsystem to enable the storage re-provisioning. In this case, the rollback protection of the storage is reset and consequently all objects previously stored are lost.

The field data of args might contain information to be passed to the subsystem.

3.1.5.11.2 Return

See `enum smw_status_code`

- Common return codes

3.1.6 Storage APIs

3.1.6.1 Introduction

The storage APIs allow user of the library to:

- Store data.
- Retrieve data.
- Delete data.

The data storing operation allows user to request the subsystem to either:

- store data as given by the user, reading back the data will in the same format as given by user.
- encrypt data then store, reading back the data will be a blob of encrypted data.
- encrypt and sign data then store, reading back the data will be a signed blob of encrypted data.
- sign data then store, reading back the data will be a signed blob of data. Knowing that data format is identical as the given by user.

Refer to the subsystem capabilities for more details of the supported features and blob format.

Signature is limited to MAC signature.

3.1.6.2 struct smw_data_attributes

struct **smw_data_attributes**

Data attributes

3.1.6.2.1 Definition

```
struct smw_data_attributes {  
    smw_attr_storage_id_t storage_id;  
    smw_attr_attributes_t attributes;  
}
```

3.1.6.2.2 Members

storage_id

Storage identifier. See *typedef smw_attr_storage_id_t*

attributes

Attributes. See *typedef smw_attr_attributes_t*

3.1.6.3 struct smw_data_descriptor

struct **smw_data_descriptor**

Data descriptor

3.1.6.3.1 Definition

```
struct smw_data_descriptor {  
    unsigned int identifier;  
    unsigned char *data;  
    unsigned int length;  
    struct smw_data_attributes attributes;  
}
```

3.1.6.3.2 Members

identifier

Data identifier

data

Pointer to the data buffer

length

Length of buffer data

attributes

Data attributes. See *typedef smw_data_attributes_t*

3.1.6.4 struct smw_encryption_args

struct **smw_encryption_args**

Encryption arguments

3.1.6.4.1 Definition

```
struct smw_encryption_args {  
    struct smw_key_descriptor **keys_desc;  
    unsigned int nb_keys;  
    smw_cipher_mode_t mode_name;  
    unsigned char *iv;  
    unsigned int iv_length;  
}
```

3.1.6.4.2 Members

keys_desc

Pointer to an array of pointers to key descriptors. See *struct smw_key_descriptor*

nb_keys

Number of entries of keys_desc

mode_name

Cipher mode name. See *typedef smw_cipher_mode_t*

iv

Pointer to initialization vector

iv_length

iv length in bytes

3.1.6.4.3 Description

Depending on mode_name, the iv is optional and represents:

- Initialization Vector (CBC, CTS)
- Initial Counter Value (CTR)
- Tweak Value (XTS)

3.1.6.5 struct smw_sign_args

struct **smw_sign_args**

Sign arguments

3.1.6.5.1 Definition

```
struct smw_sign_args {  
    struct smw_key_descriptor *key_descriptor;  
    smw_mac_algo_t algo_name;  
    smw_hash_algo_t hash_name;  
}
```

3.1.6.5.2 Members

key_descriptor

Pointer to a signing Key descriptor object. See *struct smw_key_descriptor*

algo_name

MAC algorithm name. See *typedef smw_mac_algo_t*

hash_name

Hash algorithm name. See *typedef smw_hash_algo_t*

3.1.6.6 struct smw_store_data_args

struct **smw_store_data_args**

Store data arguments

3.1.6.6.1 Definition

```
struct smw_store_data_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_data_descriptor *data_descriptor;  
    struct smw_encryption_args *encryption_args;  
    struct smw_sign_args *sign_args;  
}
```

3.1.6.6.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

data_descriptor

Data descriptor. See *struct smw_data_descriptor*

encryption_args

Encryption arguments. See *struct smw_encryption_args*

sign_args

Sign arguments. See *struct smw_sign_args*

3.1.6.6.3 Description

subsystem_name designates the Secure Subsystem to be used. If this field is NULL, the default configured Secure Subsystem is used.

The encryption_args and smw_sign_args arguments are optional. If defined the operation consists respectively in encrypting and/or signing the data. The capability to encrypt and/or sign data is function of the subsystem. Refer to the *Subsystems Capabilities*.

3.1.6.7 struct smw_retrieve_data_args

struct **smw_retrieve_data_args**

Retrieve data arguments

3.1.6.7.1 Definition

```
struct smw_retrieve_data_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_data_descriptor *data_descriptor;  
}
```

3.1.6.7.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

data_descriptor

Data descriptor. See *struct smw_data_descriptor*

3.1.6.7.3 Description

subsystem_name designates the Secure Subsystem to be used. If this field is NULL, the default configured Secure Subsystem is used.

3.1.6.8 struct smw_delete_data_args

struct **smw_delete_data_args**

Delete data arguments

3.1.6.8.1 Definition

```
struct smw_delete_data_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_data_descriptor *data_descriptor;  
}
```

3.1.6.8.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

data_descriptor

Data descriptor. See *struct smw_data_descriptor*

3.1.6.8.3 Description

subsystem_name designates the Secure Subsystem to be used. If this field is NULL, the default configured Secure Subsystem is used.

3.1.6.9 struct smw_data_info_args

struct **smw_data_info_args**

Data information arguments

3.1.6.9.1 Definition

```
struct smw_data_info_args {  
    unsigned char version;  
    smw_subsystem_t subsystem_name;  
    struct smw_data_descriptor *data_descriptor;  
}
```

3.1.6.9.2 Members

version

Version of this structure

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

data_descriptor

Data descriptor. See *struct smw_data_descriptor*

3.1.6.9.3 Description

This function gets the data attributes retrieved for the subsystem owning the given data identifier.

If data is present in the internal object database, the field `subsystem_name` is not used. If data is not present in the internal object database, and the field `subsystem_name` is defined, the data information are retrieved from the subsystem. If data is not present in the internal object database, and the field `subsystem_name` is NULL, query all subsystems to get the data information until one subsystem replies.

If some data attributes are not supported, the output values are empty.

3.1.6.10 smw_store_data

enum *smw_status_code* **smw_store_data**(struct *smw_store_data_args* *args)

Store data.

Parameters

- **args** (struct *smw_store_data_args**) – Pointer to the structure that contains the store data arguments.

3.1.6.10.1 Description

Stores the data.

3.1.6.10.2 Return

See *enum smw_status_code*

- Common return codes

3.1.6.11 smw_retrieve_data

enum *smw_status_code* **smw_retrieve_data**(struct *smw_retrieve_data_args* *args)

Retrieve data.

Parameters

- **args** (struct *smw_retrieve_data_args**) – Pointer to the structure that contains the retrieve data arguments.

3.1.6.11.1 Description

Retrieves the data.

To query the required data buffer length for data retrieval, set `args->data_descriptor->data` to NULL. The function will then set the required data buffer length in `args->data_descriptor->length` and return `SMW_STATUS_OK`.

On operation completion, the `args->data_descriptor->length` is updated to the correct value when

- Data buffer length is bigger than expected. In this case, operation succeeds.
- Data buffer length is shorter than expected. In this case, operation fails and returns `SMW_STATUS_OUTPUT_TOO_SHORT`.

3.1.6.11.2 Return

See *enum smw_status_code*

- Common return codes

3.1.6.12 smw_delete_data

enum *smw_status_code* **smw_delete_data**(struct *smw_delete_data_args* *args)

Delete data.

Parameters

- **args** (struct *smw_delete_data_args**) – Pointer to the structure that contains the store data arguments.

3.1.6.12.1 Description

Deletes the data.

3.1.6.12.2 Return

See *enum smw_status_code*

- Common return codes

3.1.6.13 smw_get_data_info

enum *smw_status_code* **smw_get_data_info**(struct *smw_data_info_args* *args)

Get data information

Parameters

- **args** (struct *smw_data_info_args**) – Pointer to the data information arguments.

3.1.6.13.1 Description

Returns the data information extracts from the subsystem where data is stored combined with the internal database if data identifier is present.

The `args.subsystem_name` field returned is the subsystem name that owns the data.

3.1.6.13.2 Return

See *enum smw_status_code*

- Common return codes

3.1.7 Object management APIs

3.1.7.1 struct smw_object_descriptor

struct **smw_object_descriptor**

Generic SMW object descriptor

3.1.7.1.1 Definition

```
struct smw_object_descriptor {
    unsigned int id;
    smw_object_type_t type;
    smw_subsystem_t subsystem_name;
    smw_attr_attributes_t persistency;
    char *label;
    char *user_id;
    unsigned int group;
    union {
        struct smw_key_descriptor key;
        struct smw_data_descriptor data;
    } ;
}
```

3.1.7.1.2 Members

id

Object identifier

type

Defines the object type. See *typedef smw_object_type_t*

subsystem_name

Secure Subsystem name. See *typedef smw_subsystem_t*

persistency

Object persistency attributes. See *typedef smw_attr_attributes_t*.

label

Object description

user_id

User defined ID in base64 null terminated string.

group

Key group (may not be used by all subsystems)

{unnamed_union}

anonymous

key

Key descriptor. See *struct smw_key_descriptor*

data

Data descriptor. See *struct smw_data_descriptor*

3.1.7.1.3 Description

Depending on the type of object (key or data), the object persistency is retrieved from either the `key.attributes` or the `data.attributes`.

3.1.7.2 struct smw_find_object_db_args

struct **smw_find_object_db_args**

Object find operation arguments

3.1.7.2.1 Definition

```
struct smw_find_object_db_args {  
    unsigned char version;  
    struct smw_object_descriptor *object_descriptor;  
    void *ctx;  
}
```

3.1.7.2.2 Members

version

Version of this structure

object_descriptor

Object descriptor. See *struct smw_object_descriptor*.

ctx

Find operation opaque context.

3.1.7.2.3 Description

The `ctx` is allocated by the init step of the find operation and destroyed by the final step of the find operation. In case of one-shot operation finding the object by its id, the `ctx` is not used.

3.1.7.3 smw_update_object_db

enum *smw_status_code* **smw_update_object_db**(struct *smw_object_descriptor* *descriptor)

Update a database object.

Parameters

-
- **descriptor** (struct *smw_object_descriptor**) – Object descriptor.
See *struct smw_object_descriptor*.

3.1.7.3.1 Description

This function updates the Object attribute list.

3.1.7.3.2 Return

See *enum smw_status_code*

- Common return codes

3.1.7.4 smw_find_object_db

enum smw_status_code **smw_find_object_db**(struct *smw_find_object_db_args** args)

Find an object by its id.

Parameters

- **args** (struct *smw_find_object_db_args**) – Pointer to the structure that contains the find object arguments.

3.1.7.4.1 Description

This function return the object descriptor matching the id. The find operation context is not used in this operation.

3.1.7.4.2 Return

See *enum smw_status_code*

- Common return codes

3.1.7.5 smw_find_object_db_init

enum smw_status_code **smw_find_object_db_init**(struct *smw_find_object_db_args** args)

Initialize the find context.

Parameters

- **args** (struct *smw_find_object_db_args**) – Pointer to the structure that contains the find object arguments.

3.1.7.5.1 Description

This function allocates and initialises the find context. If this API returns success, *smw_find_object_db_final()* must be called to free the allocated context operation.

3.1.7.5.2 Return

See *enum smw_status_code*

- Common return codes

3.1.7.6 smw_find_object_db_next

enum *smw_status_code* **smw_find_object_db_next**(struct *smw_find_object_db_args* *args)

Find next matching Object.

Parameters

- **args** (struct *smw_find_object_db_args**) – Pointer to the structure that contains the find object arguments.

3.1.7.6.1 Description

This function returns the next find Object.

3.1.7.6.2 Return

See *enum smw_status_code*

- Common return codes

3.1.7.7 smw_find_object_db_final

enum *smw_status_code* **smw_find_object_db_final**(struct *smw_find_object_db_args* *args)

Finalize the find context.

Parameters

- **args** (struct *smw_find_object_db_args**) – Pointer to the structure that contains the find object arguments.

3.1.7.7.1 Description

This function destroys the given find context.

3.1.7.7.2 Return

See *enum smw_status_code*

- Common return codes

3.1.8 TLS Helper APIs

3.1.8.1 struct smw_tls13_expand_label_args

struct **smw_tls13_expand_label_args**

TLS1.3 “HKDF-Expand-Label” arguments structure

3.1.8.1.1 Definition

```
struct smw_tls13_expand_label_args {  
    unsigned char version;  
    unsigned int length;  
    unsigned char *label;  
    unsigned int label_length;  
    unsigned char *context;  
}
```

(continues on next page)

```

unsigned int context_length;
unsigned char *expanded_label;
unsigned int expanded_label_length;
}

```

3.1.8.1.2 Members

version

[in] Version of this structure.

length

[in] Context-specific length (usually equal to Hash.Length).

label

[in] Input label buffer, without the “tls13 “ prefix.

label_length

[in] label length in bytes.

context

[in] Input context buffer, usually the Transcript Hash.

context_length

[in] context length in bytes.

expanded_label

[in/out] Buffer that holds the expanded label.

expanded_label_length

[in/out] expanded_label length in bytes.

3.1.8.2 macro SMW_TLS13_EXPANDED_LABEL_LENGTH

SMW_TLS13_EXPANDED_LABEL_LENGTH(arg)

Return the minimum length needed to construct a TLS1.3 expanded label.

Parameters

- **arg** – An pointer to a *struct smw_tls13_expand_label_args*.

3.1.8.2.1 Description

In order to avoid 2 calls to `smw_tls13_expand_label()`, you may use this macro to get the minimum required expanded label buffer length, after having filled in the `arg.label_length` and `arg.context_length` values.

3.1.8.2.2 Return

Minimum length needed for `arg.expanded_label_length`

3.1.8.3 smw_tls13_expand_label

enum *smw_status_code* **smw_tls13_expand_label**(struct *smw_tls13_expand_label_args* *args)

Helper function to construct the TLS1.3 “HkdfLabel”, used by the “HKDF-Expand-Label” function.

Parameters

- **args** (struct *smw_tls13_expand_label_args**) – Expand label arguments.

3.1.8.3.1 Description

smw_tls13_expand_label_args.expanded_label should be a buffer that is large enough to hold the expanded label; if that is not the case, this function will return *SMW_STATUS_OUTPUT_TOO_SHORT* and set *smw_tls13_expand_label_args.expanded_label_length* to the minimum required length. Alternatively, you may also use the *SMW_TLS13_EXPANDED_LABEL_LENGTH* macro to get the minimum required length.

This function will also prepend *smw_tls13_expand_label_args.label* with “tls13 “.

3.1.8.3.2 Return

See *enum smw_status_code*

- **SMW_STATUS_OK:**
Success
- **SMW_STATUS_TOO_LARGE_NUMBER:**
One of the input length parameters is too large
- **SMW_STATUS_OUTPUT_TOO_SHORT:**
Output buffer is too short

3.1.9 Return codes

3.1.9.1 enum smw_status_code

enum **smw_status_code**

Security Middleware status codes

3.1.9.1.1 Definition

```
enum smw_status_code {  
    SMW_STATUS_OK,  
    SMW_STATUS_INVALID_VERSION,  
    SMW_STATUS_INVALID_BUFFER,  
    SMW_STATUS_EOF,  
    SMW_STATUS_SYNTAX_ERROR,  
    SMW_STATUS_UNKNOWN_NAME,  
    SMW_STATUS_UNKNOWN_ID,  
    SMW_STATUS_TOO_LARGE_NUMBER,  
    SMW_STATUS_ALLOC_FAILURE,  
    SMW_STATUS_INVALID_PARAM,  
};
```

(continues on next page)

SMW_STATUS_VERSION_NOT_SUPPORTED,
SMW_STATUS_SUBSYSTEM_LOAD_FAILURE,
SMW_STATUS_SUBSYSTEM_UNLOAD_FAILURE,
SMW_STATUS_SUBSYSTEM_FAILURE,
SMW_STATUS_SUBSYSTEM_NOT_CONFIGURED,
SMW_STATUS_OPERATION_NOT_SUPPORTED,
SMW_STATUS_OPERATION_NOT_CONFIGURED,
SMW_STATUS_OPERATION_FAILURE,
SMW_STATUS_SIGNATURE_INVALID,
SMW_STATUS_NO_KEY_BUFFER,
SMW_STATUS_OUTPUT_TOO_SHORT,
SMW_STATUS_SIGNATURE_LEN_INVALID,
SMW_STATUS_OPS_INVALID,
SMW_STATUS_MUTEX_INIT_FAILURE,
SMW_STATUS_MUTEX_DESTROY_FAILURE,
SMW_STATUS_INVALID_TAG,
SMW_STATUS_RANGE_DUPLICATE,
SMW_STATUS_ALGO_NOT_CONFIGURED,
SMW_STATUS_CONFIG_ALREADY_LOADED,
SMW_STATUS_NO_CONFIG_LOADED,
SMW_STATUS_SUBSYSTEM_OUT_OF_MEMORY,
SMW_STATUS_SUBSYSTEM_STORAGE_NO_SPACE,
SMW_STATUS_SUBSYSTEM_STORAGE_ERROR,
SMW_STATUS_SUBSYSTEM_CORRUPT_OBJECT,
SMW_STATUS_LOAD_METHOD_DUPLICATE,
SMW_STATUS_LIBRARY_ALREADY_INIT,
SMW_STATUS_SUBSYSTEM_LOADED,
SMW_STATUS_SUBSYSTEM_NOT_LOADED,
SMW_STATUS_OBJ_DB_INIT,
SMW_STATUS_OBJ_DB_CREATE,
SMW_STATUS_OBJ_DB_UPDATE,
SMW_STATUS_OBJ_DB_DELETE,
SMW_STATUS_OBJ_DB_GET_INFO,
SMW_STATUS_KEY_POLICY_WARNING_IGNORED,
SMW_STATUS_KEY_INVALID,
SMW_STATUS_MUTEX_LOCK_FAILURE,
SMW_STATUS_MUTEX_UNLOCK_FAILURE,
SMW_STATUS_INVALID_LIBRARY_CONTEXT,
SMW_STATUS_INVALID_CONFIG_DATABASE,
SMW_STATUS_INVALID_LIFECYCLE,
SMW_STATUS_UNKNOWN_MODE_NAME,
SMW_STATUS_UNKNOWN_OP_TYPE_NAME,
SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME,
SMW_STATUS_UNKNOWN_ALGO_NAME,
SMW_STATUS_UNKNOWN_SIGN_ALGO_NAME,
SMW_STATUS_UNKNOWN_SIGN_TYPE_NAME,
SMW_STATUS_UNKNOWN_CONFIG_OP_NAME,
SMW_STATUS_UNKNOWN_LOAD_METHOD_NAME,
SMW_STATUS_UNKNOWN_KEY_OP_NAME,

(continues on next page)

```

SMW_STATUS_UNKNOWN_KEY_TYPE_NAME,
SMW_STATUS_UNKNOWN_FORMAT_NAME,
SMW_STATUS_UNKNOWN_KDF_NAME,
SMW_STATUS_UNKNOWN_TLS12_KEY_NAME,
SMW_STATUS_UNKNOWN_TLS12_ENC_NAME,
SMW_STATUS_OEM_SRKH_NOT_FUSED,
SMW_STATUS_CONFIGURATION_FAILURE,
SMW_STATUS_INVALID_IV_SIZE,
SMW_STATUS_UNKNOWN_KEY_PRIVACY_NAME,
SMW_STATUS_OBJ_DB_FIND,
SMW_STATUS_PUBLIC_EXPONENT_NOT_SUPPORTED,
SMW_STATUS_KEY_ID_ALREADY_EXIST,
SMW_STATUS_PERMITTED_ALGO_INVALID,
SMW_STATUS_OPERATION_ALREADY_INIT
};

```

3.1.9.1.2 Constants

SMW_STATUS_OK

Function returned successfully.

SMW_STATUS_INVALID_VERSION

The version of the configuration file is not supported.

SMW_STATUS_INVALID_BUFFER

The configuration file passed by OSAL to the library is not valid.

SMW_STATUS_EOF

The configuration file is syntactically too short.

SMW_STATUS_SYNTAX_ERROR

The configuration file is syntactically wrong.

SMW_STATUS_UNKNOWN_NAME

Generic status code indicating that one of the name arguments is not valid.

SMW_STATUS_UNKNOWN_ID

One of the identifier arguments is not valid.

SMW_STATUS_TOO_LARGE_NUMBER

The configuration file defines a too big numeral value.

SMW_STATUS_ALLOC_FAILURE

Internal allocation failure.

SMW_STATUS_INVALID_PARAM

Generic status code indicating that one of the argument parameter is not valid.

SMW_STATUS_VERSION_NOT_SUPPORTED

Argument version not compatible.

SMW_STATUS_SUBSYSTEM_LOAD_FAILURE

Load of the Secure Subsystem failed.

SMW_STATUS_SUBSYSTEM_UNLOAD_FAILURE

Unload of the Secure Subsystem failed.

SMW_STATUS_SUBSYSTEM_FAILURE

Secure Subsystem operation general failure.

SMW_STATUS_SUBSYSTEM_NOT_CONFIGURED

Secure Subsystem is not configured in the user configuration.

SMW_STATUS_OPERATION_NOT_SUPPORTED

Operation is not supported by the Secure Subsystem.

SMW_STATUS_OPERATION_NOT_CONFIGURED

Operation is not configured in the user configuration.

SMW_STATUS_OPERATION_FAILURE

Operation general failure. Error returned before calling the Secure Subsystem.

SMW_STATUS_SIGNATURE_INVALID

The Signature is not valid.

SMW_STATUS_NO_KEY_BUFFER

No Key buffer is set in the Key descriptor structure.

SMW_STATUS_OUTPUT_TOO_SHORT

Output buffer is too small. Output size field is updated with the expected size.

SMW_STATUS_SIGNATURE_LEN_INVALID

The Signature length is not valid.

SMW_STATUS_OPS_INVALID

OSAL operations structure is invalid.

SMW_STATUS_MUTEX_INIT_FAILURE

Mutex initialization has failed.

SMW_STATUS_MUTEX_DESTROY_FAILURE

Mutex destruction has failed.

SMW_STATUS_INVALID_TAG

Tag is invalid.

SMW_STATUS_RANGE_DUPLICATE

Size range is defined more than once for a given algorithm.

SMW_STATUS_ALGO_NOT_CONFIGURED

Size range is defined but the corresponding algorithm is not configured.

SMW_STATUS_CONFIG_ALREADY_LOADED

User configuration is already loaded. To load another one, the Unload configuration API must be called first.

SMW_STATUS_NO_CONFIG_LOADED

No user configuration is loaded.

SMW_STATUS_SUBSYSTEM_OUT_OF_MEMORY

Subsystem memory allocation failure.

SMW_STATUS_SUBSYSTEM_STORAGE_NO_SPACE

Not enough space in the secure subsystem to handle the requested operation.

SMW_STATUS_SUBSYSTEM_STORAGE_ERROR

Generic secure subsystem storage error.

SMW_STATUS_SUBSYSTEM_CORRUPT_OBJECT

An object stored in the secure subsystem is corrupted.

SMW_STATUS_LOAD_METHOD_DUPLICATE

The load/unload method is defined more than once.

SMW_STATUS_LIBRARY_ALREADY_INIT

Library is already initialized.

SMW_STATUS_SUBSYSTEM_LOADED

Secure Subsystem is loaded.

SMW_STATUS_SUBSYSTEM_NOT_LOADED

Secure Subsystem is not loaded.

SMW_STATUS_OBJ_DB_INIT

Initialization error of the object database.

SMW_STATUS_OBJ_DB_CREATE

Object database creation error.

SMW_STATUS_OBJ_DB_UPDATE

Object database update error.

SMW_STATUS_OBJ_DB_DELETE

Object database delete error.

SMW_STATUS_OBJ_DB_GET_INFO

Object database get information error.

SMW_STATUS_KEY_POLICY_WARNING_IGNORED

At least one element of the key policy is ignored.

SMW_STATUS_KEY_INVALID

Key used for the operation is not valid.

SMW_STATUS_MUTEX_LOCK_FAILURE

Mutex lock has failed.

SMW_STATUS_MUTEX_UNLOCK_FAILURE

Mutex unlock has failed.

SMW_STATUS_INVALID_LIBRARY_CONTEXT

Library context is not valid.

SMW_STATUS_INVALID_CONFIG_DATABASE

Configuration database is not valid.

SMW_STATUS_INVALID_LIFECYCLE

Device lifecycle not valid, or object not accessible in current device lifecycle.

SMW_STATUS_UNKNOWN_MODE_NAME

Mode name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_OP_TYPE_NAME

Operation type name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME

Secure Subsystem name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_ALGO_NAME

Algorithm name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_SIGN_ALGO_NAME

Signature algo name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_SIGN_TYPE_NAME

Signature type name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_CONFIG_OP_NAME

Operation name set in the configuration file is not recognized by SMW.

SMW_STATUS_UNKNOWN_LOAD_METHOD_NAME

String value of the load/unload method set in the configuration file is not recognized by SMW.

SMW_STATUS_UNKNOWN_KEY_OP_NAME

Key operation name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_KEY_TYPE_NAME

Key type name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_FORMAT_NAME

Key format name provided by the user is not recognized by SMW.

SMW_STATUS_UNKNOWN_KDF_NAME

Key derivation function name provided by the user or set in the user configuration is not recognized by SMW.

SMW_STATUS_UNKNOWN_TLS12_KEYA_NAME

TLS 1.2 Key exchange algorithm name provided by the user is not recognized by SMW.

SMW_STATUS_UNKNOWN_TLS12_ENC_NAME

TLS 1.2 encryption algorithm name provided by the user is not recognized by SMW.

SMW_STATUS_OEM_SRKH_NOT_FUSED

Device OEM SRKH is not fused.

SMW_STATUS_CONFIGURATION_FAILURE

Library configuration failure.

SMW_STATUS_INVALID_IV_SIZE

IV size used for the operation is not valid

SMW_STATUS_UNKNOWN_KEY_PRIVACY_NAME

Key privacy name set in the object database is not recognized by SMW.

SMW_STATUS_OBJ_DB_FIND

Object database find object error.

SMW_STATUS_PUBLIC_EXPONENT_NOT_SUPPORTED

Provided RSA public exponent is unsupported.

SMW_STATUS_KEY_ID_ALREADY_EXIST

Key identifier already exist.

SMW_STATUS_PERMITTED_ALGO_INVALID

Permitted algorithm missing one or more algorithm parameters.

SMW_STATUS_OPERATION_ALREADY_INIT

Operation is already initialized.

3.1.9.1.3 Status code classification

- Common return codes
 - SMW_STATUS_OK
 - SMW_STATUS_UNKNOWN_NAME
 - SMW_STATUS_UNKNOWN_ID
 - SMW_STATUS_ALLOC_FAILURE
 - SMW_STATUS_INVALID_PARAM
 - SMW_STATUS_VERSION_NOT_SUPPORTED
 - SMW_STATUS_SUBSYSTEM_LOAD_FAILURE
 - SMW_STATUS_SUBSYSTEM_UNLOAD_FAILURE
 - SMW_STATUS_SUBSYSTEM_FAILURE
 - SMW_STATUS_SUBSYSTEM_NOT_CONFIGURED
 - SMW_STATUS_OPERATION_NOT_SUPPORTED
 - SMW_STATUS_OPERATION_NOT_CONFIGURED
 - SMW_STATUS_OPERATION_FAILURE
 - SMW_STATUS_NO_KEY_BUFFER
 - SMW_STATUS_OUTPUT_TOO_SHORT
 - SMW_STATUS_SUBSYSTEM_OUT_OF_MEMORY
 - SMW_STATUS_SUBSYSTEM_STORAGE_NO_SPACE
 - SMW_STATUS_SUBSYSTEM_STORAGE_ERROR
 - SMW_STATUS_SUBSYSTEM_CORRUPT_OBJECT
 - SMW_STATUS_SUBSYSTEM_LOADED
 - SMW_STATUS_SUBSYSTEM_NOT_LOADED
 - SMW_STATUS_KEY_INVALID
 - SMW_STATUS_INVALID_LIFECYCLE
 - SMW_STATUS_INVALID_IV_SIZE
 - SMW_STATUS_UNKNOWN_MODE_NAME
 - SMW_STATUS_UNKNOWN_OP_TYPE_NAME
 - SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME
 - SMW_STATUS_UNKNOWN_ALGO_NAME
 - SMW_STATUS_UNKNOWN_SIGN_ALGO_NAME
 - SMW_STATUS_UNKNOWN_SIGN_TYPE_NAME

-
- SMW_STATUS_OPERATION_ALREADY_INIT
 - Specific return codes - Library initialization
 - SMW_STATUS_OPS_INVALID
 - SMW_STATUS_MUTEX_INIT_FAILURE
 - SMW_STATUS_MUTEX_DESTROY_FAILURE
 - SMW_STATUS_LIBRARY_ALREADY_INIT
 - SMW_STATUS_MUTEX_LOCK_FAILURE
 - SMW_STATUS_MUTEX_UNLOCK_FAILURE
 - SMW_STATUS_INVALID_LIBRARY_CONTEXT
 - SMW_STATUS_INVALID_CONFIG_DATABASE
 - SMW_STATUS_CONFIGURATION_FAILURE
 - Specific return codes - Configuration file
 - SMW_STATUS_INVALID_VERSION
 - SMW_STATUS_INVALID_BUFFER
 - SMW_STATUS_EOF
 - SMW_STATUS_SYNTAX_ERROR
 - SMW_STATUS_TOO_LARGE_NUMBER
 - SMW_STATUS_INVALID_TAG
 - SMW_STATUS_RANGE_DUPLICATE
 - SMW_STATUS_ALGO_NOT_CONFIGURED
 - SMW_STATUS_CONFIG_ALREADY_LOADED
 - SMW_STATUS_NO_CONFIG_LOADED
 - SMW_STATUS_LOAD_METHOD_DUPLICATE
 - SMW_STATUS_UNKNOWN_CONFIG_OP_NAME
 - SMW_STATUS_UNKNOWN_LOAD_METHOD_NAME
 - Specific return codes - Signature
 - SMW_STATUS_SIGNATURE_INVALID
 - SMW_STATUS_SIGNATURE_LEN_INVALID
 - Specific return codes - Object database
 - SMW_STATUS_ERROR_OBJ_DB_INIT
 - SMW_STATUS_ERROR_OBJ_DB_CREATE
 - SMW_STATUS_ERROR_OBJ_DB_UPDATE
 - SMW_STATUS_ERROR_OBJ_DB_DELETE
 - SMW_STATUS_ERROR_OBJ_DB_GET_INFO

-
- SMW_STATUS_ERROR_OBJ_DB_FIND
 - Specific return codes - Key manager
 - SMW_STATUS_KEY_POLICY_WARNING_IGNORED
 - SMW_STATUS_UNKNOWN_KEY_OP_NAME
 - SMW_STATUS_UNKNOWN_KEY_TYPE_NAME
 - SMW_STATUS_UNKNOWN_FORMAT_NAME
 - SMW_STATUS_UNKNOWN_KDF_NAME
 - SMW_STATUS_UNKNOWN_TLS12_KEY_NAME
 - SMW_STATUS_UNKNOWN_TLS12_ENC_NAME
 - SMW_STATUS_UNKNOWN_KEY_PRIVACY_NAME
 - SMW_STATUS_PUBLIC_EXPONENT_NOT_SUPPORTED
 - SMW_STATUS_KEY_ID_ALREADY_EXIST
 - SMW_STATUS_PERMITTED_ALGO_INVALID
 - Specific return codes - Device manager
 - SMW_STATUS_OEM_SRKH_NOT_FUSED

3.1.10 Attributes APIs

3.1.10.1 Introduction

The attributes API defines the bitmasks that represent attributes associated to a key or data. It also defines macros to write or read these attributes.

3.1.10.2 typedef smw_attr_algo_t

type **smw_attr_algo_t**

Algorithm parameters

3.1.10.2.1 Description

64-bits field used to define an algorithm and its parameters.

- Main algorithm
- Mode, if any
- Curve, if any
- Hash, if any
- Operation class
- Salt length, if any
- MAC length, if any
- Tag length, if any
- Signature parameters, if any

Bits					
[63:40]	[39:32]	[31:24]	[23:16]	[15:8]	[7:0]
	Additional parameters	Operation class	Hash	Mode / Curve	Main algorithm

3.1.10.2.2 Additional parameters

- MAC Truncated length

Bits[39:32]		Description
8	[7:0]	
M	Length	Length of the MAC truncated length

M is 1 if Length is minimum length, 0 otherwise.

- AEAD Tag length

Bits[39:32]		Description
8	[7:0]	
M	Length	AEAD Tag length

M is 1 if Length is minimum length, 0 otherwise.

- Signature (RSA-PSS) Salt length

Bits[39:32]		Description
8	[7:0]	
M	Length	RSA-PSS Salt length

M is 1 if Length is minimum length, 0 otherwise.

- Signature (ECDSA)

Bits[39:32]		Description
8	[7:0]	
1	–	ECDSA signature message is hashed

- Signature (EDDSA)

Bits[39:32]		Description
8	[7:0]	
H	0x01	EDDSA signature type is pre-hashed
H	0x02	EDDSA signature type is with context

H is 1 if message to sign or verify is already hashed, 0 otherwise.

3.1.10.3 typedef smw_attr_usage_t

type **smw_attr_usage_t**

Permitted usages

3.1.10.3.1 Description

32-bits field used to define the permitted usages.

- Cache (Ca)
- Copy (Co)
- Export (Ex)
- Encrypt (En)
- Decrypt (Dec)
- Sign message (Sm)
- Verify message (Vm)
- Sign hash (Sh)
- Verify hash (Vh)
- Derive (Der)

The permitted usages bitfield is described below.

Bits										
[31:10]	9	8	7	6	5	4	3	2	1	0
	Der	Vh	Sh	Vm	Sm	Dec	En	Ex	Co	Ca

3.1.10.4 typedef smw_attr_attributes_t

type **smw_attr_attributes_t**

Attributes

3.1.10.4.1 Description

32-bits field used to define the attributes.

- Persistence: Transient, persistent, permanent
- Lifecycle: Open, closed, closed locked
- R/W flags: Read only (Rl), read once (Rc)

The attributes bitfield is described below.

Bits					
[31:18]	17	16	[15:8]	[7:4]	[3:0]
	Rl	Rc	Lifecycle		Persistence

3.1.10.5 typedef smw_attr_storage_id_t

type **smw_attr_storage_id_t**

Storage identifier

3.1.10.5.1 Description

32-bits field used to define the storage identifier.

The storage identifier is related to the Secure Subsystem. It is not the purpose of this API to modify it.

3.1.10.6 SMW_ATTR_xxx_OFFSET (smw_attr_algo_t)

Parameters offsets in *typedef smw_attr_algo_t*

- SMW_ATTR_ALGO_OFFSET: Algorithm offset.
- SMW_ATTR_MODE_OFFSET: Mode offset.
- SMW_ATTR_CURVE_OFFSET: Curve offset.
- SMW_ATTR_KDF_OFFSET: Key agreement's key derivation function offset.
- SMW_ATTR_HASH_OFFSET: Hash offset.
- SMW_ATTR_CLASS_OFFSET: Class offset.
- SMW_ATTR_ADD_ATTR_OFFSET: Additional parameters offset.

3.1.10.7 SMW_ATTR_xxx_MASK (smw_attr_algo_t)

Parameters masks in *typedef smw_attr_algo_t*

- SMW_ATTR_ALGO_MASK: Algorithm mask.
- SMW_ATTR_MODE_MASK: Mode mask.
- SMW_ATTR_CURVE_MASK: Curve mask.
- SMW_ATTR_KDF_MASK: Key agreement's key derivation function mask.
- SMW_ATTR_HASH_MASK: Hash mask.

-
- SMW_ATTR_CLASS_MASK: Class mask.
 - SMW_ATTR_ADD_PARAM_MASK: Additional parameters mask.

3.1.10.8 SMW_ATTR_LENGTH_MIN_FLAG

Flag indicating the length bits represent a minimum length in `typedef smw_attr_algo_t`

3.1.10.9 SMW_ATTR_SALT_xxx

Salt length in `typedef smw_attr_algo_t`

- SMW_ATTR_SALT_OFFSET: Salt length offset.
- SMW_ATTR_SALT_MASK: Salt length mask.
- SMW_ATTR_SALT_MIN_FLAG: Salt length is a minimum salt length.

3.1.10.10 SMW_ATTR_MAC_xxx

MAC length in `typedef smw_attr_algo_t`

- SMW_ATTR_MAC_OFFSET: MAC length offset.
- SMW_ATTR_MAC_MASK: MAC length mask.
- SMW_ATTR_MAC_MIN_FLAG: MAC length is a minimum MAC length.

3.1.10.11 SMW_ATTR_TAG_xxx

Tag length in `typedef smw_attr_algo_t`

- SMW_ATTR_TAG_OFFSET: Tag length offset.
- SMW_ATTR_TAG_MASK: Tag length mask.
- SMW_ATTR_TAG_MIN_FLAG: Tag length is a minimum tag length.

3.1.10.12 SMW_ATTR_SIGN_PARAM_xxx

Asymmetric Signature parameters in `typedef smw_attr_algo_t`

- SMW_ATTR_SIGN_PARAM_OFFSET: Signature parameters offset.
- SMW_ATTR_SIGN_PARAM_MASK: Signature parameters mask.
- SMW_ATTR_SIGN_PARAM_EDDSA_NONE: No Signature EDDSA type.
- SMW_ATTR_SIGN_PARAM_EDDSA_PREHASHED: Signature EDDSA is pre-hashed type.
- SMW_ATTR_SIGN_PARAM_EDDSA_CONTEXT: Signature EDDSA is context type.

3.1.10.13 SMW_ATTR_SIGN_HASHED_FLAG

Flag indicating if the message to sign or verify given in signature operation is already hashed in `typedef smw_attr_algo_t`

3.1.10.14 SMW_ATTR_ALGO_xxx

Main algorithm identifier in `typedef smw_attr_algo_t`

- SMW_ATTR_ALGO_NONE: No algorithm defined.
- SMW_ATTR_ALGO_AES: Advanced Encryption Standard.
- SMW_ATTR_ALGO_DES: Data Encryption Standard.
- SMW_ATTR_ALGO_DES3: Triple DES.
- SMW_ATTR_ALGO_CHACHA20: ChaCha20-Poly1305 .
- SMW_ATTR_ALGO_SM4: ShāngMì 4.
- SMW_ATTR_ALGO_RSA: Rivest–Shamir–Adleman.
- SMW_ATTR_ALGO_SM2: ShāngMì 2.
- SMW_ATTR_ALGO_HMAC: Hash-based message authentication code.
- SMW_ATTR_ALGO_DSA: Digital Signature Algorithm.
- SMW_ATTR_ALGO_ECDSA: Elliptic Curve Digital Signature Algorithm.
- SMW_ATTR_ALGO_EDDSA: Edwards-curve Digital Signature Algorithm.
- SMW_ATTR_ALGO_DH: Diffie–Hellman.
- SMW_ATTR_ALGO_ECDH: Elliptic-curve Diffie–Hellman.
- SMW_ATTR_ALGO_HKDF: HMAC-based Key Derivation Function.
- SMW_ATTR_ALGO_HKDF_EXTRACT: HMAC-based Key Derivation Function Extract step.
- SMW_ATTR_ALGO_HKDF_EXPAND: HMAC-based Key Derivation Function Expand step.
- SMW_ATTR_ALGO_TLS_1_2: Transport Layer Security 1.2.
- SMW_ATTR_ALGO_TLS_1_3: Transport Layer Security 1.3.
- SMW_ATTR_ALGO_CKDF: Custom Key Derivation Function.
- SMW_ATTR_ALGO_HASH: Hash.

3.1.10.15 SMW_ATTR_MODE_xxx

Mode associated to the main algorithm in `typedef smw_attr_algo_t`

- SMW_ATTR_MODE_NONE: No mode defined.
- SMW_ATTR_MODE_ECB_NO_PAD: Electronic Code Book No Padding.
- SMW_ATTR_MODE_CBC_NO_PAD: Cipher block chaining No Padding.
- SMW_ATTR_MODE_CFB: Ciphertext Feedback.
- SMW_ATTR_MODE_CTR: Counter Mode.
- SMW_ATTR_MODE_CTS: Ciphertext Stealing.
- SMW_ATTR_MODE_OFB: Output Feedback.
- SMW_ATTR_MODE_XTS: XEX Tweakable Block Ciphertext Stealing.
- SMW_ATTR_MODE_CCM: Counter with CBC-MAC Mode.

- SMW_ATTR_MODE_GCM: Galois/Counter Mode.
- SMW_ATTR_MODE_PKCS1_1_5: Public-Key Cryptography Standards 1.5.
- SMW_ATTR_MODE_OAEP: Optimal Asymmetric Encryption Padding.
- SMW_ATTR_MODE_PSS: Probabilistic Signature Scheme.
- SMW_ATTR_MODE_PKCS5: Password-Based Cryptography Specification.
- SMW_ATTR_MODE_CMAC: Cipher-based Message Authentication Code.
- SMW_ATTR_MODE_POLY1305: Poly1305-AES.
- SMW_ATTR_MODE_CLIENT: Client (TLS 1.2).
- SMW_ATTR_MODE_SERVER: Server (TLS 1.2).
- SMW_ATTR_MODE_NO_PAD: Asymmetric Encryption with no padding.
- SMW_ATTR_MODE_ANY: Any mode.

3.1.10.16 SMW_ATTR_CURVE_xxx

Curve associated to the main algorithm in *typedef smw_attr_algo_t*

- SMW_ATTR_CURVE_NONE: No curve defined.
- SMW_ATTR_CURVE_SECP_R1: Secp R1 curve (aka NIST P).
- SMW_ATTR_CURVE_BRAINPOOL_R1: Brainpool R1 curve.
- SMW_ATTR_CURVE_BRAINPOOL_T1: Brainpool T1 curve.
- SMW_ATTR_CURVE_ED25519: Twisted Edwards25519.
- SMW_ATTR_CURVE_ED448: Twisted Edwards448.
- SMW_ATTR_CURVE_ANY: Any curve.

3.1.10.17 SMW_ATTR_HASH_xxx

Hash algorithm associated to the main algorithm in *typedef smw_attr_algo_t* or hash algorithm used in case of digest operation

- SMW_ATTR_HASH_NONE: No hash algorithm defined.
- SMW_ATTR_HASH_MD5: Message Digest 5.
- SMW_ATTR_HASH_SHA1: Secure Hash Algorithm 1.
- SMW_ATTR_HASH_SHA224: Secure Hash Algorithm 2, 224 bits.
- SMW_ATTR_HASH_SHA256: Secure Hash Algorithm 2, 256 bits.
- SMW_ATTR_HASH_SHA384: Secure Hash Algorithm 2, 384 bits.
- SMW_ATTR_HASH_SHA512: Secure Hash Algorithm 2, 512 bits.
- SMW_ATTR_HASH_SHA3_224: Secure Hash Algorithm 3, 224 bits.
- SMW_ATTR_HASH_SHA3_256: Secure Hash Algorithm 3, 256 bits.
- SMW_ATTR_HASH_SHA3_384: Secure Hash Algorithm 3, 384 bits.
- SMW_ATTR_HASH_SHA3_512: Secure Hash Algorithm 3, 512 bits.

- SMW_ATTR_HASH_SM3: ShangMi 3.
- SMW_ATTR_HASH_SHAKE128: Shake128.
- SMW_ATTR_HASH_SHAKE256: Shake1256.
- SMW_ATTR_HASH_ANY: Any hash algorithm.

3.1.10.18 SMW_ATTR_CLASS_xxx

Class of operation of the main algorithm in `typedef smw_attr_algo_t` or hash algorithm used in case of digest operation

- SMW_ATTR_CLASS_NONE: No class of operation defined.
- SMW_ATTR_CLASS_DIGEST: Digest calculation.
- SMW_ATTR_CLASS_SYMMETRIC_ENCRYPTION: Symmetric encryption.
- SMW_ATTR_CLASS_ASYMMETRIC_ENCRYPTION: Asymmetric encryption.
- SMW_ATTR_CLASS_ASYMMETRIC_SIGNATURE: Asymmetric signature.
- SMW_ATTR_CLASS_MAC: Message Authentication Code.
- SMW_ATTR_CLASS_AEAD: Authenticated Encryption with Associated Data.
- SMW_ATTR_CLASS_KEY_DERIVATION: Key derivation.
- SMW_ATTR_CLASS_KEY_ATTESTATION: Key attestation.
- SMW_ATTR_CLASS_KEY_AGREEMENT: Key agreement.

3.1.10.19 SMW_ATTR_USAGE_xxx

Key usage in `typedef smw_attr_usage_t`

- SMW_ATTR_USAGE_NONE: No key usage defined.
- SMW_ATTR_USAGE_CACHE: Permission to cache the key.
- SMW_ATTR_USAGE_COPY: Permission to copy the key.
- SMW_ATTR_USAGE_EXPORT: Permission to export the key.
- SMW_ATTR_USAGE_ENCRYPT: Permission to encrypt a message with the key.
- SMW_ATTR_USAGE_DECRYPT: Permission to decrypt a message with the key.
- SMW_ATTR_USAGE_SIGN_MESSAGE: Permission to sign a message with the key.
- SMW_ATTR_USAGE_VERIFY_MESSAGE: Permission to verify a message signature with the key.
- SMW_ATTR_USAGE_SIGN_HASH: Permission to sign a message hash with the key.
- SMW_ATTR_USAGE_VERIFY_HASH: Permission to verify a message hash with the key.
- SMW_ATTR_USAGE_DERIVE: Permission to derive other keys from this key.

3.1.10.20 SMW_ATTR_XXX_OFFSET (smw_attr_attributes_t)

Attributes offsets in `typedef smw_attr_attributes_t`

- SMW_ATTR_PERSISTENCE_OFFSET: Persistence offset.
- SMW_ATTR_LIFECYCLE_OFFSET: Lifecycle offset.
- SMW_ATTR_RW_FLAGS_OFFSET: R/W flags offset.

3.1.10.21 SMW_ATTR_XXX_MASK (smw_attr_attributes_t)

Attributes masks in `typedef smw_attr_attributes_t`

- SMW_ATTR_PERSISTENCE_MASK: Persistence mask.
- SMW_ATTR_LIFECYCLE_MASK: Lifecycle mask.
- SMW_ATTR_RW_FLAGS_MASK: R/W flags mask.

3.1.10.22 SMW_ATTR_PERSISTENCE_XXX

Persistence in `typedef smw_attr_attributes_t`

- SMW_ATTR_PERSISTENCE_TRANSIENT: Transient.
- SMW_ATTR_PERSISTENCE_PERSISTENT: Persistent.
- SMW_ATTR_PERSISTENCE_PERMANENT: Permanent.

3.1.10.23 SMW_ATTR_LIFECYCLE_XXX

Lifecycle in `typedef smw_attr_attributes_t`

- SMW_ATTR_LIFECYCLE_CURRENT: Current lifecycle.
- SMW_ATTR_LIFECYCLE_OPEN: Open.
- SMW_ATTR_LIFECYCLE_CLOSED: Closed.
- SMW_ATTR_LIFECYCLE_CLOSED_LOCKED: Close locked.

3.1.10.24 SMW_ATTR_RW_FLAGS_XXX

R/W flags in `typedef smw_attr_attributes_t`

- SMW_ATTR_RW_FLAGS_NONE: No R/W flag defined.
- SMW_ATTR_RW_FLAGS_READ_ONLY: Read only.
- SMW_ATTR_RW_FLAGS_READ_ONCE: Read once.

3.1.10.25 macro SMW_ATTR_SET_LENGTH

SMW_ATTR_SET_LENGTH(algo, length)

Set length.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.
- **length** – Length associated to algo.

3.1.10.25.1 Description

This macro sets the length.

3.1.10.25.2 Return

A valid algorithm with length bits set to `length`.

3.1.10.26 macro `SMW_ATTR_SET_MIN_LENGTH`

`SMW_ATTR_SET_MIN_LENGTH(algo, length)`

Set min length.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.
- **length** – Length associated to algo.

3.1.10.26.1 Description

This macro sets the min length.

3.1.10.26.2 Return

A valid algorithm with min length bits set to `length`.

3.1.10.27 macro `SMW_ATTR_IS_MIN_LENGTH`

`SMW_ATTR_IS_MIN_LENGTH(algo)`

Whether the min length bit is set.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.

3.1.10.27.1 Description

This macro returns whether the min length bit is set or not.

3.1.10.27.2 Return

1 if the min length bit is set, 0 otherwise.

3.1.10.28 macro `SMW_ATTR_GET_LENGTH`

`SMW_ATTR_GET_LENGTH(algo)`

Get length.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.

3.1.10.28.1 Description

This macro returns the length value set for `algo`.

3.1.10.28.2 Return

Length set for `algo`.

3.1.10.29 macro `SMW_ATTR_SET_SALT_LENGTH`

`SMW_ATTR_SET_SALT_LENGTH(algo, length)`

Set salt length.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.
- **length** – Salt length associated to `algo`.

3.1.10.29.1 Description

This macro sets the salt length.

3.1.10.29.2 Return

A valid algorithm with salt length bits set to `length`.

3.1.10.30 macro `SMW_ATTR_SET_MAC_LENGTH`

`SMW_ATTR_SET_MAC_LENGTH(algo, length)`

Set MAC length.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.
- **length** – MAC length associated to `algo`.

3.1.10.30.1 Description

This macro sets the MAC length.

3.1.10.30.2 Return

A valid algorithm with MAC length bits set to `length`.

3.1.10.31 macro `SMW_ATTR_SET_TAG_LENGTH`

`SMW_ATTR_SET_TAG_LENGTH(algo, length)`

Set tag length.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.
- **length** – Tag length associated to `algo`.

3.1.10.31.1 Description

This macro sets the tag length.

3.1.10.31.2 Return

A valid algorithm with tag length bits set to `length`.

3.1.10.32 macro `SMW_ATTR_SET_MIN_SALT_LENGTH`

`SMW_ATTR_SET_MIN_SALT_LENGTH(algo, length)`

Set min salt length.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.
- **length** – Salt length associated to algo.

3.1.10.32.1 Description

This macro sets the min salt length.

3.1.10.32.2 Return

A valid algorithm with min salt length bits set to `length`.

3.1.10.33 macro `SMW_ATTR_SET_MIN_MAC_LENGTH`

`SMW_ATTR_SET_MIN_MAC_LENGTH(algo, length)`

Set min MAC length.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.
- **length** – MAC length associated to algo.

3.1.10.33.1 Description

This macro sets the min MAC length.

3.1.10.33.2 Return

A valid algorithm with min MAC length bits set to `length`.

3.1.10.34 macro `SMW_ATTR_SET_MIN_TAG_LENGTH`

`SMW_ATTR_SET_MIN_TAG_LENGTH(algo, length)`

Set min tag length.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.
- **length** – Tag length associated to algo.

3.1.10.34.1 Description

This macro sets the min tag length.

3.1.10.34.2 Return

A valid algorithm with min tag length bits set to length.

3.1.10.35 macro `SMW_ATTR_IS_MIN_SALT_LENGTH`

`SMW_ATTR_IS_MIN_SALT_LENGTH(algo)`

Whether the min salt length bit is set.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.

3.1.10.35.1 Description

This macro returns whether the min salt length bit is set or not.

3.1.10.35.2 Return

1 if the min salt length bit is set, 0 otherwise.

3.1.10.36 macro `SMW_ATTR_IS_MIN_MAC_LENGTH`

`SMW_ATTR_IS_MIN_MAC_LENGTH(algo)`

Whether the min MAC length bit is set.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.

3.1.10.36.1 Description

This macro returns whether the min MAC length bit is set or not.

3.1.10.36.2 Return

1 if the min MAC length bit is set, 0 otherwise.

3.1.10.37 macro `SMW_ATTR_IS_MIN_TAG_LENGTH`

`SMW_ATTR_IS_MIN_TAG_LENGTH(algo)`

Whether the min tag length bit is set.

Parameters

- **algo** – A valid algorithm. See `struct smw_attr_algo_t`.

3.1.10.37.1 Description

This macro returns whether the min tag length bit is set or not.

3.1.10.37.2 Return

1 if the min tag length bit is set, 0 otherwise.

3.1.10.38 macro SMW_ATTR_GET_SALT_LENGTH

SMW_ATTR_GET_SALT_LENGTH(algo)

Get salt length.

Parameters

- **algo** – A valid algorithm. See *struct smw_attr_algo_t*.

3.1.10.38.1 Description

This macro returns the salt length value set for algo.

3.1.10.38.2 Return

Length set for algo.

3.1.10.39 macro SMW_ATTR_GET_MAC_LENGTH

SMW_ATTR_GET_MAC_LENGTH(algo)

Get MAC length.

Parameters

- **algo** – A valid algorithm. See *struct smw_attr_algo_t*.

3.1.10.39.1 Description

This macro returns the MAC length value set for algo.

3.1.10.39.2 Return

Length set for algo.

3.1.10.40 macro SMW_ATTR_GET_TAG_LENGTH

SMW_ATTR_GET_TAG_LENGTH(algo)

Get tag length.

Parameters

- **algo** – A valid algorithm. See *struct smw_attr_algo_t*.

3.1.10.40.1 Description

This macro returns the tag length value set for `algo`.

3.1.10.40.2 Return

Length set for `algo`.

3.1.10.41 macro `SMW_ATTR_SET_MSG_HASHED`

`SMW_ATTR_SET_MSG_HASHED(algo)`

Set Asymmetric signature message hashed flag.

Parameters

- `algo` – A valid algorithm. See `struct smw_attr_algo_t`.

3.1.10.41.1 Description

This macro sets the signature flag indicating input message is hashed.

3.1.10.41.2 Return

A valid algorithm with signature message flag set.

3.1.10.42 macro `SMW_ATTR_IS_MSG_HASHED`

`SMW_ATTR_IS_MSG_HASHED(algo)`

Whether signature message hashed flag is set.

Parameters

- `algo` – A valid algorithm. See `struct smw_attr_algo_t`.

3.1.10.42.1 Description

This macro returns whether the signature input message hashed flag is set or not.

3.1.10.42.2 Return

1 if the message hashed flag is set, 0 otherwise.

3.1.10.43 macro `SMW_ATTR_GET_SIGN_PARAM`

`SMW_ATTR_GET_SIGN_PARAM(algo)`

Get Asymmetric signature parameter.

Parameters

- `algo` – A valid algorithm. See `struct smw_attr_algo_t`.

3.1.10.43.1 Description

This macro returns the signature parameter set for algo.

3.1.10.43.2 Return

Signature parameter set for algo.

3.1.10.44 macro **SMW_ATTR_SET_SIGN_PARAM**

SMW_ATTR_SET_SIGN_PARAM(algo, param)

Set Asymmetric signature parameter.

Parameters

- **algo** – A valid algorithm. See *struct smw_attr_algo_t*.
- **param** – Parameter associated to algo.

3.1.10.44.1 Description

This macro sets the signature parameter.

3.1.10.44.2 Return

A valid algorithm with signature parameter set to param.

3.1.10.45 macro **SMW_ATTR_SET_SIGN_EDDSA_PREHASHED**

SMW_ATTR_SET_SIGN_EDDSA_PREHASHED(algo)

Set EDDSA pre-hashed signature.

Parameters

- **algo** – A valid algorithm. See *struct smw_attr_algo_t*.

3.1.10.45.1 Description

This macro sets the asymmetric EDDSA signature parameter pre-hashed.

3.1.10.45.2 Return

A valid algorithm with EDDSA signature parameter pre-hashed set.

3.1.10.46 macro **SMW_ATTR_IS_SIGN_EDDSA_PREHASHED**

SMW_ATTR_IS_SIGN_EDDSA_PREHASHED(algo)

Whether the signature EDDSA is pre-hashed.

Parameters

- **algo** – A valid algorithm. See *struct smw_attr_algo_t*.

3.1.10.46.1 Description

This macro returns whether the asymmetric EDDSA signature parameter pre-hashed is set or not.

3.1.10.46.2 Return

1 if the signature message is pre-hashed type, 0 otherwise.

3.1.10.47 macro **SMW_ATTR_SET_SIGN_EDDSA_CONTEXT**

SMW_ATTR_SET_SIGN_EDDSA_CONTEXT(algo)

Set EDDSA context signature.

Parameters

- **algo** – A valid algorithm. See *struct smw_attr_algo_t*.

3.1.10.47.1 Description

This macro sets the asymmetric EDDSA signature parameter context.

3.1.10.47.2 Return

A valid algorithm with EDDSA signature parameter context set.

3.1.10.48 macro **SMW_ATTR_IS_SIGN_EDDSA_CONTEXT**

SMW_ATTR_IS_SIGN_EDDSA_CONTEXT(algo)

Whether the EDDSE signature uses a context.

Parameters

- **algo** – A valid algorithm. See *struct smw_attr_algo_t*.

3.1.10.48.1 Description

This macro returns whether the asymmetric EDDSA signature parameter context is set or not.

3.1.10.48.2 Return

1 if the signature message uses a context, 0 otherwise.

3.1.10.49 macro **SMW_ATTR_ALGO_DIGEST**

SMW_ATTR_ALGO_DIGEST(hash)

Build a digest algorithm.

Parameters

- **hash** – A valid hash algorithm. See *smw_attr_algo_t*.

3.1.10.49.1 Description

This macro builds a digest algorithm using the hash algorithm.

3.1.10.49.2 Return

A valid digest algorithm.

3.1.10.50 macro **SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION**

SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION(algo, mode)

Build a symmetric encryption algorithm.

Parameters

- **algo** – A valid main algorithm for symmetric encryption. See `smw_attr_algo_t`.
- **mode** – A valid mode. See `smw_attr_algo_t`.

3.1.10.50.1 Description

This macro builds a symmetric encryption algorithm given the main algorithm `algo` and the mode.

3.1.10.50.2 Return

A valid symmetric encryption algorithm.

3.1.10.51 macro **SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_ECDSA**

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_ECDSA(curve, hash)

Build an asymmetric signature ECDSA algorithm.

Parameters

- **curve** – A valid curve. See `smw_attr_algo_t`.
- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.51.1 Description

This macro builds an asymmetric signature ECDSA algorithm given the `curve` and the hash algorithm.

3.1.10.51.2 Return

A valid asymmetric signature ECDSA algorithm.

3.1.10.52 macro **SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_EDDSA**

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_EDDSA(curve, hash, param)

Build an asymmetric signature EDDSA algorithm.

Parameters

- **curve** – A valid curve. See `smw_attr_algo_t`.
- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

-
- **param** – A valid EDDSA parameter algorithm. See `smw_attr_algo_t`

3.1.10.52.1 Description

This macro builds an asymmetric signature EDDSA algorithm given the `curve`, the hash algorithm and the `param` parameter.

The hash algorithm function of the signature scheme and may not be used.

3.1.10.52.2 Return

A valid asymmetric signature EDDSA algorithm.

3.1.10.53 macro `SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_DSA`

`SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_DSA`(hash)

Build an asymmetric signature DSA algorithm.

Parameters

- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.53.1 Description

This macro builds an asymmetric signature DSA algorithm given the `hash` algorithm.

3.1.10.53.2 Return

A valid asymmetric signature DSA algorithm.

3.1.10.54 macro `SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_RSA`

`SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_RSA`(mode, hash, salt)

Build an asymmetric signature RSA algorithm.

Parameters

- **mode** – A valid mode. See `smw_attr_algo_t`.
- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.
- **salt** – Salt length.

3.1.10.54.1 Description

This macro builds an asymmetric signature RSA algorithm given the `mode`, the hash algorithm and the `salt` length.

3.1.10.54.2 Return

A valid asymmetric signature RSA algorithm.

3.1.10.55 macro **SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_NO_LABEL**

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_NO_LABEL(hash)

Build an asymmetric signature TLS1.2 algorithm without label.

Parameters

- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.55.1 Description

This macro builds an asymmetric signature TLS1.2 algorithm given the **hash** algorithm.

3.1.10.55.2 Return

A valid asymmetric signature TLS1.2 algorithm.

3.1.10.56 macro **SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_CLIENT**

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_CLIENT(hash)

Build an asymmetric signature TLS1.2 algorithm with label *CLIENT*.

Parameters

- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.56.1 Description

This macro builds an asymmetric signature TLS1.2 algorithm with *CLIENT* label given the hash algorithm.

3.1.10.56.2 Return

A valid asymmetric signature TLS1.2 algorithm with label *CLIENT*.

3.1.10.57 macro **SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_SERVER**

SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_SERVER(hash)

Build an asymmetric signature TLS1.2 algorithm with label *SERVER*.

Parameters

- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.57.1 Description

This macro builds an asymmetric signature TLS1.2 algorithm with *SERVER* label given the hash algorithm.

3.1.10.57.2 Return

A valid asymmetric signature TLS1.2 algorithm with label *SERVER*.

3.1.10.58 macro **SMW_ATTR_ALGO_MAC**

SMW_ATTR_ALGO_MAC(algo, mode, mac)

Build a MAC algorithm.

Parameters

- **algo** – A valid main algorithm. See `smw_attr_algo_t`.
- **mode** – A valid mode. See `smw_attr_algo_t`.
- **mac** – MAC length.

3.1.10.58.1 Description

This macro builds a MAC algorithm given the main algorithm `algo`, the mode and the hash algorithm. The main algorithm `algo` is not HMAC.

3.1.10.58.2 Return

A valid MAC algorithm.

3.1.10.59 macro **SMW_ATTR_ALGO_MAC_HMAC**

SMW_ATTR_ALGO_MAC_HMAC(hash, mac)

Build a HMAC algorithm.

Parameters

- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.
- **mac** – MAC length.

3.1.10.59.1 Description

This macro builds a HMAC algorithm given the hash algorithm and the mac length.

3.1.10.59.2 Return

A valid HMAC algorithm.

3.1.10.60 macro **SMW_ATTR_ALGO_AEAD**

SMW_ATTR_ALGO_AEAD(algo, mode, tag)

Build an AEAD algorithm.

Parameters

- **algo** – A valid main algorithm. See `smw_attr_algo_t`.
- **mode** – A valid mode. See `smw_attr_algo_t`.
- **tag** – Tag length.

3.1.10.60.1 Description

This macro builds an AEAD algorithm given the main algorithm `algo`, the mode and the tag length.

3.1.10.60.2 Return

A valid AEAD algorithm.

3.1.10.61 macro `SMW_ATTR_ALGO_KEY_DERIVATION_HKDF`

`SMW_ATTR_ALGO_KEY_DERIVATION_HKDF(hash)`

Build an HKDF key derivation algorithm.

Parameters

- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.61.1 Description

This macro builds an HKDF key derivation algorithm given the `hash` algorithm.

3.1.10.61.2 Return

A valid HKDF key derivation algorithm.

3.1.10.62 macro `SMW_ATTR_ALGO_KEY_DERIVATION_DH`

`SMW_ATTR_ALGO_KEY_DERIVATION_DH(void)`

Build the DH key derivation algorithm.

Parameters

- **void** – no arguments

3.1.10.62.1 Description

This macro builds the DH key derivation algorithm.

3.1.10.62.2 Return

DH key derivation algorithm.

3.1.10.63 macro `SMW_ATTR_ALGO_KEY_DERIVATION_ECDH`

`SMW_ATTR_ALGO_KEY_DERIVATION_ECDH(void)`

Build the ECDH key derivation algorithm.

Parameters

- **void** – no arguments

3.1.10.63.1 Description

This macro builds the ECDH key derivation algorithm.

3.1.10.63.2 Return

ECDH key derivation algorithm.

3.1.10.64 macro **SMW_ATTR_ALGO_KEY_DERIVATION_EDDSA**

SMW_ATTR_ALGO_KEY_DERIVATION_EDDSA(curve)

Build an EDDSA key derivation algorithm.

Parameters

- **curve** – A valid curve. See `smw_attr_algo_t`.

3.1.10.64.1 Description

This macro builds an EDDSA key derivation algorithm given the `curve`.

3.1.10.64.2 Return

A valid EDDSA key derivation algorithm.

3.1.10.65 macro **SMW_ATTR_ALGO_KEY_DERIVATION_TLS12**

SMW_ATTR_ALGO_KEY_DERIVATION_TLS12(hash)

Build the TLS 1.2 derivation algorithm.

Parameters

- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.65.1 Description

This macro builds the TLS 1.2 key derivation algorithm.

3.1.10.65.2 Return

TLS 1.2 key derivation algorithm.

3.1.10.66 macro **SMW_ATTR_ALGO_KEY_DERIVATION_TLS13**

SMW_ATTR_ALGO_KEY_DERIVATION_TLS13(hash)

Build the TLS 1.3 derivation algorithm.

Parameters

- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.66.1 Description

This macro builds the TLS 1.3 key derivation algorithm.

3.1.10.66.2 Return

TLS 1.3 key derivation algorithm.

3.1.10.67 macro **SMW_ATTR_ALGO_KEY_ATTESTATION_MAC**

SMW_ATTR_ALGO_KEY_ATTESTATION_MAC(algo, mode, mac)

Build a MAC key attestation algorithm.

Parameters

- **algo** – A valid main algorithm. See `smw_attr_algo_t`.
- **mode** – A valid mode. See `smw_attr_algo_t`.
- **mac** – MAC length.

3.1.10.67.1 Description

This macro builds a MAC key attestation algorithm given the main algorithm `algo`, the mode and the mac length.

3.1.10.67.2 Return

A valid MAC key attestation algorithm.

3.1.10.68 macro **SMW_ATTR_ALGO_KEY_ATTESTATION_ECDSA**

SMW_ATTR_ALGO_KEY_ATTESTATION_ECDSA(curve, hash)

Build an ECDSA key attestation algorithm.

Parameters

- **curve** – A valid curve. See `smw_attr_algo_t`.
- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.68.1 Description

This macro builds an ECDSA key attestation algorithm given the curve and the hash algorithm.

3.1.10.68.2 Return

A valid ECDSA key attestation algorithm.

3.1.10.69 macro **SMW_ATTR_ALGO_KEY_AGREEMENT**

SMW_ATTR_ALGO_KEY_AGREEMENT(algo, kdf, hash)

Build a key agreement algorithm.

Parameters

- **algo** – Key agreement algorithm. See `smw_attr_algo_t`.

-
- **kdf** – Combined key derivation algorithm. See `smw_attr_algo_t`.
 - **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.69.1 Description

This macro builds a key agreement with or without combined key derivation algorithm.

3.1.10.69.2 Return

A valid key agreement algorithm with or without combined key derivation.

3.1.10.70 macro **SMW_ATTR_ALGO_ASYMMETRIC_ENCRYPTION_RSA**

SMW_ATTR_ALGO_ASYMMETRIC_ENCRYPTION_RSA(mode, hash)

Build an RSA asymmetric encryption algorithm.

Parameters

- **mode** – A valid mode. See `smw_attr_algo_t`.
- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.70.1 Description

This macro builds an RSA asymmetric encryption algorithm given the mode and the hash algorithm.

3.1.10.70.2 Return

A valid RSA asymmetric encryption algorithm.

3.1.10.71 macro **SMW_ATTR_GET_ALGO**

SMW_ATTR_GET_ALGO(algo)

Get the main algorithm.

Parameters

- **algo** – A valid algorithm. See `smw_attr_algo_t`.

3.1.10.71.1 Description

This macro extracts the main algorithm of algo.

3.1.10.71.2 Return

The main algorithm.

3.1.10.72 macro **SMW_ATTR_GET_MODE**

SMW_ATTR_GET_MODE(algo)

Get the mode.

Parameters

- **algo** – A valid algorithm. See `smw_attr_algo_t`.

3.1.10.72.1 Description

This macro extracts the mode of `algo`.

3.1.10.72.2 Return

The mode.

3.1.10.73 macro `SMW_ATTR_GET_CURVE`

`SMW_ATTR_GET_CURVE(algo)`

Get the curve.

Parameters

- **algo** – A valid algorithm. See `smw_attr_algo_t`.

3.1.10.73.1 Description

This macro extracts the curve of `algo`.

3.1.10.73.2 Return

The curve.

3.1.10.74 macro `SMW_ATTR_GET_KDF`

`SMW_ATTR_GET_KDF(algo)`

Get the key agreement's key derivation algorithm.

Parameters

- **algo** – A valid algorithm. See `smw_attr_algo_t`.

3.1.10.74.1 Description

This macro extracts the key derivation of key agreement `algo`.

3.1.10.74.2 Return

The curve.

3.1.10.75 macro `SMW_ATTR_GET_HASH`

`SMW_ATTR_GET_HASH(algo)`

Get the hash algorithm.

Parameters

- **algo** – A valid algorithm. See `smw_attr_algo_t`.

3.1.10.75.1 Description

This macro extracts the hash algorithm of `algo`.

3.1.10.75.2 Return

The hash algorithm.

3.1.10.76 macro `SMW_ATTR_GET_CLASS`

`SMW_ATTR_GET_CLASS`(`algo`)

Get the class.

Parameters

- **algo** – A valid algorithm. See `smw_attr_algo_t`.

3.1.10.76.1 Description

This macro extracts the class of `algo`.

3.1.10.76.2 Return

The class.

3.1.10.77 macro `SMW_ATTR_SET_HASH`

`SMW_ATTR_SET_HASH`(`algo`, `hash`)

Set the hash algorithm.

Parameters

- **algo** – A valid algorithm. See `smw_attr_algo_t`.
- **hash** – A valid hash algorithm. See `smw_attr_algo_t`.

3.1.10.77.1 Description

This macro sets the hash algorithm of `algo`.

3.1.10.77.2 Return

A valid algorithm.

3.1.10.78 macro `SMW_ATTR_SET_MODE`

`SMW_ATTR_SET_MODE`(`algo`, `mode`)

Set the algorithm mode.

Parameters

- **algo** – A valid algorithm. See `smw_attr_algo_t`.
- **mode** – A valid algorithm mode. See `smw_attr_algo_t`.

3.1.10.78.1 Description

This macro sets the algorithm mode of algo.

3.1.10.78.2 Return

A valid algorithm.

3.1.10.79 macro SMW_ATTR_USAGE_SET_CACHE

SMW_ATTR_USAGE_SET_CACHE(usage)

Set cache operation.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.79.1 Description

This macro sets cache operation as permitted usage.

3.1.10.79.2 Return

A valid usage.

3.1.10.80 macro SMW_ATTR_USAGE_SET_COPY

SMW_ATTR_USAGE_SET_COPY(usage)

Set copy operation.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.80.1 Description

This macro sets copy operation as permitted usage.

3.1.10.80.2 Return

A valid usage.

3.1.10.81 macro SMW_ATTR_USAGE_SET_EXPORT

SMW_ATTR_USAGE_SET_EXPORT(usage)

Set export operation.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.81.1 Description

This macro sets export operation as permitted usage.

3.1.10.81.2 Return

A valid usage.

3.1.10.82 macro **SMW_ATTR_USAGE_SET_ENCRYPT**

SMW_ATTR_USAGE_SET_ENCRYPT(usage)

Set encrypt operation.

Parameters

- **usage** – A usage. See `smw_attr_usage_t`.

3.1.10.82.1 Description

This macro sets encrypt operation as permitted usage.

3.1.10.82.2 Return

A valid usage.

3.1.10.83 macro **SMW_ATTR_USAGE_SET_DECRYPT**

SMW_ATTR_USAGE_SET_DECRYPT(usage)

Set decrypt operation.

Parameters

- **usage** – A usage. See `smw_attr_usage_t`.

3.1.10.83.1 Description

This macro sets decrypt operation as permitted usage.

3.1.10.83.2 Return

A valid usage.

3.1.10.84 macro **SMW_ATTR_USAGE_SET_SIGN_MESSAGE**

SMW_ATTR_USAGE_SET_SIGN_MESSAGE(usage)

Set sign message operation.

Parameters

- **usage** – A usage. See `smw_attr_usage_t`.

3.1.10.84.1 Description

This macro sets sign message operation as permitted usage.

3.1.10.84.2 Return

A valid usage.

3.1.10.85 macro **SMW_ATTR_USAGE_SET_VERIFY_MESSAGE**

SMW_ATTR_USAGE_SET_VERIFY_MESSAGE(usage)

Set verify message operation.

Parameters

- **usage** – A usage. See `smw_attr_usage_t`.

3.1.10.85.1 Description

This macro sets verify message operation as permitted usage.

3.1.10.85.2 Return

A valid usage.

3.1.10.86 macro **SMW_ATTR_USAGE_SET_SIGN_HASH**

SMW_ATTR_USAGE_SET_SIGN_HASH(usage)

Set sign hash operation.

Parameters

- **usage** – A usage. See `smw_attr_usage_t`.

3.1.10.86.1 Description

This macro sets sign hash operation as permitted usage.

3.1.10.86.2 Return

A valid usage.

3.1.10.87 macro **SMW_ATTR_USAGE_SET_VERIFY_HASH**

SMW_ATTR_USAGE_SET_VERIFY_HASH(usage)

Set verify hash operation.

Parameters

- **usage** – A usage. See `smw_attr_usage_t`.

3.1.10.87.1 Description

This macro sets verify hash operation as permitted usage.

3.1.10.87.2 Return

A valid usage.

3.1.10.88 macro SMW_ATTR_USAGE_SET_DERIVE

SMW_ATTR_USAGE_SET_DERIVE(usage)

Set derive operation.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.88.1 Description

This macro sets derive operation as permitted usage.

3.1.10.88.2 Return

A valid usage.

3.1.10.89 macro SMW_ATTR_USAGE_IS_CACHE

SMW_ATTR_USAGE_IS_CACHE(usage)

Whether the cache operation is permitted.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.89.1 Description

This macro returns whether the cache operation is permitted or not.

3.1.10.89.2 Return

1 if the cache operation is permitted, 0 otherwise.

3.1.10.90 macro SMW_ATTR_USAGE_IS_COPY

SMW_ATTR_USAGE_IS_COPY(usage)

Whether the copy operation is permitted.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.90.1 Description

This macro returns whether the copy operation is permitted or not.

3.1.10.90.2 Return

1 if the copy operation is permitted, 0 otherwise.

3.1.10.91 macro SMW_ATTR_USAGE_IS_EXPORT

SMW_ATTR_USAGE_IS_EXPORT(usage)

Whether the export operation is permitted.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.91.1 Description

This macro returns whether the export operation is permitted or not.

3.1.10.91.2 Return

1 if the export operation is permitted, 0 otherwise.

3.1.10.92 macro SMW_ATTR_USAGE_IS_ENCRYPT

SMW_ATTR_USAGE_IS_ENCRYPT(usage)

Whether the encrypt operation is permitted.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.92.1 Description

This macro returns whether the encrypt operation is permitted or not.

3.1.10.92.2 Return

1 if the encrypt operation is permitted, 0 otherwise.

3.1.10.93 macro SMW_ATTR_USAGE_IS_DECRYPT

SMW_ATTR_USAGE_IS_DECRYPT(usage)

Whether the decrypt operation is permitted.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.93.1 Description

This macro returns whether the decrypt operation is permitted or not.

3.1.10.93.2 Return

1 if the decrypt operation is permitted, 0 otherwise.

3.1.10.94 macro SMW_ATTR_USAGE_IS_SIGN_MESSAGE

SMW_ATTR_USAGE_IS_SIGN_MESSAGE(usage)

Whether the sign message operation is permitted.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.94.1 Description

This macro returns whether the sign message operation is permitted or not.

3.1.10.94.2 Return

1 if the sign message operation is permitted, 0 otherwise.

3.1.10.95 macro SMW_ATTR_USAGE_IS_VERIFY_MESSAGE

SMW_ATTR_USAGE_IS_VERIFY_MESSAGE(usage)

Whether the verify message operation is permitted.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.95.1 Description

This macro returns whether the verify message operation is permitted or not.

3.1.10.95.2 Return

1 if the verify message operation is permitted, 0 otherwise.

3.1.10.96 macro SMW_ATTR_USAGE_IS_SIGN_HASH

SMW_ATTR_USAGE_IS_SIGN_HASH(usage)

Whether the sign hash operation is permitted.

Parameters

- **usage** – A usage. See smw_attr_usage_t.

3.1.10.96.1 Description

This macro returns whether the sign hash operation is permitted or not.

3.1.10.96.2 Return

1 if the sign hash operation is permitted, 0 otherwise.

3.1.10.97 macro **SMW_ATTR_USAGE_IS_VERIFY_HASH**

SMW_ATTR_USAGE_IS_VERIFY_HASH(usage)

Whether the verify hash operation is permitted.

Parameters

- **usage** – A usage. See `smw_attr_usage_t`.

3.1.10.97.1 Description

This macro returns whether the verify hash operation is permitted or not.

3.1.10.97.2 Return

1 if the verify hash operation is permitted, 0 otherwise.

3.1.10.98 macro **SMW_ATTR_USAGE_IS_DERIVE**

SMW_ATTR_USAGE_IS_DERIVE(usage)

Whether the derive operation is permitted.

Parameters

- **usage** – A usage. See `smw_attr_usage_t`.

3.1.10.98.1 Description

This macro returns whether the derive operation is permitted or not.

3.1.10.98.2 Return

1 if the derive operation is permitted, 0 otherwise.

3.1.10.99 macro **SMW_ATTR_SET_PERSISTENCE**

SMW_ATTR_SET_PERSISTENCE(attr, persistence)

Set persistence.

Parameters

- **attr** – Attributes. See `smw_attr_attributes_t`.
- **persistence** – Persistence.

3.1.10.99.1 Description

This macro sets persistence.

3.1.10.99.2 Return

Attributes

3.1.10.100 macro SMW_ATTR_GET_PERSISTENCE

SMW_ATTR_GET_PERSISTENCE(attr)

Get persistence.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.100.1 Description

This macro gets persistence.

3.1.10.100.2 Return

Persistence

3.1.10.101 macro SMW_ATTR_SET_TRANSIENT

SMW_ATTR_SET_TRANSIENT(attr)

Set transient persistence.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.101.1 Description

This macro sets transient persistence.

3.1.10.101.2 Return

Attributes

3.1.10.102 macro SMW_ATTR_SET_PERSISTENT

SMW_ATTR_SET_PERSISTENT(attr)

Set persistent persistence.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.102.1 Description

This macro sets persistent persistence.

3.1.10.102.2 Return

Attributes

3.1.10.103 macro SMW_ATTR_SET_PERMANENT

SMW_ATTR_SET_PERMANENT(attr)

Set permanent persistence.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.103.1 Description

This macro sets permanent persistence.

3.1.10.103.2 Return

Attributes

3.1.10.104 macro SMW_ATTR_IS_TRANSIENT

SMW_ATTR_IS_TRANSIENT(attr)

Whether the persistence is transient.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.104.1 Description

This macro returns whether the persistence is transient or not.

3.1.10.104.2 Return

1 if the persistence is transient, 0 otherwise.

3.1.10.105 macro SMW_ATTR_IS_PERSISTENT

SMW_ATTR_IS_PERSISTENT(attr)

Whether the persistence is persistent.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.105.1 Description

This macro returns whether the persistence is persistent or not.

3.1.10.105.2 Return

1 if the persistence is persistent, 0 otherwise.

3.1.10.106 macro SMW_ATTR_IS_PERMANENT

SMW_ATTR_IS_PERMANENT(attr)

Whether the persistence is permanent.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.106.1 Description

This macro returns whether the persistence is permanent or not.

3.1.10.106.2 Return

1 if the persistence is permanent, 0 otherwise.

3.1.10.107 macro SMW_ATTR_SET_LC_CURRENT

SMW_ATTR_SET_LC_CURRENT(attr)

Set current lifecycle.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.107.1 Description

This macro sets current lifecycle.

3.1.10.107.2 Return

Attributes

3.1.10.108 macro SMW_ATTR_SET_LC_OPEN

SMW_ATTR_SET_LC_OPEN(attr)

Set open lifecycle.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.108.1 Description

This macro sets open lifecycle.

3.1.10.108.2 Return

Attributes

3.1.10.109 macro **SMW_ATTR_SET_LC_CLOSED**

SMW_ATTR_SET_LC_CLOSED(attr)

Set closed lifecycle.

Parameters

- **attr** – Attributes. See `smw_attr_attributes_t`.

3.1.10.109.1 Description

This macro sets closed lifecycle.

3.1.10.109.2 Return

Attributes

3.1.10.110 macro **SMW_ATTR_SET_LC_CLOSED_LOCKED**

SMW_ATTR_SET_LC_CLOSED_LOCKED(attr)

Set closed-locked lifecycle.

Parameters

- **attr** – Attributes. See `smw_attr_attributes_t`.

3.1.10.110.1 Description

This macro sets closed-locked lifecycle.

3.1.10.110.2 Return

Attributes

3.1.10.111 macro **SMW_ATTR_IS_LC_CURRENT**

SMW_ATTR_IS_LC_CURRENT(attr)

Whether the lifecycle is current.

Parameters

- **attr** – Attributes. See `smw_attr_attributes_t`.

3.1.10.111.1 Description

This macro returns whether the lifecycle is current or not.

3.1.10.111.2 Return

1 if the lifecycle is current, 0 otherwise.

3.1.10.112 macro SMW_ATTR_IS_LC_OPEN

SMW_ATTR_IS_LC_OPEN(attr)

Whether the lifecycle is open.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.112.1 Description

This macro returns whether the lifecycle is open or not.

3.1.10.112.2 Return

1 if the lifecycle is open, 0 otherwise.

3.1.10.113 macro SMW_ATTR_IS_LC_CLOSED

SMW_ATTR_IS_LC_CLOSED(attr)

Whether the lifecycle is closed.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.113.1 Description

This macro returns whether the lifecycle is closed or not.

3.1.10.113.2 Return

1 if the lifecycle is closed, 0 otherwise.

3.1.10.114 macro SMW_ATTR_IS_LC_CLOSED_LOCKED

SMW_ATTR_IS_LC_CLOSED_LOCKED(attr)

Whether the lifecycle is closed-locked.

Parameters

- **attr** – Attributes. See smw_attr_attributes_t.

3.1.10.114.1 Description

This macro returns whether the lifecycle is closed-locked or not.

3.1.10.114.2 Return

1 if the lifecycle is closed-locked, 0 otherwise.

3.1.10.115 macro **SMW_ATTR_SET_READ_ONLY**

SMW_ATTR_SET_READ_ONLY(attr)

Set read-only flag.

Parameters

- **attr** – Attributes. See `smw_attr_attributes_t`.

3.1.10.115.1 Description

This macro sets read-only flag.

3.1.10.115.2 Return

Attributes

3.1.10.116 macro **SMW_ATTR_SET_READ_ONCE**

SMW_ATTR_SET_READ_ONCE(attr)

Set read-once flag.

Parameters

- **attr** – Attributes. See `smw_attr_attributes_t`.

3.1.10.116.1 Description

This macro sets read-once flag.

3.1.10.116.2 Return

Attributes

3.1.10.117 macro **SMW_ATTR_IS_READ_ONLY**

SMW_ATTR_IS_READ_ONLY(attr)

Whether the read-only flag is set.

Parameters

- **attr** – Attributes. See `smw_attr_attributes_t`.

3.1.10.117.1 Description

This macro returns whether the read-only flag is set or not.

3.1.10.117.2 Return

1 if the read-only flag is set, 0 otherwise.

3.1.10.118 macro **SMW_ATTR_IS_READ_ONCE**

SMW_ATTR_IS_READ_ONCE(attr)

Whether the read-once flag is set.

Parameters

- **attr** – Attributes. See `smw_attr_attributes_t`.

3.1.10.118.1 Description

This macro returns whether the read-once flag is set or not.

3.1.10.118.2 Return

1 if the read-once flag is set, 0 otherwise.

3.1.11 Names APIs

3.1.11.1 typedef **smw_subsystem_t**

type **smw_subsystem_t**

Secure Subsystem name

3.1.11.1.1 Values

- **SMW_SUBSYSTEM_NAME_NONE**: No Secure Subsystem specified
- **SMW_SUBSYSTEM_NAME_TEE**: Trusted Execution Environment
- **SMW_SUBSYSTEM_NAME_SECO**: SECO
- **SMW_SUBSYSTEM_NAME_ELE**: Edge Lock Enclave
- **SMW_SUBSYSTEM_NAME_NB**: Number of Secure Subsystems

3.1.11.2 typedef **smw_operation_t**

type **smw_operation_t**

Security Operation name

3.1.11.2.1 Values

- **SMW_OPERATION_NAME_NONE**: No operation specified
- **SMW_OPERATION_NAME_HASH**: Hash a message
- **SMW_OPERATION_NAME_SIGN**: Sign a message

-
- SMW_OPERATION_NAME_VERIFY: Verify the signature of a message
 - SMW_OPERATION_NAME_RNG: Generate a Random Number
 - SMW_OPERATION_NAME_CIPHER: Cipher encryption or decryption
 - SMW_OPERATION_NAME_CIPHER_MULTI_PART: Cipher multi-part encryption or decryption
 - SMW_OPERATION_NAME_MAC: Generate a Message Authentication Code
 - SMW_OPERATION_NAME_AEAD: Authenticated Encryption with Associated Data
 - SMW_OPERATION_NAME_AEAD_MULTI_PART: AEAD multi-part
 - SMW_OPERATION_NAME_AEAD_UPDATE_AAD: Update AEAD Additional Authenticated Data
 - SMW_OPERATION_NAME_DEVICE_GET_UUID: Get device UUID
 - SMW_OPERATION_NAME_DEVICE_ATTESTATION: Attest device
 - SMW_OPERATION_NAME_DEVICE_LIFECYCLE: Get or set Device lifecycle
 - SMW_OPERATION_NAME_DEVICE_REPROVISION: Prepare or send a device reprovisioning message
 - SMW_OPERATION_NAME_GENERATE_KEY: Generate a key
 - SMW_OPERATION_NAME_DERIVE_KEY: Derive a key
 - SMW_OPERATION_NAME_IMPORT_KEY: Import a key
 - SMW_OPERATION_NAME_EXPORT_KEY: Export a key
 - SMW_OPERATION_NAME_DELETE_KEY: Delete a key
 - SMW_OPERATION_NAME_GET_KEY_LENGTHS: Get key length
 - SMW_OPERATION_NAME_GET_KEY_ATTRIBUTES: Get key attributes
 - SMW_OPERATION_NAME_COMMIT_KEY_STORAGE: Commit key storage
 - SMW_OPERATION_NAME_KEY_ATTESTATION: Attest key
 - SMW_OPERATION_NAME_STORAGE_STORE: Store data
 - SMW_OPERATION_NAME_STORAGE_RETRIEVE: Retrieve data
 - SMW_OPERATION_NAME_STORAGE_DELETE: Delete data
 - SMW_OPERATION_NAME_STORAGE_GET_DATA_INFO: Get data info
 - SMW_OPERATION_NAME_IS_OBJECT_PRESENT: Check if object is present in storage
 - SMW_OPERATION_NAME_HASH_MULTI_PART: Hash multi-part
 - SMW_OPERATION_NAME_ASYMM_ENCRYPT: Asymmetric encryption
 - SMW_OPERATION_NAME_ASYMM_DECRYPT: Asymmetric decryption
 - SMW_OPERATION_NAME_SIGN_MULTI_PART: Signature generation multi-part
 - SMW_OPERATION_NAME_VERIFY_MULTI_PART: Signature verification multi-part
 - SMW_OPERATION_NAME_NB: Number of operations

3.1.11.3 typedef smw_object_type_t

type **smw_object_type_t**

Object type name

3.1.11.3.1 Values

- SMW_OBJECT_TYPE_NAME_NONE: No object type specified
- SMW_OBJECT_TYPE_NAME_DATA: Data Object
- SMW_OBJECT_TYPE_NAME_SECRET_KEY: Symmetric Secret Key Object
- SMW_OBJECT_TYPE_NAME_PUBLIC_KEY: Asymmetric Public Key Object
- SMW_OBJECT_TYPE_NAME_KEY_PAIR: Asymmetric Key Pair Object
- SMW_OBJECT_TYPE_NAME_CERT: Certificate Object
- SMW_OBJECT_TYPE_NAME_NB: Number of object types

3.1.11.4 typedef smw_key_type_t

type **smw_key_type_t**

Key type name

3.1.11.4.1 Values

- SMW_KEY_TYPE_NAME_NONE: No key type specified
- SMW_KEY_TYPE_NAME_SECP_R1: ECC Secp R1 key
- SMW_KEY_TYPE_NAME_BRAINPOOL_R1: ECC Brainpool R1 key
- SMW_KEY_TYPE_NAME_BRAINPOOL_T1: ECC Brainpool T1 key
- SMW_KEY_TYPE_NAME_ED25519: Twisted Edwards 25519 key
- SMW_KEY_TYPE_NAME_AES: AES key
- SMW_KEY_TYPE_NAME_DES: DES key
- SMW_KEY_TYPE_NAME_DES3: Triple DES key
- SMW_KEY_TYPE_NAME_DSA_SM2_FP: DSA SM2 key
- SMW_KEY_TYPE_NAME_SM4: SM4 key
- SMW_KEY_TYPE_NAME_HMAC: HMAC key
- SMW_KEY_TYPE_NAME_RSA: RSA key
- SMW_KEY_TYPE_NAME_DH: DH key
- SMW_KEY_TYPE_NAME_TLS_MASTER: TLS master key
- SMW_KEY_TYPE_NAME_RAW: Raw key
- SMW_KEY_TYPE_NAME_DERIVE: Derived key
- SMW_KEY_TYPE_NAME_HKDF_IKM: HKDF IKM key
- SMW_KEY_TYPE_NAME_X25519: Montgomery Elliptic Curve25519 key

-
- SMW_KEY_TYPE_NAME_EL2GO_PROV_OEM_KEY: EdgeLock 2GO Provisioning OEM Key
 - SMW_KEY_TYPE_NAME_ED448: Twisted Edwards 448 key
 - SMW_KEY_TYPE_NAME_X448: Montgomery Elliptic Curve448 key
 - SMW_KEY_TYPE_NAME_NB: Number of key types

3.1.11.5 typedef smw_key_privacy_t

type **smw_key_privacy_t**
Key privacy name

3.1.11.5.1 Values

- SMW_KEY_PRIVACY_NAME_NONE: No key privacy specified
- SMW_KEY_PRIVACY_NAME_PUBLIC: Public key
- SMW_KEY_PRIVACY_NAME_PRIVATE: Private key
- SMW_KEY_PRIVACY_NAME_PAIR: Key pair
- SMW_KEY_PRIVACY_NAME_SHARED_SECRET: Shared secret
- SMW_KEY_PRIVACY_NAME_NB: Number of key privacies

3.1.11.6 typedef smw_hash_algo_t

type **smw_hash_algo_t**
Hash algorithm name

3.1.11.6.1 Values

- SMW_HASH_ALGO_NAME_NONE: No hash algorithm specified
- SMW_HASH_ALGO_NAME_MD5: Message Digest 5
- SMW_HASH_ALGO_NAME_SHA1: Secure Hash Algorithm 1
- SMW_HASH_ALGO_NAME_SHA224: Secure Hash Algorithm 2, 224 bits
- SMW_HASH_ALGO_NAME_SHA256: Secure Hash Algorithm 2, 256 bits
- SMW_HASH_ALGO_NAME_SHA384: Secure Hash Algorithm 2, 384 bits
- SMW_HASH_ALGO_NAME_SHA512: Secure Hash Algorithm 2, 512 bits
- SMW_HASH_ALGO_NAME_SHA3_224: Secure Hash Algorithm 3, 224 bits
- SMW_HASH_ALGO_NAME_SHA3_256: Secure Hash Algorithm 3, 256 bits
- SMW_HASH_ALGO_NAME_SHA3_384: Secure Hash Algorithm 3, 384 bits
- SMW_HASH_ALGO_NAME_SHA3_512: Secure Hash Algorithm 3, 512 bits
- SMW_HASH_ALGO_NAME_SM3: ShangMi 3
- SMW_HASH_ALGO_NAME_SHAKE_256: Secure Hash Algorithm KECCAK, 256 bits
- SMW_HASH_ALGO_NAME_NB: Number of Hash algorithms

3.1.11.7 typedef smw_mac_algo_t

type **smw_mac_algo_t**

MAC algorithm name

3.1.11.7.1 Values

- SMW_MAC_ALGO_NAME_NONE: No Message Authentication Code algorithm specified
- SMW_MAC_ALGO_NAME_CMAC: Cipher-based Message Authentication Code algorithm
- SMW_MAC_ALGO_NAME_CMAC_TRUNCATED: Cipher-based Message Authentication Code truncated algorithm
- SMW_MAC_ALGO_NAME_HMAC: Hash-Based Message Authentication Code algorithm
- SMW_MAC_ALGO_NAME_HMAC_TRUNCATED: Hash-Based Message Authentication Code truncated algorithm
- SMW_MAC_ALGO_NAME_NB: Number of Message Authentication Code algorithms

3.1.11.8 typedef smw_cipher_mode_t

type **smw_cipher_mode_t**

Cipher mode name

3.1.11.8.1 Values

- SMW_CIPHER_MODE_NAME_NONE: No cipher mode specified
- SMW_CIPHER_MODE_NAME_CBC: Cipher Block Chaining mode
- SMW_CIPHER_MODE_NAME_CFB: Cipher Feedback Block mode
- SMW_CIPHER_MODE_NAME_CTR: Counter mode
- SMW_CIPHER_MODE_NAME_CTS: Ciphertext Stealing mode
- SMW_CIPHER_MODE_NAME_ECB: Electronic Codebook Block mode
- SMW_CIPHER_MODE_NAME_XTS: XEX Tweakable Block Cipher with Ciphertext Stealing mode
- SMW_CIPHER_MODE_NAME_OFB: Output Feedback Block mode
- SMW_CIPHER_MODE_NAME_NB: Number of cipher modes

3.1.11.9 typedef smw_aead_mode_t

type **smw_aead_mode_t**

AEAD mode name

3.1.11.9.1 Values

- SMW_AEAD_MODE_NAME_NONE: No AEAD mode specified
- SMW_AEAD_MODE_NAME_CCM: Counter with CBC-MAC mode
- SMW_AEAD_MODE_NAME_GCM: Galois Counter Mode

-
- SMW_AEAD_MODE_NAME_CHACHA20_POLY1305: ChaCha20 stream cipher with Poly1305 MAC mode
 - SMW_AEAD_MODE_NAME_NB: Number of AEAD modes

3.1.11.10 typedef smw_cipher_op_type_t

type **smw_cipher_op_type_t**

Cipher operation type name

3.1.11.10.1 Values

- SMW_CIPHER_OP_TYPE_NAME_NONE: No cipher operation type specified
- SMW_CIPHER_OP_TYPE_NAME_ENCRYPT: Encrypt operation type
- SMW_CIPHER_OP_TYPE_NAME_DECRYPT: Decrypt operation type
- SMW_CIPHER_OP_TYPE_NAME_NB: Number of cipher operation types

3.1.11.11 typedef smw_aead_op_type_t

type **smw_aead_op_type_t**

AEAD operation type name

3.1.11.11.1 Values

- SMW_AEAD_OP_TYPE_NAME_NONE: No AEAD operation specified
- SMW_AEAD_OP_TYPE_NAME_ENCRYPT: AEAD encrypt operation
- SMW_AEAD_OP_TYPE_NAME_DECRYPT: AEAD decrypt operation
- SMW_AEAD_OP_TYPE_NAME_NB: Number of AEAD operations

3.1.11.12 typedef smw_key_format_t

type **smw_key_format_t**

Key format name

3.1.11.12.1 Values

- SMW_KEY_FORMAT_NAME_NONE: No key format specified
- SMW_KEY_FORMAT_NAME_HEX: Hexadecimal value (no encoding)
- SMW_KEY_FORMAT_NAME_BASE64: Base 64 encoding value
- SMW_KEY_FORMAT_NAME_NB: Number of key formats

3.1.11.13 typedef smw_signature_algo_t

type **smw_signature_algo_t**

Signature algorithm name

3.1.11.13.1 Values

- SMW_SIGNATURE_ALGO_NAME_NONE: No signature algorithm specified
- SMW_SIGNATURE_ALGO_NAME_DEFAULT: Signature algorithm is given by the key type
- SMW_SIGNATURE_ALGO_NAME_ECDSA: Elliptic Curve Digital Signature Algorithm
- SMW_SIGNATURE_ALGO_NAME_EDDSA: Edwards-curve Digital Signature Algorithm
- SMW_SIGNATURE_ALGO_NAME_DSA: Digital Signature Algorithm
- SMW_SIGNATURE_ALGO_NAME_RSA: Rivest, Shamir and Adleman signature algorithm
- SMW_SIGNATURE_ALGO_NAME_TLS_1_2: Transport Layer Security 1.2 signature algorithm
- SMW_SIGNATURE_ALGO_NAME_NB: Number of signature algorithms

3.1.11.14 typedef smw_signature_type_t

type **smw_signature_type_t**

Signature type name

3.1.11.14.1 Values

- SMW_SIGNATURE_TYPE_NAME_NONE: No signature type specified
- SMW_SIGNATURE_TYPE_NAME_DEFAULT: Default signature type
- SMW_SIGNATURE_TYPE_NAME_PKCS1_1_5: Public-Key Cryptography Standards #1 v1.5
- SMW_SIGNATURE_TYPE_NAME_PSS: Probabilistic Signature Scheme
- SMW_SIGNATURE_TYPE_NAME_CLIENT: TLS client signature
- SMW_SIGNATURE_TYPE_NAME_SERVER: TLS server signature
- SMW_SIGNATURE_TYPE_NAME_CMAC: Cipher-based Message Authentication Code
- SMW_SIGNATURE_TYPE_NAME_NB: Number of signature types

3.1.11.15 typedef smw_tls12_kec_t

type **smw_tls12_kec_t**

TLS 1.2 Key exchange algorithm name

3.1.11.15.1 Values

- SMW_TLS12_KEA_NAME_NONE: No TLS 1.2 Key exchange algorithm specified
- SMW_TLS12_KEA_NAME_DH_DSS: Diffie-Hellman signed with DSS
- SMW_TLS12_KEA_NAME_DH_RSA: Diffie-Hellman signed with RSA
- SMW_TLS12_KEA_NAME_DHE_DSS: Diffie-Hellman key Exchange signed with DSS
- SMW_TLS12_KEA_NAME_DHE_RSA: Diffie-Hellman key Exchange signed with RSA
- SMW_TLS12_KEA_NAME_ECDH_ECDSA: Elliptic Curve Diffie-Hellman signed with ECDSA

- SMW_TLS12_KEA_NAME_ECDH_RSA: Elliptic Curve Diffie-Hellman signed with RSA
- SMW_TLS12_KEA_NAME_ECDHE_ECDSA: Elliptic Curve Diffie-Hellman key Exchange signed with ECDSA
- SMW_TLS12_KEA_NAME_ECDHE_RSA: Elliptic Curve Diffie-Hellman key Exchange signed with RSA
- SMW_TLS12_KEA_NAME_RSA: Rivest–Shamir–Adleman
- SMW_TLS12_KEA_NAME_NB: Number of TLS 1.2 Key exchange algorithms

3.1.11.16 typedef smw_tls12_enc_t

type **smw_tls12_enc_t**

TLS 1.2 encryption algorithm name

3.1.11.16.1 Values

- SMW_TLS12_ENC_NAME_NONE: No TLS 1.2 encryption algorithm specified
- SMW_TLS12_ENC_NAME_3DES_EDE_CBC: Triple DES Encrypt-Decrypt-Encrypt with CBC
- SMW_TLS12_ENC_NAME_AES_128_CBC: Advanced Encryption Standard 128 bits with CBC
- SMW_TLS12_ENC_NAME_AES_128_GCM: Advanced Encryption Standard 128 bits with GCM
- SMW_TLS12_ENC_NAME_AES_256_CBC: Advanced Encryption Standard 256 bits with CBC
- SMW_TLS12_ENC_NAME_AES_256_GCM: Advanced Encryption Standard 256 bits with GCM
- SMW_TLS12_ENC_NAME_RC4_128: Rivest Cipher 4 128 bits
- SMW_TLS12_ENC_NAME_AES_128_CCM: Advanced Encryption Standard 128 bits with CCM
- SMW_TLS12_ENC_NAME_AES_256_CCM: Advanced Encryption Standard 256 bits with CCM
- SMW_TLS12_ENC_NAME_CHACHA20_POLY1305: ChaCha20 stream cipher with Poly1305 MAC
- SMW_TLS12_ENC_NAME_NB: Number of TLS 1.2 encryption algorithms

3.1.11.17 typedef smw_tls12_op_t

type **smw_tls12_op_t**

TLS 1.2 operation name

3.1.11.17.1 Values

- SMW_TLS12_OP_NAME_NONE: No TLS 1.2 operation name specified
- SMW_TLS12_OP_NAME_MASTER_SECRET: TLS 1.2 master secret
- SMW_TLS12_OP_NAME_KEY_EXPANSION: TLS 1.2 key expansion

-
- SMW_TLS12_OP_NAME_NB: Number of TLS 1.2 operation names

3.1.11.18 typedef smw_oem_master_key_op_t

type **smw_oem_master_key_op_t**

OEM Master key derivation operation name

3.1.11.18.1 Values

- SMW_OEM_MK_OP_NAME_NONE: No operation name specified
- SMW_OEM_MK_OP_NAME_DERIVE: Derive the OEM Master key
- SMW_OEM_MK_OP_NAME_PREPARE: Prepare the OEM Master key payload to sign
- SMW_OEM_MK_OP_NAME_NB: Number of operation names

3.1.11.19 typedef smw_lifecycle_t

type **smw_lifecycle_t**

Device lifecycle name

3.1.11.19.1 Values

- SMW_LIFECYCLE_NAME_NONE: No lifecycle specified
- SMW_LIFECYCLE_NAME_OPEN: Current
- SMW_LIFECYCLE_NAME_OPEN: Open
- SMW_LIFECYCLE_NAME_CLOSED: Closed
- SMW_LIFECYCLE_NAME_CLOSED_LOCKED: Closed-locked
- SMW_LIFECYCLE_NAME_OEM_RETURN: OEM return
- SMW_LIFECYCLE_NAME_NXP_RETURN: NXP return
- SMW_LIFECYCLE_NAME_NXP_NB: Number of lifecycles

3.1.11.20 typedef smw_kdf_t

type **smw_kdf_t**

Key Derivation Function name

3.1.11.20.1 Values

- SMW_KDF_NAME_NONE: No Key Derivation Function name specified
- SMW_KDF_NAME_HKDF: HMAC-Based Key Derivation Function
- SMW_KDF_NAME_HKDF_EXTRACT: HMAC-Based Key Derivation Extract step Function
- SMW_KDF_NAME_HKDF_EXPAND: HMAC-Based Key Derivation Expand step Function
- SMW_KDF_NAME_TLS12_KEY_EXCHANGE: TLS 1.2 Key Exchange
- SMW_KDF_NAME_ECDH: ECDH Key Exchange

- SMW_KDF_NAME_TLS12_OP_KEY_EXCHANGE: TLS 1.2 “Operation-based” Key Exchange
- SMW_KDF_NAME_TLS13_KEY_EXCHANGE: TLS 1.3 Key Exchange
- SMW_KDF_NAME_OEM_MASTER_KEY: OEM Master key derivation
- SMW_KDF_NAME_NB: Number of Key Derivation Functions

3.1.11.21 typedef smw_asymmetric_encryption_algo_t

type **smw_asymmetric_encryption_algo_t**

Asymmetric encryption/decryption algo name

3.1.11.21.1 Values

- SMW_ASYMMETRIC_ENCRYPTION_ALGO_NAME_NONE: No encryption/decryption algorithm specified
- SMW_ASYMMETRIC_ENCRYPTION_ALGO_NAME_RSA: RSA encryption/decryption algorithm
- SMW_ASYMMETRIC_ENCRYPTION_ALGO_NAME_NB: Number of encryption/decryption algorithms

3.1.11.22 typedef smw_asymmetric_encryption_mode_t

type **smw_asymmetric_encryption_mode_t**

Asymmetric encryption/decryption padding scheme name

3.1.11.22.1 Values

- SMW_ASYMMETRIC_ENCRYPTION_MODE_NAME_NONE: No Encryption padding scheme name specified
- SMW_ASYMMETRIC_ENCRYPTION_MODE_NAME_PKCS1_1_5: Public-Key Cryptography Standards #1 v1.5
- SMW_ASYMMETRIC_ENCRYPTION_MODE_NAME_OAEP: Optimal Asymmetric Encryption Padding
- SMW_ASYMMETRIC_ENCRYPTION_MODE_NAME_NO_PAD: Asymmetric Encryption/decryption with no padding
- SMW_ASYMMETRIC_ENCRYPTION_MODE_NAME_NB: Number of encryption padding schemes

3.1.12 Examples

3.1.12.1 Authentication Encryption/Decryption (AEAD)

3.1.12.1.1 Example 1: AEAD one-shot encryption operation

```
#define IV_LEN 12
#define AAD_LEN 20
#define DATA_LEN 32
```

(continues on next page)

```

#define CIPHER_LEN 32
#define TAG_LEN 16

int main(int argc, char *argv[])
{
    int res = SMW_STATUS_OPERATION_FAILURE;
    unsigned char iv[IV_LEN] = {...};
    unsigned char aad[AAD_LEN] = {...};
    unsigned char input[DATA_LEN] = {...};
    unsigned char output[CIPHER_LEN + TAG_LEN] = {0};
    unsigned char output_iv[IV_LEN] = {0};

    struct smw_aead_args args = {0};
    struct smw_aead_init_args init_args = {0};
    struct smw_aead_final_args final_args = {0};
    struct smw_aead_data_args data_args = {0};
    struct smw_aead_aad_args aad_args = {0};
    struct smw_key_descriptor key_desc = {0};

    init_args.subsystem_name = SMW_SUBSYSTEM_NAME_TEE;
    init_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_ENCRYPT;
    init_args.mode_name = SMW_AEAD_MODE_NAME_GCM;
    init_args.plaintext_length = DATA_LEN;
    init_args.user_iv = iv;
    init_args.user_iv_length = IV_LEN;
    init_args.iv_length = IV_LEN;
    init_args.key_desc = &key_desc;

    data_args.input_length = DATA_LEN;
    data_args.input = input;
    data_args.output_length = CIPHER_LEN + TAG_LEN;
    data_args.output = output;

    final_args.data = &data_args;
    final_args.tag_length = TAG_LEN;
    final_args.output_iv_length = IV_LEN;
    final_args.output_iv = output_iv;

    aad_args.data = aad;
    aad_args.data_length = AAD_LEN;

    // One-shot AE encryption operation
    args.init = &init_args;
    args.final = &final_args;
    args.aad = &aad_args;

    res = smw_aead(&args);
    return res;
}

```

3.1.12.1.2 Example 2: AEAD one-shot encryption operation (Tag stored in tag field)

```
#define IV_LEN 12
#define AAD_LEN 20
#define DATA_LEN 32
#define CIPHER_LEN 32
#define TAG_LEN 16

int main(int argc, char *argv[])
{
    int res = SMW_STATUS_OPERATION_FAILURE;
    unsigned char iv[IV_LEN] = {...};
    unsigned char aad[AAD_LEN] = {...};
    unsigned char input[DATA_LEN] = {...};
    unsigned char output[CIPHER_LEN] = {0};
    unsigned char tag[TAG_LEN] = {0};
    unsigned char output_iv[IV_LEN] = {0};

    struct smw_aead_args args = {0};
    struct smw_aead_init_args init_args = {0};
    struct smw_aead_final_args final_args = {0};
    struct smw_aead_data_args data_args = {0};
    struct smw_aead_aad_args aad_args = {0};
    struct smw_key_descriptor key_desc = {0};

    init_args.subsystem_name = SMW_SUBSYSTEM_NAME_TEE;
    init_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_ENCRYPT;
    init_args.mode_name = SMW_AEAD_MODE_NAME_GCM;
    init_args.plaintext_length = DATA_LEN;
    init_args.user_iv = iv;
    init_args.user_iv_length = IV_LEN;
    init_args.iv_length = IV_LEN;
    init_args.key_desc = &key_desc;

    data_args.input_length = DATA_LEN;
    data_args.input = input;
    data_args.output_length = CIPHER_LEN;
    data_args.output = output;

    final_args.data = &data_args;
    final_args.tag_length = TAG_LEN;
    final_args.tag = tag;
    final_args.output_iv_length = IV_LEN;
    final_args.output_iv = output_iv;

    aad_args.data = aad;
    aad_args.data_length = AAD_LEN;

    // One-shot AE encryption operation
    args.init = &init_args;
```

(continues on next page)

```

args.final = &final_args;
args.aad = &aad_args;

res = smw_aead(&args);
return res;
}

```

3.1.12.1.3 Example 3: AEAD one-shot decryption operation (Tag stored in tag field)

```

#define IV_LEN 12
#define AAD_LEN 20
#define DATA_LEN 32
#define CIPHER_LEN 32
#define TAG_LEN 16

int main(int argc, char *argv[])
{
    int res = SMW_STATUS_OPERATION_FAILURE;
    unsigned char iv[IV_LEN] = {...};
    unsigned char aad[AAD_LEN] = {...};
    unsigned char input[DATA_LEN] = {...};
    unsigned char output[CIPHER_LEN] = {0};
    unsigned char tag[TAG_LEN] = {...};

    struct smw_aead_args args = {0};
    struct smw_aead_init_args init_args = {0};
    struct smw_aead_final_args final_args = {0};
    struct smw_aead_data_args data_args = {0};
    struct smw_aead_aad_args aad_args = {0};
    struct smw_key_descriptor key_desc = {0};

    init_args.subsystem_name = SMW_SUBSYSTEM_NAME_TEE;
    init_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_DECRYPT;
    init_args.mode_name = SMW_AEAD_MODE_NAME_GCM;
    init_args.plaintext_length = DATA_LEN;
    init_args.user_iv = iv;
    init_args.user_iv_length = IV_LEN;
    init_args.iv_length = IV_LEN;
    init_args.key_desc = &key_desc;

    data_args.input_length = CIPHER_LEN;
    data_args.input = input;
    data_args.output_length = DATA_LEN;
    data_args.output = output;

    final_args.data = &data_args;
    final_args.tag_length = TAG_LEN;
    final_args.tag = tag;
}

```

(continues on next page)

(continued from previous page)

```
aad_args.data = aad;
aad_args.data_length = AAD_LEN;

// One-shot AE encryption operation
args.init = &init_args;
args.final = &final_args;
args.aad = &aad_args;

res = smw_aead(&args);
return res;
}
```

3.1.12.1.4 Example 4: AEAD multi-part encryption operation

```
#define IV_LEN 12
#define AAD_LEN 20
#define DATA_LEN 32
#define TAG_LEN 16
#define CIPHER_LEN 32

int main(int argc, char *argv[])
{

    int res = SMW_STATUS_OPERATION_FAILURE;

    unsigned char iv[IV_LEN] = {...};
    unsigned char aad[AAD_LEN] = {...};
    unsigned char input[DATA_LEN] = {...};
    unsigned char output[CIPHER_LEN + TAG_LEN] = {0};

    struct smw_aead_data_args data_args = {0};
    struct smw_aead_aad_args aad_args = {0};
    struct smw_aead_final_args final_args = {0};
    struct smw_aead_init_args init_args = {0};
    struct smw_key_descriptor key_desc = {0};
    struct smw_context_args op_ctx = {0};

    init_args.subsystem_name = SMW_SUBSYSTEM_NAME_TEE;
    init_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_ENCRYPT;
    init_args.mode_name = SMW_AEAD_MODE_NAME_GCM;
    init_args.aad_length = AAD_LEN;
    init_args.tag_length = TAG_LEN;
    init_args.plaintext_length = DATA_LEN;
    init_args.user_iv = iv;
    init_args.user_iv_length = IV_LEN;
    init_args.iv_length = IV_LEN;

    // Allocate memory to operation context
    res = smw_allocate_context(&op_ctx);
```

(continues on next page)

```

if (res != SMW_STATUS_OK)
    goto exit;

init_args.context = op_ctx.context;
init_args.key_desc = &key_desc;

// Initialize multi-part AEAD operation
res = smw_aead_init(&init_args);
if (res != SMW_STATUS_OK)
    goto exit;

// Add additional data to an active AEAD operation.
aad_args.data = aad;
aad_args.data_length = AAD_LEN;
aad_args.context = init_args.context;
res = smw_aead_update_aad(&aad_args);
if (res != SMW_STATUS_OK)
    goto exit;

/*
    * Encrypt 1st message fragment in an active
    * multi-part AEAD encryption operation.
*/
data_args.input_length = 16;
data_args.input = input;
data_args.output_length = 16;
data_args.output = output;
data_args.context = init_args.context;
res = smw_aead_update(&data_args);
if (res != SMW_STATUS_OK)
    goto exit;

/*
    * Encrypt 2nd message fragment in an active
    * multi-part AEAD encryption operation.
*/
data_args.input_length = 16;
data_args.input = &input[16];
data_args.output_length = 16;
data_args.output = &output[16];
data_args.context = init_args.context;
res = smw_aead_update(&data_args);
if (res != SMW_STATUS_OK)
    goto exit;

/*
    * Finish encrypting the message in an active
    * multi-part AEAD operation.
*/

```

(continues on next page)

(continued from previous page)

```
final_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_ENCRYPT;
final_args.data->context = init_args.context;
final_args.data->input = NULL;
final_args.data->input_length = 0;
final_args.data->output = &output[32];
final_args.data->output_length = TAG_LEN;
final_args.tag_length = TAG_LEN;
final_args.output_iv_length = IV_LEN;
final_args.output_iv = output_iv;

res = smw_aead_final(&final_args);

exit:
    return res;
}
```

3.1.12.1.5 Example 5: AEAD multi-part decryption operation

```
#define IV_LEN 12
#define AAD_LEN 20
#define DATA_LEN 32
#define TAG_LEN 16
#define CIPHER_LEN 32

int main(int argc, char *argv[])
{
    int res = SMW_STATUS_OPERATION_FAILURE;

    unsigned char iv[IV_LEN] = {...};
    unsigned char aad[AAD_LEN] = {...};
    unsigned char input[CIPHER_LEN + TAG_LEN] = {...};
    unsigned char output[DATA_LEN] = {0};

    struct smw_aead_data_args data_args = {0};
    struct smw_aead_aad_args aad_args = {0};
    struct smw_aead_final_args final_args = {0};
    struct smw_aead_init_args init_args = {0};
    struct smw_key_descriptor key_desc = {0};
    struct smw_context_args op_ctx = {0};

    init_args.subsystem_name = SMW_SUBSYSTEM_NAME_TEE;
    init_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_DECRYPT;
    init_args.mode_name = SMW_AEAD_MODE_NAME_GCM;
    init_args.aad_length = AAD_LEN;
    init_args.tag_length = TAG_LEN;
    init_args.plaintext_length = DATA_LEN;
    init_args.user_iv = iv;
    init_args.user_iv_length = IV_LEN;
```

(continues on next page)

```

init.args.iv_length = IV_LEN;

// Allocate memory to operation context
res = smw_allocate_context(&op_ctx);
if (res != SMW_STATUS_OK)
    goto exit;

init_args.context = op_ctx.context;
init_args.key_desc = &key_desc;

// Initialize multi-part AEAD operation
res = smw_aead_init(&init_args);
if (res != SMW_STATUS_OK)
    goto exit;

// Add additional data to an active AEAD operation.
aad_args.data = aad;
aad_args.data_length = AAD_LEN;
aad_args.context = init_args.context;
res = smw_aead_update_aad(&aad_args);
if (res != SMW_STATUS_OK)
    goto exit;

/*
 * Decrypt 1st message fragment in an active
 * multi-part AEAD decryption operation.
 */
data_args.input_length = 16;
data_args.input = input;
data_args.output_length = 16;
data_args.output = output;
data_args.context = init_args.context;
res = smw_aead_update(&data_args);
if (res != SMW_STATUS_OK)
    goto exit;

/*
 * Decrypt 2nd message fragment in an active
 * multi-part AEAD decryption operation.
 */
data_args.input_length = 16;
data_args.input = &input[16];
data_args.output_length = 16;
data_args.output = &output[16];
data_args.context = init_args.context;
res = smw_aead_update(&data_args);
if (res != SMW_STATUS_OK)
    goto exit;

```

(continues on next page)

```

/*
 * Finish authenticating and decrypting the message
 * in an active multi-part AEAD operation.
 */
final_args.op_type_name = SMW_AEAD_OP_TYPE_NAME_DECRYPT;
final_args.data.context = init_args.context;
// Pass the tag
final_args.data->input = &input[32];
final_args.data->input_length = TAG_LEN;
final_args.data->output = NULL;
final_args.data->output_length = 0;
final_args.tag_length = TAG_LEN;
res = smw_aead_final(&final_args);

exit:
    return res;
}

```

3.2 PSA APIs

3.2.1 Cryptography APIs

3.2.1.1 Values

3.2.1.1.1 Introduction

This file declares macros to build and analyze values of integral types defined in `crypto_types.h`.

3.2.1.1.2 Reference

Documentation:

PSA Cryptography API v1.2,1

Link:

<https://arm-software.github.io/psa-api/crypto/1.2/about>

3.2.1.1.3 macro `PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG`

`PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG(aead_alg)`

An AEAD algorithm with the default tag length.

Parameters

- **aead_alg** – An AEAD algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_AEAD(aead_alg)` is true).

3.2.1.1.3.1 Description

This macro can be used to construct the AEAD algorithm with default tag length from an AEAD algorithm with a shortened tag. See also `PSA_ALG_AEAD_WITH_SHORTENED_TAG()`.

3.2.1.1.3.2 Return

The corresponding AEAD algorithm with the default tag length for that algorithm.

3.2.1.1.4 macro `PSA_ALG_AEAD_TAG_LENGTH`

`PSA_ALG_AEAD_TAG_LENGTH(aead_alg)`

Retrieve the tag length of a specified AEAD algorithm.

Parameters

- **aead_alg** – An AEAD algorithm identifier (value of *typedef* `psa_algorithm_t` such that `PSA_ALG_IS_AEAD(aead_alg)` is true).

3.2.1.1.4.1 Return

The tag length specified by the input algorithm.

0 if `aead_alg` is not a supported AEAD algorithm.

3.2.1.1.5 macro `PSA_ALG_AEAD_WITH_SHORTENED_TAG`

`PSA_ALG_AEAD_WITH_SHORTENED_TAG(aead_alg, tag_length)`

Macro to build a AEAD algorithm with a shortened tag.

Parameters

- **aead_alg** – An AEAD algorithm identifier (value of *typedef* `psa_algorithm_t` such that `PSA_ALG_IS_AEAD(aead_alg)` is true).
- **tag_length** – Desired length of the authentication tag in bytes.

3.2.1.1.5.1 Description

An AEAD algorithm with a shortened tag is similar to the corresponding AEAD algorithm, but has an authentication tag that consists of fewer bytes. Depending on the algorithm, the tag length might affect the calculation of the ciphertext.

The AEAD algorithm with a default length tag can be recovered using `PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG()`.

3.2.1.1.5.2 Return

The corresponding AEAD algorithm with the specified tag length.

Unspecified if `aead_alg` is not a supported AEAD algorithm or if `tag_length` is not valid for the specified AEAD algorithm.

3.2.1.1.6 macro `PSA_ALG_AEAD_WITH_AT_LEAST_THIS_LENGTH_TAG`

`PSA_ALG_AEAD_WITH_AT_LEAST_THIS_LENGTH_TAG(aead_alg, min_tag_length)`

Macro to build an AEAD minimum-tag-length wildcard algorithm.

Parameters

- **aead_alg** – An AEAD algorithm: a value of *typedef* `psa_algorithm_t` such that `PSA_ALG_IS_AEAD(aead_alg)` is true.

-
- **min_tag_length** – Desired minimum length of the authentication tag in bytes. This must be at least 1 and at most the largest allowed tag length of the algorithm.

3.2.1.1.6.1 Description

A key with a minimum-tag-length AEAD wildcard algorithm as permitted algorithm policy can be used with all AEAD algorithms sharing the same base algorithm, and where the tag length of the specific algorithm is equal to or larger than the minimum tag length specified by the wildcard algorithm.

Note:

When setting the minimum required tag length to less than the smallest tag length allowed by the base algorithm, this effectively becomes an ‘any-tag-length-allowed’ policy for that base algorithm.

The AEAD algorithm with a default length tag can be recovered using `PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG()`.

Compatible key types:

The resulting wildcard AEAD algorithm is compatible with the same key types as the AEAD algorithm used to construct it.

3.2.1.1.6.2 Return

The corresponding AEAD wildcard algorithm with the specified minimum tag length.

Unspecified if `aead_alg` is not a supported AEAD algorithm or if `min_tag_length` is less than 1 or too large for the specified AEAD algorithm.

3.2.1.1.7 macro `PSA_ALG_DETERMINISTIC_ECDSA`

`PSA_ALG_DETERMINISTIC_ECDSA(hash_alg)`

Deterministic ECDSA signature scheme, with hashing.

Parameters

- **hash_alg** – A hash algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_HASH(hash_alg)` is true). This includes `PSA_ALG_ANY_HASH` when specifying the algorithm in a key policy.

3.2.1.1.7.1 Description

This algorithm can be used with both the message and hash signature functions.

Note:

When based on the same hash algorithm, the verification operations for `PSA_ALG_ECDSA` and `PSA_ALG_DETERMINISTIC_ECDSA` are identical. A signature created using `PSA_ALG_ECDSA` can be verified with the same key using either `PSA_ALG_ECDSA` or `PSA_ALG_DETERMINISTIC_ECDSA`. Similarly, a signature created using `PSA_ALG_DETERMINISTIC_ECDSA` can be verified with the same key using either `PSA_ALG_ECDSA` or `PSA_ALG_DETERMINISTIC_ECDSA`.

In particular, it is impossible to determine whether a signature was produced with deterministic ECDSA or with randomized ECDSA: it is only possible to verify that a signature was made with ECDSA with the private key corresponding to the public key used for the verification.

This is the deterministic ECDSA signature scheme defined by Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA) [RFC6979].

The representation of a signature is the same as with [PSA_ALG_ECDSA\(\)](#).

3.2.1.1.7.2 Return

The corresponding deterministic ECDSA signature algorithm.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.8 macro `PSA_ALG_ECDSA`

`PSA_ALG_ECDSA(hash_alg)`

The randomized ECDSA signature scheme, with hashing.

Parameters

- **`hash_alg`** – A hash algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_HASH(hash_alg)` is true). This includes `PSA_ALG_ANY_HASH` when specifying the algorithm in a key policy.

3.2.1.1.8.1 Description

This algorithm can be used with both the message and hash signature functions.

This algorithm is randomized: each invocation returns a different, equally valid signature.

Note:

When based on the same hash algorithm, the verification operations for `PSA_ALG_ECDSA` and `PSA_ALG_DETERMINISTIC_ECDSA` are identical. A signature created using `PSA_ALG_ECDSA` can be verified with the same key using either `PSA_ALG_ECDSA` or `PSA_ALG_DETERMINISTIC_ECDSA`. Similarly, a signature created using `PSA_ALG_DETERMINISTIC_ECDSA` can be verified with the same key using either `PSA_ALG_ECDSA` or `PSA_ALG_DETERMINISTIC_ECDSA`.

In particular, it is impossible to determine whether a signature was produced with deterministic ECDSA or with randomized ECDSA: it is only possible to verify that a signature was made with ECDSA with the private key corresponding to the public key used for the verification.

This signature scheme is defined by SEC 1: Elliptic Curve Cryptography [SEC1], and also by Public Key Cryptography For The Financial Services Industry: The Elliptic Curve Digital Signature Algorithm (ECDSA) [X9-62], with a random per-message secret number *k*.

The representation of the signature as a byte string consists of the concatenation of the signature values *r* and *s*. Each of *r* and *s* is encoded as an *N*-octet string, where *N* is the length of the base point of the curve in octets. Each value is represented in big-endian order, with the most significant octet first.

3.2.1.1.8.2 Return

The corresponding randomized ECDSA signature algorithm.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.9 macro `PSA_ALG_FULL_LENGTH_MAC`

`PSA_ALG_FULL_LENGTH_MAC`(`mac_alg`)

Macro to construct the MAC algorithm with a full length MAC, from a truncated MAC algorithm.

Parameters

- **mac_alg** – A MAC algorithm identifier (value of `typedef psa_algorithm_t` such that `PSA_ALG_IS_MAC(mac_alg)` is true). This can be a truncated or untruncated MAC algorithm.

3.2.1.1.9.1 Return

The corresponding MAC algorithm with a full length MAC.

Unspecified if `mac_alg` is not a supported MAC algorithm.

3.2.1.1.10 macro `PSA_ALG_AT_LEAST_THIS_LENGTH_MAC`

`PSA_ALG_AT_LEAST_THIS_LENGTH_MAC`(`mac_alg`, `min_mac_length`)

Macro to build a MAC minimum-MAC-length wildcard algorithm.

Parameters

- **mac_alg** – A MAC algorithm: a value of `typedef psa_algorithm_t` such that `PSA_ALG_IS_MAC(alg)` is true. This can be a truncated or untruncated MAC algorithm.
- **min_mac_length** – Desired minimum length of the message authentication code in bytes. This must be at most the untruncated length of the MAC and must be at least 1.

3.2.1.1.10.1 Description

A key with a minimum-MAC-length MAC wildcard algorithm as permitted algorithm policy can be used with all MAC algorithms sharing the same base algorithm, and where the (potentially truncated) MAC length of the specific algorithm is equal to or larger than the wildcard algorithm's minimum MAC length.

Note:

When setting the minimum required MAC length to less than the smallest MAC length allowed by the base algorithm, this effectively becomes an 'any-MAC-length-allowed' policy for that base algorithm.

The untruncated MAC algorithm can be recovered using `PSA_ALG_FULL_LENGTH_MAC()`.

Compatible key types:

The resulting wildcard MAC algorithm is compatible with the same key types as the MAC algorithm used to construct it.

3.2.1.1.10.2 Return

The corresponding MAC wildcard algorithm with the specified minimum MAC length.

Unspecified if `mac_alg` is not a supported MAC algorithm or if `min_mac_length` is less than 1 or too large for the specified MAC algorithm.

3.2.1.1.11 macro `PSA_ALG_GET_HASH`

`PSA_ALG_GET_HASH`(alg)

Get the hash used by a composite algorithm.

Parameters

- **alg** – An algorithm identifier (value of *typedef psa_algorithm_t*).

3.2.1.1.11.1 Description

The following composite algorithms require a hash algorithm:

- `PSA_ALG_ECDSA()`
- `PSA_ALG_HKDF()`
- `PSA_ALG_HMAC()`
- `PSA_ALG_RSA_OAEP()`
- `PSA_ALG_IS_RSA_PKCS1V15_SIGN()`
- `PSA_ALG_RSA_PSS()`
- `PSA_ALG_TLS12_PRF()`
- `PSA_ALG_TLS12_PSK_TO_MS()`
- `PSA_ALG_VENDOR_TLS13()`

3.2.1.1.11.2 Return

The underlying hash algorithm if **alg** is a composite algorithm that uses a hash algorithm.

`PSA_ALG_NONE` if **alg** is not a composite algorithm that uses a hash.

3.2.1.1.12 macro `PSA_ALG_HKDF`

`PSA_ALG_HKDF`(hash_alg)

Macro to build an HKDF algorithm.

Parameters

- **hash_alg** – A hash algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_HASH(hash_alg)` is true).

3.2.1.1.12.1 Description

This is the HMAC-based Extract-and-Expand Key Derivation Function (HKDF) specified by HMAC-based Extract-and-Expand Key Derivation Function (HKDF) [RFC5869].

This key derivation algorithm uses the following inputs:

- `PSA_KEY_DERIVATION_INPUT_SALT` is the salt used in the “extract” step. It is optional; if omitted, the derivation uses an empty salt.
- `PSA_KEY_DERIVATION_INPUT_SECRET` is the secret key used in the “extract” step.
- `PSA_KEY_DERIVATION_INPUT_INFO` is the info string used in the “expand” step.

If `PSA_KEY_DERIVATION_INPUT_SALT` is provided, it must be before `PSA_KEY_DERIVATION_INPUT_SECRET`. `PSA_KEY_DERIVATION_INPUT_INFO` can be provided at any time after setup and before starting to generate output.

Each input may only be passed once.

3.2.1.1.12.2 Return

The corresponding HKDF algorithm. For example, `PSA_ALG_HKDF(PSA_ALG_SHA_256)` is HKDF using HMAC-SHA-256.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.13 macro `PSA_ALG_HKDF_EXTRACT`

`PSA_ALG_HKDF_EXTRACT(hash_alg)`

Macro to build an HKDF-Extract algorithm.

Parameters

- **`hash_alg`** – A hash algorithm: a value of type `psa_algorithm_t` such that `PSA_ALG_IS_HASH(hash_alg)` is true.

3.2.1.1.13.1 Description

This is the Extract step of HKDF as specified by HMAC-based Extract-and-Expand Key Derivation Function (HKDF) [RFC5869] §2.2.

This key derivation algorithm uses the following inputs:

- `PSA_KEY_DERIVATION_INPUT_SALT` is the salt.
- `PSA_KEY_DERIVATION_INPUT_SECRET` is the input keying material used in the “extract” step.

The inputs are mandatory and must be passed in the order above. Each input may only be passed once.

Warning

HKDF-Extract is not meant to be used on its own. `PSA_ALG_HKDF` should be used instead if possible. `PSA_ALG_HKDF_EXTRACT` is provided as a separate algorithm for the sake of protocols that use it as a building block. It may also be a slight performance optimization in applications that use HKDF with the same salt and key but many different info strings.

Warning

HKDF processes the salt as follows: first hash it with `hash_alg` if the salt is longer than the block size of the hash algorithm; then pad with null bytes up to the block size. As a result, it is possible for distinct salt inputs to result in the same outputs. To ensure unique outputs, it is recommended to use a fixed length for salt values.

Compatible key types:

- `PSA_KEY_TYPE_DERIVE` (for the input keying material)
- `PSA_KEY_TYPE_RAW_DATA` (for the salt)

3.2.1.1.13.2 Return

The corresponding HKDF-Extract algorithm. For example, `PSA_ALG_HKDF_EXTRACT(PSA_ALG_SHA_256)` is HKDF-Extract using HMAC-SHA-256.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.14 macro `PSA_ALG_HKDF_EXPAND`

`PSA_ALG_HKDF_EXPAND(hash_alg)`

Macro to build an HKDF-Expand algorithm.

Parameters

- **hash_alg** – A hash algorithm: a value of type `psa_algorithm_t` such that `PSA_ALG_IS_HASH(hash_alg)` is true.

3.2.1.1.14.1 Description

This is the Expand step of HKDF as specified by HMAC-based Extract-and-Expand Key Derivation Function (HKDF) [RFC5869] §2.3.

This key derivation algorithm uses the following inputs:

- `PSA_KEY_DERIVATION_INPUT_SECRET` is the pseudorandom key (PRK).
- `PSA_KEY_DERIVATION_INPUT_INFO` is the info string.

The inputs are mandatory and must be passed in the order above. Each input may only be passed once.

Warning

HKDF-Expand is not meant to be used on its own. `PSA_ALG_HKDF` should be used instead if possible. `PSA_ALG_HKDF_EXPAND` is provided as a separate algorithm for the sake of protocols that use it as a building block. It may also be a slight performance optimization in applications that use HKDF with the same salt and key but many different info strings.

Compatible key types:

- `PSA_KEY_TYPE_DERIVE` (for the pseudorandom key)
- `PSA_KEY_TYPE_RAW_DATA` (for the info string)

3.2.1.1.14.2 Return

The corresponding HKDF-Expand algorithm. For example, `PSA_ALG_HKDF_EXPAND(PSA_ALG_SHA_256)` is HKDF-Expand using HMAC-SHA-256.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.15 macro `PSA_ALG_VENDOR_TLS13`

`PSA_ALG_VENDOR_TLS13(hash_alg)`

Macro to build a TLS1.3 algorithm.

Parameters

- **hash_alg** – A hash algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_HASH(hash_alg)` is true).

3.2.1.1.15.1 Description

PSA_ALG_HKDF, as defined in the PSA specification, can be used either standalone (for, simply, HKDF derivation) or as part of a more complex key derivation scheme such as TLS 1.3. However, in particular the ELE subsystem may not support standalone HKDF, but may support TLS 1.3 via a separate internal API.

In order to support TLS1.3, this vendor algorithm is introduced and is mapped to the internal values used by ELE.

3.2.1.1.15.2 This vendor algorithm uses the following inputs

- **PSA_KEY_DERIVATION_INPUT_SECRET**: The pre-shared key identifier used for the early secrets. Optional. Use this step together with the `psa_key_derivation_input_key()` function.
- **PSA_KEY_DERIVATION_INPUT_OTHER_SECRET**: When deriving a TLS1.3 secret such as “c hs traffic” or “c ap traffic”, this is the identifier of the base key ID used for ECDH. In this case, you must use this step together with the `psa_key_derivation_key_agreement()` function. Otherwise, when deriving key material or an IV (“key”, “iv”, “finished”), it is the ID of the secret derived previously. Use this step together with the `psa_key_derivation_input_key()` function.
- **PSA_KEY_DERIVATION_INPUT_INFO**: The byte array that corresponds to the expanded label, as defined in RFC 8446. Use this step together with the `psa_key_derivation_input_bytes()` function.

The expanded label contains the TLS1.3 label, which corresponds to either a TLS1.3 secret (e.g. client handshake secret), a key (e.g. client application traffic key), or an IV (e.g. server handshake traffic IV). When deriving a secret or key, use the `psa_key_derivation_output_key()`, and when deriving an IV use `psa_key_derivation_output_bytes()`.

Thus, to derive e.g. an encryption key, without using a PSK, you need to follow this call sequence (simplified):

- `psa_key_derivation_setup(PSA_ALG_VENDOR_TLS13(PSA_ALG_SHA256))`
- `psa_key_derivation_key_agreement(PSA_KEY_DERIVATION_INPUT_OTHER_SECRET, private_ec_key_id, peer_public_key)`
- `psa_key_derivation_input_bytes(PSA_KEY_DERIVATION_INPUT_INFO, expanded_label)`
- `secret_id = psa_key_derivation_output_key()`
- `psa_key_derivation_setup(PSA_ALG_VENDOR_TLS13(PSA_ALG_SHA256))`
- `psa_key_derivation_input_key(PSA_KEY_DERIVATION_INPUT_OTHER_SECRET, secret_id)`
- `psa_key_derivation_input_bytes(PSA_KEY_DERIVATION_INPUT_INFO, expanded_label)`
- `encryption_key_id = psa_key_derivation_output_key()`

3.2.1.1.16 macro PSA_ALG_HMAC

PSA_ALG_HMAC(hash_alg)

Macro to build an HMAC message-authentication-code algorithm from an underlying hash algorithm.

Parameters

- **hash_alg** – A hash algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_HASH(hash_alg) is true).

3.2.1.1.16.1 Description

For example, PSA_ALG_HMAC(PSA_ALG_SHA_256) is HMAC-SHA-256.

The HMAC construction is defined in HMAC: Keyed-Hashing for Message Authentication [RFC2104].

3.2.1.1.16.2 Return

The corresponding HMAC algorithm.

Unspecified if hash_alg is not a supported hash algorithm.

3.2.1.1.17 macro PSA_ALG_IS_AEAD

PSA_ALG_IS_AEAD(alg)

Whether the specified algorithm is an authenticated encryption with associated data (AEAD) algorithm.

Parameters

- **alg** – An algorithm identifier (value of *typedef psa_algorithm_t*).

3.2.1.1.17.1 Return

1 if alg is an AEAD algorithm, 0 otherwise. This macro can return either 0 or 1 if alg is not a supported algorithm identifier.

3.2.1.1.18 macro PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER

PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER(alg)

Whether the specified algorithm is an AEAD mode on a block cipher.

Parameters

- **alg** – An algorithm identifier (value of *typedef psa_algorithm_t*).

3.2.1.1.18.1 Return

1 if alg is an AEAD algorithm which is an AEAD mode based on a block cipher, 0 otherwise.

This macro can return either 0 or 1 if alg is not a supported algorithm identifier.

3.2.1.1.19 macro PSA_ALG_IS_ASYMMETRIC_ENCRYPTION

PSA_ALG_IS_ASYMMETRIC_ENCRYPTION(alg)

Whether the specified algorithm is an asymmetric encryption algorithm, also known as public-key encryption algorithm.

Parameters

- **alg** – An algorithm identifier (value of *typedef psa_algorithm_t*).

3.2.1.1.19.1 Return

1 if `alg` is an asymmetric encryption algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.20 macro `PSA_ALG_IS_BLOCK_CIPHER_MAC`

`PSA_ALG_IS_BLOCK_CIPHER_MAC(alg)`

Whether the specified algorithm is a MAC algorithm based on a block cipher.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.20.1 Return

1 if `alg` is a MAC algorithm based on a block cipher, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.21 macro `PSA_ALG_IS_CIPHER`

`PSA_ALG_IS_CIPHER(alg)`

Whether the specified algorithm is a symmetric cipher algorithm.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.21.1 Return

1 if `alg` is a symmetric cipher algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.22 macro `PSA_ALG_IS_DETERMINISTIC_ECDSA`

`PSA_ALG_IS_DETERMINISTIC_ECDSA(alg)`

Whether the specified algorithm is deterministic ECDSA.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.22.1 Description

See also `PSA_ALG_IS_ECDSA()` and `PSA_ALG_IS_RANDOMIZED_ECDSA()`.

3.2.1.1.22.2 Return

1 if `alg` is a deterministic ECDSA algorithm, 0 otherwise.

This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.23 macro `PSA_ALG_IS_ECDH`

`PSA_ALG_IS_ECDH(alg)`

Whether the specified algorithm is an elliptic curve Diffie-Hellman algorithm.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.23.1 Description

This includes the raw elliptic curve Diffie-Hellman algorithm as well as elliptic curve Diffie-Hellman followed by any supporter key derivation algorithm.

3.2.1.1.23.2 Return

1 if `alg` is an elliptic curve Diffie-Hellman algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported key agreement algorithm identifier.

3.2.1.1.24 macro `PSA_ALG_IS_ECDSA`

`PSA_ALG_IS_ECDSA(alg)`

Whether the specified algorithm is ECDSA.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.24.1 Return

1 if `alg` is an ECDSA algorithm, 0 otherwise.

This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.25 macro `PSA_ALG_IS_FFDH`

`PSA_ALG_IS_FFDH(alg)`

Whether the specified algorithm is a finite field Diffie-Hellman algorithm.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.25.1 Description

This includes the raw finite field Diffie-Hellman algorithm as well as finite-field Diffie-Hellman followed by any supporter key derivation algorithm.

3.2.1.1.25.2 Return

1 if `alg` is a finite field Diffie-Hellman algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported key agreement algorithm identifier.

3.2.1.1.26 macro `PSA_ALG_IS_HASH`

`PSA_ALG_IS_HASH(alg)`

Whether the specified algorithm is a hash algorithm.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.26.1 Description

See Hash algorithms for a list of defined hash algorithms.

3.2.1.1.26.2 Return

1 if `alg` is a hash algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.27 macro `PSA_ALG_IS_HASH_AND_SIGN`

`PSA_ALG_IS_HASH_AND_SIGN(alg)`

Whether the specified algorithm is a hash-and-sign algorithm that signs exactly the hash value.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.27.1 Description

This macro identifies algorithms that can be used with `psa_sign_hash()` that use the exact message hash value as an input the signature operation. This excludes hash-and-sign algorithms that require a encoded or modified hash for the signature step in the algorithm, such as `PSA_ALG_RSA_PKCS1V15_SIGN_RAW`.

3.2.1.1.27.2 Return

1 if `alg` is a hash-and-sign algorithm that signs exactly the hash value, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.28 macro `PSA_ALG_IS_HASH_EDDSA`

`PSA_ALG_IS_HASH_EDDSA(alg)`

Whether the specified algorithm is HashEdDSA.

Parameters

- **alg** – An algorithm identifier: a value of `typedef psa_algorithm_t`.

3.2.1.1.28.1 Return

1 if `alg` is a HashEdDSA algorithm, 0 otherwise.

This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.29 macro `PSA_ALG_IS_HKDF`

`PSA_ALG_IS_HKDF`(alg)

Whether the specified algorithm is an HKDF algorithm.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.29.1 Description

HKDF is a family of key derivation algorithms that are based on a hash function and the HMAC construction.

3.2.1.1.29.2 Return

1 if **alg** is an HKDF algorithm, 0 otherwise. This macro can return either 0 or 1 if **alg** is not a supported key derivation algorithm identifier.

3.2.1.1.30 macro `PSA_ALG_IS_VENDOR_TLS13`

`PSA_ALG_IS_VENDOR_TLS13`(alg)

Whether the specified algorithm is a TLS1.3 algorithm.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.31 macro `PSA_ALG_IS_HKDF_EXTRACT`

`PSA_ALG_IS_HKDF_EXTRACT`(alg)

Whether the specified algorithm is an HKDF-Extract algorithm.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.31.1 Return

1 if **alg** is an HKDF-Extract algorithm, 0 otherwise. This macro can return either 0 or 1 if **alg** is not a supported key derivation algorithm identifier.

3.2.1.1.32 macro `PSA_ALG_IS_HKDF_EXPAND`

`PSA_ALG_IS_HKDF_EXPAND`(alg)

Whether the specified algorithm is an HKDF-Expand algorithm.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.32.1 Return

1 if `alg` is an HKDF-Expand algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

3.2.1.1.33 macro `PSA_ALG_IS_HMAC`

`PSA_ALG_IS_HMAC`(`alg`)

Whether the specified algorithm is an HMAC algorithm.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.33.1 Description

HMAC is a family of MAC algorithms that are based on a hash function.

3.2.1.1.33.2 Return

1 if `alg` is an HMAC algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.34 macro `PSA_ALG_IS_KEY_AGREEMENT`

`PSA_ALG_IS_KEY_AGREEMENT`(`alg`)

Whether the specified algorithm is a key agreement algorithm.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.34.1 Return

1 if `alg` is a key agreement algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.35 macro `PSA_ALG_IS_KEY_DERIVATION`

`PSA_ALG_IS_KEY_DERIVATION`(`alg`)

Whether the specified algorithm is a key derivation algorithm.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.35.1 Return

1 if `alg` is a key derivation algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.36 macro `PSA_ALG_IS_KEY_DERIVATION_STRETCHING`

`PSA_ALG_IS_KEY_DERIVATION_STRETCHING`(alg)

Whether the specified algorithm is a key-stretching or password-hashing algorithm.

Parameters

- **alg** – An algorithm identifier: a value of *typedef psa_algorithm_t*.

3.2.1.1.36.1 Description

A key-stretching or password-hashing algorithm is a key derivation algorithm that is suitable for use with a low-entropy secret such as a password. Equivalently, it's a key derivation algorithm that uses a `PSA_KEY_DERIVATION_INPUT_PASSWORD` input step.

3.2.1.1.36.2 Return

1 if **alg** is a key-stretching or password-hashing algorithm, 0 otherwise. This macro can return either 0 or 1 if **alg** is not a supported key derivation algorithm identifier.

3.2.1.1.37 macro `PSA_ALG_IS_MAC`

`PSA_ALG_IS_MAC`(alg)

Whether the specified algorithm is a MAC algorithm.

Parameters

- **alg** – An algorithm identifier (value of *typedef psa_algorithm_t*).

3.2.1.1.37.1 Return

1 if **alg** is a MAC algorithm, 0 otherwise.

3.2.1.1.38 macro `PSA_ALG_IS_MAC_TRUNCATED`

`PSA_ALG_IS_MAC_TRUNCATED`(alg)

Whether the specified algorithm is a MAC truncated algorithm.

Parameters

- **alg** – An algorithm identifier (value of *typedef psa_algorithm_t*).

3.2.1.1.38.1 Return

1 if **alg** is a MAC truncated algorithm, 0 otherwise.

3.2.1.1.39 macro `PSA_ALG_IS_PBKDF2_HMAC`

`PSA_ALG_IS_PBKDF2_HMAC`(alg)

Whether the specified algorithm is a PBKDF2-HMAC algorithm.

Parameters

- **alg** – An algorithm identifier: a value of *typedef psa_algorithm_t*.

3.2.1.1.39.1 Return

1 if `alg` is a PBKDF2-HMAC algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

3.2.1.1.40 macro `PSA_ALG_IS_RANDOMIZED_ECDSA`

`PSA_ALG_IS_RANDOMIZED_ECDSA(alg)`

Whether the specified algorithm is randomized ECDSA.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.40.1 Description

See also `PSA_ALG_IS_ECDSA()` and `PSA_ALG_IS_DETERMINISTIC_ECDSA()`.

3.2.1.1.40.2 Return

1 if `alg` is a randomized ECDSA algorithm, 0 otherwise.

This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.41 macro `PSA_ALG_IS_STANDALONE_KEY_AGREEMENT`

`PSA_ALG_IS_STANDALONE_KEY_AGREEMENT(alg)`

Whether the specified algorithm is a standalone key agreement algorithm.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.41.1 Description

A standalone key agreement algorithm is one that does not specify a key derivation function. Usually, standalone key agreement algorithms are constructed directly with a `PSA_ALG_xxx` macro while combined key agreement algorithms are constructed with `PSA_ALG_KEY_AGREEMENT()`.

The standalone key agreement algorithm can be extracted from a combined key agreement algorithm identifier using `PSA_ALG_KEY_AGREEMENT_GET_BASE()`.

3.2.1.1.41.2 Return

1 if `alg` is a standalone key agreement algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.42 macro `PSA_ALG_IS_RAW_KEY_AGREEMENT`

`PSA_ALG_IS_RAW_KEY_AGREEMENT(alg)`

Whether the specified algorithm is a raw key agreement algorithm.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.42.1 Description

This is the original API name for `PSA_ALG_IS_STANDALONE_KEY_AGREEMENT()`.

3.2.1.1.43 macro `PSA_ALG_IS_RSA_OAEP`

`PSA_ALG_IS_RSA_OAEP(alg)`

Whether the specified algorithm is an RSA OAEP encryption algorithm.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.43.1 Return

1 if `alg` is an RSA OAEP algorithm, 0 otherwise.

This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.44 macro `PSA_ALG_IS_RSA_PKCS1V15_SIGN`

`PSA_ALG_IS_RSA_PKCS1V15_SIGN(alg)`

Whether the specified algorithm is an RSA PKCS#1 v1.5 signature algorithm.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.44.1 Return

1 if `alg` is an RSA PKCS#1 v1.5 signature algorithm, 0 otherwise.

This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.45 macro `PSA_ALG_IS_RSA_PSS`

`PSA_ALG_IS_RSA_PSS(alg)`

Whether the specified algorithm is an RSA PSS signature algorithm.

Parameters

- **alg** – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.45.1 Return

1 if `alg` is an RSA PSS signature algorithm, 0 otherwise.

This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.46 macro `PSA_ALG_IS_RSA_PSS_ANY_SALT`

`PSA_ALG_IS_RSA_PSS_ANY_SALT(alg)`

Whether the specified algorithm is an RSA PSS signature algorithm that permits any salt length.

Parameters

- **alg** – An algorithm identifier: a value of `typedef psa_algorithm_t`.

3.2.1.1.46.1 Description

An RSA PSS signature algorithm that permits any salt length is constructed using `PSA_ALG_RSA_PSS_ANY_SALT()`. See also `PSA_ALG_IS_RSA_PSS()` and `PSA_ALG_IS_RSA_PSS_STANDARD_SALT()`.

3.2.1.1.46.2 Return

1 if `alg` is an RSA PSS signature algorithm that permits any salt length, 0 otherwise.

This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.47 macro `PSA_ALG_IS_RSA_PSS_STANDARD_SALT`

`PSA_ALG_IS_RSA_PSS_STANDARD_SALT(alg)`

Whether the specified algorithm is an RSA PSS signature algorithm that requires the standard salt length.

Parameters

- `alg` – An algorithm identifier: a value of `typedef psa_algorithm_t`.

3.2.1.1.47.1 Description

An RSA PSS signature algorithm that requires the standard salt length is constructed using `PSA_ALG_RSA_PSS()`.

See also `PSA_ALG_IS_RSA_PSS()` and `PSA_ALG_IS_RSA_PSS_ANY_SALT()`.

3.2.1.1.47.2 Return

1 if `alg` is an RSA PSS signature algorithm that requires the standard salt length, 0 otherwise.

This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.48 macro `PSA_ALG_IS_SIGN`

`PSA_ALG_IS_SIGN(alg)`

Whether the specified algorithm is an asymmetric signature algorithm, also known as public-key signature algorithm.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.48.1 Return

1 if `alg` is an asymmetric signature algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.49 macro `PSA_ALG_IS_SIGN_HASH`

`PSA_ALG_IS_SIGN_HASH(alg)`

Whether the specified algorithm is a signature algorithm that can be used with `psa_sign_hash()` and `psa_verify_hash()`.

Parameters

- **alg** – An algorithm identifier (value of *typedef psa_algorithm_t*).

3.2.1.1.49.1 Return

1 if **alg** is a signature algorithm that can be used to sign a hash. 0 if **alg** is a signature algorithm that can only be used to sign a message. 0 if **alg** is not a signature algorithm. This macro can return either 0 or 1 if **alg** is not a supported algorithm identifier.

3.2.1.1.50 macro PSA_ALG_IS_SIGN_MESSAGE

PSA_ALG_IS_SIGN_MESSAGE(alg)

Whether the specified algorithm is a signature algorithm that can be used with *psa_sign_message()* and *psa_verify_message()*.

Parameters

- **alg** – An algorithm identifier (value of *typedef psa_algorithm_t*).

3.2.1.1.50.1 Return

1 if **alg** is a signature algorithm that can be used to sign a message. 0 if **alg** is a signature algorithm that can only be used to sign an already-calculated hash. 0 if **alg** is not a signature algorithm. This macro can return either 0 or 1 if **alg** is not a supported algorithm identifier.

3.2.1.1.51 macro PSA_ALG_IS_SP800_108_COUNTER_HMAC

PSA_ALG_IS_SP800_108_COUNTER_HMAC(alg)

Whether the specified algorithm is a key derivation algorithm constructed using *PSA_ALG_SP800_108_COUNTER_HMAC*(hash_alg).

Parameters

- **alg** – An algorithm identifier (value of *typedef psa_algorithm_t*).

3.2.1.1.51.1 Return

1 if **alg** is a key derivation algorithm constructed using *PSA_ALG_SP800_108_COUNTER_HMAC()*, 0 otherwise. This macro can return either 0 or 1 if **alg** is not a supported key derivation algorithm identifier.

3.2.1.1.52 macro PSA_ALG_IS_STREAM_CIPHER

PSA_ALG_IS_STREAM_CIPHER(alg)

Whether the specified algorithm is a stream cipher.

Parameters

- **alg** – An algorithm identifier (value of *typedef psa_algorithm_t*).

3.2.1.1.52.1 Description

A stream cipher is a symmetric cipher that encrypts or decrypts messages by applying a bitwise-xor with a stream of bytes that is generated from a key.

3.2.1.1.52.2 Return

1 if `alg` is a stream cipher algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier or if it is not a symmetric cipher algorithm.

3.2.1.1.53 macro `PSA_ALG_IS_TLS12_PRF`

`PSA_ALG_IS_TLS12_PRF(alg)`

Whether the specified algorithm is a TLS-1.2 PRF algorithm.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.53.1 Return

1 if `alg` is a TLS-1.2 PRF algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

3.2.1.1.54 macro `PSA_ALG_IS_TLS12_PSK_TO_MS`

`PSA_ALG_IS_TLS12_PSK_TO_MS(alg)`

Whether the specified algorithm is a TLS-1.2 PSK to MS algorithm.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.54.1 Return

1 if `alg` is a TLS-1.2 PSK to MS algorithm, 0 otherwise. This macro can return either 0 or 1 if `alg` is not a supported key derivation algorithm identifier.

3.2.1.1.55 macro `PSA_ALG_IS_WILDCARD`

`PSA_ALG_IS_WILDCARD(alg)`

Whether the specified algorithm encoding is a wildcard.

Parameters

- `alg` – An algorithm identifier (value of `typedef psa_algorithm_t`).

3.2.1.1.55.1 Description

Wildcard algorithm values can only be used to set the permitted algorithm field in a key policy, wildcard values cannot be used to perform an operation.

See `PSA_ALG_ANY_HASH` for example of how a wildcard algorithm can be used in a key policy.

3.2.1.1.55.2 Return

1 if `alg` is a wildcard algorithm encoding.

0 if `alg` is a non-wildcard algorithm encoding that is suitable for an operation.

This macro can return either 0 or 1 if `alg` is not a supported algorithm identifier.

3.2.1.1.56 macro `PSA_ALG_KEY_AGREEMENT`

`PSA_ALG_KEY_AGREEMENT(ka_alg, kdf_alg)`

Macro to build a combined algorithm that chains a key agreement with a key derivation.

Parameters

- **ka_alg** – A key agreement algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_KEY_AGREEMENT(ka_alg)` is true).
- **kdf_alg** – A key derivation algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_KEY_DERIVATION(kdf_alg)` is true).

3.2.1.1.56.1 Description

A combined key agreement algorithm is used with a multi-part key derivation operation, using a call to `psa_key_derivation_key_agreement()`.

The component parts of a key agreement algorithm can be extracted using `PSA_ALG_KEY_AGREEMENT_GET_BASE()` and `PSA_ALG_KEY_AGREEMENT_GET_KDF()`.

3.2.1.1.56.2 Return

The corresponding key agreement and derivation algorithm.

Unspecified if `ka_alg` is not a supported key agreement algorithm or `kdf_alg` is not a supported key derivation algorithm.

3.2.1.1.57 macro `PSA_ALG_KEY_AGREEMENT_GET_BASE`

`PSA_ALG_KEY_AGREEMENT_GET_BASE(alg)`

Get the raw key agreement algorithm from a full key agreement algorithm.

Parameters

- **alg** – A key agreement algorithm identifier (value of `typedef psa_algorithm_t` such that `PSA_ALG_IS_KEY_AGREEMENT(alg)` is true).

3.2.1.1.57.1 Description

See also `PSA_ALG_KEY_AGREEMENT()` and `PSA_ALG_KEY_AGREEMENT_GET_KDF()`.

3.2.1.1.57.2 Return

The underlying raw key agreement algorithm if `alg` is a key agreement algorithm.

Unspecified if `alg` is not a key agreement algorithm or if it is not supported by the implementation.

3.2.1.1.58 macro `PSA_ALG_KEY_AGREEMENT_GET_KDF`

`PSA_ALG_KEY_AGREEMENT_GET_KDF(alg)`

Get the key derivation algorithm used in a full key agreement algorithm.

Parameters

- **alg** – A key agreement algorithm identifier (value of `typedef psa_algorithm_t` such that `PSA_ALG_IS_KEY_AGREEMENT(alg)` is true).

3.2.1.1.58.1 Description

See also `PSA_ALG_KEY_AGREEMENT()` and `PSA_ALG_KEY_AGREEMENT_GET_BASE()`.

3.2.1.1.58.2 Return

The underlying key derivation algorithm if `alg` is a key agreement algorithm.

Unspecified if `alg` is not a key agreement algorithm or if it is not supported by the implementation.

3.2.1.1.59 macro `PSA_ALG_PBKDF2_HMAC`

`PSA_ALG_PBKDF2_HMAC(hash_alg)`

Macro to build a PBKDF2-HMAC password-hashing or key-stretching algorithm.

Parameters

- **hash_alg** – A hash algorithm: a value of `typedef psa_algorithm_t` such that `PSA_ALG_IS_HASH(hash_alg)` is true.

3.2.1.1.59.1 Description

PBKDF2 is specified by PKCS #5: Password-Based Cryptography Specification Version 2.1 [RFC8018] §5.2. This macro constructs a PBKDF2 algorithm that uses a pseudo-random function based on HMAC with the specified hash. This key derivation algorithm uses the following inputs, which must be provided in the following order:

- `PSA_KEY_DERIVATION_INPUT_COST` is the iteration count. This input step must be used exactly once.
- `PSA_KEY_DERIVATION_INPUT_SALT` is the salt. This input step must be used one or more times; if used several times, the inputs will be concatenated. This can be used to build the final salt from multiple sources, both public and secret (also known as pepper).
- `PSA_KEY_DERIVATION_INPUT_PASSWORD` is the password to be hashed. This input step must be used exactly once.

Compatible key types:

- `PSA_KEY_TYPE_DERIVE` (for password input)
- `PSA_KEY_TYPE_PASSWORD` (for password input)
- `PSA_KEY_TYPE_PEPPER` (for salt input)
- `PSA_KEY_TYPE_RAW_DATA` (for salt input)

- `PSA_KEY_TYPE_PASSWORD_HASH` (for key verification)

3.2.1.1.59.2 Return

The corresponding PBKDF2-HMAC-XXX algorithm. For example, `PSA_ALG_PBKDF2_HMAC(PSA_ALG_SHA_256)` is the algorithm identifier for PBKDF2-HMAC-SHA-256.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.60 macro `PSA_ALG_RSA_OAEP`

`PSA_ALG_RSA_OAEP(hash_alg)`

The RSA OAEP asymmetric encryption algorithm.

Parameters

- **`hash_alg`** – The hash algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_HASH(hash_alg)` is true) to use for MGF1.

3.2.1.1.60.1 Description

This encryption scheme is defined by [RFC8017] §7.1 under the name RSAES-OAEP, with the mask generation function MGF1 defined in [RFC8017] Appendix B.

3.2.1.1.60.2 Return

The corresponding RSA OAEP encryption algorithm.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.61 macro `PSA_ALG_RSA_PKCS1V15_SIGN`

`PSA_ALG_RSA_PKCS1V15_SIGN(hash_alg)`

The RSA PKCS#1 v1.5 message signature scheme, with hashing.

Parameters

- **`hash_alg`** – A hash algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_HASH(hash_alg)` is true). This includes `PSA_ALG_ANY_HASH` when specifying the algorithm in a key policy.

3.2.1.1.61.1 Description

This algorithm can be used with both the message and hash signature functions.

This signature scheme is defined by PKCS #1: RSA Cryptography Specifications Version 2.2 [RFC8017] §8.2 under the name RSASSA-PKCS1-v1_5.

When used with `psa_sign_hash()` or `psa_verify_hash()`, the provided hash parameter is used as H from step 2 onwards in the message encoding algorithm EMSA-PKCS1-V1_5-ENCODE() in [RFC8017] §9.2. H is usually the message digest, using the `hash_alg` hash algorithm.

3.2.1.1.61.2 Return

The corresponding RSA PKCS#1 v1.5 signature algorithm.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.62 macro `PSA_ALG_RSA_PSS`

`PSA_ALG_RSA_PSS(hash_alg)`

The RSA PSS message signature scheme, with hashing.

Parameters

- **`hash_alg`** – A hash algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_HASH(hash_alg)` is true). This includes `PSA_ALG_ANY_HASH` when specifying the algorithm in a key policy.

3.2.1.1.62.1 Description

This algorithm can be used with both the message and hash signature functions.

This algorithm is randomized: each invocation returns a different, equally valid signature.

This is the signature scheme defined by [RFC8017] §8.1 under the name RSASSA-PSS, with the following options:

- The mask generation function is MGF1 defined by [RFC8017] Appendix B.
- The salt length is equal to the length of the hash.
- The specified hash algorithm is used to hash the input message, to create the salted hash, and for the mask generation.

3.2.1.1.62.2 Return

The corresponding RSA PSS signature algorithm.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.63 macro `PSA_ALG_RSA_PSS_ANY_SALT`

`PSA_ALG_RSA_PSS_ANY_SALT(hash_alg)`

The RSA PSS message signature scheme, with hashing. This variant permits any salt length for signature verification.

Parameters

- **`hash_alg`** – A hash algorithm: a value of `typedef psa_algorithm_t` such that `PSA_ALG_IS_HASH(hash_alg)` is true. This includes `PSA_ALG_ANY_HASH` when specifying the algorithm in a key policy.

3.2.1.1.63.1 Description

This algorithm can be used with both the message and hash signature functions.

This algorithm is randomized: each invocation returns a different, equally valid signature.

This is the signature scheme defined by [RFC8017] §8.1 under the name RSASSA-PSS, with the following options:

- The mask generation function is MGF1 defined by [RFC8017] Appendix B.
- When creating a signature, the salt length is equal to the length of the hash, or the largest possible salt length for the algorithm and key size if that is smaller than the hash length.
- When verifying a signature, any salt length permitted by the RSASSA-PSS signature algorithm is accepted.
- The specified hash algorithm is used to hash the input message, to create the salted hash, and for the mask generation.

Note:

The `PSA_ALG_RSA_PSS()` algorithm is equivalent to `PSA_ALG_RSA_PSS_ANY_SALT()` when creating a signature, but is strict about the permitted salt length when verifying a signature.

Compatible key types:

- `PSA_KEY_TYPE_RSA_KEY_PAIR`
- `PSA_KEY_TYPE_RSA_PUBLIC_KEY` (signature verification only)

3.2.1.1.63.2 Return

The corresponding RSA PSS signature algorithm.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.64 macro `PSA_ALG_SP800_108_COUNTER_HMAC`

`PSA_ALG_SP800_108_COUNTER_HMAC(hash_alg)`

Macro to build a NIST SP 800-108 conformant, counter-mode KDF algorithm based on HMAC.

Parameters

- **hash_alg** – A hash algorithm: a value of type `psa_algorithm_t` such that `PSA_ALG_IS_HASH(hash_alg)` is true.

3.2.1.1.64.1 Description

This is an HMAC-based, counter mode key derivation function, using the construction recommended by NIST Special Publication 800-108r1: Recommendation for Key Derivation Using Pseudorandom Functions [SP800-108], §4.1.

This key derivation algorithm uses the following inputs:

- `PSA_KEY_DERIVATION_INPUT_SECRET` is the secret input keying material, **Kin**.
- `PSA_KEY_DERIVATION_INPUT_LABEL` is the **Label**. It is optional; if omitted, **Label** is a zero-length string. If provided, it must not contain any null bytes.
- `PSA_KEY_DERIVATION_INPUT_CONTEXT` is the **Context**. It is optional; if omitted, **Context** is a zero-length string.

Each input can only be passed once. Inputs must be passed in the order above.

This algorithm uses the output length as part of the derivation process. In the derivation this value is **L**, the required output size in bits. After setup, the initial capacity of the key derivation

operation is $2^{29} - 1$ bytes (0x1fffffff). The capacity can be set to a lower value by calling `psa_key_derivation_set_capacity()`.

When the first output is requested, the value of **L** is calculated as $L = 8 * \text{cap}$, where **cap** is the value of `psa_key_derivation_get_capacity()`. Subsequent calls to `psa_key_derivation_set_capacity()` are not permitted for this algorithm.

The derivation is constructed as described in [SP800-108] §4.1, with the iteration counter **i** and output length **L** encoded as big-endian, 32-bit values. The resulting output stream $K1 \parallel K2 \parallel K3 \parallel \dots$ is computed as:

$K_i = \text{HMAC}(K_{in}, [i]_4 \parallel \text{Label} \parallel 0x00 \parallel \text{Context} \parallel [L]_4)$, for $i=1,2,3,\dots$

Where $[x]_n$ is the big-endian, n-byte encoding of the integer x .

Compatible key types:

- `PSA_KEY_TYPE_HMAC` (for the secret key)
- `PSA_KEY_TYPE_DERIVE` (for the secret key)
- `PSA_KEY_TYPE_RAW_DATA` (for the other inputs)

3.2.1.1.64.2 Return

The corresponding key derivation algorithm. For example, the counter-mode KDF using HMAC-SHA-256 is `PSA_ALG_SP800_108_COUNTER_HMAC(PSA_ALG_SHA_256)`.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.65 macro `PSA_ALG_TLS12_PRF`

`PSA_ALG_TLS12_PRF(hash_alg)`

Macro to build a TLS-1.2 PRF algorithm.

Parameters

- **hash_alg** – A hash algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_HASH(hash_alg)` is true).

3.2.1.1.65.1 Description

TLS 1.2 uses a custom pseudorandom function (PRF) for key schedule, specified in The Transport Layer Security (TLS) Protocol Version 1.2 [RFC5246] §5. It is based on HMAC and can be used with either SHA-256 or SHA-384.

This key derivation algorithm uses the following inputs, which must be passed in the order given here:

- `PSA_KEY_DERIVATION_INPUT_SEED` is the seed.
- either `PSA_KEY_DERIVATION_INPUT_SECRET`, which is the secret key,
- or `PSA_KEY_DERIVATION_INPUT_OTHER_SECRET`, which is the output of `psa_key_derivation_key_agreement()`.
- `PSA_KEY_DERIVATION_INPUT_LABEL` is the label.

Each input may only be passed once.

For the application to TLS-1.2 master secret:

- The seed is the concatenation of ClientHello.Random + ServerHello.Random.
- The label is “master secret”.

For the application to TLS-1.2 extended master secret:

- The seed is the Session Hash.
- The label is “extended master secret”.

For the application to TLS-1.2 key expansion:

- The seed is the concatenation of ServerHello.Random + ClientHello.Random.
- The label is “key expansion”.

3.2.1.1.65.2 Return

The corresponding TLS-1.2 PRF algorithm. For example, PSA_ALG_TLS12_PRF(PSA_ALG_SHA_256) represents the TLS 1.2 PRF using HMAC-SHA-256.

Unspecified if hash_alg is not a supported hash algorithm.

3.2.1.1.66 macro PSA_ALG_TLS12_PSK_TO_MS

PSA_ALG_TLS12_PSK_TO_MS(hash_alg)

Macro to build a TLS-1.2 PSK-to-MasterSecret algorithm.

Parameters

- **hash_alg** – A hash algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_HASH(hash_alg) is true).

3.2.1.1.66.1 Description

In a pure-PSK handshake in TLS 1.2, the master secret (MS) is derived from the pre-shared key (PSK) through the application of padding (Pre-Shared Key Ciphersuites for Transport Layer Security (TLS) [RFC4279] §2) and the TLS-1.2 PRF (The Transport Layer Security (TLS) Protocol Version 1.2 [RFC5246] §5). The latter is based on HMAC and can be used with either SHA-256 or SHA-384.

This key derivation algorithm uses the following inputs, which must be passed in the order given here:

- PSA_KEY_DERIVATION_INPUT_SEED is the seed.
- PSA_KEY_DERIVATION_INPUT_SECRET is the PSK. The PSK must not be larger than PSA_TLS12_PSK_TO_MS_PSK_MAX_SIZE.
- PSA_KEY_DERIVATION_INPUT_LABEL is the label.

Each input may only be passed once.

For the application to TLS-1.2:

- The seed, which is forwarded to the TLS-1.2 PRF, is the concatenation of the ClientHello.Random + ServerHello.Random.
- The label is “master secret” or “extended master secret”.

3.2.1.1.66.2 Return

The corresponding TLS-1.2 PSK to MS algorithm. For example, `PSA_ALG_TLS12_PSK_TO_MS(PSA_ALG_SHA_256)` represents the TLS-1.2 PSK to MasterSecret derivation PRF using HMAC-SHA-256.

Unspecified if `hash_alg` is not a supported hash algorithm.

3.2.1.1.67 macro `PSA_ALG_TRUNCATED_MAC`

`PSA_ALG_TRUNCATED_MAC(mac_alg, mac_length)`

Macro to build a truncated MAC algorithm.

Parameters

- **mac_alg** – A MAC algorithm identifier (value of *typedef* `psa_algorithm_t` such that `PSA_ALG_IS_MAC(mac_alg)` is true). This can be a truncated or untruncated MAC algorithm.
- **mac_length** – Desired length of the truncated MAC in bytes. This must be at most the full length of the MAC and must be at least an implementation-specified minimum. The implementation-specified minimum must not be zero.

3.2.1.1.67.1 Description

A truncated MAC algorithm is identical to the corresponding MAC algorithm except that the MAC value for the truncated algorithm consists of only the first `mac_length` bytes of the MAC value for the untruncated algorithm.

Note:

This macro might allow constructing algorithm identifiers that are not valid, either because the specified length is larger than the untruncated MAC or because the specified length is smaller than permitted by the implementation.

Note:

It is implementation-defined whether a truncated MAC that is truncated to the same length as the MAC of the untruncated algorithm is considered identical to the untruncated algorithm for policy comparison purposes.

The full-length MAC algorithm can be recovered using `PSA_ALG_FULL_LENGTH_MAC()`.

3.2.1.1.67.2 Return

The corresponding MAC algorithm with the specified length.

Unspecified if `mac_alg` is not a supported MAC algorithm or if `mac_length` is too small or too large for the specified MAC algorithm.

3.2.1.1.68 macro `PSA_BLOCK_CIPHER_BLOCK_LENGTH`

`PSA_BLOCK_CIPHER_BLOCK_LENGTH(type)`

The block size of a block cipher.

Parameters

-
- **type** – A cipher key type (value of `typedef psa_key_type_t`).

3.2.1.1.68.1 Description

Note:

It is possible to build stream cipher algorithms on top of a block cipher, for example CTR mode (PSA_ALG_CTR). This macro only takes the key type into account, so it cannot be used to determine the size of the data that `psa_cipher_update()` might buffer for future processing in general.

Note:

This macro expression is a compile-time constant if `type` is a compile-time constant.

Warning:

This macro is permitted to evaluate its argument multiple times.

See also PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE.

3.2.1.1.68.2 Return

The block size for a block cipher, or 1 for a stream cipher. The return value is undefined if `type` is not a supported cipher key type.

3.2.1.1.69 PSA_DH_FAMILY_RFC7919

Finite-field Diffie-Hellman groups defined for TLS in RFC 7919.

This family includes groups with the following key sizes (in bits): 2048, 3072, 4096, 6144, 8192. An implementation can support all of these sizes or only a subset.

Keys in this group can only be used with the PSA_ALG_FFDH key agreement algorithm.

These groups are defined by Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for Transport Layer Security (TLS) [RFC7919] Appendix A.

3.2.1.1.70 PSA_ECC_FAMILY_BRAINPOOL_P_R1

Brainpool P random curves.

This family comprises the following curves:

- brainpoolP160r1 : key_bits = 160 (Deprecated)
- brainpoolP192r1 : key_bits = 192
- brainpoolP224r1 : key_bits = 224
- brainpoolP256r1 : key_bits = 256
- brainpoolP320r1 : key_bits = 320
- brainpoolP384r1 : key_bits = 384
- brainpoolP512r1 : key_bits = 512

They are defined in Elliptic Curve Cryptography (ECC) Brainpool Standard Curves and Curve Generation [RFC5639].

Warning

The 160-bit curve brainpoolP160r1 is weak and deprecated and is only recommended for use in legacy protocols.

3.2.1.1.71 PSA_ECC_FAMILY_FRP

Curve used primarily in France and elsewhere in Europe.

This family comprises one 256-bit curve:

- FRP256v1 : key_bits = 256

This is defined by Publication d'un paramétrage de courbe elliptique visant des applications de passeport électronique et de l'administration électronique française [FRP].

3.2.1.1.72 PSA_ECC_FAMILY_MONTGOMERY

Montgomery curves.

This family comprises the following Montgomery curves:

- Curve25519 : key_bits = 255
- Curve448 : key_bits = 448

Keys in this family can only be used with the PSA_ALG_ECDH key agreement algorithm.

Curve25519 is defined in Curve25519: new Diffie-Hellman speed records [Curve25519]. Curve448 is defined in Ed448-Goldilocks, a new elliptic curve [Curve448].

3.2.1.1.73 PSA_ECC_FAMILY_SECP_K1

SEC Koblitz curves over prime fields.

This family comprises the following curves:

- secp192k1 : key_bits = 192
- secp224k1 : key_bits = 225
- secp256k1 : key_bits = 256

They are defined in SEC 2: Recommended Elliptic Curve Domain Parameters [SEC2].

3.2.1.1.74 PSA_ECC_FAMILY_SECP_R1

SEC random curves over prime fields.

This family comprises the following curves:

- secp192r1 : key_bits = 192
- secp224r1 : key_bits = 224
- secp256r1 : key_bits = 256
- secp384r1 : key_bits = 384
- secp521r1 : key_bits = 521

They are defined in [SEC2]

3.2.1.1.75 PSA_ECC_FAMILY_SECP_R2

Warning:

This family of curves is weak and deprecated.

This family comprises the following curves:

- secp160r2 : key_bits = 160 (Deprecated)

It is defined in the superseded SEC 2: Recommended Elliptic Curve Domain Parameters, Version 1.0 [SEC2v1].

3.2.1.1.76 PSA_ECC_FAMILY_SECT_K1

SEC Koblitz curves over binary fields.

This family comprises the following curves:

- sect163k1 : key_bits = 163 (Deprecated)
- sect233k1 : key_bits = 233
- sect239k1 : key_bits = 239
- sect283k1 : key_bits = 283
- sect409k1 : key_bits = 409
- sect571k1 : key_bits = 571

They are defined in [SEC2].

Warning:

The 163-bit curve sect163k1 is weak and deprecated and is only recommended for use in legacy protocols.

3.2.1.1.77 PSA_ECC_FAMILY_SECT_R1

SEC random curves over binary fields.

This family comprises the following curves:

- sect163r1 : key_bits = 163 (Deprecated)
- sect233r1 : key_bits = 233
- sect283r1 : key_bits = 283
- sect409r1 : key_bits = 409
- sect571r1 : key_bits = 571

They are defined in [SEC2].

Warning:

The 163-bit curve sect163r1 is weak and deprecated and is only recommended for use in legacy protocols.

3.2.1.1.78 PSA_ECC_FAMILY_SECT_R2

SEC additional random curves over binary fields.

This family comprises the following curves:

- sect163r2 : key_bits = 163 (Deprecated)

It is defined in [SEC2].

Warning:

The 163-bit curve sect163r2 is weak and deprecated and is only recommended for use in legacy protocols.

3.2.1.1.79 PSA_ECC_FAMILY_TWISTED_EDWARDS

Twisted Edwards curves.

This family comprises the following twisted Edwards curves:

- Edwards25519 : key_bits = 255. This curve is birationally equivalent to Curve25519.
- Edwards448 : key_bits = 448. This curve is birationally equivalent to Curve448.

Edwards25519 is defined in Twisted Edwards curves [Ed25519]. Edwards448 is defined in Ed448-Goldilocks, a new elliptic curve [Curve448].

Compatible algorithms:

- PSA_ALG_PURE_EDDSA
- PSA_ALG_ED25519PH (Edwards25519 only)
- PSA_ALG_ED448PH (Edwards448 only)

3.2.1.1.80 PSA_KEY_DERIVATION_INPUT_CONTEXT

A context for key derivation.

Warning: Not supported

This is typically a direct input. It can also be a key of type PSA_KEY_TYPE_RAW_DATA.

3.2.1.1.81 PSA_KEY_DERIVATION_INPUT_COST

A cost parameter for password hashing or key stretching.

Warning: Not supported

This must be a direct input, passed to *psa_key_derivation_input_integer()*.

3.2.1.1.82 PSA_KEY_DERIVATION_INPUT_INFO

An information string for key derivation.

This is typically a direct input. It can also be a key of type PSA_KEY_TYPE_RAW_DATA.

3.2.1.1.83 PSA_KEY_DERIVATION_INPUT_LABEL

A label for key derivation.

This is typically a direct input. It can also be a key of type `PSA_KEY_TYPE_RAW_DATA`.

3.2.1.1.84 PSA_KEY_DERIVATION_INPUT_OTHER_SECRET

A high-entropy additional secret input for key derivation.

This is typically the shared secret resulting from a key agreement obtained via `psa_key_derivation_key_agreement()`. It may alternatively be a key of type `PSA_KEY_TYPE_DERIVE` passed to `psa_key_derivation_input_key()`, or a direct input passed to `psa_key_derivation_input_bytes()`.

3.2.1.1.85 PSA_KEY_DERIVATION_INPUT_PASSWORD

A low-entropy secret input for password hashing or key stretching.

Warning: Not supported

This is usually a key of type `PSA_KEY_TYPE_PASSWORD` passed to `psa_key_derivation_input_key()` or a direct input passed to `psa_key_derivation_input_bytes()` that is a password or passphrase. It can also be high-entropy secret, for example, a key of type `PSA_KEY_TYPE_DERIVE`, or the shared secret resulting from a key agreement.

If the secret is a direct input, the derivation operation cannot be used to derive keys: the operation will not allow a call to `psa_key_derivation_output_key()`.

3.2.1.1.86 PSA_KEY_DERIVATION_INPUT_SALT

A salt for key derivation.

This is typically a direct input. It can also be a key of type `PSA_KEY_TYPE_RAW_DATA`.

3.2.1.1.87 PSA_KEY_DERIVATION_INPUT_SECRET

A secret input for key derivation.

This is typically a key of type `PSA_KEY_TYPE_DERIVE` passed to `psa_key_derivation_input_key()`, or the shared secret resulting from a key agreement obtained via `psa_key_derivation_key_agreement()`.

The secret can also be a direct input passed to `psa_key_derivation_input_bytes()`. In this case, the derivation operation cannot be used to derive keys: the operation will only allow `psa_key_derivation_output_bytes()`, not `psa_key_derivation_output_key()`.

3.2.1.1.88 PSA_KEY_DERIVATION_INPUT_SEED

A seed for key derivation.

This is typically a direct input. It can also be a key of type `PSA_KEY_TYPE_RAW_DATA`.

3.2.1.1.89 PSA_KEY_ID_NULL

The null key identifier.

The null key identifier is always invalid, except when used without in a call to `psa_destroy_key()` which will return `PSA_SUCCESS`.

3.2.1.1.90 PSA_KEY_ID_USER_MAX

The maximum value for a key identifier chosen by the application.

3.2.1.1.91 PSA_KEY_ID_USER_MIN

The minimum value for a key identifier chosen by the application.

3.2.1.1.92 PSA_KEY_ID_VENDOR_MAX

The maximum value for a key identifier chosen by the implementation.

3.2.1.1.93 PSA_KEY_ID_VENDOR_MIN

The minimum value for a key identifier chosen by the implementation.

3.2.1.1.94 macro PSA_KEY_LIFETIME_FROM_PERSISTENCE_AND_LOCATION

`PSA_KEY_LIFETIME_FROM_PERSISTENCE_AND_LOCATION(persistence, location)`

Construct a lifetime from a persistence level and a location.

Parameters

- **persistence** – The persistence level (value of `typedef psa_key_persistence_t`).
- **location** – The location indicator (value of `typedef psa_key_location_t`).

3.2.1.1.94.1 Return

The constructed lifetime value.

3.2.1.1.95 macro PSA_KEY_LIFETIME_GET_LOCATION

`PSA_KEY_LIFETIME_GET_LOCATION(lifetime)`

Extract the location indicator from a key lifetime.

Parameters

- **lifetime** – The lifetime value to query (value of `typedef psa_key_lifetime_t`).

3.2.1.1.96 macro `PSA_KEY_LIFETIME_GET_PERSISTENCE`

`PSA_KEY_LIFETIME_GET_PERSISTENCE`(lifetime)

Extract the persistence level from a key lifetime.

Parameters

- **lifetime** – The lifetime value to query (value of *typedef* `psa_key_lifetime_t`).

3.2.1.1.97 macro `PSA_KEY_LIFETIME_IS_VOLATILE`

`PSA_KEY_LIFETIME_IS_VOLATILE`(lifetime)

Whether a key lifetime indicates that the key is volatile.

Parameters

- **lifetime** – The lifetime value to query (value of *typedef* `psa_key_lifetime_t`).

3.2.1.1.97.1 Description

A volatile key is automatically destroyed by the implementation when the application instance terminates. In particular, a volatile key is automatically destroyed on a power reset of the device.

A key that is not volatile is persistent. Persistent keys are preserved until the application explicitly destroys them or until an implementation-specific device management event occurs, for example, a factory reset.

3.2.1.1.97.2 Return

1 if the key is volatile, otherwise 0.

3.2.1.1.98 `PSA_KEY_LIFETIME_PERSISTENT`

The default lifetime for persistent keys.

A persistent key remains in storage until it is explicitly destroyed or until the corresponding storage area is wiped. This specification does not define any mechanism to wipe a storage area. Implementations are permitted to provide their own mechanism, for example, to perform a factory reset, to prepare for device refurbishment, or to uninstall an application.

This lifetime value is the default storage area for the calling application. Implementations can offer other storage areas designated by other lifetime values as implementation-specific extensions.

3.2.1.1.99 `PSA_KEY_LIFETIME_VOLATILE`

The default lifetime for volatile keys.

A volatile key only exists as long as its identifier is not destroyed. The key material is guaranteed to be erased on a power reset.

A key with this lifetime is typically stored in the RAM area of the PSA Crypto subsystem. However this is an implementation choice. If an implementation stores data about the key in a non-volatile memory, it must release all the resources associated with the key and erase the key material if the calling application terminates.

3.2.1.1.100 PSA_KEY_LOCATION_LOCAL_STORAGE

The local storage area for persistent keys.

This storage area is available on all systems that can store persistent keys without delegating the storage to a third-party cryptoprocessor.

See `typedef psa_key_location_t` for more information.

3.2.1.1.101 PSA_KEY_LOCATION_PRIMARY_SECURE_ELEMENT

The default secure element storage area for persistent keys.

This storage location is available on systems that have one or more secure elements that are able to store keys.

Vendor-defined locations must be provided by the system for storing keys in additional secure elements.

See `typedef psa_key_location_t` for more information.

3.2.1.1.102 PSA_KEY_PERSISTENCE_DEFAULT

The default persistence level for persistent keys.

See `typedef psa_key_persistence_t` for more information.

3.2.1.1.103 PSA_KEY_PERSISTENCE_READ_ONLY

A persistence level indicating that a key is never destroyed.

See `typedef psa_key_persistence_t` for more information.

3.2.1.1.104 PSA_KEY_PERSISTENCE_VOLATILE

The persistence level of volatile keys.

See `typedef psa_key_persistence_t` for more information.

3.2.1.1.105 PSA_KEY_TYPE_VENDOR

Key type where vendor bit is set defines specific vendor implementation key type.

Vendor key type list:

- `PSA_KEY_TYPE_DG_PROVISIONING_KEY`: EdgeLock 2GO Provisioning OEM Key

3.2.1.1.106 PSA_KEY_TYPE_AES

Key for a cipher, AEAD or MAC algorithm based on the AES block cipher.

The size of the key is related to the AES algorithm variant. For algorithms except the XTS block cipher mode, the following key sizes are used:

- AES-128 uses a 16-byte key : `key_bits = 128`
- AES-192 uses a 24-byte key : `key_bits = 192`
- AES-256 uses a 32-byte key : `key_bits = 256`

For the XTS block cipher mode (PSA_ALG_XTS), the following key sizes are used:

- AES-128-XTS uses two 16-byte keys : key_bits = 256
- AES-192-XTS uses two 24-byte keys : key_bits = 384
- AES-256-XTS uses two 32-byte keys : key_bits = 512

The AES block cipher is defined in FIPS Publication 197: Advanced Encryption Standard (AES) [FIPS197].

3.2.1.1.107 PSA_KEY_TYPE_ARC4

Key for the ARC4 stream cipher.

Warning:

The ARC4 cipher is weak and deprecated and is only recommended for use in legacy protocols.

The ARC4 cipher supports key sizes between 40 and 2048 bits, that are multiples of 8. (5 to 256 bytes)

Use algorithm PSA_ALG_STREAM_CIPHER to use this key with the ARC4 cipher.

3.2.1.1.108 PSA_KEY_TYPE_ARIA

Key for a cipher, AEAD or MAC algorithm based on the ARIA block cipher.

The size of the key is related to the ARIA algorithm variant. For algorithms except the XTS block cipher mode, the following key sizes are used:

- ARIA-128 uses a 16-byte key : key_bits = 128
- ARIA-192 uses a 24-byte key : key_bits = 192
- ARIA-256 uses a 32-byte key : key_bits = 256

For the XTS block cipher mode (PSA_ALG_XTS), the following key sizes are used:

- ARIA-128-XTS uses two 16-byte keys : key_bits = 256
- ARIA-192-XTS uses two 24-byte keys : key_bits = 384
- ARIA-256-XTS uses two 32-byte keys : key_bits = 512

The ARIA block cipher is defined in A Description of the ARIA Encryption Algorithm [RFC5794].

Compatible algorithms:

- PSA_ALG_CBC_MAC
- PSA_ALG_CMAC
- PSA_ALG_CTR
- PSA_ALG_CFB
- PSA_ALG_OFB
- PSA_ALG_XTS
- PSA_ALG_CBC_NO_PADDING
- PSA_ALG_CBC_PKCS7
- PSA_ALG_ECB_NO_PADDING

-
- PSA_ALG_CCM
 - PSA_ALG_GCM

3.2.1.1.109 PSA_KEY_TYPE_CAMELLIA

Key for a cipher, AEAD or MAC algorithm based on the Camellia block cipher.

The size of the key is related to the Camellia algorithm variant. For algorithms except the XTS block cipher mode, the following key sizes are used:

- Camellia-128 uses a 16-byte key : key_bits = 128
- Camellia-192 uses a 24-byte key : key_bits = 192
- Camellia-256 uses a 32-byte key : key_bits = 256

For the XTS block cipher mode (PSA_ALG_XTS), the following key sizes are used:

- Camellia-128-XTS uses two 16-byte keys : key_bits = 256
- Camellia-192-XTS uses two 24-byte keys : key_bits = 384
- Camellia-256-XTS uses two 32-byte keys : key_bits = 512

The Camellia block cipher is defined in Specification of Camellia — a 128-bit Block Cipher [NTT-CAM] and also described in A Description of the Camellia Encryption Algorithm [RFC3713].

3.2.1.1.110 PSA_KEY_TYPE_CHACHA20

Key for the ChaCha20 stream cipher or the ChaCha20-Poly1305 AEAD algorithm.

The ChaCha20 key size is 256 bits (32 bytes).

- Use algorithm PSA_ALG_STREAM_CIPHER to use this key with the ChaCha20 cipher for unauthenticated encryption. See PSA_ALG_STREAM_CIPHER for details of this algorithm.
- Use algorithm PSA_ALG_CHACHA20_POLY1305 to use this key with the ChaCha20 cipher and Poly1305 authenticator for AEAD. See PSA_ALG_CHACHA20_POLY1305 for details of this algorithm.

3.2.1.1.111 PSA_KEY_TYPE_XCHACHA20

Key for the XChaCha20 stream cipher or the XChaCha20-Poly1305 AEAD algorithm.

The XChaCha20 key size is 256 bits (32 bytes).

- Use algorithm PSA_ALG_STREAM_CIPHER to use this key with the XChaCha20 cipher for unauthenticated encryption. See PSA_ALG_STREAM_CIPHER for details of this algorithm.
- Use algorithm PSA_ALG_XCHACHA20_POLY1305 to use this key with the XChaCha20 cipher and Poly1305 authenticator for AEAD. See PSA_ALG_XCHACHA20_POLY1305 for details of this algorithm.

Compatible algorithms:

- PSA_ALG_STREAM_CIPHER
- PSA_ALG_XCHACHA20_POLY1305

3.2.1.1.112 PSA_KEY_TYPE_DERIVE

A secret for key derivation.

The key policy determines which key derivation algorithm the key can be used for.

The bit size of a secret for key derivation must be a non-zero multiple of 8. The maximum size of a secret for key derivation is IMPLEMENTATION DEFINED.

3.2.1.1.113 PSA_KEY_TYPE_DES

Key for a cipher or MAC algorithm based on DES or 3DES (Triple-DES).

The size of the key determines which DES algorithm is used:

- Single DES uses an 8-byte key : key_bits = 64
- 2-key 3DES uses a 16-byte key : key_bits = 128
- 3-key 3DES uses a 24-byte key : key_bits = 192

Warning:

Single DES and 2-key 3DES are weak and strongly deprecated and are only recommended for decrypting legacy data.

3-key 3DES is weak and deprecated and is only recommended for use in legacy protocols.

The DES and 3DES block ciphers are defined in NIST Special Publication 800-67: Recommendation for the Triple Data Encryption Algorithm (TDEA) Block Cipher [SP800-67].

3.2.1.1.114 macro PSA_KEY_TYPE_DH_GET_FAMILY

PSA_KEY_TYPE_DH_GET_FAMILY(type)

Extract the group family from a Diffie-Hellman key type.

Parameters

- **type** – A Diffie-Hellman key type (value of *typedef psa_key_type_t* such that PSA_KEY_TYPE_IS_DH(type) is true).

3.2.1.1.114.1 Return

typedef psa_dh_family_t

The Diffie-Hellman group family id, if type is a supported Diffie-Hellman key. Unspecified if type is not a supported Diffie-Hellman key.

3.2.1.1.115 macro PSA_KEY_TYPE_DH_KEY_PAIR

PSA_KEY_TYPE_DH_KEY_PAIR(group)

Finite-field Diffie-Hellman key pair: both the private key and public key.

Parameters

- **group** – A value of *typedef psa_dh_family_t* that identifies the Diffie-Hellman group family to be used.

3.2.1.1.116 macro `PSA_KEY_TYPE_DH_PUBLIC_KEY`

`PSA_KEY_TYPE_DH_PUBLIC_KEY`(group)

Finite-field Diffie-Hellman public key.

Parameters

- **group** – A value of `typedef psa_dh_family_t` that identifies the Diffie-Hellman group family to be used.

3.2.1.1.117 macro `PSA_KEY_TYPE_ECC_GET_FAMILY`

`PSA_KEY_TYPE_ECC_GET_FAMILY`(type)

Extract the curve family from an elliptic curve key type.

Parameters

- **type** – An elliptic curve key type (value of `typedef psa_key_type_t` such that `PSA_KEY_TYPE_IS_ECC(type)` is true).

3.2.1.1.117.1 Return

`typedef psa_ecc_family_t`

The elliptic curve family id, if `type` is a supported elliptic curve key. Unspecified if `type` is not a supported elliptic curve key.

3.2.1.1.118 macro `PSA_KEY_TYPE_ECC_KEY_PAIR`

`PSA_KEY_TYPE_ECC_KEY_PAIR`(curve)

Elliptic curve key pair: both the private and public key.

Parameters

- **curve** – A value of `typedef psa_ecc_family_t` that identifies the ECC curve family to be used.

3.2.1.1.119 macro `PSA_KEY_TYPE_ECC_PUBLIC_KEY`

`PSA_KEY_TYPE_ECC_PUBLIC_KEY`(curve)

Elliptic curve public key.

Parameters

- **curve** – A value of `typedef psa_ecc_family_t` that identifies the ECC curve family to be used.

3.2.1.1.120 `PSA_KEY_TYPE_HMAC`

HMAC key.

The key policy determines which underlying hash algorithm the key can be used for.

The bit size of an HMAC key must be a non-zero multiple of 8. An HMAC key is typically the same size as the output of the underlying hash algorithm. An HMAC key that is longer than the block size of the underlying hash algorithm will be hashed before use.

When an HMAC key is created that is longer than the block size, it is implementation defined whether the implementation stores the original HMAC key, or the hash of the HMAC key. If the hash of the key is stored, the key size reported by `psa_get_key_attributes()` will be the size of the hashed key.

Note:

PSA_HASH_LENGTH(alg) provides the output size of hash algorithm alg, in bytes.

PSA_HASH_BLOCK_LENGTH(alg) provides the block size of hash algorithm alg, in bytes.

3.2.1.1.121 macro PSA_KEY_TYPE_IS_ASYMMETRIC

PSA_KEY_TYPE_IS_ASYMMETRIC(type)

Whether a key type is asymmetric: either a key pair or a public key.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.121.1 Description

See RSA keys for a list of asymmetric key types.

3.2.1.1.122 macro PSA_KEY_TYPE_IS_DH

PSA_KEY_TYPE_IS_DH(type)

Whether a key type is a Diffie-Hellman key, either a key pair or a public key.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.123 macro PSA_KEY_TYPE_IS_DH_KEY_PAIR

PSA_KEY_TYPE_IS_DH_KEY_PAIR(type)

Whether a key type is a Diffie-Hellman key pair.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.124 macro PSA_KEY_TYPE_IS_DH_PUBLIC_KEY

PSA_KEY_TYPE_IS_DH_PUBLIC_KEY(type)

Whether a key type is a Diffie-Hellman public key.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.125 macro PSA_KEY_TYPE_IS_ECC

PSA_KEY_TYPE_IS_ECC(type)

Whether a key type is an elliptic curve key, either a key pair or a public key.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.126 macro `PSA_KEY_TYPE_IS_ECC_KEY_PAIR`

`PSA_KEY_TYPE_IS_ECC_KEY_PAIR`(type)

Whether a key type is an elliptic curve key pair.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.127 macro `PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY`

`PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY`(type)

Whether a key type is an elliptic curve public key.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.128 macro `PSA_KEY_TYPE_IS_KEY_PAIR`

`PSA_KEY_TYPE_IS_KEY_PAIR`(type)

Whether a key type is a key pair containing a private part and a public part.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.129 macro `PSA_KEY_TYPE_IS_PUBLIC_KEY`

`PSA_KEY_TYPE_IS_PUBLIC_KEY`(type)

Whether a key type is the public part of a key pair.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.130 macro `PSA_KEY_TYPE_IS_RSA`

`PSA_KEY_TYPE_IS_RSA`(type)

Whether a key type is an RSA key. This includes both key pairs and public keys.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.131 macro `PSA_KEY_TYPE_IS_RSA_KEY_PAIR`

`PSA_KEY_TYPE_IS_RSA_KEY_PAIR`(type)

Whether a key type is an RSA key pair.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.132 macro `PSA_KEY_TYPE_IS_RSA_PUBLIC_KEY`

`PSA_KEY_TYPE_IS_RSA_PUBLIC_KEY`(type)

Whether a key type is an RSA public key.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.133 macro `PSA_KEY_TYPE_IS_UNSTRUCTURED`

`PSA_KEY_TYPE_IS_UNSTRUCTURED`(type)

Whether a key type is an unstructured array of bytes.

Parameters

- **type** – A key type (value of `typedef psa_key_type_t`).

3.2.1.1.133.1 Description

This encompasses both symmetric keys and non-key data.

See Symmetric keys for a list of symmetric key types.

3.2.1.1.134 macro `PSA_KEY_TYPE_KEY_PAIR_OF_PUBLIC_KEY`

`PSA_KEY_TYPE_KEY_PAIR_OF_PUBLIC_KEY`(type)

The key pair type corresponding to a public key type.

Parameters

- **type** – A public key type or key pair type.

3.2.1.1.134.1 Description

If type is a key pair type, it will be left unchanged.

3.2.1.1.134.2 Return

The corresponding key pair type. If **type** is not a public key or a key pair, the return value is undefined.

3.2.1.1.135 `PSA_KEY_TYPE_NONE`

An invalid key type value.

Zero is not the encoding of any key type.

3.2.1.1.136 `PSA_KEY_TYPE_PASSWORD`

A low-entropy secret for password hashing or key derivation.

This key type is suitable for passwords and passphrases which are typically intended to be memorable by humans, and have a low entropy relative to their size. It can be used for randomly generated or derived keys with maximum or near-maximum entropy, but `PSA_KEY_TYPE_DERIVE` is more suitable for such keys. It is not suitable for passwords with extremely low entropy, such as numerical PINs.

These keys can be used in the `PSA_KEY_DERIVATION_INPUT_PASSWORD` input step of key derivation algorithms. Algorithms that accept such an input were designed to accept low-entropy secret and are known as password hashing or key stretching algorithms.

These keys cannot be used in the `PSA_KEY_DERIVATION_INPUT_SECRET` input step of key derivation algorithms, as the algorithms expect such an input to have high entropy.

The key policy determines which key derivation algorithm the key can be used for, among the permissible subset defined above.

Compatible algorithms:

- `PSA_ALG_PBKDF2_HMAC()` (password input)
- `PSA_ALG_PBKDF2_AES_CMAC_PRF_128` (password input)

3.2.1.1.137 `PSA_KEY_TYPE_PASSWORD_HASH`

A secret value that can be used to verify a password hash.

The key policy determines which key derivation algorithm the key can be used for, among the same permissible subset as for `PSA_KEY_TYPE_PASSWORD`.

Compatible algorithms:

- `PSA_ALG_PBKDF2_HMAC()` (key output and verification)
- `PSA_ALG_PBKDF2_AES_CMAC_PRF_128` (key output and verification)

3.2.1.1.138 `PSA_KEY_TYPE_PEPPER`

A secret value that can be used when computing a password hash.

The key policy determines which key derivation algorithm the key can be used for, among the subset of algorithms that can use pepper.

Compatible algorithms:

- `PSA_ALG_PBKDF2_HMAC()` (salt input)
- `PSA_ALG_PBKDF2_AES_CMAC_PRF_128` (salt input)

3.2.1.1.139 macro `PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR`

`PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type)`

The public key type corresponding to a key pair type.

Parameters

- **type** – A public key type or key pair type.

3.2.1.1.139.1 Description

If type is a public key type, it will be left unchanged.

3.2.1.1.139.2 Return

The corresponding public key type. If `type` is not a public key or a key pair, the return value is undefined.

3.2.1.1.140 PSA_KEY_TYPE_RAW_DATA

Raw data.

A “key” of this type cannot be used for any cryptographic operation. Applications can use this type to store arbitrary data in the keystore.

The bit size of a raw key must be a non-zero multiple of 8. The maximum size of a raw key is IMPLEMENTATION DEFINED.

3.2.1.1.141 PSA_KEY_TYPE_RSA_KEY_PAIR

RSA key pair: both the private and public key.

3.2.1.1.142 PSA_KEY_TYPE_RSA_PUBLIC_KEY

RSA public key.

3.2.1.1.143 PSA_KEY_TYPE_SM4

Key for a cipher, AEAD or MAC algorithm based on the SM4 block cipher.

For algorithms except the XTS block cipher mode, the SM4 key size is 128 bits (16 bytes).

For the XTS block cipher mode (PSA_ALG_XTS), the SM4 key size is 256 bits (two 16-byte keys).

The SM4 block cipher is defined in GB/T 32907-2016: Information security technology — SM4 block cipher algorithm [PRC-SM4] and also described in The SM4 Blockcipher Algorithm And Its Modes Of Operations [IETF-SM4].

3.2.1.1.144 PSA_KEY_USAGE_CACHE

Permission for the implementation to cache the key.

This flag allows the implementation to make additional copies of the key material that are not in storage and not for the purpose of an ongoing operation. Applications can use it as a hint to keep the key around for repeated access.

An application can request that cached key material is removed from memory by calling [`psa\_purge\_key\(\)`](#).

The presence of this usage flag when creating a key is a hint:

- An implementation is not required to cache keys that have this usage flag.
- An implementation must not report an error if it does not cache keys.

If this usage flag is not present, the implementation must ensure key material is removed from memory as soon as it is not required for an operation or for maintenance of a volatile key.

This flag must be preserved when reading back the attributes for all keys, regardless of key type or implementation behavior.

3.2.1.1.145 PSA_KEY_USAGE_COPY

Permission to copy the key.

This flag allows the use of `psa_copy_key()` to make a copy of the key with the same policy or a more restrictive policy.

For lifetimes for which the key is located in a secure element which enforce the non-exportability of keys, copying a key outside the secure element also requires the usage flag `PSA_KEY_USAGE_EXPORT`. Copying the key inside the secure element is permitted with just `PSA_KEY_USAGE_COPY` if the secure element supports it. For keys with the lifetime `PSA_KEY_LIFETIME_VOLATILE` or `PSA_KEY_LIFETIME_PERSISTENT`, the usage flag `PSA_KEY_USAGE_COPY` is sufficient to permit the copy.

3.2.1.1.146 PSA_KEY_USAGE_DECRYPT

Permission to decrypt a message with the key. This flag allows the key to be used for a symmetric decryption operation, for an AEAD decryption-and-verification operation, or for an asymmetric decryption operation, if otherwise permitted by the key's type and policy. The flag must be present on keys used with the following APIs:

- `psa_cipher_decrypt()`
- `psa_cipher_decrypt_setup()`
- `psa_aead_decrypt()`
- `psa_aead_decrypt_setup()`
- `psa_asymmetric_decrypt()`

For a key pair, this concerns the private key.

3.2.1.1.147 PSA_KEY_USAGE_DERIVE

Permission to derive other keys from this key.

This flag allows the key to be used for a key derivation operation or for a key agreement operation, if otherwise permitted by the key's type and policy. The flag must be present on keys used with the following APIs:

- `psa_key_derivation_input_key()`
- `psa_key_derivation_key_agreement()`
- `psa_raw_key_agreement()`

3.2.1.1.148 PSA_KEY_USAGE_ENCRYPT

Permission to encrypt a message with the key.

This flag allows the key to be used for a symmetric encryption operation, for an AEAD encryption-and-authentication operation, or for an asymmetric encryption operation, if otherwise permitted by the key's type and policy. The flag must be present on keys used with the following APIs:

- `psa_cipher_encrypt()`
- `psa_cipher_encrypt_setup()`
- `psa_aead_encrypt()`

-
- [psa_aead_encrypt_setup\(\)](#)
 - [psa_asymmetric_encrypt\(\)](#)

For a key pair, this concerns the public key.

3.2.1.1.149 PSA_KEY_USAGE_EXPORT

Permission to export the key.

This flag allows the use of [psa_export_key\(\)](#) to export a key from the cryptoprocessor. A public key or the public part of a key pair can always be exported regardless of the value of this permission flag.

This flag can also be required to copy a key using [psa_copy_key\(\)](#) outside of a secure element. See also PSA_KEY_USAGE_COPY.

If a key does not have export permission, implementations must not allow the key to be exported in plain form from the cryptoprocessor, whether through [psa_export_key\(\)](#) or through a proprietary interface. The key might still be exportable in a wrapped form, i.e. in a form where it is encrypted by another key.

3.2.1.1.150 PSA_KEY_USAGE_SIGN_HASH

Permission to sign a message hash with the key.

This flag allows the key to be used to sign a message hash as part of an asymmetric signature operation, if otherwise permitted by the key's type and policy. The flag must be present on keys used when calling [psa_sign_hash\(\)](#).

This flag automatically sets PSA_KEY_USAGE_SIGN_MESSAGE: if an application sets the flag PSA_KEY_USAGE_SIGN_HASH when creating a key, then the key always has the permissions conveyed by PSA_KEY_USAGE_SIGN_MESSAGE, and the flag PSA_KEY_USAGE_SIGN_MESSAGE will also be present when the application queries the usage flags of the key.

For a key pair, this concerns the private key.

3.2.1.1.151 PSA_KEY_USAGE_SIGN_MESSAGE

Permission to sign a message with the key.

This flag allows the key to be used for a MAC calculation operation or for an asymmetric message signature operation, if otherwise permitted by the key's type and policy. The flag must be present on keys used with the following APIs:

- [psa_mac_compute\(\)](#)
- [psa_mac_sign_setup\(\)](#)
- [psa_sign_message\(\)](#)

For a key pair, this concerns the private key.

3.2.1.1.152 PSA_KEY_USAGE_VERIFY_DERIVATION

Permission to verify the result of a key derivation, including password hashing.

This flag allows the key to be used in a key derivation operation, if otherwise permitted by the key's type and policy.

This flag must be present on keys used with `psa_key_derivation_verify_key()`.

If this flag is present on all keys used in calls to `psa_key_derivation_input_key()` for a key derivation operation, then it permits calling `psa_key_derivation_verify_bytes()` or `psa_key_derivation_verify_key()` at the end of the operation.

3.2.1.1.153 PSA_KEY_USAGE_VERIFY_HASH

Permission to verify a message hash with the key.

This flag allows the key to be used to verify a message hash as part of an asymmetric signature verification operation, if otherwise permitted by the key's type and policy. The flag must be present on keys used when calling `psa_verify_hash()`.

This flag automatically sets `PSA_KEY_USAGE_VERIFY_MESSAGE`: if an application sets the flag `PSA_KEY_USAGE_VERIFY_HASH` when creating a key, then the key always has the permissions conveyed by `PSA_KEY_USAGE_VERIFY_MESSAGE`, and the flag `PSA_KEY_USAGE_VERIFY_MESSAGE` will also be present when the application queries the usage flags of the key.

For a key pair, this concerns the public key.

3.2.1.1.154 PSA_KEY_USAGE_VERIFY_MESSAGE

Permission to verify a message signature with the key.

This flag allows the key to be used for a MAC verification operation or for an asymmetric message signature verification operation, if otherwise permitted by the key's type and policy. The flag must be present on keys used with the following APIs:

- `psa_mac_verify()`
- `psa_mac_verify_setup()`
- `psa_verify_message()`

For a key pair, this concerns the public key.

3.2.1.2 Sizes

3.2.1.2.1 Introduction

This file contains the definitions of macros that are useful to compute buffer sizes. The signatures and semantics of these macros are standardized, but the definitions are not, because they depend on the available algorithms and, in some cases, on permitted tolerances on buffer sizes.

3.2.1.2.2 Reference

Documentation:

PSA Cryptography API v1.2.1

Link:

<https://arm-software.github.io/psa-api/crypto/1.2/about>

3.2.1.2.3 macro `PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE`

`PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE(ciphertext_length)`

A sufficient output buffer size for `psa_aead_decrypt()`, for any of the supported key types and AEAD algorithms.

Parameters

- **ciphertext_length** – Size of the ciphertext in bytes.

3.2.1.2.3.1 Description

If the size of the plaintext buffer is at least this large, it is guaranteed that `psa_aead_decrypt()` will not fail due to an insufficient buffer size.

See also `PSA_AEAD_DECRYPT_OUTPUT_SIZE()`.

3.2.1.2.4 macro `PSA_AEAD_DECRYPT_OUTPUT_SIZE`

`PSA_AEAD_DECRYPT_OUTPUT_SIZE(key_type, alg, ciphertext_length)`

The maximum size of the output of `psa_aead_decrypt()`, in bytes.

Parameters

- **key_type** – A symmetric key type that is compatible with algorithm `alg`.
- **alg** – An AEAD algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_AEAD(alg)` is true).
- **ciphertext_length** – Size of the ciphertext in bytes.

3.2.1.2.4.1 Description

If the size of the plaintext buffer is at least this large, it is guaranteed that `psa_aead_decrypt()` will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the plaintext might be smaller.

See also `PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE()`.

3.2.1.2.4.2 Return

The AEAD plaintext size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0. An implementation can return either 0 or a correct size for a key type and AEAD algorithm that it recognizes, but does not support.

3.2.1.2.5 macro `PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE`

`PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE(plaintext_length)`

A sufficient output buffer size for `psa_aead_encrypt()`, for any of the supported key types and AEAD algorithms.

Parameters

- **plaintext_length** – Size of the plaintext in bytes.

3.2.1.2.5.1 Description

If the size of the ciphertext buffer is at least this large, it is guaranteed that `psa_aead_encrypt()` will not fail due to an insufficient buffer size.

See also `PSA_AEAD_ENCRYPT_OUTPUT_SIZE()`.

3.2.1.2.6 macro `PSA_AEAD_ENCRYPT_OUTPUT_SIZE`

`PSA_AEAD_ENCRYPT_OUTPUT_SIZE(key_type, alg, plaintext_length)`

The maximum size of the output of `psa_aead_encrypt()`, in bytes.

Parameters

- **key_type** – A symmetric key type that is compatible with algorithm `alg`.
- **alg** – An AEAD algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_AEAD(alg)` is true).
- **plaintext_length** – Size of the plaintext in bytes.

3.2.1.2.6.1 Description

If the size of the ciphertext buffer is at least this large, it is guaranteed that `psa_aead_encrypt()` will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the ciphertext might be smaller.

See also `PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE()`.

3.2.1.2.6.2 Return

The AEAD ciphertext size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0. An implementation can return either 0 or a correct size for a key type and AEAD algorithm that it recognizes, but does not support.

3.2.1.2.7 `PSA_AEAD_FINISH_OUTPUT_MAX_SIZE`

A sufficient ciphertext buffer size for `psa_aead_finish()`, for any of the supported key types and AEAD algorithms.

Warning: Not supported

See also `PSA_AEAD_FINISH_OUTPUT_SIZE()`.

3.2.1.2.8 macro `PSA_AEAD_FINISH_OUTPUT_SIZE`

`PSA_AEAD_FINISH_OUTPUT_SIZE(key_type, alg)`

A sufficient ciphertext buffer size for `psa_aead_finish()`.

Parameters

- **key_type** – A symmetric key type that is compatible with algorithm `alg`.
- **alg** – An AEAD algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_AEAD(alg)` is true).

3.2.1.2.8.1 Description

Warning: Not supported

If the size of the ciphertext buffer is at least this large, it is guaranteed that `psa_aead_finish()` will not fail due to an insufficient ciphertext buffer size. The actual size of the output might be smaller in any given call.

See also `PSA_AEAD_FINISH_OUTPUT_MAX_SIZE`.

3.2.1.2.8.2 Return

A sufficient ciphertext buffer size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0. An implementation can return either 0 or a correct size for a key type and AEAD algorithm that it recognizes, but does not support.

3.2.1.2.9 macro `PSA_AEAD_NONCE_LENGTH`

`PSA_AEAD_NONCE_LENGTH(key_type, alg)`

The default nonce size for an AEAD algorithm, in bytes.

Parameters

- **key_type** – A symmetric key type that is compatible with algorithm `alg`.
- **alg** – An AEAD algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_AEAD(alg)` is true).

3.2.1.2.9.1 Description

This macro can be used to allocate a buffer of sufficient size to store the nonce output from `psa_aead_generate_nonce()`.

See also `PSA_AEAD_NONCE_MAX_SIZE`.

3.2.1.2.9.2 Return

The default nonce size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0. An implementation can return either 0 or a correct size for a key type and AEAD algorithm that it recognizes, but does not support.

3.2.1.2.10 `PSA_AEAD_NONCE_MAX_SIZE`

The maximum nonce size for all supported AEAD algorithms, in bytes.

See also `PSA_AEAD_NONCE_LENGTH()`.

3.2.1.2.11 macro `PSA_AEAD_TAG_LENGTH`

`PSA_AEAD_TAG_LENGTH(key_type, key_bits, alg)`

The length of a tag for an AEAD algorithm, in bytes.

Parameters

- **key_type** – The type of the AEAD key.
- **key_bits** – The size of the AEAD key in bits.

-
- **alg** – An AEAD algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_AEAD(alg) is true).

3.2.1.2.11.1 Description

This macro can be used to allocate a buffer of sufficient size to store the tag output from *psa_aead_finish()*.

See also PSA_AEAD_TAG_MAX_SIZE.

3.2.1.2.11.2 Return

The tag length for the specified algorithm and key. If the AEAD algorithm does not have an identified tag that can be distinguished from the rest of the ciphertext, return 0. If the AEAD algorithm is not recognized, return 0. An implementation can return either 0 or a correct size for an AEAD algorithm that it recognizes, but does not support.

3.2.1.2.12 PSA_AEAD_TAG_MAX_SIZE

The maximum tag size for all supported AEAD algorithms, in bytes.

See also *PSA_AEAD_TAG_LENGTH()*.

3.2.1.2.13 macro PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE

PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE(input_length)

A sufficient output buffer size for *psa_aead_update()*, for any of the supported key types and AEAD algorithms.

Parameters

- **input_length** – Size of the input in bytes.

3.2.1.2.13.1 Description

Warning: Not supported

If the size of the output buffer is at least this large, it is guaranteed that *psa_aead_update()* will not fail due to an insufficient buffer size.

See also *PSA_AEAD_UPDATE_OUTPUT_SIZE()*.

3.2.1.2.14 macro PSA_AEAD_UPDATE_OUTPUT_SIZE

PSA_AEAD_UPDATE_OUTPUT_SIZE(key_type, alg, input_length)

A sufficient output buffer size for *psa_aead_update()*.

Parameters

- **key_type** – A symmetric key type that is compatible with algorithm alg.
- **alg** – An AEAD algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_AEAD(alg) is true).
- **input_length** – Size of the input in bytes.

3.2.1.2.14.1 Description

Warning: Not supported

If the size of the output buffer is at least this large, it is guaranteed that `psa_aead_update()` will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also `PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE`.

3.2.1.2.14.2 Return

A sufficient output buffer size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0. An implementation can return either 0 or a correct size for a key type and AEAD algorithm that it recognizes, but does not support.

3.2.1.2.15 PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE

A sufficient plaintext buffer size for `psa_aead_verify()`, for any of the supported key types and AEAD algorithms.

Warning: Not supported

See also `PSA_AEAD_VERIFY_OUTPUT_SIZE()`.

3.2.1.2.16 macro PSA_AEAD_VERIFY_OUTPUT_SIZE

`PSA_AEAD_VERIFY_OUTPUT_SIZE(key_type, alg)`

A sufficient plaintext buffer size for `psa_aead_verify()`.

Parameters

- **key_type** – A symmetric key type that is compatible with algorithm `alg`.
- **alg** – An AEAD algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_AEAD(alg)` is true).

3.2.1.2.16.1 Description

Warning: Not supported

If the size of the plaintext buffer is at least this large, it is guaranteed that `psa_aead_verify()` will not fail due to an insufficient plaintext buffer size. The actual size of the output might be smaller in any given call.

See also `PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE`.

3.2.1.2.16.2 Return

A sufficient plaintext buffer size for the specified key type and algorithm. If the key type or AEAD algorithm is not recognized, or the parameters are incompatible, return 0. An implementation can return either 0 or a correct size for a key type and AEAD algorithm that it recognizes, but does not support.

3.2.1.2.17 PSA_ASYMMETRIC_DECRYPT_OUTPUT_MAX_SIZE

A sufficient output buffer size for `psa_asymmetric_decrypt()`, for any supported asymmetric decryption.

Warning: Not supported

See also `PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE()`.

3.2.1.2.18 macro PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE

`PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE(key_type, key_bits, alg)`

Sufficient output buffer size for `psa_asymmetric_decrypt()`.

Parameters

- **key_type** – An asymmetric key type, either a key pair or a public key.
- **key_bits** – The size of the key in bits.
- **alg** – The asymmetric encryption algorithm.

3.2.1.2.18.1 Description

Warning: Not supported

This macro returns a sufficient buffer size for a plaintext produced using a key of the specified type and size, with the specified algorithm. Note that the actual size of the plaintext might be smaller, depending on the algorithm.

Warning:

This function might evaluate its arguments multiple times or zero times. Providing arguments that have side effects will result in implementation-specific behavior, and is non-portable.

See also `PSA_ASYMMETRIC_DECRYPT_OUTPUT_MAX_SIZE`.

3.2.1.2.18.2 Return

If the parameters are valid and supported, return a buffer size in bytes that guarantees that `psa_asymmetric_decrypt()` will not fail with `PSA_ERROR_BUFFER_TOO_SMALL`. If the parameters are a valid combination that is not supported by the implementation, this macro must return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

3.2.1.2.19 PSA_ASYMMETRIC_ENCRYPT_OUTPUT_MAX_SIZE

A sufficient output buffer size for `psa_asymmetric_encrypt()`, for any supported asymmetric encryption.

Warning: Not supported

See also `PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE()`.

3.2.1.2.20 macro PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE

`PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE(key_type, key_bits, alg)`

Sufficient output buffer size for `psa_asymmetric_encrypt()`.

Parameters

- **key_type** – An asymmetric key type, either a key pair or a public key.
- **key_bits** – The size of the key in bits.
- **alg** – The asymmetric encryption algorithm.

3.2.1.2.20.1 Description

Warning: Not supported

This macro returns a sufficient buffer size for a ciphertext produced using a key of the specified type and size, with the specified algorithm. Note that the actual size of the ciphertext might be smaller, depending on the algorithm.

Warning:

This function might evaluate its arguments multiple times or zero times. Providing arguments that have side effects will result in implementation-specific behavior, and is non-portable.

See also `PSA_ASYMMETRIC_ENCRYPT_OUTPUT_MAX_SIZE`.

3.2.1.2.20.2 Return

If the parameters are valid and supported, return a buffer size in bytes that guarantees that `psa_asymmetric_encrypt()` will not fail with `PSA_ERROR_BUFFER_TOO_SMALL`. If the parameters are a valid combination that is not supported by the implementation, this macro must return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

3.2.1.2.21 PSA_BLOCK_CIPHER_BLOCK_MAX_SIZE

The maximum size of a block cipher supported by the implementation.

See also `PSA_BLOCK_CIPHER_BLOCK_LENGTH()`.

3.2.1.2.22 macro PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE

`PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE(input_length)`

A sufficient output buffer size for `psa_cipher_decrypt()`, for any of the supported key types and cipher algorithms.

Parameters

- **input_length** – Size of the input in bytes.

3.2.1.2.22.1 Description

If the size of the output buffer is at least this large, it is guaranteed that `psa_cipher_decrypt()` will not fail due to an insufficient buffer size.

See also `PSA_CIPHER_DECRYPT_OUTPUT_SIZE()`.

3.2.1.2.23 macro PSA_CIPHER_DECRYPT_OUTPUT_SIZE

`PSA_CIPHER_DECRYPT_OUTPUT_SIZE(key_type, alg, input_length)`

The maximum size of the output of `psa_cipher_decrypt()`, in bytes.

Parameters

- **key_type** – A symmetric key type that is compatible with algorithm `alg`.
- **alg** – A cipher algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_CIPHER(alg)` is true).
- **input_length** – Size of the input in bytes.

3.2.1.2.23.1 Description

If the size of the output buffer is at least this large, it is guaranteed that `psa_cipher_decrypt()` will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the output might be smaller.

See also `PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE`.

3.2.1.2.23.2 Return

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0. An implementation can return either 0 or a correct size for a key type and cipher algorithm that it recognizes, but does not support.

3.2.1.2.24 macro `PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE`

`PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE(input_length)`

A sufficient output buffer size for `psa_cipher_encrypt()`, for any of the supported key types and cipher algorithms.

Parameters

- **input_length** – Size of the input in bytes.

3.2.1.2.24.1 Description

If the size of the output buffer is at least this large, it is guaranteed that `psa_cipher_encrypt()` will not fail due to an insufficient buffer size.

See also `PSA_CIPHER_ENCRYPT_OUTPUT_SIZE()`.

3.2.1.2.25 macro `PSA_CIPHER_ENCRYPT_OUTPUT_SIZE`

`PSA_CIPHER_ENCRYPT_OUTPUT_SIZE(key_type, alg, input_length)`

The maximum size of the output of `psa_cipher_encrypt()`, in bytes.

Parameters

- **key_type** – A symmetric key type that is compatible with algorithm `alg`.
- **alg** – A cipher algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_CIPHER(alg)` is true).
- **input_length** – Size of the input in bytes.

3.2.1.2.25.1 Description

If the size of the output buffer is at least this large, it is guaranteed that `psa_cipher_encrypt()` will not fail due to an insufficient buffer size. Depending on the algorithm, the actual size of the output might be smaller.

See also `PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE`.

3.2.1.2.25.2 Return

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0. An implementation can return either 0 or a correct size for a key type and cipher algorithm that it recognizes, but does not support.

3.2.1.2.26 PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE

A sufficient ciphertext buffer size for `psa_cipher_finish()`, for any of the supported key types and cipher algorithms.

Warning: Not supported

See also `PSA_CIPHER_FINISH_OUTPUT_SIZE()`.

3.2.1.2.27 macro PSA_CIPHER_FINISH_OUTPUT_SIZE

`PSA_CIPHER_FINISH_OUTPUT_SIZE(key_type, alg)`

A sufficient ciphertext buffer size for `psa_cipher_finish()`.

Parameters

- **key_type** – A symmetric key type that is compatible with algorithm `alg`.
- **alg** – A cipher algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_CIPHER(alg)` is true).

3.2.1.2.27.1 Description

Warning: Not supported

If the size of the ciphertext buffer is at least this large, it is guaranteed that `psa_cipher_finish()` will not fail due to an insufficient ciphertext buffer size. The actual size of the output might be smaller in any given call.

See also `PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE`.

3.2.1.2.27.2 Return

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0. An implementation can return either 0 or a correct size for a key type and cipher algorithm that it recognizes, but does not support.

3.2.1.2.28 macro `PSA_CIPHER_IV_LENGTH`

`PSA_CIPHER_IV_LENGTH(key_type, alg)`

The default IV size for a cipher algorithm, in bytes.

Parameters

- **key_type** – A symmetric key type that is compatible with algorithm `alg`.
- **alg** – A cipher algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_CIPHER(alg)` is true).

3.2.1.2.28.1 Description

The IV that is generated as part of a call to `psa_cipher_encrypt()` is always the default IV length for the algorithm.

This macro can be used to allocate a buffer of sufficient size to store the IV output from `psa_cipher_generate_iv()` when using a multi-part cipher operation.

See also `PSA_CIPHER_IV_MAX_SIZE`.

3.2.1.2.28.2 Return

The default IV size for the specified key type and algorithm. If the algorithm does not use an IV, return 0. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0. An implementation can return either 0 or a correct size for a key type and cipher algorithm that it recognizes, but does not support.

3.2.1.2.29 `PSA_CIPHER_IV_MAX_SIZE`

The maximum IV size for all supported cipher algorithms, in bytes.

See also `PSA_CIPHER_IV_LENGTH()`.

3.2.1.2.30 macro `PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE`

`PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE(input_length)`

A sufficient output buffer size for `psa_cipher_update()`, for any of the supported key types and cipher algorithms.

Parameters

- **input_length** – Size of the input in bytes.

3.2.1.2.30.1 Description

Warning: Not supported

If the size of the output buffer is at least this large, it is guaranteed that `psa_cipher_update()` will not fail due to an insufficient buffer size.

See also `PSA_CIPHER_UPDATE_OUTPUT_SIZE()`.

3.2.1.2.31 macro `PSA_CIPHER_UPDATE_OUTPUT_SIZE`

`PSA_CIPHER_UPDATE_OUTPUT_SIZE`(key_type, alg, input_length)

A sufficient output buffer size for `psa_cipher_update()`.

Parameters

- **key_type** – A symmetric key type that is compatible with algorithm `alg`.
- **alg** – A cipher algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_CIPHER(alg)` is true).
- **input_length** – Size of the input in bytes.

3.2.1.2.31.1 Description

Warning: Not supported

If the size of the output buffer is at least this large, it is guaranteed that `psa_cipher_update()` will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also `PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE`.

3.2.1.2.31.2 Return

A sufficient output size for the specified key type and algorithm. If the key type or cipher algorithm is not recognized, or the parameters are incompatible, return 0. An implementation can return either 0 or a correct size for a key type and cipher algorithm that it recognizes, but does not support.

3.2.1.2.32 macro `PSA_EXPORT_KEY_OUTPUT_SIZE`

`PSA_EXPORT_KEY_OUTPUT_SIZE`(key_type, key_bits)

Sufficient output buffer size for `psa_export_key()`.

Parameters

- **key_type** – A supported key type.
- **key_bits** – The size of the key in bits.

3.2.1.2.32.1 Description

The following code illustrates how to allocate enough memory to export a key by querying the key type and size at runtime.

```
psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
psa_status_t status;
status = psa_get_key_attributes(key, &attributes);
if (status != PSA_SUCCESS)
    handle_error(...);
psa_key_type_t key_type = psa_get_key_type(&attributes);
size_t key_bits = psa_get_key_bits(&attributes);
size_t buffer_size = PSA_EXPORT_KEY_OUTPUT_SIZE(key_type, key_bits);
psa_reset_key_attributes(&attributes);
uint8_t *buffer = malloc(buffer_size);
```

(continues on next page)


```

if (buffer == NULL)
    handle_error(...);
size_t buffer_length;
status = psa_export_key(key, buffer, buffer_size, &buffer_length);
if (status != PSA_SUCCESS)
    handle_error(...);

```

See also `PSA_EXPORT_KEY_PAIR_MAX_SIZE` and `PSA_EXPORT_PUBLIC_KEY_MAX_SIZE`.

3.2.1.2.32.2 Return

If the parameters are valid and supported, return a buffer size in bytes that guarantees that `psa_export_key()` or `psa_export_public_key()` will not fail with `PSA_ERROR_BUFFER_TOO_SMALL`. If the parameters are a valid combination that is not supported by the implementation, this macro must return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

3.2.1.2.33 `PSA_EXPORT_KEY_PAIR_MAX_SIZE`

Sufficient buffer size for exporting any asymmetric key pair.

This value must be a sufficient buffer size when calling `psa_export_key()` to export any asymmetric key pair that is supported by the implementation, regardless of the exact key type and key size.

See also `PSA_EXPORT_KEY_OUTPUT_SIZE()`.

3.2.1.2.34 `PSA_EXPORT_PUBLIC_KEY_MAX_SIZE`

Sufficient buffer size for exporting any asymmetric public key.

This value must be a sufficient buffer size when calling `psa_export_key()` or `psa_export_public_key()` to export any asymmetric public key that is supported by the implementation, regardless of the exact key type and key size.

See also `PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE()`.

3.2.1.2.35 macro `PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE`

`PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE(key_type, key_bits)`

Sufficient output buffer size for `psa_export_public_key()`.

Parameters

- **key_type** – A public key or key pair key type.
- **key_bits** – The size of the key in bits.

3.2.1.2.35.1 Description

The following code illustrates how to allocate enough memory to export a public key by querying the key type and size at runtime.

```

psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
psa_status_t status;
status = psa_get_key_attributes(key, &attributes);
if (status != PSA_SUCCESS)
    handle_error(...);
psa_key_type_t key_type = psa_get_key_type(&attributes);
size_t key_bits = psa_get_key_bits(&attributes);
size_t buffer_size = PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE(key_type, key_bits);
psa_reset_key_attributes(&attributes);
uint8_t *buffer = malloc(buffer_size);
if (buffer == NULL)
    handle_error(...);
size_t buffer_length;
status = psa_export_public_key(key, buffer, buffer_size, &buffer_length);
if (status != PSA_SUCCESS)
    handle_error(...);

```

See also `PSA_EXPORT_PUBLIC_KEY_MAX_SIZE`.

3.2.1.2.35.2 Return

If the parameters are valid and supported, return a buffer size in bytes that guarantees that `psa_export_public_key()` will not fail with `PSA_ERROR_BUFFER_TOO_SMALL`. If the parameters are a valid combination that is not supported by the implementation, this macro must return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

If the parameters are valid and supported, it is recommended that this macro returns the same result as `PSA_EXPORT_KEY_OUTPUT_SIZE(PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(key_type), key_bits)`.

3.2.1.2.36 macro `PSA_HASH_BLOCK_LENGTH`

`PSA_HASH_BLOCK_LENGTH`(alg)

The input block size of a hash algorithm, in bytes.

Parameters

- **alg** – A hash algorithm (`PSA_ALG_XXX` value such that `PSA_ALG_IS_HASH(alg)` is true).

3.2.1.2.36.1 Description

Hash algorithms process their input data in blocks. Hash operations will retain any partial blocks until they have enough input to fill the block or until the operation is finished.

This affects the output from `psa_hash_suspend()`.

3.2.1.2.36.2 Return

The block size in bytes for the specified hash algorithm. If the hash algorithm is not recognized, return 0. An implementation can return either 0 or the correct size for a hash algorithm that it recognizes, but does not support.

3.2.1.2.37 macro `PSA_HASH_LENGTH`

`PSA_HASH_LENGTH(alg)`

The size of the output of `psa_hash_compute()` and `psa_hash_finish()`, in bytes.

Parameters

- **alg** – A hash algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_HASH(alg)` is true), or an HMAC algorithm (`PSA_ALG_HMAC(hash_alg)` where `hash_alg` is a hash algorithm).

3.2.1.2.37.1 Description

This is also the hash length that `psa_hash_compare()` and `psa_hash_verify()` expect.

See also `PSA_HASH_MAX_SIZE`.

3.2.1.2.37.2 Return

The hash length for the specified hash algorithm. If the hash algorithm is not recognized, return 0. An implementation can return either 0 or the correct size for a hash algorithm that it recognizes, but does not support.

3.2.1.2.38 `PSA_HASH_MAX_SIZE`

Maximum size of a hash.

This macro must expand to a compile-time constant integer. It is recommended that this value is the maximum size of a hash supported by the implementation, in bytes. The value must not be smaller than this maximum.

See also `PSA_HASH_LENGTH()`.

3.2.1.2.39 `PSA_HASH_SUSPEND_ALGORITHM_FIELD_LENGTH`

The size of the algorithm field that is part of the output of `psa_hash_suspend()`, in bytes.

Warning: Not supported

Applications can use this value to unpack the hash suspend state that is output by `psa_hash_suspend()`.

3.2.1.2.40 macro `PSA_HASH_SUSPEND_HASH_STATE_FIELD_LENGTH`

`PSA_HASH_SUSPEND_HASH_STATE_FIELD_LENGTH(alg)`

The size of the hash-state field that is part of the output of `psa_hash_suspend()`, in bytes.

Parameters

- **alg** – A hash algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_HASH(alg)` is true).

3.2.1.2.40.1 Description

Applications can use this value to unpack the hash suspend state that is output by `psa_hash_suspend()`.

3.2.1.2.40.2 Return

The size, in bytes, of the hash-state field of the hash suspend state for the specified hash algorithm. If the hash algorithm is not recognized, return 0. An implementation can return either 0 or the correct size for a hash algorithm that it recognizes, but does not support.

3.2.1.2.41 macro `PSA_HASH_SUSPEND_INPUT_LENGTH_FIELD_LENGTH`

`PSA_HASH_SUSPEND_INPUT_LENGTH_FIELD_LENGTH(alg)`

The size of the input-length field that is part of the output of `psa_hash_suspend()`, in bytes.

Parameters

- **alg** – A hash algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_HASH(alg)` is true).

3.2.1.2.41.1 Description

Applications can use this value to unpack the hash suspend state that is output by `psa_hash_suspend()`.

3.2.1.2.41.2 Return

The size, in bytes, of the input-length field of the hash suspend state for the specified hash algorithm. If the hash algorithm is not recognized, return 0. An implementation can return either 0 or the correct size for a hash algorithm that it recognizes, but does not support.

3.2.1.2.42 `PSA_HASH_SUSPEND_OUTPUT_MAX_SIZE`

A sufficient hash suspend state buffer size for `psa_hash_suspend()`, for any supported hash algorithms.

Warning: Not supported

See also `PSA_HASH_SUSPEND_OUTPUT_SIZE()`.

3.2.1.2.43 macro `PSA_HASH_SUSPEND_OUTPUT_SIZE`

`PSA_HASH_SUSPEND_OUTPUT_SIZE(alg)`

A sufficient hash suspend state buffer size for `psa_hash_suspend()`.

Parameters

- **alg** – A hash algorithm (PSA_ALG_XXX value such that `PSA_ALG_IS_HASH(alg)` is true).

3.2.1.2.43.1 Description

If the size of the hash state buffer is at least this large, it is guaranteed that `psa_hash_suspend()` will not fail due to an insufficient buffer size. The actual size of the output might be smaller in any given call.

See also `PSA_HASH_SUSPEND_OUTPUT_MAX_SIZE`.

3.2.1.2.43.2 Return

A sufficient output size for the algorithm. If the hash algorithm is not recognized, or is not supported by `psa_hash_suspend()`, return 0. An implementation can return either 0 or a correct size for a hash algorithm that it recognizes, but does not support.

For a supported hash algorithm `alg`, the following expression is true:

```
PSA_HASH_SUSPEND_OUTPUT_SIZE(alg) == PSA_HASH_SUSPEND_ALGORITHM_FIELD_LENGTH +
                                     PSA_HASH_SUSPEND_INPUT_LENGTH_FIELD_
                                     ↪LENGTH(alg) +
                                     PSA_HASH_SUSPEND_HASH_STATE_FIELD_
                                     ↪LENGTH(alg) +
                                     PSA_HASH_BLOCK_LENGTH(alg) - 1
```

3.2.1.2.44 macro PSA_MAC_TRUNCATED_LENGTH

PSA_MAC_TRUNCATED_LENGTH(`alg`)

Size of the truncated MAC algorithm in bytes.

Parameters

- **alg** – A MAC algorithm (such that `PSA_ALG_IS_MAC_TRUNCATED(alg)` is true).

3.2.1.2.44.1 Return

The MAC truncated length for the specified algorithm. 0 if the algorithm is not a MAC or a truncated MAC algorithm.

3.2.1.2.45 macro PSA_HMAC_LENGTH

PSA_HMAC_LENGTH(`alg`)

Size of the HMAC output length in bytes.

Parameters

- **alg** – A MAC algorithm (such that `PSA_ALG_IS_HMAC(alg)` is true).

3.2.1.2.45.1 Return

The MAC length for the specified algorithm. 0 if the MAC algorithm is not HMAC.

3.2.1.2.46 macro PSA_MAC_LENGTH

PSA_MAC_LENGTH(`key_type`, `key_bits`, `alg`)

The size of the output of `psa_mac_compute()` and `psa_mac_sign_finish()`, in bytes.

Parameters

- **key_type** – The type of the MAC key.
- **key_bits** – The size of the MAC key in bits.
- **alg** – A MAC algorithm (such that `PSA_ALG_IS_MAC(alg)` is true).

3.2.1.2.46.1 Description

This is also the MAC length that *psa_mac_verify()* and *psa_mac_verify_finish()* expect.

See also `PSA_MAC_MAX_SIZE`.

3.2.1.2.46.2 Return

The MAC length for the specified algorithm with the specified key parameters.

0 if the MAC algorithm is not recognized.

Either 0 or the correct length for a MAC algorithm that the implementation recognizes, but does not support.

3.2.1.2.47 PSA_MAC_MAX_SIZE

Maximum size of a MAC.

This macro must expand to a compile-time constant integer. The maximum MAC size is based on the maximum hash size supported by HMAC

See also *PSA_MAC_LENGTH()*.

3.2.1.2.48 PSA_RAW_KEY_AGREEMENT_OUTPUT_MAX_SIZE

Maximum size of the output from *psa_raw_key_agreement()*.

Warning: Not supported

This macro must expand to a compile-time constant integer. It is recommended that this value is the maximum size of the output any raw key agreement algorithm supported by the implementation, in bytes. The value must not be smaller than this maximum.

See also *PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE()*.

3.2.1.2.49 macro PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE

PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE(key_type, key_bits)

Sufficient output buffer size for *psa_raw_key_agreement()*.

Parameters

- **key_type** – A supported key type.
- **key_bits** – The size of the key in bits.

3.2.1.2.49.1 Description

Warning: Not supported

This macro returns a compile-time constant if its arguments are compile-time constants.

Warning:

This function might evaluate its arguments multiple times or zero times. Providing arguments that have side effects will result in implementation-specific behavior, and is non-portable.

See also `PSA_RAW_KEY_AGREEMENT_OUTPUT_MAX_SIZE`.

3.2.1.2.49.2 Return

If the parameters are valid and supported, return a buffer size in bytes that guarantees that `psa_raw_key_agreement()` will not fail with `PSA_ERROR_BUFFER_TOO_SMALL`. If the parameters are a valid combination that is not supported by the implementation, this macro must return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

3.2.1.2.50 PSA_ECC_SIGNATURE_SIZE

Size of an elliptic curve signature.

key_bits: The size of the key in bits.

3.2.1.2.51 PSA_SIGNATURE_MAX_SIZE

Maximum size of an asymmetric signature.

This macro must expand to a compile-time constant integer. It is recommended that this value is the maximum size of an asymmetric signature supported by the implementation, in bytes. The value must not be smaller than this maximum.

3.2.1.2.52 macro PSA_SIGN_OUTPUT_SIZE

PSA_SIGN_OUTPUT_SIZE(key_type, key_bits, alg)

Sufficient signature buffer size for `psa_sign_message()` and `psa_sign_hash()`.

Parameters

- **key_type** – An asymmetric key type. This can be a key pair type or a public key type.
- **key_bits** – The size of the key in bits.
- **alg** – The signature algorithm.

3.2.1.2.52.1 Description

This macro returns a sufficient buffer size for a signature using a key of the specified type and size, with the specified algorithm. Note that the actual size of the signature might be smaller, as some algorithms produce a variable-size signature.

See also `PSA_SIGNATURE_MAX_SIZE`.

3.2.1.2.52.2 Return

If the parameters are valid and supported, return a buffer size in bytes that guarantees that `psa_sign_message()` and `psa_sign_hash()` will not fail with `PSA_ERROR_BUFFER_TOO_SMALL`. If the parameters are a valid combination that is not supported by the implementation, this macro must return either a sensible size or 0. If the parameters are not valid, the return value is unspecified.

3.2.1.2.53 PSA_TLS12_ECJPAKE_TO_PMS_OUTPUT_SIZE

The size of the output from the TLS 1.2 ECJPAKE-to-PMS key-derivation algorithm, in bytes.

This value can be used when extracting the result of a key-derivation operation that was set up with the `PSA_ALG_TLS12_ECJPAKE_TO_PMS` algorithm.

3.2.1.2.54 PSA_TLS12_PSK_TO_MS_PSK_MAX_SIZE

This macro returns the maximum supported length of the PSK for the TLS-1.2 PSK-to-MS key derivation.

Warning: Not supported

This implementation-defined value specifies the maximum length for the PSK input used with a `PSA_ALG_TLS12_PSK_TO_MS()` key agreement algorithm.

Quoting Pre-Shared Key Ciphersuites for Transport Layer Security (TLS) [RFC4279] §5.3:

TLS implementations supporting these cipher suites MUST support arbitrary PSK identities up to 128 octets in length, and arbitrary PSKs up to 64 octets in length. Supporting longer identities and keys is RECOMMENDED.

Therefore, it is recommended that implementations define `PSA_TLS12_PSK_TO_MS_PSK_MAX_SIZE` with a value greater than or equal to 64.

3.2.1.3 Types

3.2.1.3.1 Introduction

This file declares types that encode errors, algorithms, key types, policies, etc.

3.2.1.3.2 Reference

Documentation:

PSA Cryptography API v1.2.1

Link:

<https://arm-software.github.io/psa-api/crypto/1.2/about>

3.2.1.3.3 typedef `psa_algorithm_t`

type `psa_algorithm_t`

Encoding of a cryptographic algorithm.

3.2.1.3.3.1 Description

This is a structured bitfield that identifies the category and type of algorithm. The range of algorithm identifier values is divided as follows:

- **0x00000000**
Reserved as an invalid algorithm identifier.
- **0x00000001 - 0x7ffffff**
Specification-defined algorithm identifiers. Algorithm identifiers defined by this standard always have bit 31 clear. Unallocated algorithm identifier values in this range are reserved for future use.

- **0x80000000 - 0xffffffff**

Implementation-defined algorithm identifiers. Implementations that define additional algorithms must use an encoding with bit 31 set. The related support macros will be easier to write if these algorithm identifier encodings also respect the bitwise structure used by standard encodings.

For algorithms that can be applied to multiple key types, this identifier does not encode the key type. For example, for symmetric ciphers based on a block cipher, `typedef psa_algorithm_t` encodes the block cipher mode and the padding mode while the block cipher itself is encoded via `typedef psa_key_type_t`.

3.2.1.3.3.2 Values

- PSA_ALG_ANY_HASH
- PSA_ALG_CBC_MAC
- PSA_ALG_CBC_NO_PADDING
- PSA_ALG_CBC_PKCS7
- PSA_ALG_CCM
- PSA_ALG_CFB
- PSA_ALG_CHACHA20_POLY1305
- PSA_ALG_CMAC
- PSA_ALG_CTR
- PSA_ALG_ECB_NO_PADDING
- PSA_ALG_ECDH
- PSA_ALG_ECDSA_ANY
- PSA_ALG_FFDH
- PSA_ALG_GCM
- PSA_ALG_MD2
- PSA_ALG_MD4
- PSA_ALG_MD5
- PSA_ALG_NONE
- PSA_ALG_OFB
- PSA_ALG_RIPEMD160
- PSA_ALG_RSA_PKCS1V15_CRYPT
- PSA_ALG_RSA_PKCS1V15_SIGN_RAW
- PSA_ALG_SHA3_224
- PSA_ALG_SHA3_256
- PSA_ALG_SHA3_384
- PSA_ALG_SHA3_512

- PSA_ALG_SHA_1
- PSA_ALG_SHA_224
- PSA_ALG_SHA_256
- PSA_ALG_SHA_384
- PSA_ALG_SHA_512
- PSA_ALG_SHA_512_224
- PSA_ALG_SHA_512_256
- PSA_ALG_SM3
- PSA_ALG_STREAM_CIPHER
- PSA_ALG_XTS

3.2.1.3.4 typedef `psa_dh_family_t`

type `psa_dh_family_t`

The type of PSA finite-field Diffie-Hellman group family identifiers.

3.2.1.3.4.1 Description

The group family identifier is required to create an Diffie-Hellman key using the `PSA_KEY_TYPE_DH_KEY_PAIR()` or `PSA_KEY_TYPE_DH_PUBLIC_KEY()` macros.

The specific Diffie-Hellman group within a family is identified by the `key_bits` attribute of the key.

The range of Diffie-Hellman group family identifier values is divided as follows:

- **0x00 - 0x7f**
DH group family identifiers defined by this standard. Unallocated values in this range are reserved for future use.
- **0x80 - 0xff**
Implementations that define additional families must use an encoding in this range.

3.2.1.3.5 typedef `psa_ecc_family_t`

type `psa_ecc_family_t`

The type of PSA elliptic curve family identifiers.

3.2.1.3.5.1 Description

The curve identifier is required to create an ECC key using the `PSA_KEY_TYPE_ECC_KEY_PAIR()` or `PSA_KEY_TYPE_ECC_PUBLIC_KEY()` macros.

The specific ECC curve within a family is identified by the `key_bits` attribute of the key.

The range of Elliptic curve family identifier values is divided as follows:

- **0x00 - 0x7f**
ECC family identifiers defined by this standard. Unallocated values in this range are reserved for future use.

-
- **0x80 - 0xff**

Implementations that define additional families must use an encoding in this range.

3.2.1.3.6 typedef `psa_key_derivation_step_t`

type `psa_key_derivation_step_t`

Encoding of the step of a key derivation.

3.2.1.3.7 typedef `psa_key_id_t`

type `psa_key_id_t`

Key identifier.

3.2.1.3.7.1 Description

A key identifier can be a permanent name for a persistent key, or a transient reference to a volatile key.

3.2.1.3.8 typedef `psa_key_lifetime_t`

type `psa_key_lifetime_t`

Encoding of key lifetimes.

3.2.1.3.8.1 Description

The lifetime of a key indicates where it is stored and which application and system actions will create and destroy it.

Lifetime values have the following structure:

- Bits[7:0]: Persistence level

This value indicates what device management actions can cause it to be destroyed. In particular, it indicates whether the key is volatile or persistent. See [typedef `psa\_key\_persistence\_t`](#) for more information.

`PSA_KEY_LIFETIME_GET_PERSISTENCE(lifetime)` returns the persistence level for a key lifetime value.

- Bits[31:8]: Location indicator

This value indicates where the key material is stored (or at least where it is accessible in cleartext) and where operations on the key are performed. See [typedef `psa\_key\_location\_t`](#) for more information.

`PSA_KEY_LIFETIME_GET_LOCATION(lifetime)` returns the location indicator for a key lifetime value.

Volatile keys (`PSA_KEY_LIFETIME_VOLATILE`) are automatically destroyed when the application instance terminates or on a power reset of the device. Persistent keys are preserved until the application explicitly destroys them or until an implementation-specific device management event occurs, for example, a factory reset.

Persistent keys (`PSA_KEY_LIFETIME_PERSISTENT`) have a unique key identifier of type [typedef `psa\_key\_id\_t`](#) per application instantiating the library. This identifier remains valid throughout the lifetime of the key, even if the application instance that created the key terminates.

3.2.1.3.9 typedef `psa_key_location_t`

type `psa_key_location_t`

Encoding of key location indicators.

3.2.1.3.9.1 Description

If an implementation of this API can make calls to external cryptoprocessors such as secure elements, the location of a key indicates which secure element performs the operations on the key. If the key material is not stored persistently inside the secure element, it must be stored in a wrapped form such that only the secure element can access the key material in cleartext.

Values for location indicators defined by this specification are shown below.

Location indicator	Definition
0	Primary local storage. The primary local storage is typically the same storage area that contains the key metadata.
1	Primary secure element. SECO or ELE Secure Subsystems are primary secure elements. As a guideline, secure elements may provide higher resistance against side channel and physical attacks than the primary local storage, but may have restrictions on supported key types, sizes, policies and operations and may have different performance characteristics.
2 - 0x7ffff	Other locations defined by a PSA specification. The PSA Cryptography API does not currently assign any meaning to these locations, but future versions of this specification or other PSA specifications may do so.
0x800000 - 0xfffff	Vendor-defined locations. No PSA specification will assign a meaning to locations in this range.

3.2.1.3.9.2 Note

Key location indicators are 24-bit values. Key management interfaces operate on lifetimes (see `typedef psa_key_lifetime_t`), and encode the location as the upper 24 bits of a 32-bit value.

3.2.1.3.10 typedef `psa_key_persistence_t`

type `psa_key_persistence_t`

Encoding of key persistence levels.

3.2.1.3.10.1 Description

What distinguishes different persistence levels is which device management events can cause keys to be destroyed. For example, power reset, transfer of device ownership, or a factory reset are device management events that can affect keys at different persistence levels. The specific management events which affect persistent keys at different levels is outside the scope of the PSA Cryptography specification.

Values for persistence levels defined by this specification are shown below.

Persistence level	Definition
0 = PSA_KEY_PERSISTENCE_VOLATILE	Volatile key. A volatile key is automatically destroyed by the implementation when the application instance terminates. In particular, a volatile key is automatically destroyed on a power reset of the device.
1 = PSA_KEY_PERSISTENCE_DEFAULT	Persistent key with a default lifetime. Applications should use this value if they have no specific needs that are only met by implementation-specific features.
2 - 127	Persistent key with a PSA-specified lifetime. The PSA Cryptography specification does not define the meaning of these values, but other PSA specifications may do so.
128 - 254	Persistent key with a vendor-specified lifetime. No PSA specification will define the meaning of these values, so implementations may choose the meaning freely. As a guideline, higher persistence levels should cause a key to survive more management events than lower levels.
255 = PSA_KEY_PERSISTENCE_READ_ONLY	Read-only or write-once key. A key with this persistence level cannot be destroyed. Note that keys that are read-only due to policy restrictions rather than due to physical limitations should not have this persistence level.

3.2.1.3.10.2 Note

Key persistence levels are 8-bit values. Key management interfaces operate on lifetimes (see [typedef psa_key_lifetime_t](#)), and encode the persistence value as the lower 8 bits of a 32-bit value.

3.2.1.3.11 typedef `psa_key_type_t`

type `psa_key_type_t`

Encoding of a key type.

3.2.1.3.11.1 Description

This is a structured bitfield that identifies the category and type of key. The range of key type values is divided as follows:

- **PSA_KEY_TYPE_NONE == 0**
Reserved as an invalid key type.
- **0x0001 - 0x7fff**
Specification-defined key types. Key types defined by this standard always have bit 15 clear. Unallocated key type values in this range are reserved for future use.
- **0x8000 - 0xffff**
No additional key type is defined.

3.2.1.3.12 typedef `psa_key_usage_t`

type `psa_key_usage_t`

Encoding of permitted usage on a key.

3.2.1.4 Structures

3.2.1.4.1 Introduction

This file contains the definitions of the data structures exposed by the PSA Cryptography API.

3.2.1.4.2 Reference

Documentation:

PSA Cryptography API v1.2.1

Link:

<https://arm-software.github.io/psa-api/crypto/1.2/about>

3.2.1.4.3 `PSA_AEAD_OPERATION_INIT`

This macro returns a suitable initializer for an AEAD operation object of type *typedef* `psa_aead_operation_t`.

3.2.1.4.4 `PSA_CIPHER_OPERATION_INIT`

This macro returns a suitable initializer for a cipher operation object of type *typedef* `psa_cipher_operation_t`.

3.2.1.4.5 `PSA_HASH_OPERATION_INIT`

This macro returns a suitable initializer for a hash operation object of type *typedef* `psa_hash_operation_t`.

3.2.1.4.6 PSA_KEY_DERIVATION_OPERATION_INIT

This macro returns a suitable initializer for a key derivation operation object of type *typedef psa_key_derivation_operation_t*.

3.2.1.4.7 PSA_MAC_OPERATION_INIT

This macro returns a suitable initializer for a MAC operation object of type *typedef psa_mac_operation_t*.

3.2.1.5 Functions

3.2.1.5.1 Introduction

This file contains the declarations of the crypto functions supported by the PSA Cryptography API.

3.2.1.5.2 Reference

Documentation:

PSA Cryptography API v1.2.1

Link:

<https://arm-software.github.io/psa-api/crypto/1.2/about>

3.2.1.5.3 typedef psa_aead_operation_t

type **psa_aead_operation_t**

The type of the state object for multi-part AEAD operations.

3.2.1.5.3.1 Description

Before calling any function on an AEAD operation object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_aead_operation_t operation;  
memset(&operation, 0, sizeof(operation));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_aead_operation_t operation;
```

- Initialize the object to the initializer PSA_AEAD_OPERATION_INIT, for example:

```
psa_aead_operation_t operation = PSA_AEAD_OPERATION_INIT;
```

- Assign the result of the function *psa_aead_operation_init()* to the object, for example:

```
psa_aead_operation_t operation;
```

This is an implementation-defined type. Application should not make any assumptions about the content of this object.

3.2.1.5.4 typedef `psa_cipher_operation_t`

type `psa_cipher_operation_t`

The type of the state object for multi-part cipher operations.

3.2.1.5.4.1 Description

Before calling any function on a cipher operation object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_cipher_operation_t operation;  
memset(&operation, 0, sizeof(operation));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_cipher_operation_t operation;
```

- Initialize the object to the initializer `PSA_CIPHER_OPERATION_INIT`, for example:

```
psa_cipher_operation_t operation = PSA_CIPHER_OPERATION_INIT;
```

- Assign the result of the function `psa_cipher_operation_init()` to the object, for example:

```
psa_cipher_operation_t operation;  
operation = psa_cipher_operation_init();
```

This is an implementation-defined type. Application should not make any assumptions about the content of this object.

3.2.1.5.5 typedef `psa_hash_operation_t`

type `psa_hash_operation_t`

The type of the state object for multi-part hash operations.

3.2.1.5.5.1 Description

Before calling any function on a hash operation object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_hash_operation_t operation;  
memset(&operation, 0, sizeof(operation));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_hash_operation_t operation;
```

- Initialize the object to the initializer `PSA_HASH_OPERATION_INIT`, for example:

```
psa_hash_operation_t operation = PSA_HASH_OPERATION_INIT;
```

- Assign the result of the function `psa_hash_operation_init()` to the object, for example:

```
psa_hash_operation_t operation;  
operation = psa_hash_operation_init();
```

This is an implementation-defined type. Application should not make any assumptions about the content of this object.

3.2.1.5.6 typedef `psa_key_attributes_t`

type **`psa_key_attributes_t`**

The type of an object containing key attributes.

3.2.1.5.6.1 Description

This is the object that represents the metadata of a key object. Metadata that can be stored in attributes includes:

- The location of the key in storage, indicated by its key identifier and its lifetime.
- The key's policy, comprising usage flags and a specification of the permitted algorithm(s).
- Information about the key itself: the key type and its size.
- Implementation specific attributes.

The actual key material is not considered an attribute of a key. Key attributes do not contain information that is generally considered highly confidential.

This is an implementation-defined type. Application should not make any assumptions about the content of this object.

Each attribute of this object is set with a function `psa_set_key_xxx()` and retrieved with a function `psa_get_key_xxx()`.

An attribute object can contain references to auxiliary resources, for example pointers to allocated memory or indirect references to pre-calculated values. In order to free such resources, the application must call `psa_reset_key_attributes()`. As an exception, calling `psa_reset_key_attributes()` on an attribute object is optional if the object has only been modified by the following functions since it was initialized or last reset with `psa_reset_key_attributes()`:

- `psa_set_key_id()`
- `psa_set_key_lifetime()`
- `psa_set_key_type()`
- `psa_set_key_bits()`
- `psa_set_key_usage_flags()`
- `psa_set_key_algorithm()`

Before calling any function on a key attribute object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_key_attributes_t attributes;
memset(&attributes, 0, sizeof(attributes));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_key_attributes_t attributes;
```

- Initialize the object to the initializer `PSA_KEY_ATTRIBUTES_INIT`, for example:

```
psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
```

- Assign the result of the function `psa_key_attributes_init()` to the object, for example:

```
psa_key_attributes_t attributes;
attributes = psa_key_attributes_init();
```

A freshly initialized attribute object contains the following values:

Attribute	Value
lifetime	<code>PSA_KEY_LIFETIME_VOLATILE</code> .
key identifier	<code>PSA_KEY_ID_NULL</code> - which is not a valid key identifier.
type	<code>PSA_KEY_TYPE_NONE</code> - meaning that the type is unspecified.
key size	0 - meaning that the size is unspecified.
usage flags	0 - which allows no usage except exporting a public key.
algorithm	<code>PSA_ALG_NONE</code> - which does not allow cryptographic usage, but allows exporting.

Usage:

A typical sequence to create a key is as follows:

1. Create and initialize an attribute object.
2. If the key is persistent, call `psa_set_key_id()`. Also call `psa_set_key_lifetime()` to place the key in a non-default location.
3. Set the key policy with `psa_set_key_usage_flags()` and `psa_set_key_algorithm()`.
4. Set the key type with `psa_set_key_type()`. Skip this step if copying an existing key with `psa_copy_key()`.
5. When generating a random key with `psa_generate_key()` or deriving a key with `psa_key_derivation_output_key()`, set the desired key size with `psa_set_key_bits()`.
6. Call a key creation function: `psa_import_key()`, `psa_generate_key()`, `psa_key_derivation_output_key()` or `psa_copy_key()`. This function reads the attribute object, creates a key with these attributes, and outputs an identifier for the newly created key.

-
7. Optionally call `psa_reset_key_attributes()`, now that the attribute object is no longer needed. Currently this call is not required as the attributes defined in this specification do not require additional resources beyond the object itself.

A typical sequence to query a key's attributes is as follows:

1. Call `psa_get_key_attributes()`.
2. Call `psa_get_key_xxx()` functions to retrieve the required attribute(s).
3. Call `psa_reset_key_attributes()` to free any resources that can be used by the attribute object.

Once a key has been created, it is impossible to change its attributes.

3.2.1.5.7 typedef `psa_key_derivation_operation_t`

type `psa_key_derivation_operation_t`

The type of the state object for key derivation operations.

3.2.1.5.7.1 Description

Before calling any function on a key derivation operation object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_key_derivation_operation_t operation;  
memset(&operation, 0, sizeof(operation));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_key_derivation_operation_t operation;
```

- Initialize the object to the initializer `PSA_KEY_DERIVATION_OPERATION_INIT`, for example:

```
psa_key_derivation_operation_t operation = PSA_KEY_DERIVATION_OPERATION_  
↪INIT;
```

- Assign the result of the function `psa_key_derivation_operation_init()` to the object, for example:

```
psa_key_derivation_operation_t operation;  
operation = psa_key_derivation_operation_init();
```

This is an implementation-defined type. Application should not make any assumptions about the content of this object.

3.2.1.5.8 typedef `psa_mac_operation_t`

type `psa_mac_operation_t`

The type of the state object for multi-part MAC operations.

3.2.1.5.8.1 Description

Before calling any function on a MAC operation object, the application must initialize it by any of the following means:

- Set the object to all-bits-zero, for example:

```
psa_mac_operation_t operation;  
memset(&operation, 0, sizeof(operation));
```

- Initialize the object to logical zero values by declaring the object as static or global without an explicit initializer, for example:

```
static psa_mac_operation_t operation;
```

- Initialize the object to the initializer `PSA_MAC_OPERATION_INIT`, for example:

```
psa_mac_operation_t operation = PSA_MAC_OPERATION_INIT;
```

- Assign the result of the function `psa_mac_operation_init()` to the object, for example:

```
psa_mac_operation_t operation;  
operation = psa_mac_operation_init();
```

This is an implementation-defined type. Application should not make any assumptions about the content of this object.

3.2.1.5.9 PSA_CRYPTO_API_VERSION_MAJOR

The major version of this implementation of the PSA Crypto API.

3.2.1.5.10 PSA_CRYPTO_API_VERSION_MINOR

The minor version of this implementation of the PSA Crypto API.

3.2.1.5.11 PSA_KEY_DERIVATION_UNLIMITED_CAPACITY

Use the maximum possible capacity for a key derivation operation.

Use this value as the capacity argument when setting up a key derivation to specify that the operation will use the maximum possible capacity. The value of the maximum possible capacity depends on the key derivation algorithm.

3.2.1.5.12 `psa_aead_abort`

psa_status_t **psa_aead_abort**(*psa_aead_operation_t* *operation)

Abort an AEAD operation.

Parameters

- **operation** (*psa_aead_operation_t**) – Initialized AEAD operation.

3.2.1.5.12.1 Description

Warning: Not supported

Aborting an operation frees all associated resources except for the operation object itself. Once aborted, the operation object can be reused for another operation by calling `psa_aead_encrypt_setup()` or `psa_aead_decrypt_setup()` again.

This function can be called any time after the operation object has been initialized as described in `typedef psa_aead_operation_t`.

In particular, calling `psa_aead_abort()` after the operation has been terminated by a call to `psa_aead_abort()`, `psa_aead_finish()` or `psa_aead_verify()` is safe and has no effect.

3.2.1.5.12.2 Return

- `PSA_SUCCESS`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- **`PSA_ERROR_BAD_STATE`:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.13 `psa_aead_decrypt`

`psa_status_t` **`psa_aead_decrypt`**(`psa_key_id_t` key, `psa_algorithm_t` alg, const uint8_t *nonce, size_t nonce_length, const uint8_t *additional_data, size_t additional_data_length, const uint8_t *ciphertext, size_t ciphertext_length, uint8_t *plaintext, size_t plaintext_size, size_t *plaintext_length)

Process an authenticated decryption operation.

Parameters

- **key** (`psa_key_id_t`) – Identifier of the key to use for the operation. It must allow the usage `PSA_KEY_USAGE_DECRYPT`.
- **alg** (`psa_algorithm_t`) – The AEAD algorithm to compute (PSA_ALG_XXX value such that `PSA_ALG_IS_AEAD(alg)` is true).
- **nonce** (const uint8_t*) – Nonce or IV to use.
- **nonce_length** (size_t) – Size of the nonce buffer in bytes. This must be appropriate for the selected algorithm. The default nonce size is `PSA_AEAD_NONCE_LENGTH(key_type, alg)` where `key_type` is the type of key.
- **additional_data** (const uint8_t*) – Additional data that has been authenticated but not encrypted.
- **additional_data_length** (size_t) – Size of `additional_data` in bytes.
- **ciphertext** (const uint8_t*) – Data that has been authenticated and encrypted. For algorithms where the encrypted data and the authentication

tag are defined as separate inputs, the buffer must contain the encrypted data followed by the authentication tag.

- **ciphertext_length** (size_t) – Size of ciphertext in bytes.
- **plaintext** (uint8_t*) – Output buffer for the decrypted data.
- **plaintext_size** (size_t) – Size of the plaintext buffer in bytes.
- **plaintext_length** (size_t*) – On success, the size of the output in the plaintext buffer.

3.2.1.5.13.1 Description

Parameter `plaintext_size` must be appropriate for the selected algorithm and key:

- A sufficient output size is `PSA_AEAD_DECRYPT_OUTPUT_SIZE(key_type, alg, ciphertext_length)` where `key_type` is the type of key.
- `PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE(ciphertext_length)` evaluates to the maximum plaintext size of any supported AEAD decryption.

3.2.1.5.13.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_INVALID_SIGNATURE:**
The ciphertext is not authentic.
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_DECRYPT` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with `alg`.
- **PSA_ERROR_NOT_SUPPORTED:**
`alg` is not supported or is not an AEAD algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_BUFFER_TOO_SMALL:**
`plaintext_size` is too small. [`PSA\_AEAD\_DECRYPT\_OUTPUT\_SIZE\(\)`](#) or [`PSA\_AEAD\_DECRYPT\_OUTPUT\_MAX\_SIZE\(\)`](#) can be used to determine the required buffer size.
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.14 `psa_aead_decrypt_setup`

`psa_status_t` **psa_aead_decrypt_setup**(`psa_aead_operation_t` *operation, `psa_key_id_t` key, `psa_algorithm_t` alg)

Set the key for a multi-part authenticated decryption operation.

Parameters

- **operation** (`psa_aead_operation_t`*) – The operation object to set up. It must have been initialized as per the documentation for `typedef psa_aead_operation_t` and not yet in use.
- **key** (`psa_key_id_t`) – Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_DECRYPT`.
- **alg** (`psa_algorithm_t`) – The AEAD algorithm to compute (PSA_ALG_XXX value such that `PSA_ALG_IS_AEAD(alg)` is true).

3.2.1.5.14.1 Description

Warning: Not supported

The sequence of operations to decrypt a message with authentication is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for `typedef psa_aead_operation_t`, e.g. `PSA_AEAD_OPERATION_INIT`.
3. Call `psa_aead_decrypt_setup()` to specify the algorithm and key.
4. If needed, call `psa_aead_set_lengths()` to specify the length of the inputs to the subsequent calls to `psa_aead_update_ad()` and `psa_aead_update()`. See the documentation of `psa_aead_set_lengths()` for details.
5. Call `psa_aead_set_nonce()` with the nonce for the decryption.
6. Call `psa_aead_update_ad()` zero, one or more times, passing a fragment of the non-encrypted additional authenticated data each time.
7. Call `psa_aead_update()` zero, one or more times, passing a fragment of the ciphertext to decrypt each time.
8. Call `psa_aead_verify()`.

If an error occurs at any step after a call to `psa_aead_decrypt_setup()`, the operation will need to be reset by a call to `psa_aead_abort()`. The application can call `psa_aead_abort()` at any time after the operation has been initialized.

After a successful call to `psa_aead_decrypt_setup()`, the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to `psa_aead_verify()`.
- A call to `psa_aead_abort()`.

3.2.1.5.14.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be inactive.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the PSA_KEY_USAGE_DECRYPT flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with alg.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not an AEAD algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.15 `psa_aead_encrypt`

psa_status_t **psa_aead_encrypt**(*psa_key_id_t* key, *psa_algorithm_t* alg, const uint8_t *nonce, size_t nonce_length, const uint8_t *additional_data, size_t additional_data_length, const uint8_t *plaintext, size_t plaintext_length, uint8_t *ciphertext, size_t ciphertext_size, size_t *ciphertext_length)

Process an authenticated encryption operation.

Parameters

- **key** (*psa_key_id_t*) – Identifier of the key to use for the operation. It must allow the usage PSA_KEY_USAGE_ENCRYPT.
- **alg** (*psa_algorithm_t*) – The AEAD algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_AEAD(alg) is true).
- **nonce** (const uint8_t*) – Nonce or IV to use.
- **nonce_length** (size_t) – Size of the nonce buffer in bytes. This must be appropriate for the selected algorithm. The default nonce size is

PSA_AEAD_NONCE_LENGTH(key_type, alg) where key_type is the type of key.

- **additional_data** (const uint8_t*) – Additional data that will be authenticated but not encrypted.
- **additional_data_length** (size_t) – Size of additional_data in bytes.
- **plaintext** (const uint8_t*) – Data that will be authenticated and encrypted.
- **plaintext_length** (size_t) – Size of plaintext in bytes.
- **ciphertext** (uint8_t*) – Output buffer for the authenticated and encrypted data. The additional data is not part of this output. For algorithms where the encrypted data and the authentication tag are defined as separate outputs, the authentication tag is appended to the encrypted data.
- **ciphertext_size** (size_t) – Size of the ciphertext buffer in bytes.
- **ciphertext_length** (size_t*) – On success, the size of the output in the ciphertext buffer.

3.2.1.5.15.1 Description

Parameter ciphertext_size must be appropriate for the selected algorithm and key:

- A sufficient output size is PSA_AEAD_ENCRYPT_OUTPUT_SIZE(key_type, alg, plaintext_length) where key_type is the type of key.
- PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE(plaintext_length) evaluates to the maximum ciphertext size of any supported AEAD encryption.

3.2.1.5.15.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the PSA_KEY_USAGE_ENCRYPT flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with alg.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not an AEAD algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_BUFFER_TOO_SMALL:**
ciphertext_size is too small. [PSA_AEAD_ENCRYPT_OUTPUT_SIZE\(\)](#) or [PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE\(\)](#) can be used to determine the required buffer size.
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**

- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **`PSA_ERROR_BAD_STATE:`**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.16 `psa_aead_encrypt_setup`

`psa_status_t` **`psa_aead_encrypt_setup`**(`psa_aead_operation_t` *operation, `psa_key_id_t` key, `psa_algorithm_t` alg)

Set the key for a multi-part authenticated encryption operation.

Parameters

- **operation** (`psa_aead_operation_t`*) – The operation object to set up. It must have been initialized as per the documentation for `typedef psa_aead_operation_t` and not yet in use.
- **key** (`psa_key_id_t`) – Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_ENCRYPT`.
- **alg** (`psa_algorithm_t`) – The AEAD algorithm to compute (PSA_ALG_XXX value such that `PSA_ALG_IS_AEAD(alg)` is true).

3.2.1.5.16.1 Description

Warning: Not supported

The sequence of operations to encrypt a message with authentication is as follows:

- Allocate an operation object which will be passed to all the functions listed here.
- Initialize the operation object with one of the methods described in the documentation for `typedef psa_aead_operation_t`, e.g. `PSA_AEAD_OPERATION_INIT`.
- Call `psa_aead_encrypt_setup()` to specify the algorithm and key.
- If needed, call `psa_aead_set_lengths()` to specify the length of the inputs to the subsequent calls to `psa_aead_update_ad()` and `psa_aead_update()`. See the documentation of `psa_aead_set_lengths()` for details.
- Call either `psa_aead_generate_nonce()` or `psa_aead_set_nonce()` to generate or set the nonce. It is recommended to use `psa_aead_generate_nonce()` unless the protocol being implemented requires a specific nonce value.
- Call `psa_aead_update_ad()` zero, one or more times, passing a fragment of the non-encrypted additional authenticated data each time.
- Call `psa_aead_update()` zero, one or more times, passing a fragment of the message to encrypt each time.
- Call `psa_aead_finish()`.

If an error occurs at any step after a call to `psa_aead_encrypt_setup()`, the operation will need to be reset by a call to `psa_aead_abort()`. The application can call `psa_aead_abort()` at any time after the operation has been initialized.

After a successful call to `psa_aead_encrypt_setup()`, the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to `psa_aead_finish()`.
- A call to `psa_aead_abort()`.

3.2.1.5.16.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be inactive.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_ENCRYPT` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with alg.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not an AEAD algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.17 `psa_aead_finish`

`psa_status_t` **psa_aead_finish**(`psa_aead_operation_t` *operation, uint8_t *ciphertext, size_t ciphertext_size, size_t *ciphertext_length, uint8_t *tag, size_t tag_size, size_t *tag_length)

Finish encrypting a message in an AEAD operation.

Parameters

- **operation** (`psa_aead_operation_t`*) – Active AEAD operation.

- **ciphertext** (uint8_t*) – Buffer where the last part of the ciphertext is to be written.
- **ciphertext_size** (size_t) – Size of the ciphertext buffer in bytes.
- **ciphertext_length** (size_t*) – On success, the number of bytes of returned ciphertext.
- **tag** (uint8_t*) – Buffer where the authentication tag is to be written.
- **tag_size** (size_t) – Size of the tag buffer in bytes.
- **tag_length** (size_t*) – On success, the number of bytes that make up the returned tag.

3.2.1.5.17.1 Description

Warning: Not supported

The operation must have been set up with [psa_aead_encrypt_setup\(\)](#).

This function finishes the authentication of the additional data formed by concatenating the inputs passed to preceding calls to [psa_aead_update_ad\(\)](#) with the plaintext formed by concatenating the inputs passed to preceding calls to [psa_aead_update\(\)](#).

This function has two output buffers:

- **ciphertext** contains trailing ciphertext that was buffered from preceding calls to [psa_aead_update\(\)](#).
- **tag** contains the authentication tag.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_aead_abort\(\)](#).

Parameter **ciphertext_size** must be appropriate for the selected algorithm and key:

- A sufficient output size is `PSA_AEAD_FINISH_OUTPUT_SIZE(key_type, alg)` where `key_type` is the type of key and `alg` is the algorithm that were used to set up the operation.
- `PSA_AEAD_FINISH_OUTPUT_MAX_SIZE` evaluates to the maximum output size of any supported AEAD algorithm.

Parameter **tag_size** must be appropriate for the selected algorithm and key:

- The exact tag size is `PSA_AEAD_TAG_LENGTH(key_type, key_bits, alg)` where `key_type` and `key_bits` are the type and bit-size of the key, and `alg` is the algorithm that were used in the call to [psa_aead_encrypt_setup\(\)](#).
- `PSA_AEAD_TAG_MAX_SIZE` evaluates to the maximum tag size of any supported AEAD algorithm.

3.2.1.5.17.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE**
The operation state is not valid: it must be an active encryption operation with a nonce set.

- **PSA_ERROR_BUFFER_TOO_SMALL**

The size of the ciphertext or tag buffer is too small. `PSA_AEAD_FINISH_OUTPUT_SIZE()` or `PSA_AEAD_FINISH_OUTPUT_MAX_SIZE` can be used to determine the required ciphertext buffer size. `PSA_AEAD_TAG_LENGTH()` or `PSA_AEAD_TAG_MAX_SIZE` can be used to determine the required tag buffer size.

- **PSA_ERROR_INVALID_ARGUMENT**

The total length of input to `psa_aead_update_ad()` so far is less than the additional data length that was previously specified with `psa_aead_set_lengths()`.

- **PSA_ERROR_INVALID_ARGUMENT**

The total length of input to `psa_aead_update()` so far is less than the plaintext length that was previously specified with `psa_aead_set_lengths()`.

- **PSA_ERROR_INSUFFICIENT_MEMORY**

- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_HARDWARE_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.18 `psa_aead_generate_nonce`

psa_status_t **psa_aead_generate_nonce**(*psa_aead_operation_t* *operation, uint8_t *nonce, size_t nonce_size, size_t *nonce_length)

Generate a random nonce for an authenticated encryption operation.

Parameters

- **operation** (*psa_aead_operation_t**) – Active AEAD operation.
- **nonce** (uint8_t*) – Buffer where the generated nonce is to be written.
- **nonce_size** (size_t) – Size of the nonce buffer in bytes. This must be at least `PSA_AEAD_NONCE_LENGTH(key_type, alg)` where `key_type` and `alg` are type of key and the algorithm respectively that were used to set up the AEAD operation.
- **nonce_length** (size_t*) – On success, the number of bytes of the generated nonce.

3.2.1.5.18.1 Description

Warning: Not supported

This function generates a random nonce for the authenticated encryption operation with an appropriate size for the chosen algorithm, key type and key size.

The application must call `psa_aead_encrypt_setup()` before calling this function. If applicable for the algorithm, the application must call `psa_aead_set_lengths()` before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_aead_abort()`.

3.2.1.5.18.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be an active AEAD encryption operation, with no nonce set.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: this is an algorithm which requires `psa_aead_set_lengths()` to be called before setting the nonce.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the nonce buffer is too small. `PSA_AEAD_NONCE_LENGTH()` or `PSA_AEAD_NONCE_MAX_SIZE` can be used to determine the required buffer size.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.19 `psa_aead_operation_init`

`psa_aead_operation_t` `psa_aead_operation_init`(void)

Return an initial value for an AEAD operation object.

Parameters

- **void** – no arguments

3.2.1.5.19.1 Description

Warning: Not supported

3.2.1.5.19.2 Return

typedef *psa_aead_operation_t*

3.2.1.5.20 *psa_aead_set_lengths*

psa_status_t ***psa_aead_set_lengths***(*psa_aead_operation_t* *operation, size_t ad_length, size_t plaintext_length)

Declare the lengths of the message and additional data for AEAD.

Parameters

- **operation** (*psa_aead_operation_t**) – Active AEAD operation.
- **ad_length** (size_t) – Size of the non-encrypted additional authenticated data in bytes.
- **plaintext_length** (size_t) – Size of the plaintext to encrypt in bytes.

3.2.1.5.20.1 Description

Warning: Not supported

The application must call this function before calling *psa_aead_set_nonce()* or *psa_aead_generate_nonce()*, if the algorithm for the operation requires it. If the algorithm does not require it, calling this function is optional, but if this function is called then the implementation must enforce the lengths.

- For PSA_ALG_CCM, calling this function is required.
- For the other AEAD algorithms defined in this specification, calling this function is not required.
- For vendor-defined algorithm, refer to the vendor documentation.

If this function returns an error status, the operation enters an error state and must be aborted by calling *psa_aead_abort()*.

3.2.1.5.20.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active, and *psa_aead_set_nonce()* and *psa_aead_generate_nonce()* must not have been called yet.
- **PSA_ERROR_INVALID_ARGUMENT:**
At least one of the lengths is not acceptable for the chosen algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by *psa_crypto_init()*. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.21 `psa_aead_set_nonce`

psa_status_t **psa_aead_set_nonce**(*psa_aead_operation_t* *operation, const uint8_t *nonce, size_t nonce_length)

Set the nonce for an authenticated encryption or decryption operation.

Parameters

- **operation** (*psa_aead_operation_t**) – Active AEAD operation.
- **nonce** (const uint8_t*) – Buffer containing the nonce to use.
- **nonce_length** (size_t) – Size of the nonce in bytes. This must be a valid nonce size for the chosen algorithm. The default nonce size is `PSA_AEAD_NONCE_LENGTH(key_type, alg)` where `key_type` and `alg` are type of key and the algorithm respectively that were used to set up the AEAD operation.

3.2.1.5.21.1 Description

This function sets the nonce for the authenticated encryption or decryption operation.

Warning: Not supported

The application must call `psa_aead_encrypt_setup()` or `psa_aead_decrypt_setup()` before calling this function. If applicable for the algorithm, the application must call `psa_aead_set_lengths()` before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_aead_abort()`.

Note:

When encrypting, `psa_aead_generate_nonce()` is recommended instead of using this function, unless implementing a protocol that requires a non-random IV.

3.2.1.5.21.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active, with no nonce set.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: this is an algorithm which requires `psa_aead_set_lengths()` to be called before setting the nonce.
- **PSA_ERROR_INVALID_ARGUMENT:**
The size of nonce is not acceptable for the chosen algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**

- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **`PSA_ERROR_BAD_STATE`:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.22 `psa_aead_update`

psa_status_t **`psa_aead_update`**(*psa_aead_operation_t* *operation, const uint8_t *input, size_t input_length, uint8_t *output, size_t output_size, size_t *output_length)

Encrypt or decrypt a message fragment in an active AEAD operation.

Parameters

- **operation** (*psa_aead_operation_t**) – Active AEAD operation.
- **input** (const uint8_t*) – Buffer containing the message fragment to encrypt or decrypt.
- **input_length** (size_t) – Size of the input buffer in bytes.
- **output** (uint8_t*) – Buffer where the output is to be written.
- **output_size** (size_t) – Size of the output buffer in bytes.
- **output_length** (size_t*) – On success, the number of bytes that make up the returned output.

3.2.1.5.22.1 Description

Warning: Not supported

The following must occur before calling this function:

- Call either `psa_aead_encrypt_setup()` or `psa_aead_decrypt_setup()`. The choice of setup function determines whether this function encrypts or decrypts its input.
- Set the nonce with `psa_aead_generate_nonce()` or `psa_aead_set_nonce()`.
- Call `psa_aead_update_ad()` to pass all the additional data.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_aead_abort()`.

This function does not require the input to be aligned to any particular block boundary. If the implementation can only process a whole block at a time, it must consume all the input provided, but it might delay the end of the corresponding output until a subsequent call to `psa_aead_update()`, `psa_aead_finish()` or `psa_aead_verify()` provides sufficient input. The amount of data that can be delayed in this way is bounded by `PSA_AEAD_UPDATE_OUTPUT_SIZE()`.

Parameter `output_size` must be appropriate for the selected algorithm and key:

- A sufficient output size is `PSA_AEAD_UPDATE_OUTPUT_SIZE(key_type, alg, input_length)` where `key_type` is the type of key and `alg` is the algorithm that were used to set up the operation.
- `PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE(input_length)` evaluates to the maximum output size of any supported AEAD algorithm.

3.2.1.5.22.2 Return

- **PSA_SUCCESS:**

Success.

- **Warning:**

When decrypting, do not use the output until `psa_aead_verify()` succeeds.

See the detailed warning.

- **PSA_ERROR_BAD_STATE:**

The operation state is not valid: it must be active, have a nonce set, and have lengths set if required by the algorithm.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the output buffer is too small. `PSA_AEAD_UPDATE_OUTPUT_SIZE()` or `PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE()` can be used to determine the required buffer size.

- **PSA_ERROR_INVALID_ARGUMENT:**

The total length of input to `psa_aead_update_ad()` so far is less than the additional data length that was previously specified with `psa_aead_set_lengths()`.

- **PSA_ERROR_INVALID_ARGUMENT:**

The total input length overflows the plaintext length that was previously specified with `psa_aead_set_lengths()`.

- **PSA_ERROR_INSUFFICIENT_MEMORY**

- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_HARDWARE_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.23 `psa_aead_update_ad`

`psa_status_t` **psa_aead_update_ad**(`psa_aead_operation_t` *operation, const uint8_t *input, size_t input_length)

Pass additional data to an active AEAD operation.

Parameters

- **operation** (`psa_aead_operation_t`*) – Active AEAD operation.
- **input** (const uint8_t*) – Buffer containing the fragment of additional data.
- **input_length** (size_t) – Size of the input buffer in bytes.

3.2.1.5.23.1 Description

Warning: Not supported

Additional data is authenticated, but not encrypted.

This function can be called multiple times to pass successive fragments of the additional data. This function must not be called after passing data to encrypt or decrypt with `psa_aead_update()`.

3.2.1.5.23.2 The following must occur before calling this function

- Call either `psa_aead_encrypt_setup()` or `psa_aead_decrypt_setup()`.
- Set the nonce with `psa_aead_generate_nonce()` or `psa_aead_set_nonce()`.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_aead_abort()`.

3.2.1.5.23.3 Return

- **PSA_SUCCESS:**
Success.
Warning:
When decrypting, do not trust the input until `psa_aead_verify()` succeeds.
See the detailed warning.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active, have a nonce set, have lengths set if required by the algorithm, and `psa_aead_update()` must not have been called yet.
- **PSA_ERROR_INVALID_ARGUMENT:**
The total input length overflows the additional data length that was previously specified with `psa_aead_set_lengths()`.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.24 `psa_aead_verify`

```
psa_status_t psa_aead_verify(psa_aead_operation_t *operation, uint8_t *plaintext, size_t  
plaintext_size, size_t *plaintext_length, const uint8_t *tag, size_t  
tag_length)
```

Finish authenticating and decrypting a message in an AEAD operation.

Parameters

- **operation** (*psa_aead_operation_t**) – Active AEAD operation.
- **plaintext** (*uint8_t**) – Buffer where the last part of the plaintext is to be written. This is the remaining data from previous calls to *psa_aead_update()* that could not be processed until the end of the input.
- **plaintext_size** (*size_t*) – Size of the plaintext buffer in bytes.
- **plaintext_length** (*size_t**) – On success, the number of bytes of returned plaintext.
- **tag** (*const uint8_t**) – Buffer containing the authentication tag.
- **tag_length** (*size_t*) – Size of the tag buffer in bytes.

3.2.1.5.24.1 Description

Warning: Not supported

The operation must have been set up with *psa_aead_decrypt_setup()*.

This function finishes the authenticated decryption of the message components:

- The additional data consisting of the concatenation of the inputs passed to preceding calls to *psa_aead_update_ad()*.
- The ciphertext consisting of the concatenation of the inputs passed to preceding calls to *psa_aead_update()*.
- The tag passed to this function call.

If the authentication tag is correct, this function outputs any remaining plaintext and reports success. If the authentication tag is not correct, this function returns `PSA_ERROR_INVALID_SIGNATURE`.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling *psa_aead_abort()*.

Note:

Implementations must make the best effort to ensure that the comparison between the actual tag and the expected tag is performed in constant time.

Parameter `plaintext_size` must be appropriate for the selected algorithm and key:

- A sufficient output size is `PSA_AEAD_VERIFY_OUTPUT_SIZE(key_type, alg)` where `key_type` is the type of key and `alg` is the algorithm that were used to set up the operation.
- `PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE` evaluates to the maximum output size of any supported AEAD algorithm.

3.2.1.5.24.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_SIGNATURE:**
The calculations were successful, but the authentication tag is not correct.

- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be an active decryption operation with a nonce set.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the plaintext buffer is too small. `PSA_AEAD_VERIFY_OUTPUT_SIZE()` or `PSA_AEAD_VERIFY_OUTPUT_MAX_SIZE` can be used to determine the required buffer size.
- **PSA_ERROR_INVALID_ARGUMENT:**
The total length of input to `psa_aead_update_ad()` so far is less than the additional data length that was previously specified with `psa_aead_set_lengths()`.
- **PSA_ERROR_INVALID_ARGUMENT:**
The total length of input to `psa_aead_update()` so far is less than the plaintext length that was previously specified with `psa_aead_set_lengths()`.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.25 `psa_asymmetric_decrypt`

`psa_status_t` **psa_asymmetric_decrypt**(`psa_key_id_t` key, `psa_algorithm_t` alg, const uint8_t *input, size_t input_length, const uint8_t *salt, size_t salt_length, uint8_t *output, size_t output_size, size_t *output_length)

Decrypt a short message with a private key.

Parameters

- **key** (`psa_key_id_t`) – Identifier of the key to use for the operation. It must be an asymmetric key pair. It must allow the usage `PSA_KEY_USAGE_DECRYPT`.
- **alg** (`psa_algorithm_t`) – An asymmetric encryption algorithm that is compatible with the type of key.
- **input** (const uint8_t*) – The message to decrypt.
- **input_length** (size_t) – Size of the input buffer in bytes.
- **salt** (const uint8_t*) – A salt or label, if supported by the encryption algorithm. If the algorithm does not support a salt, pass NULL. If the algorithm supports an optional salt, pass NULL to indicate that there is no salt.

- **salt_length** (size_t) – Size of the salt buffer in bytes. If salt is NULL, pass 0.
- **output** (uint8_t*) – Buffer where the decrypted message is to be written.
- **output_size** (size_t) – Size of the output buffer in bytes.
- **output_length** (size_t*) – On success, the number of bytes that make up the returned output.

3.2.1.5.25.1 Description

Warning: Not supported

For PSA_ALG_RSA_PKCS1V15_CRYPT, no salt is supported.

Parameter output_size must be appropriate for the selected algorithm and key:

- The required output size is PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE(key_type, key_bits, alg) where key_type and key_bits are the type and bit-size respectively of key.
- PSA_ASYMMETRIC_DECRYPT_OUTPUT_MAX_SIZE evaluates to the maximum output size of any supported asymmetric decryption.

3.2.1.5.25.2 Return

- PSA_SUCCESS
- PSA_ERROR_INVALID_HANDLE
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the PSA_KEY_USAGE_DECRYPT flag, or it does not permit the requested algorithm.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the output buffer is too small. [PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE\(\)](#) or PSA_ASYMMETRIC_DECRYPT_OUTPUT_MAX_SIZE can be used to determine the required buffer size.
- PSA_ERROR_NOT_SUPPORTED
- PSA_ERROR_INVALID_ARGUMENT
- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID
- PSA_ERROR_INSUFFICIENT_ENTROPY
- PSA_ERROR_INVALID_PADDING

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.26 `psa_asymmetric_encrypt`

```
psa_status_t psa_asymmetric_encrypt(psa_key_id_t key, psa_algorithm_t alg, const uint8_t
                                     *input, size_t input_length, const uint8_t *salt, size_t
                                     salt_length, uint8_t *output, size_t output_size, size_t
                                     *output_length)
```

Encrypt a short message with a public key.

Parameters

- **key** (*psa_key_id_t*) – Identifier of the key to use for the operation. It must be a public key or an asymmetric key pair. It must allow the usage `PSA_KEY_USAGE_ENCRYPT`.
- **alg** (*psa_algorithm_t*) – An asymmetric encryption algorithm that is compatible with the type of key.
- **input** (const uint8_t*) – The message to encrypt.
- **input_length** (size_t) – Size of the input buffer in bytes.
- **salt** (const uint8_t*) – A salt or label, if supported by the encryption algorithm. If the algorithm does not support a salt, pass NULL. If the algorithm supports an optional salt, pass NULL to indicate that there is no salt.
- **salt_length** (size_t) – Size of the salt buffer in bytes. If salt is NULL, pass 0.
- **output** (uint8_t*) – Buffer where the encrypted message is to be written.
- **output_size** (size_t) – Size of the output buffer in bytes.
- **output_length** (size_t*) – On success, the number of bytes that make up the returned output.

3.2.1.5.26.1 Description

Warning: Not supported

For `PSA_ALG_RSA_PKCS1V15_CRYPT`, no salt is supported.

Parameter `output_size` must be appropriate for the selected algorithm and key:

- The required output size is `PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE(key_type, key_bits, alg)` where `key_type` and `key_bits` are the type and bit-size respectively of `key`.
- `PSA_ASYMMETRIC_ENCRYPT_OUTPUT_MAX_SIZE` evaluates to the maximum output size of any supported asymmetric encryption.

3.2.1.5.26.2 Return

- `PSA_SUCCESS`
- `PSA_ERROR_INVALID_HANDLE`
- **`PSA_ERROR_NOT_PERMITTED:`**
The key does not have the `PSA_KEY_USAGE_ENCRYPT` flag, or it does not permit the requested algorithm.
- **`PSA_ERROR_BUFFER_TOO_SMALL:`**
The size of the output buffer is too small. `PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE()` or `PSA_ASYMMETRIC_ENCRYPT_OUTPUT_MAX_SIZE` can be used to determine the required buffer size.
- `PSA_ERROR_NOT_SUPPORTED`
- `PSA_ERROR_INVALID_ARGUMENT`
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- `PSA_ERROR_INSUFFICIENT_ENTROPY`
- **`PSA_ERROR_BAD_STATE:`**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.27 `psa_cipher_abort`

`psa_status_t` **`psa_cipher_abort`**(`psa_cipher_operation_t` *operation)

Abort a cipher operation.

Parameters

- **operation** (`psa_cipher_operation_t`*) – Initialized cipher operation.

3.2.1.5.27.1 Description

Warning: Not supported

Aborting an operation frees all associated resources except for the operation object itself. Once aborted, the operation object can be reused for another operation by calling `psa_cipher_encrypt_setup()` or `psa_cipher_decrypt_setup()` again.

This function can be called any time after the operation object has been initialized as described in `typedef psa_cipher_operation_t`.

In particular, calling `psa_cipher_abort()` after the operation has been terminated by a call to `psa_cipher_abort()` or `psa_cipher_finish()` is safe and has no effect.

3.2.1.5.27.2 Return

- PSA_SUCCESS
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.28 `psa_cipher_decrypt`

`psa_status_t` **psa_cipher_decrypt**(`psa_key_id_t` key, `psa_algorithm_t` alg, const uint8_t *input, size_t input_length, uint8_t *output, size_t output_size, size_t *output_length)

Decrypt a message using a symmetric cipher.

Parameters

- **key** (`psa_key_id_t`) – Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage PSA_KEY_USAGE_DECRYPT.
- **alg** (`psa_algorithm_t`) – The cipher algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_CIPHER(alg) is true).
- **input** (const uint8_t*) – Buffer containing the message to decrypt. This consists of the IV followed by the ciphertext proper.
- **input_length** (size_t) – Size of the input buffer in bytes.
- **output** (uint8_t*) – Buffer where the plaintext is to be written.
- **output_size** (size_t) – Size of the output buffer in bytes.
- **output_length** (size_t*) – On success, the number of bytes that make up the output.

3.2.1.5.28.1 Description

This function decrypts a message encrypted with a symmetric cipher.

The input to this function must contain the IV followed by the ciphertext, as output by `psa_cipher_encrypt()`. The IV must be PSA_CIPHER_IV_LENGTH(key_type, alg) bytes in length, where key_type is the type of key.

Use the multi-part operation interface with a `typedef psa_cipher_operation_t` object to decrypt data which is not in the expected input format.

Parameter output_size must be appropriate for the selected algorithm and key:

- A sufficient output size is PSA_CIPHER_DECRYPT_OUTPUT_SIZE(key_type, alg, input_length) where key_type is the type of key.
- PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE(input_length) evaluates to the maximum output size of any supported cipher decryption.

3.2.1.5.28.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_DECRYPT` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with alg.
- **PSA_ERROR_INVALID_ARGUMENT:**
The input_length is not valid for the algorithm and key type. For example, the algorithm is a based on block cipher and requires a whole number of blocks, but the total input size is not a multiple of the block size.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not a cipher algorithm.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
output_size is too small. `PSA_CIPHER_DECRYPT_OUTPUT_SIZE()` or `PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE()` can be used to determine the required buffer size.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.29 `psa_cipher_decrypt_setup`

`psa_status_t` **psa_cipher_decrypt_setup**(`psa_cipher_operation_t` *operation, `psa_key_id_t` key, `psa_algorithm_t` alg)

Set the key for a multi-part symmetric decryption operation.

Parameters

- **operation** (`psa_cipher_operation_t`*) – The operation object to set up. It must have been initialized as per the documentation for `typedef psa_cipher_operation_t` and not yet in use.
- **key** (`psa_key_id_t`) – Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_DECRYPT`.

-
- **alg** (*psa_algorithm_t*) – The cipher algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_CIPHER(alg) is true).

3.2.1.5.29.1 Description

Warning: Not supported

The sequence of operations to decrypt a message with a symmetric cipher is as follows:

- Allocate an operation object which will be passed to all the functions listed here.
- Initialize the operation object with one of the methods described in the documentation for *typedef psa_cipher_operation_t*, e.g. PSA_CIPHER_OPERATION_INIT.
- Call *psa_cipher_decrypt_setup()* to specify the algorithm and key.
- Call *psa_cipher_set_iv()* with the initialization vector (IV) for the decryption, if the algorithm requires one. This must match the IV used for the encryption.
- Call *psa_cipher_update()* zero, one or more times, passing a fragment of the message each time.
- Call *psa_cipher_finish()*.

If an error occurs at any step after a call to *psa_cipher_decrypt_setup()*, the operation will need to be reset by a call to *psa_cipher_abort()*. The application can call *psa_cipher_abort()* at any time after the operation has been initialized.

After a successful call to *psa_cipher_decrypt_setup()*, the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to *psa_cipher_finish()*.
- A call to *psa_cipher_abort()*.

3.2.1.5.29.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the PSA_KEY_USAGE_DECRYPT flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with alg.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not a cipher algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be inactive.
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.30 `psa_cipher_encrypt`

psa_status_t **psa_cipher_encrypt**(*psa_key_id_t* key, *psa_algorithm_t* alg, const uint8_t *input, size_t input_length, uint8_t *output, size_t output_size, size_t *output_length)

Encrypt a message using a symmetric cipher.

Parameters

- **key** (*psa_key_id_t*) – Identifier of the key to use for the operation. It must allow the usage `PSA_KEY_USAGE_ENCRYPT`.
- **alg** (*psa_algorithm_t*) – The cipher algorithm to compute (PSA_ALG_XXX value such that `PSA_ALG_IS_CIPHER(alg)` is true).
- **input** (const uint8_t*) – Buffer containing the message to encrypt.
- **input_length** (size_t) – Size of the input buffer in bytes.
- **output** (uint8_t*) – Buffer where the output is to be written. The output contains the IV followed by the ciphertext proper.
- **output_size** (size_t) – Size of the output buffer in bytes.
- **output_length** (size_t*) – On success, the number of bytes that make up the output.

3.2.1.5.30.1 Description

This function encrypts a message with a random initialization vector (IV). The length of the IV is `PSA_CIPHER_IV_LENGTH(key_type, alg)` where `key_type` is the type of key. The output of `psa_cipher_encrypt()` is the IV followed by the ciphertext.

Use the multi-part operation interface with a `typedef psa_cipher_operation_t` object to provide other forms of IV or to manage the IV and ciphertext independently.

Parameter `output_size` must be appropriate for the selected algorithm and key:

- A sufficient output size is `PSA_CIPHER_ENCRYPT_OUTPUT_SIZE(key_type, alg, input_length)` where `key_type` is the type of key.
- `PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE(input_length)` evaluates to the maximum output size of any supported cipher encryption.

3.2.1.5.30.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_ENCRYPT` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with alg.
- **PSA_ERROR_INVALID_ARGUMENT:**
The input_length is not valid for the algorithm and key type. For example, the algorithm is a based on block cipher and requires a whole number of blocks, but the total input size is not a multiple of the block size.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not a cipher algorithm.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
output_size is too small. `PSA_CIPHER_ENCRYPT_OUTPUT_SIZE()` or `PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE()` can be used to determine the required buffer size.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.31 `psa_cipher_encrypt_setup`

psa_status_t **psa_cipher_encrypt_setup**(*psa_cipher_operation_t* *operation, *psa_key_id_t* key, *psa_algorithm_t* alg)

Set the key for a multi-part symmetric encryption operation.

Parameters

- **operation** (*psa_cipher_operation_t**) – The operation object to set up. It must have been initialized as per the documentation for `typedef psa_cipher_operation_t` and not yet in use.
- **key** (*psa_key_id_t*) – Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_ENCRYPT`.

-
- **alg** (*psa_algorithm_t*) – The cipher algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_CIPHER(alg) is true).

3.2.1.5.31.1 Description

Warning: Not supported

The sequence of operations to encrypt a message with a symmetric cipher is as follows:

- Allocate an operation object which will be passed to all the functions listed here.
- Initialize the operation object with one of the methods described in the documentation for *typedef psa_cipher_operation_t*, e.g. PSA_CIPHER_OPERATION_INIT.
- Call *psa_cipher_encrypt_setup()* to specify the algorithm and key.
- Call either *psa_cipher_generate_iv()* or *psa_cipher_set_iv()* to generate or set the initialization vector (IV), if the algorithm requires one. It is recommended to use *psa_cipher_generate_iv()* unless the protocol being implemented requires a specific IV value.
- Call *psa_cipher_update()* zero, one or more times, passing a fragment of the message each time.
- Call *psa_cipher_finish()*.

If an error occurs at any step after a call to *psa_cipher_encrypt_setup()*, the operation will need to be reset by a call to *psa_cipher_abort()*. The application can call *psa_cipher_abort()* at any time after the operation has been initialized.

After a successful call to *psa_cipher_encrypt_setup()*, the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to *psa_cipher_finish()*.
- A call to *psa_cipher_abort()*.

3.2.1.5.31.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the PSA_KEY_USAGE_ENCRYPT flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with alg.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not a cipher algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be inactive.
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.32 `psa_cipher_finish`

`psa_status_t` **psa_cipher_finish**(`psa_cipher_operation_t` *operation, uint8_t *output, size_t output_size, size_t *output_length)

Finish encrypting or decrypting a message in a cipher operation.

Parameters

- **operation** (`psa_cipher_operation_t`*) – Active cipher operation.
- **output** (uint8_t*) – Buffer where the output is to be written.
- **output_size** (size_t) – Size of the output buffer in bytes.
- **output_length** (size_t*) – On success, the number of bytes that make up the returned output.

3.2.1.5.32.1 Description

Warning: Not supported

The application must call `psa_cipher_encrypt_setup()` or `psa_cipher_decrypt_setup()` before calling this function. The choice of setup function determines whether this function encrypts or decrypts its input.

This function finishes the encryption or decryption of the message formed by concatenating the inputs passed to preceding calls to `psa_cipher_update()`.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_cipher_abort()`.

Parameter `output_size` must be appropriate for the selected algorithm and key:

- A sufficient output size is `PSA_CIPHER_FINISH_OUTPUT_SIZE(key_type, alg)` where `key_type` is the type of key and `alg` is the algorithm that were used to set up the operation.
- `PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE` evaluates to the maximum output size of any supported cipher algorithm.

3.2.1.5.32.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_ARGUMENT:**
The total input size passed to this operation is not valid for this particular algorithm. For

example, the algorithm is based on block cipher and requires a whole number of blocks, but the total input size is not a multiple of the block size.

- **PSA_ERROR_INVALID_PADDING:**

This is a decryption operation for an algorithm that includes padding, and the ciphertext does not contain valid padding.

- **PSA_ERROR_BAD_STATE:**

The operation state is not valid: it must be active, with an IV set if required for the algorithm.

- **PSA_ERROR_BUFFER_TOO_SMALL:**

The size of the output buffer is too small. `PSA_CIPHER_FINISH_OUTPUT_SIZE()` or `PSA_CIPHER_FINISH_OUTPUT_MAX_SIZE` can be used to determine the required buffer size.

- **PSA_ERROR_INSUFFICIENT_MEMORY**

- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_HARDWARE_FAILURE**

- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**

- **PSA_ERROR_DATA_CORRUPT**

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.33 `psa_cipher_generate_iv`

`psa_status_t` **psa_cipher_generate_iv**(`psa_cipher_operation_t` *operation, uint8_t *iv, size_t iv_size, size_t *iv_length)

Generate an initialization vector (IV) for a symmetric encryption operation.

Parameters

- **operation** (`psa_cipher_operation_t`*) – Active cipher operation.
- **iv** (uint8_t*) – Buffer where the generated IV is to be written.
- **iv_size** (size_t) – Size of the iv buffer in bytes. This must be at least `PSA_CIPHER_IV_LENGTH(key_type, alg)` where `key_type` and `alg` are type of key and the algorithm respectively that were used to set up the cipher operation.
- **iv_length** (size_t*) – On success, the number of bytes of the generated IV.

3.2.1.5.33.1 Description

Warning: Not supported

This function generates a random IV, nonce or initial counter value for the encryption operation as appropriate for the chosen algorithm, key type and key size.

The generated IV is always the default length for the key and algorithm: `PSA_CIPHER_IV_LENGTH(key_type, alg)`, where `key_type` is the type of key and `alg` is the algorithm that were used to set up the operation. To generate different lengths of IV, use [`psa\_generate\_random\(\)`](#) and [`psa\_cipher\_set\_iv\(\)`](#).

If the cipher algorithm does not use an IV, calling this function returns a `PSA_ERROR_BAD_STATE` error. For these algorithms, `PSA_CIPHER_IV_LENGTH(key_type, alg)` will be zero.

The application must call [`psa\_cipher\_encrypt\_setup\(\)`](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [`psa\_cipher\_abort\(\)`](#).

3.2.1.5.33.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
Either:
 - The cipher algorithm does not use an IV.
 - The operation state is not valid: it must be active, with no IV set.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the iv buffer is too small. [`PSA\_CIPHER\_IV\_LENGTH\(\)`](#) or `PSA_CIPHER_IV_MAX_SIZE` can be used to determine the required buffer size.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by [`psa\_crypto\_init\(\)`](#). It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.34 `psa_cipher_operation_init`

[`psa\_cipher\_operation\_t`](#) **`psa_cipher_operation_init`**(void)

Return an initial value for a cipher operation object.

Parameters

- **void** – no arguments

3.2.1.5.34.1 Return

typedef psa_cipher_operation_t

3.2.1.5.35 psa_cipher_set_iv

psa_status_t **psa_cipher_set_iv**(*psa_cipher_operation_t* *operation, const uint8_t *iv, size_t iv_length)

Set the initialization vector (IV) for a symmetric encryption or decryption operation.

Parameters

- **operation** (*psa_cipher_operation_t**) – Active cipher operation.
- **iv** (const uint8_t*) – Buffer containing the IV to use.
- **iv_length** (size_t) – Size of the IV in bytes.

3.2.1.5.35.1 Description

Warning: Not supported

This function sets the IV, nonce or initial counter value for the encryption or decryption operation.

If the cipher algorithm does not use an IV, calling this function returns a `PSA_ERROR_BAD_STATE` error. For these algorithms, `PSA_CIPHER_IV_LENGTH(key_type, alg)` will be zero.

The application must call `psa_cipher_encrypt_setup()` or `psa_cipher_decrypt_setup()` before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_cipher_abort()`.

Note:

When encrypting, `psa_cipher_generate_iv()` is recommended instead of using this function, unless implementing a protocol that requires a non-random IV.

3.2.1.5.35.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
Either:
 - The cipher algorithm does not use an IV.
 - The operation state is not valid: it must be an active cipher encrypt operation, with no IV set.
- **PSA_ERROR_INVALID_ARGUMENT:**
The size of iv is not acceptable for the chosen algorithm, or the chosen algorithm does not use an IV.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**

- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **`PSA_ERROR_BAD_STATE:`**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.36 `psa_cipher_update`

`psa_status_t` **`psa_cipher_update`**(`psa_cipher_operation_t` *operation, const uint8_t *input, size_t input_length, uint8_t *output, size_t output_size, size_t *output_length)

Encrypt or decrypt a message fragment in an active cipher operation.

Parameters

- **operation** (`psa_cipher_operation_t`*) – Active cipher operation.
- **input** (const uint8_t*) – Buffer containing the message fragment to encrypt or decrypt.
- **input_length** (size_t) – Size of the input buffer in bytes.
- **output** (uint8_t*) – Buffer where the output is to be written.
- **output_size** (size_t) – Size of the output buffer in bytes.
- **output_length** (size_t*) – On success, the number of bytes that make up the returned output.

3.2.1.5.36.1 Description

Warning: Not supported

The following must occur before calling this function:

1. Call either `psa_cipher_encrypt_setup()` or `psa_cipher_decrypt_setup()`. The choice of setup function determines whether this function encrypts or decrypts its input.
2. If the algorithm requires an IV, call `psa_cipher_generate_iv()` or `psa_cipher_set_iv()`. `psa_cipher_generate_iv()` is recommended when encrypting.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_cipher_abort()`.

Parameter `output_size` must be appropriate for the selected algorithm and key:

- A sufficient output size is `PSA_CIPHER_UPDATE_OUTPUT_SIZE(key_type, alg, input_length)` where `key_type` is the type of key and `alg` is the algorithm that were used to set up the operation.
- `PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE(input_length)` evaluates to the maximum output size of any supported cipher algorithm.

3.2.1.5.36.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active, with an IV set if required for the algorithm.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the output buffer is too small. [PSA_CIPHER_UPDATE_OUTPUT_SIZE\(\)](#) or [PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE\(\)](#) can be used to determine the required buffer size.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by [psa_crypto_init\(\)](#). It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.37 `psa_copy_key`

psa_status_t **psa_copy_key**(*psa_key_id_t* source_key, const *psa_key_attributes_t* *attributes, *psa_key_id_t* *target_key)

Make a copy of a key.

Parameters

- **source_key** (*psa_key_id_t*) – The key to copy. It must allow the usage `PSA_KEY_USAGE_COPY`. If a private or secret key is being copied outside of a secure element it must also allow `PSA_KEY_USAGE_EXPORT`.
- **attributes** (const *psa_key_attributes_t**) – The attributes for the new key.
- **target_key** (*psa_key_id_t**) – On success, an identifier for the newly created key. `PSA_KEY_ID_NULL` on failure.

3.2.1.5.37.1 Description

Warning: Not supported

Copy key material from one location to another.

This function is primarily useful to copy a key from one location to another, as it populates a key using the material from another key which can have a different lifetime.

This function can be used to share a key with a different party.

The policy on the source key must have the usage flag `PSA_KEY_USAGE_COPY` set. This flag is sufficient to permit the copy if the key has the lifetime `PSA_KEY_LIFETIME_VOLATILE` or `PSA_KEY_LIFETIME_PERSISTENT`. Some secure elements do not provide a way to copy a key without making it extractable from the secure element. If a key is located in such a secure element, then the key must have both usage flags `PSA_KEY_USAGE_COPY` and `PSA_KEY_USAGE_EXPORT` in order to make a copy of the key outside the secure element.

The resulting key can only be used in a way that conforms to both the policy of the original key and the policy specified in the `attributes` parameter:

- The usage flags on the resulting key are the bitwise-and of the usage flags on the source policy and the usage flags in `attributes`.
- If both permit the same algorithm or wildcard-based algorithm, the resulting key has the same permitted algorithm.
- If either of the policies permits an algorithm and the other policy allows a wildcard-based permitted algorithm that includes this algorithm, the resulting key uses this permitted algorithm.
- If the policies do not permit any algorithm in common, this function fails with the status `PSA_ERROR_INVALID_ARGUMENT`.

The effect of this function on implementation-defined attributes is implementation-defined.

This function uses the `attributes` as follows:

- The key type and size can be 0. If either is nonzero, it must match the corresponding attribute of the source key.
- The key location (the lifetime and, for persistent keys, the key identifier) is used directly.
- The key policy (usage flags and permitted algorithm) are combined from the source key and `attributes` so that both sets of restrictions apply, as described in the documentation of this function.

Note:

This is an input parameter: it is not updated with the final key attributes. The final attributes of the new key can be queried by calling `psa_get_key_attributes()` with the key's identifier.

3.2.1.5.37.2 Return

- **PSA_SUCCESS:**
Success. If the new key is persistent, the key material and the key's metadata have been saved to persistent storage.
- **PSA_ERROR_INVALID_HANDLE:**
`source_key` is invalid.
- **PSA_ERROR_ALREADY_EXISTS:**
This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
- **PSA_ERROR_INVALID_ARGUMENT:**
The lifetime or identifier in `attributes` are invalid.
- **PSA_ERROR_INVALID_ARGUMENT:**
The key policies from `source_key` and specified in `attributes` are incompatible.

- **PSA_ERROR_INVALID_ARGUMENT:**
attributes specifies a key type or key size which does not match the attributes of source key.
- **PSA_ERROR_NOT_PERMITTED:**
source_key does not have the PSA_KEY_USAGE_COPY usage flag.
- **PSA_ERROR_NOT_PERMITTED:**
source_key does not have the PSA_KEY_USAGE_EXPORT usage flag and its lifetime does not allow copying it to the target's lifetime.
- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_INSUFFICIENT_STORAGE
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID
- PSA_ERROR_CORRUPTION_DETECTED
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.38 `psa_crypto_init`

`psa_status_t` **psa_crypto_init**(void)

Library initialization.

Parameters

- **void** – no arguments

3.2.1.5.38.1 Description

Applications must call this function before calling any other function in this module.

Applications are permitted to call this function more than once. Once a call succeeds, subsequent calls are guaranteed to succeed.

If the application calls other functions before calling `psa_crypto_init()`, the behavior is undefined. In this situation:

- Implementations are encouraged to either perform the operation as if the library had been initialized or to return PSA_ERROR_BAD_STATE or some other applicable error.
- Implementations must not return a success status if the lack of initialization might have security implications, for example due to improper seeding of the random number generator.

3.2.1.5.38.2 Return

- **PSA_SUCCESS**
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_INSUFFICIENT_ENTROPY**

3.2.1.5.39 `psa_destroy_key`

psa_status_t **psa_destroy_key**(*psa_key_id_t* key)

Destroy a key.

Parameters

- **key** (*psa_key_id_t*) – Identifier of the key to erase. If this is **PSA_KEY_ID_NULL**, do nothing and return **PSA_SUCCESS**.

3.2.1.5.39.1 Description

This function destroys a key from both volatile memory and, if applicable, non-volatile storage. Implementations must make a best effort to ensure that the key material cannot be recovered.

This function also erases any metadata such as policies and frees resources associated with the key.

Destroying the key makes the key identifier invalid, and the key identifier must not be used again by the application.

If a key is currently in use in a multi-part operation, then destroying the key will cause the multi-part operation to fail.

3.2.1.5.39.2 Return

- **PSA_SUCCESS:**
key was a valid key identifier and the key material that it referred to has been erased. Alternatively, key is **PSA_KEY_ID_NULL**.
- **PSA_ERROR_NOT_PERMITTED:**
The key cannot be erased because it is read-only, either due to a policy or due to physical restrictions.
- **PSA_ERROR_INVALID_HANDLE:**
key is not a valid handle nor **PSA_KEY_ID_NULL**.
- **PSA_ERROR_COMMUNICATION_FAILURE**
There was a failure in communication with the cryptoprocessor. The key material might still be present in the cryptoprocessor.
- **PSA_ERROR_STORAGE_FAILURE:**
The storage operation failed. Implementations must make a best effort to erase key material even in this situation, however, it might be impossible to guarantee that the key material is not recoverable in such cases.

- **PSA_ERROR_DATA_CORRUPT:**

The storage is corrupted. Implementations must make a best effort to erase key material even in this situation, however, it might be impossible to guarantee that the key material is not recoverable in such cases.

- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_CORRUPTION_DETECTED:**

An unexpected condition which is not a storage corruption or a communication failure occurred. The cryptoprocessor might have been compromised.

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.40 `psa_export_key`

`psa_status_t` **psa_export_key**(`psa_key_id_t` key, `uint8_t *`data, `size_t` data_size, `size_t *`data_length)

Export a key in binary format.

Parameters

- **key** (`psa_key_id_t`) – Identifier of the key to export. It must allow the usage `PSA_KEY_USAGE_EXPORT`, unless it is a public key.
- **data** (`uint8_t *`) – Buffer where the key data is to be written.
- **data_size** (`size_t`) – Size of the data buffer in bytes.
- **data_length** (`size_t *`) – On success, the number of bytes that make up the key data.

3.2.1.5.40.1 Description

Warning: Export of any private key is not supported for now.

The output of this function can be passed to `psa_import_key()` to create an equivalent object.

If the implementation of `psa_import_key()` supports other formats beyond the format specified here, the output from `psa_export_key()` must use the representation specified here, not the original representation.

For standard key types, the output format is as follows:

- For symmetric keys, excluding HMAC keys, the format is the raw bytes of the key.
- For HMAC keys that are shorter than, or equal in size to, the underlying hash algorithm block size, the format is the raw bytes of the key.

For HMAC keys that are longer than the underlying hash algorithm block size, the format is an implementation defined choice between the following formats:

1. The raw bytes of the key.
2. The raw bytes of the hash of the key, using the underlying hash algorithm.

See also `PSA_KEY_TYPE_HMAC`.

- For DES, the key data consists of 8 bytes. The parity bits must be correct.
- For Triple-DES, the format is the concatenation of the two or three DES keys.

- For RSA key pairs, with key type `PSA_KEY_TYPE_RSA_KEY_PAIR`, the format is the non-encrypted DER encoding of the representation defined by in PKCS #1: RSA Cryptography Specifications Version 2.2 [RFC8017] as `RSAPrivateKey`, version 0.

```

RSAPrivateKey ::= SEQUENCE {
    version          INTEGER,  -- must be 0
    modulus          INTEGER,  -- n
    publicExponent   INTEGER,  -- e
    privateExponent  INTEGER,  -- d
    prime1           INTEGER,  -- p
    prime2           INTEGER,  -- q
    exponent1        INTEGER,  -- d mod (p-1)
    exponent2        INTEGER,  -- d mod (q-1)
    coefficient       INTEGER,  -- (inverse of q) mod p
}

```

Note:

Although it is possible to define an RSA key pair or private key using a subset of these elements, the output from `psa_export_key()` for an RSA key pair must include all of these elements.

- For elliptic curve key pairs, with key types for which `PSA_KEY_TYPE_IS_ECC_KEY_PAIR()` is true, the format is a representation of the private value.

- For Weierstrass curve families `PSA_ECC_FAMILY_SECT_XX`, `PSA_ECC_FAMILY_SECP_XX`, `PSA_ECC_FAMILY_FRP` and `PSA_ECC_FAMILY_BRAINPOOL_P_R1`, the content of the `privateKey` field of the `ECPrivateKey` format defined by Elliptic Curve Private Key Structure [RFC5915].

This is a $\lceil m/8 \rceil$ -byte string in big-endian order where m is the key size in bits.

- For curve family `PSA_ECC_FAMILY_MONTGOMERY`, the scalar value of the ‘private key’ in little-endian order as defined by Elliptic Curves for Security [RFC7748] §6. The value must have the forced bits set to zero or one as specified by `decodeScalar25519()` and `decodeScalar448()` in [RFC7748] §5.

This is a $\lceil m/8 \rceil$ -byte string where m is the key size in bits. This is 32 bytes for Curve25519, and 56 bytes for Curve448.

- For Diffie-Hellman key exchange key pairs, with key types for which `PSA_KEY_TYPE_IS_DH_KEY_PAIR()` is true, the format is the representation of the private key x as a big-endian byte string. The length of the byte string is the private key size in bytes, and leading zeroes are not stripped.
- For public keys, with key types for which `PSA_KEY_TYPE_IS_PUBLIC_KEY()` is true, the format is the same as for `psa_export_public_key()`.

The policy on the key must have the usage flag `PSA_KEY_USAGE_EXPORT` set.

Parameter `data_size` must be appropriate for the key:

- The required output size is `PSA_EXPORT_KEY_OUTPUT_SIZE(type, bits)` where `type` is the key type and `bits` is the key size in bits.
- `PSA_EXPORT_KEY_PAIR_MAX_SIZE` evaluates to the maximum output size of any supported key pair.

- `PSA_EXPORT_PUBLIC_KEY_MAX_SIZE` evaluates to the maximum output size of any supported public key.
- This API defines no maximum size for symmetric keys. Arbitrarily large data items can be stored in the key store, for example certificates that correspond to a stored private key or input material for key derivation.

3.2.1.5.40.2 Return

- `PSA_SUCCESS`
- `PSA_ERROR_INVALID_HANDLE`
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_EXPORT` flag.
- `PSA_ERROR_NOT_SUPPORTED`
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the data buffer is too small. `PSA_EXPORT_KEY_OUTPUT_SIZE()` or `PSA_EXPORT_KEY_PAIR_MAX_SIZE` can be used to determine the required buffer size.
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.41 `psa_export_public_key`

psa_status_t **psa_export_public_key**(*psa_key_id_t* key, uint8_t *data, size_t data_size, size_t *data_length)

Export a public key or the public part of a key pair in binary format.

Parameters

- **key** (*psa_key_id_t*) – Identifier of the key to export.
- **data** (uint8_t*) – Buffer where the key data is to be written.
- **data_size** (size_t) – Size of the data buffer in bytes.
- **data_length** (size_t*) – On success, the number of bytes that make up the key data.

3.2.1.5.41.1 Description

Warning: Not supported

The output of this function can be passed to `psa_import_key()` to create an object that is equivalent to the public key.

If the implementation of `psa_import_key()` supports other formats beyond the format specified here, the output from `psa_export_public_key()` must use the representation specified here, not the original representation.

For standard key types, the output format is as follows:

- For RSA public keys, with key type `PSA_KEY_TYPE_RSA_PUBLIC_KEY`, the DER encoding of the representation defined by Algorithms and Identifiers for the Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile [RFC3279] §2.3.1 as `RSAPublicKey`.

```
RSAPublicKey ::= SEQUENCE {  
    modulus          INTEGER,      -- n  
    publicExponent   INTEGER      } -- e
```

- For elliptic curve key pairs, with key types for which `PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY()` is true, the format depends on the key family:
 - For Weierstrass curve families `PSA_ECC_FAMILY_SECT_XX`, `PSA_ECC_FAMILY_SECP_XX`, `PSA_ECC_FAMILY_FRP` and `PSA_ECC_FAMILY_BRAINPOOL_P_R1`, the uncompressed representation of an elliptic curve point as an octet string defined in SEC 1: Elliptic Curve Cryptography [SEC1] §2.3.3. If m is the bit size associated with the curve, i.e. the bit size of q for a curve over F_q . The representation consists of:
 - * The byte 0x04;
 - * x_P as a $\text{ceiling}(m/8)$ -byte string, big-endian;
 - * y_P as a $\text{ceiling}(m/8)$ -byte string, big-endian.
 - For curve family `PSA_ECC_FAMILY_MONTGOMERY`, the scalar value of the ‘public key’ in little-endian order as defined by Elliptic Curves for Security [RFC7748] §6. This is a $\text{ceiling}(m/8)$ -byte string where m is the key size in bits.
 - * This is 32 bytes for Curve25519, computed as `X25519(private_key, 9)`.
 - * This is 56 bytes for Curve448, computed as `X448(private_key, 5)`.
- For Diffie-Hellman key exchange public keys, with key types for which `PSA_KEY_TYPE_IS_DH_PUBLIC_KEY` is true, the format is the representation of the public key $y = g^x \bmod p$ as a big-endian byte string. The length of the byte string is the length of the base prime p in bytes.

Exporting a public key object or the public part of a key pair is always permitted, regardless of the key’s usage flags.

Parameter `data_size` must be appropriate for the key:

- The required output size is `PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE(type, bits)` where `type` is the key type and `bits` is the key size in bits.

- `PSA_EXPORT_PUBLIC_KEY_MAX_SIZE` evaluates to the maximum output size of any supported public key or public part of a key pair.

3.2.1.5.41.2 Return

- `PSA_SUCCESS`
- `PSA_ERROR_INVALID_HANDLE`
- **`PSA_ERROR_INVALID_ARGUMENT`:**
The key is neither a public key nor a key pair.
- `PSA_ERROR_NOT_SUPPORTED`
- **`PSA_ERROR_BUFFER_TOO_SMALL`:**
The size of the data buffer is too small. `PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE()` or `PSA_EXPORT_PUBLIC_KEY_MAX_SIZE` can be used to determine the required buffer size.
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- **`PSA_ERROR_BAD_STATE`:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.42 `psa_generate_key`

`psa_status_t` **`psa_generate_key`**(const `psa_key_attributes_t` *attributes, `psa_key_id_t` *key)

Generate a key or key pair.

Parameters

- **attributes** (const `psa_key_attributes_t` *) – The attributes for the new key.
- **key** (`psa_key_id_t` *) – On success, an identifier for the newly created key. `PSA_KEY_ID_NULL` on failure.

3.2.1.5.42.1 Description

The key is generated randomly. Its location, policy, type and size are taken from attributes.

Implementations must reject an attempt to generate a key of size 0.

The following type-specific considerations apply:

-
- For RSA keys (PSA_KEY_TYPE_RSA_KEY_PAIR), the public exponent is 65537. The modulus is a product of two probabilistic primes between 2^{n-1} and 2^n where n is the bit size specified in the attributes.

This function uses the `attributes` as follows:

- The key type is required. It cannot be an asymmetric public key.
- The key size is required. It must be a valid size for the key type.
- The key permitted-algorithm policy is required for keys that will be used for a cryptographic operation, see Permitted algorithms.
- The key usage flags define what operations are permitted with the key, see Key usage flags.
- The key lifetime and identifier are required for a persistent key.

Note:

This is an input parameter: it is not updated with the final key attributes. The final attributes of the new key can be queried by calling `psa_get_key_attributes()` with the key's identifier.

3.2.1.5.42.2 Return

- **PSA_SUCCESS:**
Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
- **PSA_ERROR_ALREADY_EXISTS:**
This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
- **PSA_ERROR_NOT_SUPPORTED:**
The key type or key size is not supported, either by the implementation in general or in this particular persistent location.
- **PSA_ERROR_INVALID_ARGUMENT:**
The key attributes, as a whole, are invalid.
- **PSA_ERROR_INVALID_ARGUMENT:**
The key type is an asymmetric public key type.
- **PSA_ERROR_INVALID_ARGUMENT:**
The key size is not a valid size for the key type.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_INSUFFICIENT_ENTROPY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_INSUFFICIENT_STORAGE**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.43 `psa_generate_random`

psa_status_t **psa_generate_random**(uint8_t *output, size_t output_size)

Generate random bytes.

Parameters

- **output** (uint8_t*) – Output buffer for the generated data.
- **output_size** (size_t) – Number of bytes to generate and output.

3.2.1.5.43.1 Description

Warning:

This function can fail! Callers **MUST** check the return status and **MUST NOT** use the content of the output buffer if the return status is not `PSA_SUCCESS`.

Note:

To generate a key, use `psa_generate_key()` instead.

3.2.1.5.43.2 Return

- `PSA_SUCCESS`
- `PSA_ERROR_NOT_SUPPORTED`
- `PSA_ERROR_INSUFFICIENT_ENTROPY`
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.44 `psa_get_key_algorithm`

psa_algorithm_t **psa_get_key_algorithm**(const *psa_key_attributes_t* *attributes)

Retrieve the permitted algorithm policy from key attributes.

Parameters

- **attributes** (const *psa_key_attributes_t**) – The key attribute object to query.

3.2.1.5.44.1 Description

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.44.2 Return

`typedef psa_algorithm_t`

The algorithm stored in the attribute object.

3.2.1.5.45 `psa_get_key_attributes`

`psa_status_t` **`psa_get_key_attributes`**(`psa_key_id_t` key, `psa_key_attributes_t` *attributes)

Retrieve the attributes of a key.

Parameters

- **key** (`psa_key_id_t`) – Identifier of the key to query.
- **attributes** (`psa_key_attributes_t`*) – On entry, *attributes must be in a valid state. On successful return, it contains the attributes of the key. On failure, it is equivalent to a freshly-initialized attribute object.

3.2.1.5.45.1 Description

This function first resets the attribute object as with `psa_reset_key_attributes()`. It then copies the attributes of the given key into the given attribute object.

Note:

This function clears any previous content from the attribute object and therefore expects it to be in a valid state. In particular, if this function is called on a newly allocated attribute object, the attribute object must be initialized before calling this function.

Note:

This function might allocate memory or other resources. Once this function has been called on an attribute object, `psa_reset_key_attributes()` must be called to free these resources.

3.2.1.5.45.2 Return

- `PSA_SUCCESS`
- `PSA_ERROR_INVALID_HANDLE`
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`

- `PSA_ERROR_DATA_INVALID`

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.46 `psa_get_key_bits`

`size_t` **psa_get_key_bits**(const `psa_key_attributes_t` *attributes)

Retrieve the key size from key attributes.

Parameters

- **attributes** (const `psa_key_attributes_t`*) – The key attribute object to query.

3.2.1.5.46.1 Description

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.46.2 Return

`size_t`

The key size stored in the attribute object, in bits.

3.2.1.5.47 `psa_get_key_id`

`psa_key_id_t` **psa_get_key_id**(const `psa_key_attributes_t` *attributes)

Retrieve the key identifier from key attributes.

Parameters

- **attributes** (const `psa_key_attributes_t`*) – The key attribute object to query.

3.2.1.5.47.1 Description

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.47.2 Return

typedef psa_key_id_t

The persistent identifier stored in the attribute object. This value is unspecified if the attribute object declares the key as volatile.

3.2.1.5.48 **psa_get_key_lifetime**

psa_key_lifetime_t **psa_get_key_lifetime**(const *psa_key_attributes_t* *attributes)

Retrieve the lifetime from key attributes.

Parameters

- **attributes** (const *psa_key_attributes_t**) – The key attribute object to query.

3.2.1.5.48.1 Description

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.48.2 Return

typedef psa_key_lifetime_t

The lifetime value stored in the attribute object.

3.2.1.5.49 **psa_get_key_type**

psa_key_type_t **psa_get_key_type**(const *psa_key_attributes_t* *attributes)

Retrieve the key type from key attributes.

Parameters

- **attributes** (const *psa_key_attributes_t**) – The key attribute object to query.

3.2.1.5.49.1 Description

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.49.2 Return

typedef psa_key_type_t

The key type stored in the attribute object.

3.2.1.5.50 `psa_get_key_usage_flags`

psa_key_usage_t **psa_get_key_usage_flags**(const *psa_key_attributes_t* *attributes)

Retrieve the usage flags from key attributes.

Parameters

- **attributes** (const *psa_key_attributes_t**) – The key attribute object to query.

3.2.1.5.50.1 Description

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.50.2 Return

typedef psa_key_usage_t

The usage flags stored in the attribute object.

3.2.1.5.51 `psa_hash_abort`

psa_status_t **psa_hash_abort**(*psa_hash_operation_t* *operation)

Abort a hash operation.

Parameters

- **operation** (*psa_hash_operation_t**) – Initialized hash operation.

3.2.1.5.51.1 Description

Warning: Not supported

Aborting an operation frees all associated resources except for the operation object itself. Once aborted, the operation object can be reused for another operation by calling `psa_hash_setup()` again.

This function can be called any time after the operation object has been initialized by one of the methods described in *typedef psa_hash_operation_t*.

In particular, calling `psa_hash_abort()` after the operation has been terminated by a call to `psa_hash_abort()`, `psa_hash_finish()` or `psa_hash_verify()` is safe and has no effect.

3.2.1.5.51.2 Return

- PSA_SUCCESS
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.52 `psa_hash_clone`

`psa_status_t` **psa_hash_clone**(const `psa_hash_operation_t` *source_operation,
`psa_hash_operation_t` *target_operation)

Clone a hash operation.

Parameters

- **source_operation** (const `psa_hash_operation_t`*) – The active hash operation to clone.
- **target_operation** (`psa_hash_operation_t`*) – The operation object to set up. It must be initialized but not active.

3.2.1.5.52.1 Description

Warning: Not supported

This function copies the state of an ongoing hash operation to a new operation object. In other words, this function is equivalent to calling `psa_hash_setup()` on `target_operation` with the same algorithm that `source_operation` was set up for, then `psa_hash_update()` on `target_operation` with the same input that was passed to `source_operation`. After this function returns, the two objects are independent, i.e. subsequent calls involving one of the objects do not affect the other object.

3.2.1.5.52.2 Return

- PSA_SUCCESS
- **PSA_ERROR_BAD_STATE:**
The `source_operation` state is not valid: it must be active.
- **PSA_ERROR_BAD_STATE:**
The `target_operation` state is not valid: it must be inactive.
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_INSUFFICIENT_MEMORY
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.53 `psa_hash_compare`

psa_status_t **psa_hash_compare**(*psa_algorithm_t* alg, const uint8_t *input, size_t input_length, const uint8_t *hash, size_t hash_length)

Calculate the hash (digest) of a message and compare it with a reference value.

Parameters

- **alg** (*psa_algorithm_t*) – The hash algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_HASH(alg) is true).
- **input** (const uint8_t*) – Buffer containing the message to hash.
- **input_length** (size_t) – Size of the input buffer in bytes.
- **hash** (const uint8_t*) – Buffer containing the expected hash value.
- **hash_length** (size_t) – Size of the hash buffer in bytes.

3.2.1.5.53.1 Return

- **PSA_SUCCESS:**
The expected hash is identical to the actual hash of the input.
- **PSA_ERROR_INVALID_SIGNATURE:**
The hash of the message was calculated successfully, but it differs from the expected hash.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not a hash algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
input_length or hash_length do not match the hash size for alg
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.54 `psa_hash_compute`

psa_status_t **psa_hash_compute**(*psa_algorithm_t* alg, const uint8_t *input, size_t input_length, uint8_t *hash, size_t hash_size, size_t *hash_length)

Calculate the hash (digest) of a message.

Parameters

- **alg** (*psa_algorithm_t*) – The hash algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_HASH(alg) is true).
- **input** (const uint8_t*) – Buffer containing the message to hash.
- **input_length** (size_t) – Size of the input buffer in bytes.

- **hash** (uint8_t*) – Buffer where the hash is to be written.
- **hash_size** (size_t) – Size of the hash buffer in bytes. This must be at least PSA_HASH_LENGTH(alg).
- **hash_length** (size_t*) – On success, the number of bytes that make up the hash value. This is always PSA_HASH_LENGTH(alg).

3.2.1.5.54.1 Description

Note:

To verify the hash of a message against an expected value, use [psa_hash_compare\(\)](#) instead.

3.2.1.5.54.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not a hash algorithm.
- **PSA_ERROR_INVALID_ARGUMENT**
- **PSA_ERROR_BUFFER_TOO_SMALL:**
hash_size is too small. [PSA_HASH_LENGTH\(\)](#) can be used to determine the required buffer size.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by [psa_crypto_init\(\)](#). It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.55 psa_hash_finish

psa_status_t **psa_hash_finish**(*psa_hash_operation_t* *operation, uint8_t *hash, size_t hash_size, size_t *hash_length)

Finish the calculation of the hash of a message.

Parameters

- **operation** (*psa_hash_operation_t**) – Active hash operation.
- **hash** (uint8_t*) – Buffer where the hash is to be written.
- **hash_size** (size_t) – Size of the hash buffer in bytes. This must be at least PSA_HASH_LENGTH(alg) where alg is the algorithm that the operation performs.

-
- **hash_length** (size_t*) – On success, the number of bytes that make up the hash value. This is always PSA_HASH_LENGTH(alg) where alg is the hash algorithm that the operation performs.

3.2.1.5.55.1 Description

Warning: Not supported

The application must call `psa_hash_setup()` or `psa_hash_resume()` before calling this function. This function calculates the hash of the message formed by concatenating the inputs passed to preceding calls to `psa_hash_update()`.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_hash_abort()`.

Warning:

It is not recommended to use this function when a specific value is expected for the hash. Call `psa_hash_verify()` instead with the expected hash value.

Comparing integrity or authenticity data such as hash values with a function such as `memcmp()` is risky because the time taken by the comparison might leak information about the hashed data which could allow an attacker to guess a valid hash and thereby bypass security controls.

3.2.1.5.55.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the hash buffer is too small. `PSA_HASH_LENGTH()` can be used to determine the required buffer size.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.56 psa_hash_operation_init

`psa_hash_operation_t` **psa_hash_operation_init**(void)

Return an initial value for a hash operation object.

Parameters

- **void** – no arguments

3.2.1.5.56.1 Return

typedef *psa_hash_operation_t*

3.2.1.5.57 *psa_hash_resume*

psa_status_t **psa_hash_resume**(*psa_hash_operation_t* *operation, const uint8_t *hash_state, size_t hash_state_length)

Set up a multi-part hash operation using the hash suspend state from a previously suspended hash operation.

Parameters

- **operation** (*psa_hash_operation_t**) – The operation object to set up. It must have been initialized as per the documentation for *typedef psa_hash_operation_t* and not yet in use.
- **hash_state** (const uint8_t*) – A buffer containing the suspended hash state which is to be resumed. This must be in the format output by *psa_hash_suspend()*, which is described in Hash suspend state format.
- **hash_state_length** (size_t) – Length of hash_state in bytes.

3.2.1.5.57.1 Description

Warning: Not supported

See *psa_hash_suspend()* for an example of how to use this function to suspend and resume a hash operation.

After a successful call to *psa_hash_resume()*, the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to *psa_hash_finish()*, *psa_hash_verify()* or *psa_hash_suspend()*.
- A call to *psa_hash_abort()*.

3.2.1.5.57.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_NOT_SUPPORTED:**
The provided hash suspend state is for an algorithm that is not supported.
- **PSA_ERROR_INVALID_ARGUMENT:**
hash_state does not correspond to a valid hash suspend state. See Hash suspend state format for the definition.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be inactive.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.58 `psa_hash_setup`

`psa_status_t` **psa_hash_setup**(`psa_hash_operation_t` *operation, `psa_algorithm_t` alg)

Set up a multi-part hash operation.

Parameters

- **operation** (`psa_hash_operation_t`*) – The operation object to set up. It must have been initialized as per the documentation for `typedef psa_hash_operation_t` and not yet in use.
- **alg** (`psa_algorithm_t`) – The hash algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_HASH(alg) is true).

3.2.1.5.58.1 Description

Warning: Not supported

The sequence of operations to calculate a hash (message digest) is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for `typedef psa_hash_operation_t`, e.g. `PSA_HASH_OPERATION_INIT`.
3. Call `psa_hash_setup()` to specify the algorithm.
4. Call `psa_hash_update()` zero, one or more times, passing a fragment of the message each time. The hash that is calculated is the hash of the concatenation of these messages in order.
5. To calculate the hash, call `psa_hash_finish()`. To compare the hash with an expected value, call `psa_hash_verify()`. To suspend the hash operation and extract the current state, call `psa_hash_suspend()`.

If an error occurs at any step after a call to `psa_hash_setup()`, the operation will need to be reset by a call to `psa_hash_abort()`. The application can call `psa_hash_abort()` at any time after the operation has been initialized.

After a successful call to `psa_hash_setup()`, the application must eventually terminate the operation. The following events terminate an operation:

- A successful call to `psa_hash_finish()` or `psa_hash_verify()` or `psa_hash_suspend()`.
- A call to `psa_hash_abort()`.

3.2.1.5.58.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not a supported hash algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
alg is not a hash algorithm.

- **PSA_ERROR_BAD_STATE:**

The operation state is not valid: it must be inactive.

- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.59 `psa_hash_suspend`

`psa_status_t` **psa_hash_suspend**(`psa_hash_operation_t` *operation, `uint8_t` *hash_state, `size_t` hash_state_size, `size_t` *hash_state_length)

Halt the hash operation and extract the intermediate state of the hash computation.

Parameters

- **operation** (`psa_hash_operation_t`*) – Active hash operation.
- **hash_state** (`uint8_t`*) – Buffer where the hash suspend state is to be written.
- **hash_state_size** (`size_t`) – Size of the hash_state buffer in bytes.
- **hash_state_length** (`size_t`*) – On success, the number of bytes that make up the hash suspend state.

3.2.1.5.59.1 Description

Warning: Not supported

The application must call `psa_hash_setup()` or `psa_hash_resume()` before calling this function. This function extracts an intermediate state of the hash computation of the message formed by concatenating the inputs passed to preceding calls to `psa_hash_update()`.

This function can be used to halt a hash operation, and then resume the hash operation at a later time, or in another application, by transferring the extracted hash suspend state to a call to `psa_hash_resume()`.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_hash_abort()`.

Hash suspend and resume is not defined for the SHA3 family of hash algorithms. Hash suspend state defines the format of the output from `psa_hash_suspend()`.

Warning:

Applications must not use any of the hash suspend state as if it was a hash output. Instead, the suspend state must only be used to resume a hash operation, and `psa_hash_finish()` or `psa_hash_verify()` can then calculate or verify the final hash value.

Parameter `hash_state_size` must be appropriate for the selected algorithm:

- A sufficient output size is `PSA_HASH_SUSPEND_OUTPUT_SIZE(alg)` where `alg` is the algorithm that was used to set up the operation.

- **PSA_HASH_SUSPEND_OUTPUT_MAX_SIZE** evaluates to the maximum output size of any supported hash algorithm.

3.2.1.5.59.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the `hash_state` buffer is too small. [PSA_HASH_SUSPEND_OUTPUT_SIZE\(\)](#) or **PSA_HASH_SUSPEND_OUTPUT_MAX_SIZE** can be used to determine the required buffer size.
- **PSA_ERROR_NOT_SUPPORTED:**
The hash algorithm being computed does not support suspend and resume.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by [psa_crypto_init\(\)](#). It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.60 psa_hash_update

psa_status_t **psa_hash_update**(*psa_hash_operation_t* *operation, const uint8_t *input, size_t input_length)

Add a message fragment to a multi-part hash operation.

Parameters

- **operation** (*psa_hash_operation_t**) – Active hash operation.
- **input** (const uint8_t*) – Buffer containing the message fragment to hash.
- **input_length** (size_t) – Size of the input buffer in bytes.

3.2.1.5.60.1 Description

Warning: Not supported

The application must call [psa_hash_setup\(\)](#) or [psa_hash_resume\(\)](#) before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling [psa_hash_abort\(\)](#).

3.2.1.5.60.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.61 `psa_hash_verify`

psa_status_t **psa_hash_verify**(*psa_hash_operation_t* *operation, const uint8_t *hash, size_t hash_length)

Finish the calculation of the hash of a message and compare it with an expected value.

Parameters

- **operation** (*psa_hash_operation_t**) – Active hash operation.
- **hash** (const uint8_t*) – Buffer containing the expected hash value.
- **hash_length** (size_t) – Size of the hash buffer in bytes.

3.2.1.5.61.1 Description

Warning: Not supported

The application must call `psa_hash_setup()` before calling this function. This function calculates the hash of the message formed by concatenating the inputs passed to preceding calls to `psa_hash_update()`. It then compares the calculated hash with the expected hash passed as a parameter to this function.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_hash_abort()`.

Note:

Implementations must make the best effort to ensure that the comparison between the actual hash and the expected hash is performed in constant time.

3.2.1.5.61.2 Return

- **PSA_SUCCESS:**
The expected hash is identical to the actual hash of the message.
- **PSA_ERROR_INVALID_SIGNATURE:**
The hash of the message was calculated successfully, but it differs from the expected hash.

- **PSA_ERROR_BAD_STATE:**

The operation state is not valid: it must be active.

- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.62 `psa_import_key`

`psa_status_t` **psa_import_key**(const `psa_key_attributes_t` *attributes, const uint8_t *data, size_t data_length, `psa_key_id_t` *key)

Import a key in binary format.

Parameters

- **attributes** (const `psa_key_attributes_t` *) – The attributes for the new key.
- **data** (const uint8_t *) – Buffer containing the key data. The content of this buffer is interpreted according to the type declared in attributes. All implementations must support at least the format described in the documentation of `psa_export_key()` or `psa_export_public_key()` for the chosen type. Implementations can support other formats, but be conservative in interpreting the key data: it is recommended that implementations reject content if it might be erroneous, for example, if it is the wrong type or is truncated.
- **data_length** (size_t) – Size of the data buffer in bytes.
- **key** (`psa_key_id_t` *) – On success, an identifier for the newly created key. PSA_KEY_ID_NULL on failure.

3.2.1.5.62.1 Description

Warning: Import of private key may be not supported depending on the secure subsystem in use.

This function supports any output from `psa_export_key()`. Refer to the documentation of `psa_export_public_key()` for the format of public keys and to the documentation of `psa_export_key()` for the format for other key types.

The key data determines the key size. The attributes can optionally specify a key size; in this case it must match the size determined from the key data. A key size of 0 in attributes indicates that the key size is solely determined by the key data.

Implementations must reject an attempt to import a key of size 0.

This specification defines a single format for each key type. Implementations can optionally support other formats in addition to the standard format. It is recommended that implementations that support other formats ensure that the formats are clearly unambiguous, to minimize the risk that an invalid input is accidentally interpreted according to a different format.

Note:

The PSA Crypto API does not support asymmetric private key objects outside of a key pair. To import a private key, the attributes must specify the corresponding key pair type. Depending on the key type, either the import format contains the public key data or the implementation will reconstruct the public key from the private key as needed.

This function uses the `attributes` as follows:

- The key type is required, and determines how the data buffer is interpreted.
- The key size is always determined from the data buffer. If the key size in attributes is nonzero, it must be equal to the size determined from data.
- The key permitted-algorithm policy is required for keys that will be used for a cryptographic operation, see Permitted algorithms.
- The key usage flags define what operations are permitted with the key, see Key usage flags.
- The key lifetime and identifier are required for a persistent key.

Note:

This is an input parameter: it is not updated with the final key attributes. The final attributes of the new key can be queried by calling `psa_get_key_attributes()` with the key's identifier.

3.2.1.5.62.2 Return

- **PSA_SUCCESS:**
Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
- **PSA_ERROR_ALREADY_EXISTS:**
This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
- **PSA_ERROR_NOT_SUPPORTED:**
The key type or key size is not supported, either by the implementation in general or in this particular persistent location.
- **PSA_ERROR_INVALID_ARGUMENT:**
The key attributes, as a whole, are invalid.
- **PSA_ERROR_INVALID_ARGUMENT:**
The key data is not correctly formatted.
- **PSA_ERROR_INVALID_ARGUMENT:**
The size in `attributes` is nonzero and does not match the size of the key data.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_INSUFFICIENT_STORAGE**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_HARDWARE_FAILURE**

-
- `PSA_ERROR_CORRUPTION_DETECTED`
 - **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.63 `psa_key_attributes_init`

psa_key_attributes_t **psa_key_attributes_init**(void)

Return an initial value for a key attribute object.

Parameters

- **void** – no arguments

3.2.1.5.63.1 Return

typedef psa_key_attributes_t

3.2.1.5.64 `psa_key_derivation_abort`

psa_status_t **psa_key_derivation_abort**(*psa_key_derivation_operation_t* *operation)

Abort a key derivation operation.

Parameters

- **operation** (*psa_key_derivation_operation_t**) – The operation to abort.

3.2.1.5.64.1 Description

Aborting an operation frees all associated resources except for the operation object itself. Once aborted, the operation object can be reused for another operation by calling `psa_key_derivation_setup()` again.

This function can be called at any time after the operation object has been initialized as described in *typedef psa_key_derivation_operation_t*.

In particular, it is valid to call `psa_key_derivation_abort()` twice, or to call `psa_key_derivation_abort()` on an operation that has not been set up.

3.2.1.5.64.2 Return

- `PSA_SUCCESS`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.65 `psa_key_derivation_get_capacity`

psa_status_t **psa_key_derivation_get_capacity**(const *psa_key_derivation_operation_t* *operation, size_t *capacity)

Retrieve the current capacity of a key derivation operation.

Parameters

- **operation** (const *psa_key_derivation_operation_t**) – The operation to query.
- **capacity** (size_t*) – On success, the capacity of the operation.

3.2.1.5.65.1 Description

Warning: Not supported

The capacity of a key derivation is the maximum number of bytes that it can return. Reading N bytes of output from a key derivation operation reduces its capacity by at least N. The capacity can be reduced by more than N in the following situations:

- Calling `psa_key_derivation_output_key()` can reduce the capacity by more than the key size, depending on the type of key being generated. See `psa_key_derivation_output_key()` for details of the key derivation process.
- When the `typedef psa_key_derivation_operation_t` object is operating as a deterministic random bit generator (DRBG), which reduces capacity in whole blocks, even when less than a block is read.

3.2.1.5.65.2 Return

- `PSA_SUCCESS`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active.
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.66 `psa_key_derivation_input_bytes`

psa_status_t **psa_key_derivation_input_bytes**(*psa_key_derivation_operation_t* *operation, *psa_key_derivation_step_t* step, const uint8_t *data, size_t data_length)

Provide an input for key derivation or key agreement.

Parameters

- **operation** (*psa_key_derivation_operation_t**) – The key derivation operation object to use. It must have been set up with

`psa_key_derivation_setup()` and must not have produced any output yet.

- **step** (`psa_key_derivation_step_t`) – Which step the input data is for.
- **data** (`const uint8_t*`) – Input data to use.
- **data_length** (`size_t`) – Size of the data buffer in bytes.

3.2.1.5.66.1 Description

Which inputs are required and in what order depends on the algorithm. Refer to the documentation of each key derivation or key agreement algorithm for information.

This function passes direct inputs, which is usually correct for non-secret inputs. To pass a secret input, which is normally in a key object, call `psa_key_derivation_input_key()` instead of this function. Refer to the documentation of individual step types (PSA_KEY_DERIVATION_INPUT_xxx values of `typedef psa_key_derivation_step_t`) for more information.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_key_derivation_abort()`.

3.2.1.5.66.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_ARGUMENT:**
step is not compatible with the operation's algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
step does not allow direct inputs.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid for this input step. This can happen if the application provides a step out of order or repeats a step that may not be repeated.
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.67 `psa_key_derivation_input_integer`

psa_status_t **psa_key_derivation_input_integer**(*psa_key_derivation_operation_t* *operation, *psa_key_derivation_step_t* step, uint64_t value)

Provide a numeric input for key derivation or key agreement.

Parameters

- **operation** (*psa_key_derivation_operation_t**) – The key derivation operation object to use. It must have been set up with `psa_key_derivation_setup()` and must not have produced any output yet.
- **step** (*psa_key_derivation_step_t*) – Which step the input data is for.
- **value** (uint64_t) – The value of the numeric input.

3.2.1.5.67.1 Description

Warning: Not supported

Which inputs are required and in what order depends on the algorithm. However, when an algorithm requires a particular order, numeric inputs usually come first as they tend to be configuration parameters. Refer to the documentation of each key derivation or key agreement algorithm for information.

This function is used for inputs which are fixed-size non-negative integers.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_key_derivation_abort()`.

3.2.1.5.67.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The following conditions can result in this error:
 - The operation state is not valid for this input `step`. This can happen if the application provides a step out of order or repeats a step that may not be repeated.
 - The library requires initializing by a call to `psa_crypto_init()`.
- **PSA_ERROR_INVALID_ARGUMENT:**
The following conditions can result in this error:
 - `step` is not compatible with the operation's algorithm.
 - `step` does not allow numerical inputs.
 - `value` is not valid for `step` in the operation's algorithm.
- **PSA_ERROR_NOT_SUPPORTED:**
The following conditions can result in this error:
 - `step` is not supported with the operation's algorithm.
 - `value` is not supported for `step` in the operation's algorithm.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`

3.2.1.5.68 `psa_key_derivation_input_key`

psa_status_t **psa_key_derivation_input_key**(*psa_key_derivation_operation_t* *operation,
psa_key_derivation_step_t step, *psa_key_id_t* key)

Provide an input for key derivation in the form of a key.

Parameters

- **operation** (*psa_key_derivation_operation_t**) – The key derivation operation object to use. It must have been set up with `psa_key_derivation_setup()` and must not have produced any output yet.
- **step** (*psa_key_derivation_step_t*) – Which step the input data is for.
- **key** (*psa_key_id_t*) – Identifier of the key. It must have an appropriate type for step and must allow the usage `PSA_KEY_USAGE_DERIVE`.

3.2.1.5.68.1 Description

Which inputs are required and in what order depends on the algorithm. Refer to the documentation of each key derivation or key agreement algorithm for information.

This function obtains input from a key object, which is usually correct for secret inputs or for non-secret personalization strings kept in the key store. To pass a non-secret parameter which is not in the key store, call `psa_key_derivation_input_bytes()` instead of this function. Refer to the documentation of individual step types (`PSA_KEY_DERIVATION_INPUT_XXX` values of type `typedef psa_key_derivation_step_t`) for more information.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_key_derivation_abort()`.

3.2.1.5.68.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_DERIVE` flag.
- **PSA_ERROR_INVALID_ARGUMENT:**
step is not compatible with the operation's algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
step does not allow key inputs of the given type or does not allow key inputs at all.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid for this input step. This can happen if the application provides a step out of order or repeats a step that may not be repeated.
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.69 `psa_key_derivation_key_agreement`

psa_status_t `psa_key_derivation_key_agreement`(*psa_key_derivation_operation_t* *operation, *psa_key_derivation_step_t* step, *psa_key_id_t* private_key, const uint8_t *peer_key, size_t peer_key_length)

Perform a key agreement and use the shared secret as input to a key derivation.

Parameters

- **operation** (*psa_key_derivation_operation_t**) – The key derivation operation object to use. It must have been set up with `psa_key_derivation_setup()` with a key agreement and derivation algorithm alg (`PSA_ALG_XXX` value such that `PSA_ALG_IS_KEY_AGREEMENT(alg)` is true and `PSA_ALG_IS_RAW_KEY_AGREEMENT(alg)` is false). The operation must be ready for an input of the type given by step.
- **step** (*psa_key_derivation_step_t*) – Which step the input data is for.
- **private_key** (*psa_key_id_t*) – Identifier of the private key to use. It must allow the usage `PSA_KEY_USAGE_DERIVE`.
- **peer_key** (const uint8_t*) – Public key of the peer. The peer key must be in the same format that `psa_import_key()` accepts for the public key type corresponding to the type of `private_key`. That is, this function performs the equivalent of `psa_import_key(..., peer_key, peer_key_length)` where with key attributes indicating the public key type corresponding to the type of `private_key`. For example, for EC keys, this means that `peer_key` is interpreted as a point on the curve that the private key is on. The standard formats for public keys are documented in the documentation of `psa_export_public_key()`.
- **peer_key_length** (size_t) – Size of `peer_key` in bytes.

3.2.1.5.69.1 Description

A key agreement algorithm takes two inputs: a private key `private_key` a public key `peer_key`. The result of this function is passed as input to a key derivation. The output of this key derivation can be extracted by reading from the resulting operation to produce keys and other cryptographic material.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_key_derivation_abort()`.

3.2.1.5.69.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid for this key agreement step.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_DERIVE` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
`private_@key` is not compatible with `alg`, or `peer_key` is not valid for `alg` or not compatible with `private_key`.
- **PSA_ERROR_NOT_SUPPORTED:**
`alg` is not supported or is not a key derivation algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
step does not allow an input resulting from a key agreement.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.70 `psa_key_derivation_operation_init`

`psa_key_derivation_operation_t` **`psa_key_derivation_operation_init`**(void)

Return an initial value for a key derivation operation object.

Parameters

- **void** – no arguments

3.2.1.5.70.1 Return

`typedef psa_key_derivation_operation_t`

3.2.1.5.71 psa_key_derivation_output_bytes

`psa_status_t psa_key_derivation_output_bytes(psa_key_derivation_operation_t *operation, uint8_t *output, size_t output_length)`

Read some data from a key derivation operation.

Parameters

- **operation** (`psa_key_derivation_operation_t*`) – The key derivation operation object to read from.
- **output** (`uint8_t*`) – Buffer where the output will be written.
- **output_length** (`size_t`) – Number of bytes to output.

3.2.1.5.71.1 Description

This function calculates output bytes from a key derivation algorithm and returns those bytes. If the key derivation's output is viewed as a stream of bytes, this function consumes the requested number of bytes from the stream and returns them to the caller. The operation's capacity decreases by the number of bytes read.

If this function returns an error status other than `PSA_ERROR_INSUFFICIENT_DATA`, the operation enters an error state and must be aborted by calling `psa_key_derivation_abort()`.

3.2.1.5.71.2 Return

- `PSA_SUCCESS`
- **PSA_ERROR_INSUFFICIENT_DATA:**
The operation's capacity was less than `output_length` bytes. Note that in this case, no output is written to the `output` buffer. The operation's capacity is set to 0, thus subsequent calls to this function will not succeed, even with a smaller output buffer.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active and completed all required input steps.
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.72 `psa_key_derivation_output_key`

psa_status_t **psa_key_derivation_output_key**(const *psa_key_attributes_t* *attributes,
psa_key_derivation_operation_t *operation,
psa_key_id_t *key)

Derive a key from an ongoing key derivation operation.

Parameters

- **attributes** (const *psa_key_attributes_t* *) – The attributes for the new key.
- **operation** (*psa_key_derivation_operation_t* *) – The key derivation operation object to read from.
- **key** (*psa_key_id_t* *) – On success, an identifier for the newly created key. `PSA_KEY_ID_NULL` on failure.

3.2.1.5.72.1 Description

This function calculates output bytes from a key derivation algorithm and uses those bytes to generate a key deterministically. The key's location, policy, type and size are taken from **attributes**.

If the key derivation's output is viewed as a stream of bytes, this function consumes the required number of bytes from the stream. The operation's capacity decreases by the number of bytes used to derive the key.

If this function returns an error status other than `PSA_ERROR_INSUFFICIENT_DATA`, the operation enters an error state and must be aborted by calling `psa_key_derivation_abort()`.

How much output is produced and consumed from the operation, and how the key is derived, depends on the key type. The table below describes the required key derivation procedures for standard key derivation algorithms. Implementations can use other methods for implementation-specific algorithms.

In all cases, the data that is read is discarded from the operation. The operation's capacity is decreased by the number of bytes read.

Key type	Key type details and derivation procedure
AES	PSA_KEY_TYPE_AES
ARC4	PSA_KEY_TYPE_ARC4
CAMELLIA	PSA_KEY_TYPE_CAMELLIA
ChaCha20	PSA_KEY_TYPE_CHACHA20
SM4	PSA_KEY_TYPE_SM4
Secrets for derivation	PSA_KEY_TYPE_DERIVE
HMAC	PSA_KEY_TYPE_HMAC For key types for which the key is an arbitrary sequence of bytes of a given size, this function is functionally equivalent to calling <code>psa_key_derivation_output_bytes()</code> and passing the resulting output to <code>psa_import_key()</code> . However, this function has a security benefit: if the implementation provides an isolation boundary then the key material is not exposed outside the isolation boundary. As a consequence, for these key types, this function always consumes exactly (bits/8) bytes from the operation.
DES	PSA_KEY_TYPE_DES, 64 bits. This function generates a key using the following process: 1. Draw an 8-byte string. 2. Set/clear the parity bits in each byte. 3. If the result is a forbidden weak key, discard the result and return to step 1. 4. Output the string.
2-key 3DES	PSA_KEY_TYPE_DES, 192 bits.
3-key 3DES	PSA_KEY_TYPE_DES, 128 bits. The two or three keys are generated by repeated application of the process used to generate a DES key. For example, for 3-key 3DES, if the first 8 bytes specify a weak key and the next 8 bytes do not, discard the first 8 bytes, use the next 8 bytes as the first key, and continue reading output from the operation to derive the other two keys.
Finite-field Diffie-Hellman keys	PSA_KEY_TYPE_DH_KEY_PAIR(<i>dh_family</i>) where <i>dh_family</i> designates any Diffie-Hellman family.
ECC keys on a Weierstrass elliptic curve	PSA_KEY_TYPE_ECC_KEY_PAIR(<i>ecc_family</i>) where <i>ecc_family</i> designates a Weierstrass curve family. These key types require the generation of a private key, which is an integer in the range [1, N - 1], where N is the boundary of the private key domain: N is the prime p for Diffie-Hellman, or the order of the curve's base point for ECC.

For algorithms that take an input step `PSA_KEY_DERIVATION_INPUT_SECRET`, the input to that step must be provided with `psa_key_derivation_input_key()`. Future versions of this specification might include additional restrictions on the derived key based on the attributes and strength of the secret key.

This function uses the `attributes` as follows:

- The key type is required. It cannot be an asymmetric public key.
- The key size is required. It must be a valid size for the key type.
- The key permitted-algorithm policy is required for keys that will be used for a cryptographic operation, see Permitted algorithms.
- The key usage flags define what operations are permitted with the key, see Key usage flags.
- The key lifetime and identifier are required for a persistent key.

Note:

This is an input parameter: it is not updated with the final key attributes. The final attributes of the new key can be queried by calling `psa_get_key_attributes()` with the key's identifier.

3.2.1.5.72.2 Return

- **PSA_SUCCESS:**
Success. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
- **PSA_ERROR_ALREADY_EXISTS:**
This is an attempt to create a persistent key, and there is already a persistent key with the
given identifier.
- **PSA_ERROR_INSUFFICIENT_DATA:**
There was not enough data to create the desired key. Note that in this case, no output is written to the output buffer. The operation's capacity is set to 0, thus subsequent calls to this function will not succeed, even with a smaller output buffer.
- **PSA_ERROR_NOT_SUPPORTED:**
The key type or key size is not supported, either by the implementation in general or in this particular location.
- **PSA_ERROR_INVALID_ARGUMENT:**
The key attributes, as a whole, are invalid.
- **PSA_ERROR_INVALID_ARGUMENT:**
The key type is an asymmetric public key type.
- **PSA_ERROR_INVALID_ARGUMENT:**
The key size is not a valid size for the key type.
- **PSA_ERROR_NOT_PERMITTED:**
The `PSA_KEY_DERIVATION_INPUT_SECRET` input was neither provided through a key nor the result of a key agreement.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active and completed all required input steps.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_INSUFFICIENT_STORAGE`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.73 `psa_key_derivation_set_capacity`

psa_status_t **psa_key_derivation_set_capacity**(*psa_key_derivation_operation_t* *operation, size_t capacity)

Set the maximum capacity of a key derivation operation.

Parameters

- **operation** (*psa_key_derivation_operation_t**) – The key derivation operation object to modify.
- **capacity** (size_t) – The new capacity of the operation. It must be less or equal to the operation's current capacity.

3.2.1.5.73.1 Description

Warning: Not supported

The capacity of a key derivation operation is the maximum number of bytes that the key derivation operation can return from this point onwards.

3.2.1.5.73.2 Return

- `PSA_SUCCESS`
- **PSA_ERROR_INVALID_ARGUMENT:**
capacity is larger than the operation's current capacity. In this case, the operation object remains valid and its capacity remains unchanged.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active.
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.74 `psa_key_derivation_setup`

`psa_status_t` **psa_key_derivation_setup**(`psa_key_derivation_operation_t` *operation,
`psa_algorithm_t` alg)

Set up a key derivation operation.

Parameters

- **operation** (`psa_key_derivation_operation_t`*) – The key derivation operation object to set up. It must have been initialized but not set up yet.
- **alg** (`psa_algorithm_t`) – The key derivation algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_KEY_DERIVATION(alg) is true).

3.2.1.5.74.1 Description

A key derivation algorithm takes some inputs and uses them to generate a byte stream in a deterministic way. This byte stream can be used to produce keys and other cryptographic material.

To derive a key:

1. Start with an initialized object of `typedef psa_key_derivation_operation_t`.
2. Call `psa_key_derivation_setup()` to select the algorithm.
3. Provide the inputs for the key derivation by calling `psa_key_derivation_input_bytes()` or `psa_key_derivation_input_key()` as appropriate. Which inputs are needed, in what order, whether keys are permitted, and what type of keys depends on the algorithm.
4. Optionally set the operation's maximum capacity with `psa_key_derivation_set_capacity()`. This can be done before, in the middle of, or after providing inputs. For some algorithms, this step is mandatory because the output depends on the maximum capacity.
5. To derive a key, call `psa_key_derivation_output_key()`. To derive a byte string for a different purpose, call `psa_key_derivation_output_bytes()`. Successive calls to these functions use successive output bytes calculated by the key derivation algorithm.
6. Clean up the key derivation operation object with `psa_key_derivation_abort()`.

If this function returns an error, the key derivation operation object is not changed.

If an error occurs at any step after a call to `psa_key_derivation_setup()`, the operation will need to be reset by a call to `psa_key_derivation_abort()`.

Implementations must reject an attempt to derive a key of size 0.

3.2.1.5.74.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_ARGUMENT:**
alg is not a key derivation algorithm.

- **PSA_ERROR_NOT_SUPPORTED:**

alg is not supported or is not a key derivation algorithm.

- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID

- **PSA_ERROR_BAD_STATE:**

The operation state is not valid: it must be inactive.

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.75 `psa_key_derivation_verify_bytes`

psa_status_t `psa_key_derivation_verify_bytes`(*psa_key_derivation_operation_t* *operation,
const uint8_t *expected_output, size_t
output_length)

Compare output data from a key derivation operation to an expected value.

Parameters

- **operation** (*psa_key_derivation_operation_t**) – The key derivation operation object to read from.
- **expected_output** (const uint8_t*) – Buffer containing the expected derivation output.
- **output_length** (size_t) – Length of the expected output. This is also the number of bytes that will be read.

3.2.1.5.75.1 Description

Warning: Not supported

This function calculates output bytes from a key derivation algorithm and compares those bytes to an expected value. If the key derivation's output is viewed as a stream of bytes, this function destructively reads `output_length` bytes from the stream before comparing them with `expected_output`. The operation's capacity decreases by the number of bytes read.

This is functionally equivalent to the following code:

```
uint8_t tmp[output_length];  
psa_key_derivation_output_bytes(operation, tmp, output_length);  
if (memcmp(expected_output, tmp, output_length) != 0)  
    return PSA_ERROR_INVALID_SIGNATURE;
```

However, calling `psa_key_derivation_verify_bytes()` works even if the key's policy does not allow output of the bytes.

If this function returns an error status other than `PSA_ERROR_INSUFFICIENT_DATA` or `PSA_ERROR_INVALID_SIGNATURE`, the operation enters an error state and must be aborted by calling `psa_key_derivation_abort()`.

Note:

Implementations must make the best effort to ensure that the comparison between the actual key derivation output and the expected output is performed in constant time.

3.2.1.5.75.2 Return

- **PSA_SUCCESS:**
Success. The output of the key derivation operation matches `expected_output`.
- **PSA_ERROR_BAD_STATE:**
The following conditions can result in this error:
 - The operation state is not valid: it must be active, with all required input steps complete.
 - The library requires initializing by a call to `psa_crypto_init()`.
- **PSA_ERROR_NOT_PERMITTED:**
One of the inputs is a key whose policy does not permit `PSA_KEY_USAGE_VERIFY_DERIVATION`.
- **PSA_ERROR_INVALID_SIGNATURE:**
The output of the key derivation operation does not match the value in `expected_output`.
- **PSA_ERROR_INSUFFICIENT_DATA:**
The operation's capacity was less than `output_length` bytes. In this case, the operation's capacity is set to zero — subsequent calls to this function will not succeed, even with a smaller expected output length.
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`

3.2.1.5.76 `psa_key_derivation_verify_key`

`psa_status_t` **psa_key_derivation_verify_key**(`psa_key_derivation_operation_t` *operation, `psa_key_id_t` expected)

Compare output data from a key derivation operation to an expected value stored in a key.

Parameters

- **operation** (`psa_key_derivation_operation_t`*) – The key derivation operation object to read from.
- **expected** (`psa_key_id_t`) – A key of type `PSA_KEY_TYPE_PASSWORD_HASH` containing the expected output.

The key must allow the usage `PSA_KEY_USAGE_VERIFY_DERIVATION`, and the permitted algorithm must match the operation's algorithm. The value of this key is typically computed by a previous call to `psa_key_derivation_output_key()`.

3.2.1.5.76.1 Description

Warning: Not supported

This function calculates output bytes from a key derivation algorithm and compares those bytes to an expected value, provided as key of type `PSA_KEY_TYPE_PASSWORD_HASH`. If the key derivation's output is viewed as a stream of bytes, this function destructively reads the number of bytes corresponding to the length of the expected key from the stream before comparing them with the key value. The operation's capacity decreases by the number of bytes read.

This is functionally equivalent to exporting the expected key and calling `psa_key_derivation_verify_bytes()` on the result, except that it works when the key cannot be exported.

If this function returns an error status other than `PSA_ERROR_INSUFFICIENT_DATA` or `PSA_ERROR_INVALID_SIGNATURE`, the operation enters an error state and must be aborted by calling `psa_key_derivation_abort()`.

Note:

Implementations must make the best effort to ensure that the comparison between the actual key derivation output and the expected output is performed in constant time.

3.2.1.5.76.2 Return

- **PSA_SUCCESS:**
Success. The output of the key derivation operation matches the expected key value.
- **PSA_ERROR_BAD_STATE:**
The following conditions can result in this error:
 - The operation state is not valid: it must be active, with all required input steps complete.
 - The library requires initializing by a call to `psa_crypto_init()`.
- **PSA_ERROR_INVALID_HANDLE:**
expected is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**
The following conditions can result in this error:
 - The key does not have the `PSA_KEY_USAGE_VERIFY_DERIVATION` flag, or it does not permit the requested algorithm.
 - One of the inputs is a key whose policy does not permit `PSA_KEY_USAGE_VERIFY_DERIVATION`.
- **PSA_ERROR_INVALID_SIGNATURE:**
The output of the key derivation operation does not match the value of the expected key.
- **PSA_ERROR_INSUFFICIENT_DATA:**
The operation's capacity was less than the length of the expected key. In this case, the operation's capacity is set to zero — subsequent calls to this function will not succeed, even with a smaller expected key length.

- **PSA_ERROR_INVALID_ARGUMENT:**
The key type is not PSA_KEY_TYPE_PASSWORD_HASH.
- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID

3.2.1.5.77 `psa_mac_abort`

psa_status_t **psa_mac_abort**(*psa_mac_operation_t* *operation)

Abort a MAC operation.

Parameters

- **operation** (*psa_mac_operation_t**) – Initialized MAC operation.

3.2.1.5.77.1 Description

Warning: Not supported

Aborting an operation frees all associated resources except for the operation object itself. Once aborted, the operation object can be reused for another operation by calling `psa_mac_sign_setup()` or `psa_mac_verify_setup()` again.

This function can be called any time after the operation object has been initialized by one of the methods described in `typedef psa_mac_operation_t`.

In particular, calling `psa_mac_abort()` after the operation has been terminated by a call to `psa_mac_abort()`, `psa_mac_sign_finish()` or `psa_mac_verify_finish()` is safe and has no effect.

3.2.1.5.77.2 Return

- PSA_SUCCESS
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- **PSA_ERROR_BAD_STATE**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.78 `psa_mac_compute`

psa_status_t **psa_mac_compute**(*psa_key_id_t* key, *psa_algorithm_t* alg, const uint8_t *input, size_t input_length, uint8_t *mac, size_t mac_size, size_t *mac_length)

Calculate the message authentication code (MAC) of a message.

Parameters

- **key** (*psa_key_id_t*) – Identifier of the key to use for the operation. It must allow the usage `PSA_KEY_USAGE_SIGN_MESSAGE`.
- **alg** (*psa_algorithm_t*) – The MAC algorithm to compute (PSA_ALG_XXX value such that `PSA_ALG_IS_MAC(alg)` is true).
- **input** (`const uint8_t*`) – Buffer containing the input message.
- **input_length** (`size_t`) – Size of the input buffer in bytes.
- **mac** (`uint8_t*`) – Buffer where the MAC value is to be written.
- **mac_size** (`size_t`) – Size of the mac buffer in bytes.
- **mac_length** (`size_t*`) – On success, the number of bytes that make up the MAC value.

3.2.1.5.78.1 Description

Note:

To verify the MAC of a message against an expected value, use `psa_mac_verify()` instead. Beware that comparing integrity or authenticity data such as MAC values with a function such as `memcmp()` is risky because the time taken by the comparison might leak information about the MAC value which could allow an attacker to guess a valid MAC and thereby bypass security controls.

Parameter `mac_size` must be appropriate for the selected algorithm and key:

- The exact MAC size is `PSA_MAC_LENGTH(key_type, key_bits, alg)` where `key_type` and `key_bits` are attributes of the key used to compute the MAC.
- `PSA_MAC_MAX_SIZE` evaluates to the maximum MAC size of any supported MAC algorithm.

3.2.1.5.78.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_SIGN_MESSAGE` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with alg.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not a MAC algorithm.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the mac buffer is too small. `PSA_MAC_LENGTH()` or `PSA_MAC_MAX_SIZE` can be used to determine the required buffer size.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE:**
The key could not be retrieved from storage.
- **PSA_ERROR_DATA_CORRUPT:**
The key could not be retrieved from storage.
- **PSA_ERROR_DATA_INVALID:**
The key could not be retrieved from storage.
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.79 `psa_mac_operation_init`

`psa_mac_operation_t` **psa_mac_operation_init**(void)

Return an initial value for a MAC operation object.

Parameters

- **void** – no arguments

3.2.1.5.79.1 Return

`typedef psa_mac_operation_t`

3.2.1.5.80 `psa_mac_sign_finish`

`psa_status_t` **psa_mac_sign_finish**(`psa_mac_operation_t` *operation, uint8_t *mac, size_t mac_size, size_t *mac_length)

Finish the calculation of the MAC of a message.

Parameters

- **operation** (`psa_mac_operation_t`*) – Active MAC operation.
- **mac** (uint8_t*) – Buffer where the MAC value is to be written.
- **mac_size** (size_t) – Size of the mac buffer in bytes.
- **mac_length** (size_t*) – On success, the number of bytes that make up the MAC value. This is always `PSA_MAC_FINAL_SIZE(key_type, key_bits, alg)` where `key_type` and `key_bits` are the type and bit-size respectively of the key and `alg` is the MAC algorithm that is calculated.

3.2.1.5.80.1 Description

Warning: Not supported

The application must call `psa_mac_sign_setup()` before calling this function. This function calculates the MAC of the message formed by concatenating the inputs passed to preceding calls to `psa_mac_update()`.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_mac_abort()`.

Warning:

It is not recommended to use this function when a specific value is expected for the MAC. Call `psa_mac_verify_finish()` instead with the expected MAC value.

Comparing integrity or authenticity data such as MAC values with a function such as `memcmp()` is risky because the time taken by the comparison might leak information about the hashed data which could allow an attacker to guess a valid MAC and thereby bypass security controls.

Parameter `mac_size` must be appropriate for the selected algorithm and key:

- The exact MAC size is `PSA_MAC_LENGTH(key_type, key_bits, alg)` where `key_type` and `key_bits` are attributes of the key, and `alg` is the algorithm used to compute the MAC.
- `PSA_MAC_MAX_SIZE` evaluates to the maximum MAC size of any supported MAC algorithm.

3.2.1.5.80.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be an active mac sign operation.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the mac buffer is too small. `PSA_MAC_LENGTH()` or `PSA_MAC_MAX_SIZE` can be used to determine the required buffer size.
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.81 `psa_mac_sign_setup`

`psa_status_t` **psa_mac_sign_setup**(`psa_mac_operation_t` *operation, `psa_key_id_t` key, `psa_algorithm_t` alg)

Set up a multi-part MAC calculation operation.

Parameters

- **operation** (`psa_mac_operation_t`*) – The operation object to set up. It must have been initialized as per the documentation for `psa_mac_operation_t` and not yet in use.

- **key** (*psa_key_id_t*) – Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_SIGN_MESSAGE`.
- **alg** (*psa_algorithm_t*) – The MAC algorithm to compute (PSA_ALG_XXX value such that `PSA_ALG_IS_MAC(alg)` is true).

3.2.1.5.81.1 Description

Warning: Not supported

This function sets up the calculation of the message authentication code (MAC) of a byte string. To verify the MAC of a message against an expected value, use *psa_mac_verify_setup()* instead.

The sequence of operations to calculate a MAC is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for *typedef psa_mac_operation_t*, e.g. `PSA_MAC_OPERATION_INIT`.
3. Call *psa_mac_sign_setup()* to specify the algorithm and key.
4. Call *psa_mac_update()* zero, one or more times, passing a fragment of the message each time. The MAC that is calculated is the MAC of the concatenation of these messages in order.
5. At the end of the message, call *psa_mac_sign_finish()* to finish calculating the MAC value and retrieve it.

If an error occurs at any step after a call to *psa_mac_sign_setup()*, the operation will need to be reset by a call to *psa_mac_abort()*. The application can call *psa_mac_abort()* at any time after the operation has been initialized.

After a successful call to *psa_mac_sign_setup()*, the application must eventually terminate the operation through one of the following methods:

- A successful call to *psa_mac_sign_finish()*.
- A call to *psa_mac_abort()*.

3.2.1.5.81.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_SIGN_MESSAGE` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with alg.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not a MAC algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**

- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE:**
The key could not be retrieved from storage.
- **PSA_ERROR_DATA_CORRUPT:**
The key could not be retrieved from storage.
- **PSA_ERROR_DATA_INVALID:**
The key could not be retrieved from storage.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be inactive.
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.82 `psa_mac_update`

`psa_status_t` **psa_mac_update**(`psa_mac_operation_t` *operation, const uint8_t *input, size_t input_length)

Add a message fragment to a multi-part MAC operation.

Parameters

- **operation** (`psa_mac_operation_t`*) – Active MAC operation.
- **input** (const uint8_t*) – Buffer containing the message fragment to add to the MAC calculation.
- **input_length** (size_t) – Size of the input buffer in bytes.

3.2.1.5.82.1 Description

Warning: Not supported

The application must call `psa_mac_sign_setup()` or `psa_mac_verify_setup()` before calling this function.

If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_mac_abort()`.

3.2.1.5.82.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be active.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.83 `psa_mac_verify`

`psa_status_t` **psa_mac_verify**(`psa_key_id_t` key, `psa_algorithm_t` alg, const uint8_t *input, size_t input_length, const uint8_t *mac, size_t mac_length)

Calculate the MAC of a message and compare it with a reference value.

Parameters

- **key** (`psa_key_id_t`) – Identifier of the key to use for the operation. It must allow the usage `PSA_KEY_USAGE_VERIFY_MESSAGE`.
- **alg** (`psa_algorithm_t`) – The MAC algorithm to compute (PSA_ALG_XXX value such that `PSA_ALG_IS_MAC(alg)` is true).
- **input** (const uint8_t*) – Buffer containing the input message.
- **input_length** (size_t) – Size of the input buffer in bytes.
- **mac** (const uint8_t*) – Buffer containing the expected MAC value.
- **mac_length** (size_t) – Size of the mac buffer in bytes.

3.2.1.5.83.1 Return

- **PSA_SUCCESS:**
The expected MAC is identical to the actual MAC of the input.
- **PSA_ERROR_INVALID_SIGNATURE:**
The MAC of the message was calculated successfully, but it differs from the expected value.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_VERIFY_MESSAGE` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with alg.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not a MAC algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**

- **PSA_ERROR_STORAGE_FAILURE:**

The key could not be retrieved from storage.

- **PSA_ERROR_DATA_CORRUPT:**

The key could not be retrieved from storage.

- **PSA_ERROR_DATA_INVALID:**

The key could not be retrieved from storage.

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.84 `psa_mac_verify_finish`

`psa_status_t` **psa_mac_verify_finish**(`psa_mac_operation_t` *operation, const uint8_t *mac, size_t mac_length)

Finish the calculation of the MAC of a message and compare it with an expected value.

Parameters

- **operation** (`psa_mac_operation_t`*) – Active MAC operation.
- **mac** (const uint8_t*) – Buffer containing the expected MAC value.
- **mac_length** (size_t) – Size of the mac buffer in bytes.

3.2.1.5.84.1 Description

Warning: Not supported

The application must call `psa_mac_verify_setup()` before calling this function. This function calculates the MAC of the message formed by concatenating the inputs passed to preceding calls to `psa_mac_update()`. It then compares the calculated MAC with the expected MAC passed as a parameter to this function.

When this function returns successfully, the operation becomes inactive. If this function returns an error status, the operation enters an error state and must be aborted by calling `psa_mac_abort()`.

Note:

Implementations must make the best effort to ensure that the comparison between the actual MAC and the expected MAC is performed in constant time.

3.2.1.5.84.2 Return

- **PSA_SUCCESS:**
The expected MAC is identical to the actual MAC of the message.
- **PSA_ERROR_INVALID_SIGNATURE:**
The MAC of the message was calculated successfully, but it differs from the expected MAC.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be an active mac verify operation.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**

- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.85 `psa_mac_verify_setup`

`psa_status_t` **psa_mac_verify_setup**(`psa_mac_operation_t` *operation, `psa_key_id_t` key, `psa_algorithm_t` alg)

Set up a multi-part MAC verification operation.

Parameters

- **operation** (`psa_mac_operation_t`*) – The operation object to set up. It must have been initialized as per the documentation for `typedef psa_mac_operation_t` and not yet in use.
- **key** (`psa_key_id_t`) – Identifier of the key to use for the operation. It must remain valid until the operation terminates. It must allow the usage `PSA_KEY_USAGE_VERIFY_MESSAGE`.
- **alg** (`psa_algorithm_t`) – The MAC algorithm to compute (PSA_ALG_XXX value such that `PSA_ALG_IS_MAC(alg)` is true).

3.2.1.5.85.1 Description

Warning: Not supported

This function sets up the verification of the message authentication code (MAC) of a byte string against an expected value.

The sequence of operations to verify a MAC is as follows:

1. Allocate an operation object which will be passed to all the functions listed here.
2. Initialize the operation object with one of the methods described in the documentation for `typedef psa_mac_operation_t`, e.g. `PSA_MAC_OPERATION_INIT`.
3. Call `psa_mac_verify_setup()` to specify the algorithm and key.
4. Call `psa_mac_update()` zero, one or more times, passing a fragment of the message each time. The MAC that is calculated is the MAC of the concatenation of these messages in order.
5. At the end of the message, call `psa_mac_verify_finish()` to finish calculating the actual MAC of the message and verify it against the expected value.

If an error occurs at any step after a call to `psa_mac_verify_setup()`, the operation will need to be reset by a call to `psa_mac_abort()`. The application can call `psa_mac_abort()` at any time after the operation has been initialized.

After a successful call to `psa_mac_verify_setup()`, the application must eventually terminate the operation through one of the following methods:

- A successful call to `psa_mac_verify_finish()`.
- A call to `psa_mac_abort()`.

3.2.1.5.85.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the `PSA_KEY_USAGE_VERIFY_MESSAGE` flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
key is not compatible with alg.
- **PSA_ERROR_NOT_SUPPORTED:**
alg is not supported or is not a MAC algorithm.
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE:**
The key could not be retrieved from storage
- **PSA_ERROR_DATA_CORRUPT:**
The key could not be retrieved from storage.
- **PSA_ERROR_DATA_INVALID:**
The key could not be retrieved from storage.
- **PSA_ERROR_BAD_STATE:**
The operation state is not valid: it must be inactive.
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.86 `psa_purge_key`

`psa_status_t` **psa_purge_key**(`psa_key_id_t` key)

Remove non-essential copies of key material from memory.

Parameters

- **key** (`psa_key_id_t`) – Identifier of the key to purge.

3.2.1.5.86.1 Description

Warning: Not supported

For keys that have been created with the `PSA_KEY_USAGE_CACHE` usage flag, an implementation is permitted to make additional copies of the key material that are not in storage and not for the purpose of ongoing operations.

This function will remove these extra copies of the key material from memory.

This function is not required to remove key material from memory in any of the following situations:

- The key is currently in use in a cryptographic operation.
- The key is volatile.

3.2.1.5.86.2 Return

- **PSA_SUCCESS:**
The key material will have been removed from memory if it is not currently required.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.87 `psa_key_agreement`

psa_status_t **psa_key_agreement**(*psa_key_id_t* private_key, const uint8_t *peer_key, size_t peer_key_length, *psa_algorithm_t* alg, const *psa_key_attributes_t* *attributes, *psa_key_id_t* *key)

Perform a key agreement and return the shared secret as a derivation key.

Parameters

- **private_key** (*psa_key_id_t*) – Identifier of the private key to use. It must permit the usage `PSA_KEY_USAGE_DERIVE`.
- **peer_key** (const uint8_t*) – Public key of the peer. The peer key data is parsed with the type `PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR(type)` where type is the type of `private_key`, and with the same bit-size as `private_key`. The peer key must be in the format that `psa_import_key()` accepts for this public key type. These formats are described in Key formats.
- **peer_key_length** (size_t) – Size of `peer_key` in bytes.
- **alg** (*psa_algorithm_t*) – The standalone key agreement algorithm to compute: a value of type `psa_algorithm_t` such that `PSA_ALG_IS_STANDALONE_KEY_AGREEMENT(alg)` is true.
- **attributes** (const *psa_key_attributes_t**) – The attributes for the new key.

-
- **key** (*psa_key_id_t**) – On success, an identifier for the newly created key. PSA_KEY_ID_NULL on failure.

3.2.1.5.87.1 Description

Warning: Not supported

A key agreement algorithm takes two inputs: a private key `private_key`, and a public key `peer_key`. The result of this function is a shared secret, returned as a derivation key. This key can be input to a key derivation operation using `psa_key_derivation_input_key()`.

Warning

The shared secret resulting from a key agreement algorithm such as finite-field Diffie-Hellman or elliptic curve Diffie-Hellman has biases. This makes it unsuitable for use as key material, for example, as an AES key. Instead, it is recommended that a key derivation algorithm is applied to the result, to derive unbiased cryptographic keys.

This function uses the `attributes` as follows:

- The key type must be one of PSA_KEY_TYPE_DERIVE, PSA_KEY_TYPE_RAW_DATA, PSA_KEY_TYPE_HMAC, or PSA_KEY_TYPE_PASSWORD. Implementations must support the PSA_KEY_TYPE_DERIVE and PSA_KEY_TYPE_RAW_DATA key types.
- The size of the returned key is always the bit-size of the shared secret, rounded up to a whole number of bytes. The key size in `attributes` can be zero; if it is nonzero, it must be equal to the output size of the key agreement, in bits. The output size, in bits, of the key agreement is $8 * \text{PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE}(\text{type}, \text{bits})$, where `type` and `bits` are the type and bit-size of `private_key`.
- The key permitted-algorithm policy is required for keys that will be used for a cryptographic operation, see Permitted algorithms.
- The key usage flags define what operations are permitted with the key, see Key usage flags.
- The key lifetime and identifier are required for a persistent key.

Note:

This is an input parameter: it is not updated with the final key attributes. The final attributes of the new key can be queried by calling `psa_get_key_attributes()` with the key's identifier.

3.2.1.5.87.2 Return

- **PSA_SUCCESS:**
Success. The new key contains the share secret. If the key is persistent, the key material and the key's metadata have been saved to persistent storage.
- **PSA_ERROR_BAD_STATE:**
The library requires initializing by a call to `psa_crypto_init()`.
- **PSA_ERROR_INVALID_HANDLE:**
`private_key` is not a valid key identifier.
- **PSA_ERROR_NOT_PERMITTED:**
The following conditions can result in this error:
 - `private_key` does not have the PSA_KEY_USAGE_DERIVE flag, or it does not permit the requested algorithm.

-
- The implementation does not permit creating a key with the specified `attributes` due to some implementation-specific policy.
 - **PSA_ERROR_ALREADY_EXISTS:**
This is an attempt to create a persistent key, and there is already a persistent key with the given identifier.
 - **PSA_ERROR_INVALID_ARGUMENT:**
The following conditions can result in this error:
 - `alg` is not a key agreement algorithm.
 - `private_key` is not compatible with `alg`.
 - `peer_key` is not a valid public key corresponding to `private_key`.
 - The output key attributes in `attributes` are not valid:
 - * The key type is not valid for key agreement output.
 - * The key size is nonzero, and is not the size of the shared secret.
 - * The key lifetime is invalid.
 - * The key identifier is not valid for the key lifetime.
 - * The key usage flags include invalid values.
 - * The key's permitted-usage algorithm is invalid.
 - * The key attributes, as a whole, are invalid.
 - **PSA_ERROR_NOT_SUPPORTED:**
The following conditions can result in this error:
 - `alg` is not supported or is not a key agreement algorithm.
 - `private_key` is not supported for use with `alg`.
 - The output key attributes, as a whole, are not supported, either by the implementation in general or in the specified storage location.
 - **PSA_ERROR_INSUFFICIENT_MEMORY**
 - **PSA_ERROR_INSUFFICIENT_STORAGE**
 - **PSA_ERROR_COMMUNICATION_FAILURE**
 - **PSA_ERROR_CORRUPTION_DETECTED**
 - **PSA_ERROR_STORAGE_FAILURE**
 - **PSA_ERROR_DATA_CORRUPT**
 - **PSA_ERROR_DATA_INVALID**

3.2.1.5.88 `psa_raw_key_agreement`

psa_status_t **psa_raw_key_agreement**(*psa_algorithm_t* alg, *psa_key_id_t* private_key, const uint8_t *peer_key, size_t peer_key_length, uint8_t *output, size_t output_size, size_t *output_length)

Perform a key agreement and return the raw shared secret.

Parameters

- **alg** (*psa_algorithm_t*) – The key agreement algorithm to compute (PSA_ALG_XXX value such that PSA_ALG_IS_RAW_KEY_AGREEMENT(alg) is true).
- **private_key** (*psa_key_id_t*) – Identifier of the private key to use. It must allow the usage PSA_KEY_USAGE_DERIVE.
- **peer_key** (const uint8_t*) – Public key of the peer. It must be in the same format that *psa_import_key()* accepts. The standard formats for public keys are documented in the documentation of *psa_export_public_key()*.
- **peer_key_length** (size_t) – Size of peer_key in bytes.
- **output** (uint8_t*) – Buffer where the raw shared secret is to be written.
- **output_size** (size_t) – Size of the output buffer in bytes.
- **output_length** (size_t*) – On success, the number of bytes that make up the returned output.

3.2.1.5.88.1 Description

Warning: Not supported

Warning:

The raw result of a key agreement algorithm such as finite-field Diffie-Hellman or elliptic curve Diffie-Hellman has biases, and is not suitable for use as key material. Instead it is recommended that the result is used as input to a key derivation algorithm. To chain a key agreement with a key derivation, use *psa_key_derivation_key_agreement()* and other functions from the key derivation interface.

Parameter output_size must be appropriate for the keys:

- The required output size is PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE(type, bits) where type is the type of private_key and bits is the bit-size of either private_key or the peer_key.
- PSA_RAW_KEY_AGREEMENT_OUTPUT_MAX_SIZE evaluates to the maximum output size of any supported raw key agreement algorithm.

3.2.1.5.88.2 Return

- **PSA_SUCCESS:**
Success.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the PSA_KEY_USAGE_DERIVE flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_ARGUMENT:**
alg is not a key agreement algorithm
- **PSA_ERROR_INVALID_ARGUMENT:**
private_@key is not compatible with alg, or peer_key is not valid for alg or not compatible with private_key.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the output buffer is too small. *PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE()*

or `PSA_RAW_KEY_AGREEMENT_OUTPUT_MAX_SIZE` can be used to determine the required buffer size.

- **PSA_ERROR_NOT_SUPPORTED:**

`alg` is not a supported key agreement algorithm.

- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.89 `psa_reset_key_attributes`

void **psa_reset_key_attributes**(*psa_key_attributes_t* *attributes)

Reset a key attribute object to a freshly initialized state.

Parameters

- **attributes** (*psa_key_attributes_t**) – The attribute object to reset.

3.2.1.5.89.1 Description

The attribute object must be initialized as described in the documentation of the type `typedef psa_key_attributes_t` before calling this function. Once the object has been initialized, this function can be called at any time.

This function frees any auxiliary resources that the object might contain.

3.2.1.5.89.2 Return

void

3.2.1.5.90 `psa_set_key_algorithm`

void **psa_set_key_algorithm**(*psa_key_attributes_t* *attributes, *psa_algorithm_t* alg)

Declare the permitted algorithm policy for a key.

Parameters

- **attributes** (*psa_key_attributes_t**) – The attribute object to write to.
- **alg** (*psa_algorithm_t*) – The permitted algorithm to write.

3.2.1.5.90.1 Description

The permitted algorithm policy of a key encodes which algorithm or algorithms are permitted to be used with this key.

This function overwrites any permitted algorithm policy previously set in `attributes`.

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.90.2 Return

void

3.2.1.5.91 `psa_set_key_bits`

void **psa_set_key_bits**(*psa_key_attributes_t* *attributes, size_t bits)

Declare the size of a key.

Parameters

- **attributes** (*psa_key_attributes_t**) – The attribute object to write to.
- **bits** (size_t) – The key size in bits. If this is 0, the key size in `attributes` becomes unspecified. Keys of size 0 are not supported.

3.2.1.5.91.1 Description

This function overwrites any key size previously set in `attributes`.

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.91.2 Return

void

3.2.1.5.92 `psa_set_key_id`

void **psa_set_key_id**(*psa_key_attributes_t* *attributes, *psa_key_id_t* id)

Declare a key as persistent and set its key identifier.

Parameters

- **attributes** (*psa_key_attributes_t**) – The attribute object to write to.

- **id** (*psa_key_id_t*) – The persistent identifier for the key.

3.2.1.5.92.1 Description

The application must choose a value for **id** between `PSA_KEY_ID_USER_MIN` and `PSA_KEY_ID_USER_MAX`.

If the attribute object currently declares the key as volatile, which is the default lifetime of an attribute object, this function sets the lifetime attribute to `PSA_KEY_LIFETIME_PERSISTENT`.

This function does not access storage, it merely stores the given value in the attribute object. The persistent key will be written to storage when the attribute object is passed to a key creation function such as *psa_import_key()*, *psa_generate_key()*, *psa_key_derivation_output_key()* or *psa_copy_key()*.

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.92.2 Return

void

3.2.1.5.93 psa_set_key_lifetime

void **psa_set_key_lifetime**(*psa_key_attributes_t* *attributes, *psa_key_lifetime_t* lifetime)

Set the location of a persistent key.

Parameters

- **attributes** (*psa_key_attributes_t**) – The attribute object to write to.
- **lifetime** (*psa_key_lifetime_t*) – The lifetime for the key. If this is `PSA_KEY_LIFETIME_VOLATILE`, the key will be volatile, and the key identifier attribute is reset to `PSA_KEY_ID_NULL`.

3.2.1.5.93.1 Description

To make a key persistent, give it a persistent key identifier by using *psa_set_key_id()*. By default, a key that has a persistent identifier is stored in the default storage area identifier by `PSA_KEY_LIFETIME_PERSISTENT`. Call this function to choose a storage area, or to explicitly declare the key as volatile.

This function does not access storage, it merely stores the given value in the attribute object. The persistent key will be written to storage when the attribute object is passed to a key creation function such as *psa_import_key()*, *psa_generate_key()*, *psa_key_derivation_output_key()* or *psa_copy_key()*.

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.93.2 Return

void

3.2.1.5.94 psa_set_key_type

void **psa_set_key_type**(*psa_key_attributes_t* *attributes, *psa_key_type_t* type)

Declare the type of a key.

Parameters

- **attributes** (*psa_key_attributes_t**) – The attribute object to write to.
- **type** (*psa_key_type_t*) – The key type to write. If this is `PSA_KEY_TYPE_NONE`, the key type in **attributes** becomes unspecified.

3.2.1.5.94.1 Description

This function overwrites any key type previously set in **attributes**.

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.94.2 Return

void

3.2.1.5.95 psa_set_key_usage_flags

void **psa_set_key_usage_flags**(*psa_key_attributes_t* *attributes, *psa_key_usage_t* usage_flags)

Declare usage flags for a key.

Parameters

- **attributes** (*psa_key_attributes_t**) – The attribute object to write to.
- **usage_flags** (*psa_key_usage_t*) – `psa_set_key_usage_flags` The usage flags to write.

3.2.1.5.95.1 Description

Usage flags are part of a key's policy. They encode what kind of operations are permitted on the key. For more details, see Key policies.

This function overwrites any usage flags previously set in **attributes**.

Implementation note:

This is a simple accessor function that is not required to validate its inputs. The following approaches can be used to provide an efficient implementation:

- This function can be declared as static or inline, instead of using the default external linkage.
- This function can be provided as a function-like macro. In this form, the macro must evaluate each of its arguments exactly once, as if it was a function call.

3.2.1.5.95.2 Return

void

3.2.1.5.96 `psa_sign_hash`

psa_status_t **psa_sign_hash**(*psa_key_id_t* key, *psa_algorithm_t* alg, const uint8_t *hash, size_t hash_length, uint8_t *signature, size_t signature_size, size_t *signature_length)

Sign an already-calculated hash with a private key.

Parameters

- **key** (*psa_key_id_t*) – Identifier of the key to use for the operation. It must be an asymmetric key pair. The key must allow the usage `PSA_KEY_USAGE_SIGN_HASH`.
- **alg** (*psa_algorithm_t*) – An asymmetric signature algorithm that separates the hash and sign operations (`PSA_ALG_XXX` value such that `PSA_ALG_IS_SIGN_HASH(alg)` is true), that is compatible with the type of key.
- **hash** (const uint8_t*) – The input to sign. This is usually the hash of a message. See the detailed description of this function and the description of individual signature algorithms for a detailed description of acceptable inputs.
- **hash_length** (size_t) – Size of the hash buffer in bytes.
- **signature** (uint8_t*) – Buffer where the signature is to be written.
- **signature_size** (size_t) – Size of the signature buffer in bytes.
- **signature_length** (size_t*) – On success, the number of bytes that make up the returned signature value.

3.2.1.5.96.1 Description

With most signature mechanisms that follow the hash-and-sign paradigm, the hash input to this function is the hash of the message to sign. The hash algorithm is encoded in the signature algorithm.

Some hash-and-sign mechanisms apply a padding or encoding to the hash. In such cases, the encoded hash must be passed to this function. The current version of this specification defines one such signature algorithm: `PSA_ALG_RSA_PKCS1V15_SIGN_RAW`.

Note:

To perform a hash-and-sign algorithm, the hash must be calculated before passing it to this function. This can be done by calling `psa_hash_compute()` or with a multi-part hash operation. Alternatively, to hash and sign a message in a single call, use `psa_sign_message()`.

Parameter `signature_size` must be appropriate for the selected algorithm and key:

- The required signature size is `PSA_SIGN_OUTPUT_SIZE(key_type, key_bits, alg)` where `key_type` and `key_bits` are the type and bit-size respectively of `key`.
- `PSA_SIGNATURE_MAX_SIZE` evaluates to the maximum signature size of any supported signature algorithm.

3.2.1.5.96.2 Return

- `PSA_SUCCESS`
- `PSA_ERROR_INVALID_HANDLE`
- **`PSA_ERROR_NOT_PERMITTED:`**
The key does not have the `PSA_KEY_USAGE_SIGN_HASH` flag, or it does not permit the requested algorithm.
- **`PSA_ERROR_BUFFER_TOO_SMALL:`**
The size of the signature buffer is too small. `PSA_SIGN_OUTPUT_SIZE()` or `PSA_SIGNATURE_MAX_SIZE` can be used to determine the required buffer size.
- `PSA_ERROR_NOT_SUPPORTED`
- `PSA_ERROR_INVALID_ARGUMENT`
- `PSA_ERROR_INSUFFICIENT_MEMORY`
- `PSA_ERROR_COMMUNICATION_FAILURE`
- `PSA_ERROR_HARDWARE_FAILURE`
- `PSA_ERROR_CORRUPTION_DETECTED`
- `PSA_ERROR_STORAGE_FAILURE`
- `PSA_ERROR_DATA_CORRUPT`
- `PSA_ERROR_DATA_INVALID`
- `PSA_ERROR_INSUFFICIENT_ENTROPY`
- **`PSA_ERROR_BAD_STATE:`**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.97 `psa_sign_message`

psa_status_t **psa_sign_message**(*psa_key_id_t* key, *psa_algorithm_t* alg, const uint8_t *input, size_t input_length, uint8_t *signature, size_t signature_size, size_t *signature_length)

Sign a message with a private key. For hash-and-sign algorithms, this includes the hashing step.

Parameters

- **key** (*psa_key_id_t*) – Identifier of the key to use for the operation. It must be an asymmetric key pair. The key must allow the usage `PSA_KEY_USAGE_SIGN_MESSAGE`.

-
- **alg** (*psa_algorithm_t*) – An asymmetric signature algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_SIGN_MESSAGE(alg) is true), that is compatible with the type of key.
 - **input** (const uint8_t*) – The input message to sign.
 - **input_length** (size_t) – Size of the input buffer in bytes.
 - **signature** (uint8_t*) – Buffer where the signature is to be written.
 - **signature_size** (size_t) – Size of the signature buffer in bytes.
 - **signature_length** (size_t*) – On success, the number of bytes that make up the returned signature value.

3.2.1.5.97.1 Description

Note:

To perform a multi-part hash-and-sign signature algorithm, first use a multi-part hash operation and then pass the resulting hash to *psa_sign_hash()*. PSA_ALG_GET_HASH(alg) can be used to determine the hash algorithm to use.

Parameter **signature_size** must be appropriate for the selected algorithm and key:

- The required signature size is PSA_SIGN_OUTPUT_SIZE(key_type, key_bits, alg) where key_type and key_bits are the type and bit-size respectively of key.
- PSA_SIGNATURE_MAX_SIZE evaluates to the maximum signature size of any supported signature algorithm.

3.2.1.5.97.2 Return

- PSA_SUCCESS
- PSA_ERROR_INVALID_HANDLE
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the PSA_KEY_USAGE_SIGN_MESSAGE flag, or it does not permit the requested algorithm.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
The size of the signature buffer is too small. *PSA_SIGN_OUTPUT_SIZE()* or PSA_SIGNATURE_MAX_SIZE can be used to determine the required buffer size.
- PSA_ERROR_NOT_SUPPORTED
- PSA_ERROR_INVALID_ARGUMENT
- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID

- **PSA_ERROR_INSUFFICIENT_ENTROPY**

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.98 `psa_verify_hash`

`psa_status_t` **psa_verify_hash**(`psa_key_id_t` key, `psa_algorithm_t` alg, const uint8_t *hash, size_t hash_length, const uint8_t *signature, size_t signature_length)

Verify the signature of a hash or short message using a public key.

Parameters

- **key** (`psa_key_id_t`) – Identifier of the key to use for the operation. It must be a public key or an asymmetric key pair. The key must allow the usage `PSA_KEY_USAGE_VERIFY_HASH`.
- **alg** (`psa_algorithm_t`) – An asymmetric signature algorithm that separates the hash and sign operations (`PSA_ALG_XXX` value such that `PSA_ALG_IS_SIGN_HASH(alg)` is true), that is compatible with the type of key.
- **hash** (const uint8_t*) – The input whose signature is to be verified. This is usually the hash of a message. See the detailed description of this function and the description of individual signature algorithms for a detailed description of acceptable inputs.
- **hash_length** (size_t) – Size of the hash buffer in bytes.
- **signature** (const uint8_t*) – Buffer containing the signature to verify.
- **signature_length** (size_t) – Size of the signature buffer in bytes.

3.2.1.5.98.1 Description

With most signature mechanisms that follow the hash-and-sign paradigm, the hash input to this function is the hash of the message to sign. The hash algorithm is encoded in the signature algorithm.

Some hash-and-sign mechanisms apply a padding or encoding to the hash. In such cases, the encoded hash must be passed to this function. The current version of this specification defines one such signature algorithm: `PSA_ALG_RSA_PKCS1V15_SIGN_RAW`.

Note:

To perform a hash-and-sign verification algorithm, the hash must be calculated before passing it to this function. This can be done by calling `psa_hash_compute()` or with a multi-part hash operation. Alternatively, to hash and verify a message signature in a single call, use `psa_verify_message()`.

3.2.1.5.98.2 Return

- **PSA_SUCCESS:**
The signature is valid.
- **PSA_ERROR_INVALID_HANDLE**

- **PSA_ERROR_NOT_PERMITTED:**

The key does not have the PSA_KEY_USAGE_VERIFY_HASH flag, or it does not permit the requested algorithm.

- **PSA_ERROR_INVALID_SIGNATURE:**

The calculation was performed successfully, but the passed signature is not a valid signature.

- PSA_ERROR_NOT_SUPPORTED
- PSA_ERROR_INVALID_ARGUMENT
- PSA_ERROR_INSUFFICIENT_MEMORY
- PSA_ERROR_COMMUNICATION_FAILURE
- PSA_ERROR_HARDWARE_FAILURE
- PSA_ERROR_CORRUPTION_DETECTED
- PSA_ERROR_STORAGE_FAILURE
- PSA_ERROR_DATA_CORRUPT
- PSA_ERROR_DATA_INVALID

- **PSA_ERROR_BAD_STATE:**

The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.1.5.99 `psa_verify_message`

`psa_status_t` **psa_verify_message**(`psa_key_id_t` key, `psa_algorithm_t` alg, const uint8_t *input, size_t input_length, const uint8_t *signature, size_t signature_length)

Verify the signature of a message with a public key, using a hash-and-sign verification algorithm.

Parameters

- **key** (`psa_key_id_t`) – Identifier of the key to use for the operation. It must be a public key or an asymmetric key pair. The key must allow the usage PSA_KEY_USAGE_VERIFY_MESSAGE.
- **alg** (`psa_algorithm_t`) – An asymmetric signature algorithm (PSA_ALG_XXX value such that PSA_ALG_IS_SIGN_MESSAGE(alg) is true), that is compatible with the type of key.
- **input** (const uint8_t*) – The message whose signature is to be verified.
- **input_length** (size_t) – Size of the input buffer in bytes.
- **signature** (const uint8_t*) – Buffer containing the signature to verify.
- **signature_length** (size_t) – Size of the signature buffer in bytes.

3.2.1.5.99.1 Description

Note:

To perform a multi-part hash-and-sign signature verification algorithm, first use a multi-part hash operation to hash the message and then pass the resulting hash to `psa_verify_hash()`. PSA_ALG_GET_HASH(alg) can be used to determine the hash algorithm to use.

3.2.1.5.99.2 Return

- **PSA_SUCCESS:**
The signature is valid.
- **PSA_ERROR_INVALID_HANDLE**
- **PSA_ERROR_NOT_PERMITTED:**
The key does not have the PSA_KEY_USAGE_VERIFY_MESSAGE flag, or it does not permit the requested algorithm.
- **PSA_ERROR_INVALID_SIGNATURE:**
The calculation was performed successfully, but the passed signature is not a valid signature.
- **PSA_ERROR_NOT_SUPPORTED**
- **PSA_ERROR_INVALID_ARGUMENT**
- **PSA_ERROR_INSUFFICIENT_MEMORY**
- **PSA_ERROR_COMMUNICATION_FAILURE**
- **PSA_ERROR_HARDWARE_FAILURE**
- **PSA_ERROR_CORRUPTION_DETECTED**
- **PSA_ERROR_STORAGE_FAILURE**
- **PSA_ERROR_DATA_CORRUPT**
- **PSA_ERROR_DATA_INVALID**
- **PSA_ERROR_BAD_STATE:**
The library has not been previously initialized by `psa_crypto_init()`. It is implementation-dependent whether a failure to initialize results in this error code.

3.2.2 Initial Attestation APIs

3.2.2.1 Introduction

The PSA Attestation API is a standard interface provided by the PSA Root of Trust. The definition of the PSA Root of Trust is described in the PSA Security Model (PSA-SM - <https://www.arm.com/architecture/security-features>).

The API can be used either to directly sign data or as a way to bootstrap trust in other attestation schemes. PSA provides a framework and the minimal generic security features allowing OEM and service providers to integrate various attestation schemes on top of the PSA Root of Trust.

3.2.2.2 Reference

Documentation:

PSA Attestation API v1.0.2

Link:

https://armkeil.blob.core.windows.net/developer/Files/pdf/PlatformSecurityArchitecture/Implement/IHI0085-PSA_Attestation_API-1.0.2.pdf

3.2.2.3 psa_initial_attest_get_token

psa_status_t **psa_initial_attest_get_token**(const uint8_t *auth_challenge, size_t challenge_size, uint8_t *token_buf, size_t token_buf_size, size_t *token_size)

Retrieve the Initial Attestation Token.

Parameters

- **auth_challenge** (const uint8_t*) – Buffer with a challenge object. The challenge object is data provided by the caller. For example, it may be a cryptographic nonce or a hash of data (such as an external object record). If a hash of data is provided then it is the caller's responsibility to ensure that the data is protected against replay attacks (for example, by including a cryptographic nonce within the data).
- **challenge_size** (size_t) – Size of the buffer **auth_challenge** in bytes. The size must always be a supported challenge size. Supported challenge sizes are defined by the `PSA_INITIAL_ATTEST_CHALLENGE_SIZE_XXX` constant.
- **token_buf** (uint8_t*) – Output buffer where the attestation token is to be written.
- **token_buf_size** (size_t) – Size of **token_buf**. The expected size can be determined by using [*psa_initial_attest_get_token_size\(\)*](#).
- **token_size** (size_t*) – Output variable for the actual token size.

3.2.2.3.1 Description

Warning: Not supported

Retrieves the Initial Attestation Token. A challenge can be passed as an input to mitigate replay attacks.

3.2.2.3.2 Return

- **PSA_SUCCESS:**
Action was performed successfully.
- **PSA_ERROR_SERVICE_FAILURE:**
The implementation failed to fully initialize.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
token_buf is too small for the attestation token.
- **PSA_ERROR_INVALID_ARGUMENT:**
The challenge size is not supported.
- **PSA_ERROR_GENERIC_ERROR:**
An unspecified internal error has occurred.

3.2.2.4 psa_initial_attest_get_token_size

psa_status_t **psa_initial_attest_get_token_size**(size_t challenge_size, size_t *token_size)

Calculate the size of an Initial Attestation Token.

Parameters

- **challenge_size** (size_t) – Size of a challenge object in bytes. This must be a supported challenge size as defined by the PSA_INITIAL_ATTEST_CHALLENGE_SIZE_XXX constant.
- **token_size** (size_t*) – Output variable for the token size.

3.2.2.4.1 Description

Warning: Not supported

Retrieve the exact size of the Initial Attestation Token in bytes, given a specific challenge size.

3.2.2.4.2 Return

- **PSA_SUCCESS:**
Action was performed successfully.
- **PSA_ERROR_SERVICE_FAILURE:**
The implementation failed to fully initialize.
- **PSA_ERROR_INVALID_ARGUMENT:**
The challenge size is not supported.
- **PSA_ERROR_GENERIC_ERROR:**
An unspecified internal error has occurred.

3.2.2.5 psa_attest_key

psa_status_t **psa_attest_key**(*psa_key_id_t* key, const uint8_t *auth_challenge, size_t *challenge_size, uint8_t *cert_buf, size_t cert_buf_size, size_t *cert_size)

Retrieve a Key Attestation.

Parameters

- **key** (*psa_key_id_t*) – Key identifier.
- **auth_challenge** (const uint8_t*) – Buffer with a challenge object. The challenge object is data provided by the caller. For example, it may be a cryptographic nonce or a hash of data (such as an external object record). If a hash of data is provided then it is the caller's responsibility to ensure that the data is protected against replay attacks (for example, by including a cryptographic nonce within the data).
- **challenge_size** (size_t*) – Size of a challenge object in bytes. This must be a supported challenge size as defined by the PSA_INITIAL_ATTEST_CHALLENGE_SIZE_XXX constant.
- **cert_buf** (uint8_t*) – Output variable for the Key Attestation certificate.
- **cert_buf_size** (size_t) – Maximum size of the Key Attestation certificate.
- **cert_size** (size_t*) – Output variable for the actual Key Attestation certificate size.

3.2.2.5.1 Description

Warning: Not supported

Retrieves the Key Attestation certificate. A challenge can be passed as an input to mitigate replay attacks.

3.2.2.5.2 Return

- **PSA_SUCCESS:**
Action was performed successfully.
- **PSA_ERROR_SERVICE_FAILURE:**
The implementation failed to fully initialize.
- **PSA_ERROR_INVALID_ARGUMENT:**
The challenge size is not supported.
- **PSA_ERROR_GENERIC_ERROR:**
An unspecified internal error has occurred.

3.2.2.6 psa_attest_key_get_size

psa_status_t **psa_attest_key_get_size**(*psa_key_id_t* key, size_t challenge_size, size_t *cert_size)

Calculate the size of a Key Attestation certificate.

Parameters

- **key** (*psa_key_id_t*) – Key identifier.
- **challenge_size** (size_t) – Size of a challenge object in bytes. This must be a supported challenge size as defined by the PSA_INITIAL_ATTEST_CHALLENGE_SIZE_XXX constant.
- **cert_size** (size_t*) – Output variable for the certificate size.

3.2.2.6.1 Description

Warning: Not supported

Retrieve the exact size of the Key Attestation certificate in bytes, given a specific challenge size.

3.2.2.6.2 Return

- **PSA_SUCCESS:**
Action was performed successfully.
- **PSA_ERROR_SERVICE_FAILURE:**
The implementation failed to fully initialize.
- **PSA_ERROR_INVALID_ARGUMENT:**
The challenge size is not supported.
- **PSA_ERROR_GENERIC_ERROR:**
An unspecified internal error has occurred.

3.2.3 Storage APIs

3.2.3.1 Storage common APIs

3.2.3.1.1 Introduction

This file defines common definitions for PSA storage.

3.2.3.1.2 Reference

Documentation:

PSA Storage API v1.0.0 section 5.1 General Definitions

Link:

https://armkeil.blob.core.windows.net/developer/Files/pdf/PlatformSecurityArchitecture/Implement/IHI0087-PSA_Storage_API-1.0.0.pdf

3.2.3.1.3 typedef `psa_storage_create_flags_t`

type `psa_storage_create_flags_t`

Storage create flags

3.2.3.1.3.1 Description

Flags used when creating a data entry.

3.2.3.1.3.2 Values

- **PSA_STORAGE_FLAG_NONE:**
No flags to pass.
- **PSA_STORAGE_FLAG_WRITE_ONCE:**
The data associated with the uid will not be able to be modified or deleted. Intended to be used to set bits in `typedef psa_storage_create_flags_t`.
- **PSA_STORAGE_FLAG_NO_CONFIDENTIALITY:**
The data associated with the uid is public and therefore does not require confidentiality. It therefore only needs to be integrity protected.
- **PSA_STORAGE_FLAG_NO_REPLAY_PROTECTION:**
The data associated with the uid does not require replay protection. This may permit faster storage - but it permits an attacker with physical access to revert to an earlier version of the data.

3.2.3.1.4 typedef `psa_storage_uid_t`

type `psa_storage_uid_t`

Storage uid

3.2.3.1.4.1 Description

A type for uid used for identifying data.

3.2.3.1.5 struct psa_storage_info_t

struct **psa_storage_info_t**

Storage info

3.2.3.1.5.1 Definition

```
struct psa_storage_info_t {  
    size_t capacity;  
    size_t size;  
    psa_storage_create_flags_t flags;  
}
```

3.2.3.1.5.2 Members

capacity

The allocated capacity of the storage associated with a uid.

size

The size of the data associated with a uid.

flags

The flags set when the uid was created.

3.2.3.2 Internal trusted storage APIs

3.2.3.2.1 Introduction

The PSA Internal Trusted Storage API (PITS API) is a more specialized API. Uses of this API will be less common. It is intended to be used for assets that must be placed inside internal flash. Some examples of assets that require this are replay protection values for external storage and keys for use by components of the PSA Root of Trust.

3.2.3.2.2 Reference

Documentation:

PSA Storage API v1.0.0 section 5.2 Internal Trusted Storage API

Link:

https://armkeil.blob.core.windows.net/developer/Files/pdf/PlatformSecurityArchitecture/Implement/IHI0087-PSA_Storage_API-1.0.0.pdf

3.2.3.2.3 PSA_ITS_API_VERSION_MAJOR

The major version number of the PSA ITS API.

It will be incremented on significant updates that may include breaking changes.

3.2.3.2.4 PSA_ITS_API_VERSION_MINOR

The minor version number of the PSA ITS API.

It will be incremented in small updates that are unlikely to include breaking changes.

3.2.3.2.5 **psa_its_set**

psa_status_t **psa_its_set**(*psa_storage_uid_t* uid, size_t data_length, const void *p_data,
psa_storage_create_flags_t create_flags)

Create a new, or modify an existing, uid/value pair.

Parameters

- **uid** (*psa_storage_uid_t*) – The identifier for the data.
- **data_length** (size_t) – The size in bytes of the data in p_data.
- **p_data** (const void*) – A buffer containing the data.
- **create_flags** (*psa_storage_create_flags_t*) – The flags that the data will be stored with.

3.2.3.2.5.1 Description

Stores data in the internal storage.

3.2.3.2.5.2 Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_NOT_PERMITTED:**
The operation failed because the provided uid value was already created with PSA_STORAGE_FLAG_WRITE_ONCE.
- **PSA_ERROR_NOT_SUPPORTED:**
The operation failed because one or more of the flags provided in create_flags is not supported or is not valid.
- **PSA_ERROR_INSUFFICIENT_STORAGE:**
The operation failed because there was insufficient space on the storage medium.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_INVALID_ARGUMENT:**
The operation failed because one of the provided pointers (e.g. p_data) is invalid, for example is NULL or references memory the caller cannot access.

3.2.3.2.6 **psa_its_get**

psa_status_t **psa_its_get**(*psa_storage_uid_t* uid, size_t data_offset, size_t data_size, void *p_data,
size_t *p_data_length)

Retrieve data associated with a provided UID.

Parameters

- **uid** (*psa_storage_uid_t*) – The uid value.
- **data_offset** (size_t) – The starting offset of the data requested.
- **data_size** (size_t) – The amount of data requested.
- **p_data** (void*) – On success, the buffer where the data will be placed.

- **p_data_length** (size_t*) – On success, this will contain size of the data placed in p_data.

3.2.3.2.6.1 Description

Retrieves up to data_size bytes of the data associated with uid, starting at data_offset bytes from the beginning of the data. Upon successful completion, the data will be placed in the p_data buffer, which must be at least data_size bytes in size. The length of the data returned will be in p_data_length. If data_size is 0, the contents of p_data_length will be set to zero.

3.2.3.2.6.2 Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_DOES_NOT_EXIST:**
The operation failed because the provided uid value was not found in the storage.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_INVALID_ARGUMENT:**
The operation failed because one of the provided arguments (e.g. p_data, p_data_length) is invalid, for example is NULL or references memory the caller cannot access. In addition, this can also happen if data_offset is larger than the size of the data associated with uid.

3.2.3.2.7 psa_its_get_info

psa_status_t **psa_its_get_info**(*psa_storage_uid_t* uid, struct *psa_storage_info_t* *p_info)

Retrieve the metadata about the provided uid.

Parameters

- **uid** (*psa_storage_uid_t*) – The uid value.
- **p_info** (struct *psa_storage_info_t**) – A pointer to the *struct psa_storage_info_t* that will be populated with the metadata.

3.2.3.2.7.1 Description

Retrieves the metadata stored for a given uid as a *struct psa_storage_info_t*.

3.2.3.2.7.2 Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_DOES_NOT_EXIST:**
The operation failed because the provided uid value was not found in the storage.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_INVALID_ARGUMENT:**
The operation failed because one of the provided pointers (e.g. p_info) is invalid, for example is NULL or references memory the caller cannot access.

3.2.3.2.8 psa_its_remove

psa_status_t **psa_its_remove**(*psa_storage_uid_t* uid)

Remove the provided key and its associated data from the storage.

Parameters

- **uid** (*psa_storage_uid_t*) – The uid value.

3.2.3.2.8.1 Description

Deletes the data from internal storage.

3.2.3.2.8.2 Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_INVALID_ARGUMENT:**
The operation failed because one or more of the given arguments were invalid (null pointer, wrong flags and so on).
- **PSA_ERROR_DOES_NOT_EXIST:**
The operation failed because the provided key value was not found in the storage.
- **PSA_ERROR_NOT_PERMITTED:**
The operation failed because the provided key value was created with `PSA_STORAGE_FLAG_WRITE_ONCE`.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).

3.2.3.3 Protected storage APIs

3.2.3.3.1 Introduction

The PSA Protected Storage API (PS API) is the general-purpose API that most developers should use. It is intended to be used to protect storage media that are external to the MCU package.

3.2.3.3.2 Reference

Documentation:

PSA Storage API v1.0.0 section 5.3 Protected Storage API

Link:

https://armkeil.blob.core.windows.net/developer/Files/pdf/PlatformSecurityArchitecture/Implement/IHI0087-PSA_Storage_API-1.0.0.pdf

3.2.3.3.3 PSA_PS_API_VERSION_MAJOR

The major version number of the PSA PS API.

It will be incremented on significant updates that may include breaking changes.

3.2.3.3.4 PSA_PS_API_VERSION_MINOR

The minor version number of the PSA PS API.

It will be incremented in small updates that are unlikely to include breaking changes.

3.2.3.3.5 psa_ps_set

psa_status_t **psa_ps_set**(*psa_storage_uid_t* uid, size_t data_length, const void *p_data,
psa_storage_create_flags_t create_flags)

Create a new or modify an existing key/value pair.

Parameters

- **uid** (*psa_storage_uid_t*) – The identifier for the data.
- **data_length** (size_t) – The size in bytes of the data in p_data.
- **p_data** (const void*) – A buffer containing the data.
- **create_flags** (*psa_storage_create_flags_t*) – The flags indicating the properties of the data.

3.2.3.3.5.1 Description

Warning: Not supported

The newly created asset has a capacity and size that are equal to data_length.

3.2.3.3.5.2 Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_NOT_PERMITTED:**
The operation failed because the provided uid value was already created with PSA_STORAGE_FLAG_WRITE_ONCE.
- **PSA_ERROR_INVALID_ARGUMENT:**
The operation failed because one or more of the given arguments were invalid.
- **PSA_ERROR_NOT_SUPPORTED:**
The operation failed because one or more of the flags provided in create_flags is not supported or is not valid.
- **PSA_ERROR_INSUFFICIENT_STORAGE:**
The operation failed because there was insufficient space on the storage medium.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_GENERIC_ERROR:**
The operation failed because of an unspecified internal failure.

3.2.3.3.6 psa_ps_get

psa_status_t **psa_ps_get**(*psa_storage_uid_t* uid, size_t data_offset, size_t data_size, void *p_data, size_t *p_data_length)

Retrieve data associated with a provided uid.

Parameters

- **uid** (*psa_storage_uid_t*) – The uid value.
- **data_offset** (size_t) – The starting offset of the data requested.
- **data_size** (size_t) – The amount of data requested.
- **p_data** (void*) – On success, the buffer where the data will be placed.
- **p_data_length** (size_t*) – On success, will contain size of the data placed in p_data.

3.2.3.3.6.1 Description

Warning: Not supported

Retrieves up to data_size bytes of the data associated with uid, starting at data_offset bytes from the beginning of the data. Upon successful completion, the data will be placed in the p_data buffer, which must be at least data_size bytes in size. The length of the data returned will be in p_data_length. If data_size is 0, the contents of p_data_length will be set to zero.

3.2.3.3.6.2 Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_INVALID_ARGUMENT:**
The operation failed because one of the provided arguments (e.g. p_data, p_data_length) is invalid, for example is NULL or references memory the caller cannot access. In addition, this can also happen if data_offset is larger than the size of the data associated with uid.
- **PSA_ERROR_DOES_NOT_EXIST:**
The operation failed because the provided uid value was not found in the storage.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_GENERIC_ERROR:**
The operation failed because of an unspecified internal failure.
- **PSA_ERROR_DATA_CORRUPT:**
The operation failed because of an authentication failure when attempting to get the key.
- **PSA_ERROR_INVALID_SIGNATURE:**
The operation failed because the data associated with the uid failed authentication.

3.2.3.3.7 psa_ps_get_info

psa_status_t **psa_ps_get_info**(*psa_storage_uid_t* uid, struct *psa_storage_info_t* *p_info)

Retrieve the metadata about the provided uid.

Parameters

- **uid** (*psa_storage_uid_t*) – The identifier for the data.
- **p_info** (struct *psa_storage_info_t**) – A pointer to the *psa_storage_info_t* struct that will be populated with the metadata.

3.2.3.3.7.1 Description

Warning: Not supported

Retrieves the metadata stored for a given uid as a *struct psa_storage_info_t*.

3.2.3.3.7.2 Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_INVALID_ARGUMENT:**
The operation failed because one or more of the given arguments were invalid (null pointer, wrong flags and so on).
- **PSA_ERROR_DOES_NOT_EXIST:**
The operation failed because the provided uid value was not found in the storage.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_GENERIC_ERROR:**
The operation failed because of an unspecified internal failure.
- **PSA_ERROR_DATA_CORRUPT:**
The operation failed because of an authentication failure when attempting to get the key.
- **PSA_ERROR_INVALID_SIGNATURE:**
The operation failed because the data associated with the uid failed authentication.

3.2.3.3.8 psa_ps_remove

psa_status_t **psa_ps_remove**(*psa_storage_uid_t* uid)

Remove the provided uid and its associated data from the storage.

Parameters

- **uid** (*psa_storage_uid_t*) – The identifier for the data to be removed.

3.2.3.3.8.1 Description

Warning: Not supported

Removes previously stored data and any associated metadata, including rollback protection data.

3.2.3.3.8.2 Return

- **PSA_SUCCESS:**
The operation completed successfully.
- **PSA_ERROR_INVALID_ARGUMENT:**
The operation failed because one or more of the given arguments were invalid (null pointer, wrong flags and so on).
- **PSA_ERROR_DOES_NOT_EXIST:**
The operation failed because the provided uid value was not found in the storage.
- **PSA_ERROR_NOT_PERMITTED:**
The operation failed because the provided uid value was created with PSA_STORAGE_FLAG_WRITE_ONCE.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_GENERIC_ERROR:**
The operation failed because of an unspecified internal failure.

3.2.3.3.9 psa_ps_create

psa_status_t **psa_ps_create**(*psa_storage_uid_t* uid, size_t capacity, *psa_storage_create_flags_t* create_flags)

Create an asset based on parameters.

Parameters

- **uid** (*psa_storage_uid_t*) – A unique identifier for the asset.
- **capacity** (size_t) – The allocated capacity, in bytes, of the uid.
- **create_flags** (*psa_storage_create_flags_t*) – Flags indicating properties of the storage.

3.2.3.3.9.1 Description

Warning: Not supported

Reserves storage for the specified uid. Upon success, the capacity of the storage is capacity, and the size is 0. It is only necessary to call this function for assets that will be written with the *psa_ps_set_extended()* function. If only *psa_ps_set()* is needed, calls to this function are redundant.

This function cannot be used to replace an existing asset, and attempting to do so will return PSA_ERROR_ALREADY_EXISTS.

If the PSA_STORAGE_FLAG_WRITE_ONCE flag is passed, *psa_ps_create()* will return PSA_ERROR_NOT_SUPPORTED.

This function is supported only if the *psa_ps_get_support()* returns PSA_STORAGE_SUPPORT_SET_EXTENDED.

3.2.3.3.9.2 Return

- **PSA_SUCCESS:**
The storage was successfully reserved.
- **PSA_ERROR_STORAGE_FAILURE:**
The operation failed because the physical storage has failed (Fatal error).
- **PSA_ERROR_INSUFFICIENT_STORAGE:**
capacity is bigger than the current available space.
- **PSA_ERROR_NOT_SUPPORTED:**
The function is not implemented or one or more create_flags are not supported.
- **PSA_ERROR_INVALID_ARGUMENT:**
uid was 0 or create_flags specified flags that are not defined in the API.
- **PSA_ERROR_GENERIC_ERROR:**
The operation has failed due to an unspecified error.
- **PSA_ERROR_ALREADY_EXISTS:**
Storage for the specified uid already exists.

3.2.3.3.10 psa_ps_set_extended

psa_status_t **psa_ps_set_extended**(*psa_storage_uid_t* uid, size_t data_offset, size_t data_length, const void *p_data)

Set partial data into an asset based on parameters

Parameters

- **uid** (*psa_storage_uid_t*) – The unique identifier for the asset.
- **data_offset** (size_t) – Offset within the asset to start the write.
- **data_length** (size_t) – The size in bytes of the data in p_data to write.
- **p_data** (const void*) – Pointer to a buffer which contains the data to write.

3.2.3.3.10.1 Description

Warning: Not supported

Sets partial data into an asset based on the given uid, data_offset, data_length and p_data.

Before calling this function, the storage must have been reserved with a call to *psa_ps_create()*. It can also be used to overwrite data in an asset that was created with a call to *psa_ps_set()*.

Calling this function with data_length = 0 is permitted. This makes no change to the stored data.

This function can overwrite existing data and/or extend it up to the capacity for the uid specified in *psa_ps_create*, but cannot create gaps. That is, it has preconditions:

- data_offset <= size
- data_offset + data_length <= capacity

and postconditions:

- size = max(size, data_offset + data_length)

- capacity unchanged.

This function is supported only if the `psa_ps_get_support()` returns `PSA_STORAGE_SUPPORT_SET_EXTENDED`.

3.2.3.3.10.2 Return

- **PSA_SUCCESS:**
The asset exists, the input parameters are correct and the data is correctly written in the physical storage.
- **PSA_ERROR_STORAGE_FAILURE:**
The data was not written correctly in the physical storage.
- **PSA_ERROR_INVALID_ARGUMENT:**
The operation failed because one or more of the preconditions listed above regarding `data_offset`, `size`, or `data_length` was violated.
- **PSA_ERROR_DOES_NOT_EXIST:**
The specified `uid` was not found.
- **PSA_ERROR_NOT_SUPPORTED:**
The implementation of the API does not support this function.
- **PSA_ERROR_GENERIC_ERROR:**
The operation failed due to an unspecified error.
- **PSA_ERROR_DATA_CORRUPT:**
The operation failed because the existing data has been corrupted.
- **PSA_ERROR_INVALID_SIGNATURE:**
The operation failed because the existing data failed authentication (MAC check failed).
- **PSA_ERROR_NOT_PERMITTED:**
The operation failed because it was attempted on an asset which was written with the flag `PSA_STORAGE_FLAG_WRITE_ONCE`.

3.2.3.3.11 `psa_ps_get_support`

`uint32_t psa_ps_get_support(void)`

Get implemented optional features

Parameters

- **void** – no arguments

3.2.3.3.11.1 Description

Warning: Not supported

Returns a bitmask with flags set for all of the optional features supported by the implementation.

Currently defined flags are limited to:

- `PSA_STORAGE_SUPPORT_SET_EXTENDED`

3.2.3.3.11.2 Return

uint32_t

3.2.4 Return codes

3.2.4.1 Introduction

This file defines the error codes returned by the PSA Cryptography API

3.2.4.2 Reference

Documentation:

PSA Cryptography API v1.2.1

Link:

<https://arm-software.github.io/psa-api/crypto/1.2/about>

3.2.4.3 typedef psa_status_t

type **psa_status_t**

Function return status.

3.2.4.3.1 Description

This is either `PSA_SUCCESS`, which is zero, indicating success; or a small negative value indicating that an error occurred. Errors are encoded as one of the `PSA_ERROR_XXX` values defined here.

3.2.4.3.2 Values

- **PSA_SUCCESS:**
The action was completed successfully.
- **PSA_ERROR_ALREADY_EXISTS:**
Asking for an item that already exists.

It is recommended that implementations return this error code when attempting to write to a location where a key is already present.
- **PSA_ERROR_BAD_STATE:**
The requested action cannot be performed in the current state.

Multi-part operations return this error when one of the functions is called out of sequence. Refer to the function descriptions for permitted sequencing of functions.

Implementations must not return this error code to indicate that a key identifier is invalid, but must return `PSA_ERROR_INVALID_HANDLE` instead.
- **PSA_ERROR_BUFFER_TOO_SMALL:**
An output buffer is too small.

Applications can call the `PSA_XXX_SIZE` macro listed in the function description to determine a sufficient buffer size.

It is recommended that implementations only return this error code in cases when performing the operation with a larger output buffer would succeed. However, implementations can also return this error if a function has invalid or unsupported parameters in addition to an insufficient output buffer size.

- **PSA_ERROR_COMMUNICATION_FAILURE:**

There was a communication failure inside the implementation.

This can indicate a communication failure between the application and an external cryptoprocessor or between the cryptoprocessor and an external volatile or persistent memory. A communication failure can be transient or permanent depending on the cause.

Warning:

If a function returns this error, it is undetermined whether the requested action has completed. Returning `PSA_SUCCESS` is recommended on successful completion whenever possible, however functions can return `PSA_ERROR_COMMUNICATION_FAILURE` if the requested action was completed successfully in an external cryptoprocessor but there was a breakdown of communication before the cryptoprocessor could report the status to the application.

- **PSA_ERROR_CORRUPTION_DETECTED:**

A tampering attempt was detected.

If an application receives this error code, there is no guarantee that previously accessed or computed data was correct and remains confidential. In this situation, it is recommended that applications perform no further security functions and enter a safe failure state.

Implementations can return this error code if they detect an invalid state that cannot happen during normal operation and that indicates that the implementation's security guarantees no longer hold. Depending on the implementation architecture and on its security and safety goals, the implementation might forcibly terminate the application.

This error code is intended as a last resort when a security breach is detected and it is unsure whether the keystore data is still protected. Implementations must only return this error code to report an alarm from a tampering detector, to indicate that the confidentiality of stored data can no longer be guaranteed, or to indicate that the integrity of previously returned data is now considered compromised. Implementations must not use this error code to indicate a hardware failure that merely makes it impossible to perform the requested operation, instead use `PSA_ERROR_COMMUNICATION_FAILURE`, `PSA_ERROR_STORAGE_FAILURE`, `PSA_ERROR_HARDWARE_FAILURE`, `PSA_ERROR_INSUFFICIENT_ENTROPY` or other applicable error code.

This error indicates an attack against the application. Implementations must not return this error code as a consequence of the behavior of the application itself.

- **PSA_ERROR_DATA_CORRUPT:**

Stored data has been corrupted.

This error indicates that some persistent storage has suffered corruption. It does not indicate the following situations, which have specific error codes:

- A corruption of volatile memory - use `PSA_ERROR_CORRUPTION_DETECTED`.
- A communication error between the cryptoprocessor and its external storage - use `PSA_ERROR_COMMUNICATION_FAILURE`.
- When the storage is in a valid state but is full - use `PSA_ERROR_INSUFFICIENT_STORAGE`.
- When the storage fails for other reasons - use `PSA_ERROR_STORAGE_FAILURE`.
- When the stored data is not valid - use `PSA_ERROR_DATA_INVALID`.

Note that a storage corruption does not indicate that any data that was previously read is invalid. However this previously read data might no longer be readable from storage.

When a storage failure occurs, it is no longer possible to ensure the global integrity of the keystore. Depending on the global integrity guarantees offered by the implementation, access to other data might fail even if the data is still readable but its integrity cannot be guaranteed.

It is recommended to only use this error code to report when a storage component indicates that the stored data is corrupt, or fails an integrity check. For example, in situations that the PSA Storage API [PSA-ITS] reports `PSA_ERROR_DATA_CORRUPT` or `PSA_ERROR_INVALID_SIGNATURE`.

- **PSA_ERROR_DATA_INVALID:**

Data read from storage is not valid for the implementation.

This error indicates that some data read from storage does not have a valid format. It does not indicate the following situations, which have specific error codes:

- When the storage or stored data is corrupted - use `PSA_ERROR_DATA_CORRUPT`.
- When the storage fails for other reasons - use `PSA_ERROR_STORAGE_FAILURE`.
- An invalid argument to the API - use `PSA_ERROR_INVALID_ARGUMENT`.

This error is typically a result of an integration failure, where the implementation reading the data is not compatible with the implementation that stored the data.

It is recommended to only use this error code to report when data that is successfully read from storage is invalid.

- **PSA_ERROR_DOES_NOT_EXIST:**

Asking for an item that doesn't exist.

Implementations must not return this error code to indicate that a key identifier is invalid, but must return `PSA_ERROR_INVALID_HANDLE` instead.

- **PSA_ERROR_GENERIC_ERROR:**

An error occurred that does not correspond to any defined failure cause.

Implementations can use this error code if none of the other standard error codes are applicable.

- **PSA_ERROR_HARDWARE_FAILURE:**

A hardware failure was detected.

A hardware failure can be transient or permanent depending on the cause.

- **PSA_ERROR_INSUFFICIENT_DATA:**

Return this error when there's insufficient data when attempting to read from a resource.

- **PSA_ERROR_INSUFFICIENT_ENTROPY:**

There is not enough entropy to generate random data needed for the requested action.

This error indicates a failure of a hardware random generator. Application writers must note that this error can be returned not only by functions whose purpose is to generate random data, such as key, IV or nonce generation, but also by functions that execute an algorithm with a randomized result, as well as functions that use randomization of intermediate computations as a countermeasure to certain attacks.

It is recommended that implementations do not return this error after `psa_crypto_init()` has succeeded. This can be achieved if the implementation generates sufficient entropy during

initialization and subsequently a cryptographically secure pseudorandom generator (PRNG) is used. However, implementations might return this error at any time, for example, if a policy requires the PRNG to be reseeded during normal operation.

- **PSA_ERROR_INSUFFICIENT_MEMORY:**

There is not enough runtime memory.

If the action is carried out across multiple security realms, this error can refer to available memory in any of the security realms.

- **PSA_ERROR_INSUFFICIENT_STORAGE:**

There is not enough persistent storage.

Functions that modify the key storage return this error code if there is insufficient storage space on the host media. In addition, many functions that do not otherwise access storage might return this error code if the implementation requires a mandatory log entry for the requested action and the log storage space is full.

- **PSA_ERROR_INVALID_ARGUMENT:**

The parameters passed to the function are invalid.

Implementations can return this error any time a parameter or combination of parameters are recognized as invalid.

Implementations must not return this error code to indicate that a key identifier is invalid, but must return `PSA_ERROR_INVALID_HANDLE` instead.

- **PSA_ERROR_INVALID_HANDLE:**

The key identifier is not valid.

- **PSA_ERROR_INVALID_PADDING:**

The decrypted padding is incorrect.

Warning:

In some protocols, when decrypting data, it is essential that the behavior of the application does not depend on whether the padding is correct, down to precise timing. Protocols that use authenticated encryption are recommended for use by applications, rather than plain encryption. If the application must perform a decryption of unauthenticated data, the application writer must take care not to reveal whether the padding is invalid.

Implementations must handle padding carefully, aiming to make it impossible for an external observer to distinguish between valid and invalid padding. In particular, it is recommended that the timing of a decryption operation does not depend on the validity of the padding.

- **PSA_ERROR_INVALID_SIGNATURE:**

The signature, MAC or hash is incorrect.

Verification functions return this error if the verification calculations completed successfully, and the value to be verified was determined to be incorrect.

If the value to verify has an invalid size, implementations can return either `PSA_ERROR_INVALID_ARGUMENT` or `PSA_ERROR_INVALID_SIGNATURE`.

- **PSA_ERROR_NOT_PERMITTED:**

The requested action is denied by a policy.

It is recommended that implementations return this error code when the parameters are recognized as valid and supported, and a policy explicitly denies the requested operation.

If a subset of the parameters of a function call identify a forbidden operation, and another subset of the parameters are not valid or not supported, it is unspecified whether the function returns `PSA_ERROR_NOT_PERMITTED`, `PSA_ERROR_NOT_SUPPORTED` or `PSA_ERROR_INVALID_ARGUMENT`.

- **PSA_ERROR_NOT_SUPPORTED:**

The requested operation or a parameter is not supported by this implementation.

It is recommended that implementations return this error code when an enumeration parameter such as a key type, algorithm, etc. is not recognized. If a combination of parameters is recognized and identified as not valid, return `PSA_ERROR_INVALID_ARGUMENT` instead.

- **PSA_ERROR_STORAGE_FAILURE:**

There was a storage failure that might have led to data loss.

This error indicates that some persistent storage could not be read or written by the implementation. It does not indicate the following situations, which have specific error codes:

- A corruption of volatile memory - use `PSA_ERROR_CORRUPTION_DETECTED`.
- A communication error between the cryptoprocessor and its external storage - use `PSA_ERROR_COMMUNICATION_FAILURE`.
- When the storage is in a valid state but is full - use `PSA_ERROR_INSUFFICIENT_STORAGE`.
- When the storage or stored data is corrupted - use `PSA_ERROR_DATA_CORRUPT`.
- When the stored data is not valid - use `PSA_ERROR_DATA_INVALID`.

A storage failure does not indicate that any data that was previously read is invalid. However this previously read data might no longer be readable from storage.

When a storage failure occurs, it is no longer possible to ensure the global integrity of the keystore. Depending on the global integrity guarantees offered by the implementation, access to other data might fail even if the data is still readable but its integrity cannot be guaranteed.

It is recommended to only use this error code to report a permanent storage corruption. However application writers must keep in mind that transient errors while reading the storage might be reported using this error code.

SUBSYSTEMS CAPABILITIES

4.1 ELE capabilities

4.1.1 Key manager

Table 4.1: ELE Key type

Key type	Key security size(s)	Devices			
		8ULP	91/93	943	95
AES	128 / 192 / 256	X	X	X	X
Secp R1	224 / 256 / 384 / 521	X	X	X	X
Brainpool R1	224 / 256 / 384	X	X	X	X
HMAC	224 / 256 / 384 / 512	X	X	X	X
RSA	2048 / 3072 / 4096		X	X	X
ED25519	255		X		X ¹
ED448	448		X		X ¹
X25519	255		X		X
X448	448		X		X
EL2GO_PROV_OEM_KEY	N/A		X		X

Note

The *EL2GO_PROV_OEM_KEY* key type is limited to the EdgeLock 2GO key import of the OEM Shared secret.

Operations supported:

- Generate
- *Key Import*
- Export (only public key in HEX or Base64 format)

¹ i.MX95 Pre-Hashed signature is not supported.

- Delete
- *Key Derivation*
- Get key attributes
- Get key buffers' length
- Get key security size
- Get key type name
- Commit key storage
- *Key attestation*

Key group:

The SMW Library is managing the ELE key group automatically. The library is selecting a key group depending if a key is persistent/permanent or transient.

- Persistent/Permanent keys are in key groups from 0 to 49.
- Transient keys are in key groups from 50 to 99.

4.1.1.1 Key policy

When creating a new key, the key policy must be specified through the operation key attributes.

The following [Table 4.2](#) lists all key usages applicable in ELE subsystem. A key policy defines one or more key usage.

Table 4.2: ELE Key usages

USAGE	Description
ENCRYPT	Permission to encrypt a message
DECRYPT	Permission to decrypt a message
SIGN_MESSAGE	Permission to sign a message
SIGN_HASH	Permission to sign a message hashed
VERIFY_MESSAGE	Permission to verify the signature of a message
VERIFY_HASH	Permission to verify the signature of message hashed
DERIVE	Permission to derive other keys from this key

The following [Table 4.3](#) lists all permitted algorithms applicable in ELE subsystem. Only one permitted algorithm is allowed per key.

Table 4.3: ELE Key permitted algorithm

ALGO	HASH	MIN_LENGTH LENGTH	Comment
HMAC	SHA256	From 8 to 32 bytes	If not specified length is 32 bytes
	SHA384	From 8 to 48 bytes	If not specified length is 48 bytes
CBC_NO_PADDING	N/A	N/A	
CFB	N/A	N/A	
CTR	N/A	N/A	
ECB_NO_PADDING	N/A	N/A	
OFB	N/A	N/A	Not supported on i.MX8ULP
ALL_CIPHER	N/A	N/A	Support all ciphers including CMAC
CCM	N/A	N/A	
ALL_AEAD	N/A	N/A	Support all AEAD
RSA PKCS1V15 Asymmetric Signature	N/A	N/A	Support all hash
	SHA1	N/A	
	SHA224	N/A	
	SHA256	N/A	
	SHA384	N/A	
	SHA512	N/A	
RSA PSS Asymmetric Signature	N/A	N/A	Support all hash
	SHA1	N/A	
	SHA224	N/A	
	SHA256	N/A	
	SHA384	N/A	
	SHA512	N/A	
RSA PKCS1V15 Asymmetric Encryption	N/A	N/A	
RSA OAEP Asymmetric Encryption	N/A	N/A	Support all hash
	SHA1	N/A	
	SHA224	N/A	
	SHA256	N/A	
	SHA384	N/A	
	SHA512	N/A	

continues on next page

Table 4.3 – continued from previous page

ALGO	HASH	MIN_LENGTH LENGTH	Comment
RSA PKCS1 ANY Asymmetric Encryption	N/A	N/A	Support any RSA Asymmetric Encryption
ECDSA	SHA224	N/A	
	SHA256	N/A	
	SHA384	N/A	
	SHA512	N/A	
CMAC	N/A	N/A	
ATTEST_CMAC	N/A	N/A	Attestation restricted key
ATTEST_ECDSA	SHA224	N/A	Attestation restricted key
	SHA256	N/A	
	SHA384	N/A	
	SHA512	N/A	
ALL EDDSA	ANY	N/A	
PURE EDDSA	N/A	N/A	
ED25519PH	N/A	N/A	Not supported on i.MX95
ED448PH	N/A	N/A	Not supported on i.MX95
TLS1_3_MASTER_SECRET	SHA256	N/A	Derivation restricted key
	SHA384	N/A	
	N/A	N/A	Support all hash
ECDH HKDF	SHA256	N/A	Key agreement
	SHA384	N/A	
	ANY	N/A	
CKDF	N/A	N/A	Custom Key Derivation Function. Allow importing EL2GO OEM Secret Key

4.1.2 Hash

Table 4.4: ELE Hash

Hash Algorithm	Devices	
	8ULP	9x
MD5		X
SHA1		X
SHA224	X	X
SHA256	X	X
SHA384	X	X
SHA512	X	X
SHA3_224		X
SHA3_256		X
SHA3_384		X
SHA3_512		X
SHAKE256		X

Operations supported:

- One shot and multi-part

Note

The SHAKE256 is an extendable-output function (XOF) where digest length can be any length as detailed in the *FIPS 202* <<https://doi.org/10.6028/NIST.FIPS.202>>.

Note

In case of multi-part operation, the ELE Secure Enclave doesn't support additional input data during the final operation.

4.1.3 Signature

Table 4.5: ELE Signature

Signature Type	Key type	Key security size(s)	Hash algorithm
ECDSA	Secp R1	224 / 256 / 384 / 521	• SHA224 • SHA256 • SHA384 • SHA512
	Brainpool R1	224 / 256 / 384	• SHA224 • SHA256 • SHA384
RSA_PKCS1V15	RSA	2048 / 3072 / 4096	• SHA224 • SHA256 • SHA384 • SHA512
RSA_PSS	RSA	2048 / 3072 / 4096	• SHA224 • SHA256 • SHA384 • SHA512
PURE_EDDSA	ED25519	255	None (Message)
	ED448	448	None (Message)
ED25519PH	ED25519	255	None (pre-hashed)
ED448PH	ED448	448	None (pre-hashed)

Operations supported:

- Sign One-shot and Multi-part
- Verify One-shot and Multi-part

Note

Message to sign/verify is full or hashed depending on the algorithm 64-bits word definition additional parameters (bits[39:32]).

Caution

Signature Multi-part is supported only if the hash operation is enabled.

4.1.3.1 Sign operation

The following key policies must defined:

- Usage:
 - SIGN_MESSAGE to sign a message to be hashed
 - SIGN_HASH to sign a message already hashed
- Algorithm:
 - ECDSA Signature with hash or a message already hashed as listed in [Table 4.5](#)
 - RSA Signature, PKCS1 v1.5 and PSS with hash as listed in [Table 4.5](#). Not supported on i.MX8ULP
 - EDDSA Signature with hash or a message already hashed as listed in [Table 4.5](#). Not supported on i.MX8ULP, i.MX943 and i.MX95.
- Opaque key or a plaintext key buffer are supported. Signature generation using plaintext key buffer:
 - For an EC key pair (Secp R1 and Twisted edwards), private key buffer must be set.
 - For RSA key pair, private key and modulus buffers must be set. ELE subsystem only supports RSA key pairs with default public exponent of 0x010001.

4.1.3.2 Verify operation

The following key policies must defined if a key identifier is used:

- Usage:
 - VERIFY_MESSAGE to verify the signature of a message to be hashed
 - VERIFY_HASH to verify the signature of a message already hashed
- Algorithm:
 - ECDSA Signature with hash or a message already hashed as listed in [Table 4.5](#)
 - RSA PKCS1 v1.5 and PSS with hash as listed in [Table 4.5](#). Not supported on i.MX8ULP
 - EDDSA Signature with hash or a message already hashed as listed in [Table 4.5](#). Not supported on i.MX8ULP, i.MX943 and i.MX95.
- Opaque key or a plaintext key buffer are supported. Signature verification using plaintext key buffer:
 - For an EC key pair (Secp R1 and Twisted edwards), public key buffer must be set.
 - For RSA key, public exponent and modulus buffer must be set. ELE subsystem only supports RSA key pairs with default public exponent of 0x010001.

4.1.4 Random

Length: 1 to UINT32_MAX

4.1.5 MAC

Table 4.6: ELE MAC

MAC Type	Key type	Key security size(s)
CMAC	AES	128 / 192 / 256
HMAC	HMAC	224 / 256 / 384 / 512

The MAC size can be truncated if the key permitted algorithm limits the MAC output length.

Operations supported:

- Compute MAC
- Verify MAC

4.1.5.1 Compute MAC operation

MAC generation operation can compute either a full MAC length or a truncated MAC length. The operation algorithm and key permitted algorithm allows to select the MAC length to be generated.

ELE subsystem supports MAC operation using either a key ID or a plaintext key buffer.

Table 4.7: ELE MAC - Compute

MAC Length	Algorithm	Hash	Key policy
Full MAC	CMAC	N/A	Usage: SIGN_MESSAGE Algorithm: CMAC
	HMAC	SHA256 SHA384	Usage: SIGN_MESSAGE Algorithm: HMAC with HASH=[256/384]
Truncated MAC Minimum length	CMAC_TRUNCATED	N/A	Usage: SIGN_MESSAGE Algorithm: CMAC with MIN_LENGTH=[min]
	HMAC_TRUNCATED	SHA256 SHA384	Usage: SIGN_MESSAGE Algorithm: HMAC with HASH=[256/384] and MIN_LENGTH=[min]
Truncated MAC Fix length	CMAC_TRUNCATED	N/A	Usage: SIGN_MESSAGE Algorithm: CMAC with LENGTH=[length]
	HMAC_TRUNCATED	SHA256 SHA384	Usage: SIGN_MESSAGE Algorithm: HMAC with HASH=[256/384] and LENGTH=[min]

4.1.5.2 Verify MAC operation

MAC verification operation can verify either a full MAC length or a truncated MAC length. The operation algorithm and key permitted algorithm allows to select the MAC length to be generated.

ELE subsystem supports MAC operation using either a key ID or a plaintext key buffer.

Table 4.8: ELE MAC - Verify

MAC Length	Algorithm	Hash	Key policy
Full MAC	CMAC	N/A	Usage: VERIFY_MESSAGE Algorithm: CMAC
	HMAC	SHA256 SHA384	Usage: VERIFY_MESSAGE Algorithm: HMAC with HASH=[256/384]
Truncated MAC Minimum length	CMAC_TRUNCATED	N/A	Usage: VERIFY_MESSAGE Algorithm: CMAC with MIN_LENGTH=[min]
	HMAC_TRUNCATED	SHA256 SHA384	Usage: VERIFY_MESSAGE Algorithm: HMAC with HASH=[256/384] and MIN_LENGTH=[min]
Truncated MAC Fix length	CMAC_TRUNCATED	N/A	Usage: VERIFY_MESSAGE Algorithm: CMAC with LENGTH=[length]
	HMAC_TRUNCATED	SHA256 SHA384	Usage: VERIFY_MESSAGE Algorithm: HMAC with HASH=[256/384] and LENGTH=[min]

4.1.6 Cipher

Table 4.9: ELE Cipher

Cipher Mode	Key type	Key security size(s)
ECB No Padding	AES	128 / 192 / 256
CBC No Padding		
CTR		
CFB		
OFB		

One-shot operations supported:

- Encrypt
- Decrypt

4.1.6.1 Encrypt operation

The following key policies must defined:

- Usage: ENCRYPT
- Algorithm:
 - ECB_NO_PADDING
 - CBC_NO_PADDING
 - CFB
 - CTR
 - OFB
 - ALL_CIPHER (any cipher mode)

4.1.6.2 Decrypt operation

The following key policies must defined if a key identifier is used:

- Usage: DECRYPT
- Algorithm:
 - ECB_NO_PADDING
 - CBC_NO_PADDING
 - CFB
 - CTR
 - OFB
 - ALL_CIPHER (any cipher mode)

4.1.7 AEAD

Table 4.10: ELE AEAD

AEAD Mode	Key type	IV length (bytes)	Tag length (bytes)
CCM	AES	12	16
GCM ²		Encryption: • 0 (subsystem generates full IV) • 4 (subsystem generates 8 bytes of IV) • 12 (user supplied full IV)	16
		Decryption: • 12 (user supplied full IV)	
CHACHA20_POLY1305 ³		12	16

Note

- AEAD is not supported on i.MX943.
- The ELE subsystem supports AEAD encryption and decryption using either a key identifier or a plaintext key buffer.

One-shot operations supported:

- AEAD Encryption
- AEAD Decryption

4.1.8 Asymmetric encryption and decryption

Table 4.11: ELE Asymmetric encryption and decryption

Encryption mode	Key type	Key security size(s)	Hash algorithm
OAEP	RSA	2048 / 3072 / 4096	• SHA1 • SHA224 • SHA256 • SHA384 • SHA512
PKCS1V15	RSA	2048 / 3072 / 4096	N/A

² Not supported on i.MX8ULP

³ Supported on i.MX91, i.MX93 and i.MX95.

Operations supported:

- Encryption
- Decryption

Note

- Asymmetric encryption and decryption operations are not supported on i.MX8ULP and i.MX943.

4.1.9 Device management

The following operations are available:

- Device Attestation
- Device UUID (in big endian format)
- Device lifecycle

4.1.9.1 Device Attestation

The device attestation requires a challenge value to guaranty the certificate request. The challenge value maximum length depends of the device as listed in the following table.

Table 4.12: ELE Attestation Challenge

Device	Challenge Length in bytes
8ULP	4
9x	16

4.1.9.2 Device lifecycle

The device lifecycle operations supported are get and set device lifecycle. The following table lists the device lifecycle supported when executing a get or set device lifecycle.

Warning

Changing the device lifecycle (set operation) is not revertable. Refer to the device documentation to get more details about the lifecycle.

Table 4.13: ELE Device lifecycle

Lifecycle	Get	Set	Comment
OPEN	Yes	Yes	
CLOSED	Yes	Yes	A signed image is required to boot
CLOSED_LOCKED	Yes	Yes	A signed image is required to boot
OEM_RETURN	No	Yes	Device is no more OEM usable and must be returned to NXP
NXP_RETURN	No	Yes	Device is no more usable and must be returned to NXP

4.1.10 Operation context

Operations supported:

- Allocate
- Cancel

4.1.11 Data Storage manager

Data Storage manager allows to store and retrieve data. The data ID is a 32-bits value with the exception of the 0xF00000E0 reserved for EdgeLock 2GO claimcode.

The subsystem allows to:

- store, retrieve and delete user data.
- encrypt and sign data (Table 4.14) before storing it and retrieve a TLV blob (Table 4.15).
- set encrypted and signed data as READ_ONCE, meaning that when data is retrieved the subsystem deletes the data.

Note

- Data size is limited to 2048 bytes.
- Data size must be aligned on a cipher block in case of data encryption. in other word, user must pad to the data.
- Data lifecycle can be defined only when storing encrypted/signed data.

Table 4.14: ELE Data Encrypt/Sign

Encryption	IV	Signature
ECB No Padding	N/A	CMAC
CBC No Padding	Yes	
CFB	Yes	
CTR	Yes	

Table 4.15: ELE Data blob (encrypted and signed)

Tag	Length (bytes)	Value/Description
0x41	16	Device UUID in big endian format.
0x45	16	Value of the IV used to encrypt data in case encryption algorithm use an IV. The IV can be either given as input by the user or randomly generated by the subsystem (user must the IV buffer and its length to 0).
0x46	Variable	Encrypted data. Maximum length is 2048 bytes.

4.1.12 Key attestation

Key attestation only applies to the public key of asymmetric keys.

The subsystem allows to attest a public key of an asymmetric key present in the subsystem key storage.

The public key attestation results in a certificate encoded as a signed TLV buffer, in the table below ([Table 4.16](#)):

Table 4.16: Key attestation certificate

Tag	Length (bytes)	Value/Description
0x41	4	Identifier, in the subsystem, of the key to attest.
0x42	4	Key permitted algorithm.
0x43	4	Key usage flags.
0x44	2	Type of key.
0x45	4	Key security size in bits.
0x46	4	Lifetime.
0x50	Variable	Input value chosen by the user, could be a random or the current data/time.
0x51	4	Key identifier in the subsystem of the key used to attest the public key (sign this TLV).
0x52	Variable	Public key buffer.
0x53	1	Information about the key origin: • 0x01: Key has been generated by the device. • 0x02: Key has been imported into the device.
0x54	16	Device UUID.
0x55	4	Key lifecycle usage flags: • OPEN: 0x01. • CLOSED: 0x02. • CLOSED_LOCKED: 0x04.
0x56	4	Algorithm used to sign the blob itself. Field “Signature” of this blob. Possible values are: • 0x01: CMAC. • 0x02: ECDSA (hash of the payload must be the same as the signature key size).
0x5E	Variable	Signature of all previous fields of this blob including the signature tag (0x5E) and signature length fields.

4.1.13 Storage Re-provisioning

This operation allows to reset and re-fill a new non-volatile storage that has been rollback protected (using the commit operation). This operation requires a specific tool to sign the ELE Secure Enclave re-provisioning message. The signature key is correlated to the OEM SRKH fused.

The NXP SPSDK tool can be used to sign the message and can be installed from [SPSK releases](#). The documentation is available [here](#). The command used to sign is `nxpimage`.

The SMW library offers the possibility to create the message payload to be signed (see :ref:smw_device_reprovision_prepare). The :ref:smw_device_reprovision API will take as input the

resulting signed message.

Note

The OEM SRKH must be fused.

4.1.14 Key Derivation

4.1.14.1 TLS 1.2 (TLS1-PRF)

The subsystem supports generating the master secret, encryption/decryption keys and IVs, and verify data. ELE does not allow some operations on data with length less than the key size, so the key size of the base key used for ECDH(E) dictates which ciphersuites can be used. For example, if the key size is 384 bits, you may only use ciphersuites that use SHA384.

Only ECDH(E) key exchange is supported, with SECP_R1 key type, and the following ciphersuites:

Table 4.17: ELE supported ciphersuites

SMW encryption name	OpenSSL equivalent
SMW_TLS12_ENC_NAME_AES_128_CBC	ECDHE-ECDSA-AES128-SHA256
SMW_TLS12_ENC_NAME_AES_128_CCM	ECDHE-ECDSA-AES128-CCM
SMW_TLS12_ENC_NAME_AES_128_GCM	ECDHE-ECDSA-AES128-GCM-SHA256
SMW_TLS12_ENC_NAME_AES_256_CBC	ECDHE-ECDSA-AES256-SHA384
SMW_TLS12_ENC_NAME_AES_256_CCM	ECDHE-ECDSA-AES256-CCM
SMW_TLS12_ENC_NAME_AES_256_GCM	ECDHE-ECDSA-AES256-GCM-SHA384
SMW_TLS12_ENC_NAME_CHACHA20_POLY1305	ECDHE-ECDSA-CHACHA20-POLY1305

4.1.14.2 TLS 1.3 (TLS13-KDF)

The early secret, ECDH shared secret, handshake secret and master secret are computed internally and not exported. The SECP_R1, X25519 and X448 key types are supported. The subsystem supports derivation of the following TLS1.3 secrets:

Table 4.18: ELE TLS1.3 secrets

TLS1.3 secret name	Label without null termination
Binder key	“ext binder” or “res binder”
Client early traffic secret	“c e traffic”
Early exporter master secret	“e exp master”
Client handshake traffic secret	“c hs traffic”
Server handshake traffic secret	“s hs traffic”
Client application traffic secret 0	“c ap traffic”
Server application traffic secret 0	“s ap traffic”
Exporter master secret	“exp master”
Resumption master secret	“res master”

After the required secret is computed, the TLS1.3 API may be used to further derive other keys and IVs, as required by [RFC8446](#).

Any keys that are derived from these secrets need to have the proper attributes set before doing the derivation. For example, from “s hs traffic”, you may derive an AES-128-GCM key to decrypt data and an HMAC-256 key to compute the Finished data. In both cases, the key type, size, algorithm and usage need to be set for the derived key.

Note

Only supported on i.MX91, i.MX93 and i.MX95

4.1.14.3 OEM Master key

The OEM Master key is a key agreement between a secure host ECDSA key and a device ECDSA key. The resulting key is a symmetric key used to derive two keys:

- A wrap key to wrap secure key to import
- A sign key to sign EdgeLock Enclave blob including the wrapped key and its attributes.

The OEM Master key requests a EdgeLock Enclave signed message to authenticate the key attributes and derivation parameters.

The SMW library offers the possibility to pre-fill the message payload to be signed (see :ref:smw_kdf_oem_master_key_args).

The NXP SPSDK tool can be used to sign the message and can be installed from [SPSDK releases](#). The documentation is available [here](#). The command used to sign is `nxpimage`.

The OEM Master key attributes are almost pre-defined, the following [Table 4.19](#) lists the value to set in the signed payload and the SMW key attributes.

The signed payload defined also a `derived key group` field where the key will be stored. Value must be in the range [0:99] SMW defined 2 groups range function of the key persistency:

- 0:49 Groups of the persistent keys
- 50:99 Groups of the transient keys

It's impossible during runtime to predict the group number where the key could be stored if other keys are already present in the ELE NVM Secure Storage. If the key group can't be known in advance, it's advised to use the SMW operation pre-filling the payload content (see *struct smw_kdf_oem_master_key_args*) that will try to find a key group number where the key could be stored. Note, even that, it's not 100% guaranty that the key group will have enough place to store the key at the time the key derivation operation will be requested.

It's advised to store this key as persistent key with a known key identifier. In case of transient key the ELE FW will assign the key identifier after key derivation operation success.

Table 4.19: OEM Master key attributes

Key attributes	ELE value	SMW value
Key type	0x9200	SMW_KEY_TYPE_NAME_DERIVE
Security size (bits)	256	256
Usage	0x00004000	SMW_ATTR_USAGE_DERIVE
Permitted algorithm	0x0800109	SMW_ATTR_ALGO_KEY_DERIVATION_HKDF(SHA256)
Lifetime	0x00000000	SMW_ATTR_PERSISTENCE_TRANSIENT flag
	0x00000001	SMW_ATTR_PERSISTENCE_PERSISTENT flag
Lifecycle flags	0x00	SMW_ATTR_LIFECYCLE_CURRENT
	0x01	SMW_ATTR_LIFECYCLE_OPEN
	0x02	SMW_ATTR_LIFECYCLE_CLOSED
	0x03	SMW_ATTR_LIFECYCLE_CLOSED_LOCKED

Note

The OEM SRKH must be fused.

Note

Only supported on i.MX8ULP, i.MX91 and i.MX93

4.1.14.4 Key Import

ELE supports only key importation using trusted blob like EdgeLock 2GO blob and EdgeLock Enclave blob.

4.1.14.5 EdgeLock 2GO blob

This service is a NXP service involving the EdgeLock 2GO server. The server allows to provision key and data using blob created function of the device UUID and OEM SRKH fused in the device. Visit [EdgeLock 2GO](#)

4.1.14.6 EdgeLock Enclave blob

This service is a NXP service provided by the EdgeLock Enclave Firmware. A EdgeLock Enclave blob is used to import the secure key. The blob contains the key attributes, the wrapped key (AES CBC pad or) and a blob signature (CMAC).

The wrapping key and the signing key are both derived from the *OEM Master key*. On device both keys are derived and kept internal, on host side, the OEM Master key, the wrapping key and signing key must be derived using the HMAC two steps standard [RFC5869](#).

OEM import wrap and cmac keys derivation parameters:

- The salt used in the first step: extract can be either:
 - A zeros string. Its length, in bytes, is equal to the hash algorithm (used in the key derivation process) length byte size.
 - Device SRKH. As salt length is the hash algorithm (used in the key derivation process) length byte size, device SRKH value could be truncated or concatenated with 0s to match the required salt length.
- The IKM is the *OEM Master key* shared secret key
- The fixed info in a byte string imposed by EdgeLock Secure Enclave Firmware. It differs switch derived key:
 - For wrapping key, fixed info is: “oemelefwkeyimportwrap256” string (quotation marks not included).
 - For signing key, fixed info is: “oemelefwkeyimportcmac256” string (quotation marks not included).
- The key derivation algorithm is imposed by EdgeLock Secure Enclave Firmware as HKDF_SHA256.

The EdgeLock Enclave blob is encoded as described in [Table 4.20](#).

Table 4.20: EdgeLock Enclave blob

Field	TAG	Length (bytes)	Description / Value	
Magic	0x40	21	EdgeLock Secure Enclave identification blob. Value is the hexadecimal string “edgelockenclaveimport”. [65 64 67 65 6c 6f 63 6b 65 6e 63 6c 61 76 65 69 6d 70 6f 72 74]	
Key ID	0x41	4	Key identifier in the subsystem: • 0x0 if the key is transient. • between 0x1 and 0x3FFFFFFF if the key is persistent.	
Key properties	Algorithm	0x42	4	ELE key permitted algorithm.
	Usage	0x43	4	ELE key usage flags.
	Type	0x44	2	ELE Type of key.
	Bits	0x45	4	Key security size in bits.
	Lifetime	0x46	4	ELE Lifetime: • 0xC0020000, transient key • 0xC0020001, persistent key • 0xC00200FF, permanent key
Key lifecycle	0x47	4	ELE device lifecycle flags when key is usable: • OPEN: 0x01 • CLOSED: 0x02 • CLOSED_LOCKED: 0x04	
OEM Master key identifier	0x50	4	OEM Master key identifier resulting <i>OEM Master key</i> .	
Wrapping algorithm	0x51	4	Wrapping algorithm of the key blob. This field is required to distinguish between different flavors of wrapping algorithms. Possible values are: • 0x01: RFC 3394 wrapping. • 0x02: AES-CBC wrapping (padding cipher, ISO7816-4 Appendix C).	
IV	0x52	16	IV to use for CBC wrapping. Not used if wrapping algorithm not equal 0x02.	
Signing algorithm	0x54	4	Algorithm used to sign the blob itself. Field Signature of this blob. It must be 0x01 (CMAC).	
Wrapped private key	0x55	Variable	Private key data in encrypted format as defined by the Wrapping Algorithm. Key used to do the encryption must be wrapping key derived from the OEM Master key.	

4.1.14.6.1 Host example to derive OEM Master key

Here's an example of host source code C to derive the OEM Master key

In this example:

- [MbedTLS Library](#) is used as third party cryptographic service.
- ECDSA NIST 256 bits keypair are used on device and host side as based key of the OEM Master key agreement.
- ECDH HKDF SHA256 algorithm is used for the key agreement operation.

Note

The following code is not complete, failure must be handled correctly and resource freed. The following examples aim to give the guidelines to write application on host and device.

1. On device, generates and exports an ECDSA NIST 256 bits keypair.

The SMW API :ref:smw_generate_key is used to generate the key. The key is stored as a transient key. The public key is exported if the operation success. The public key is the device peer key that will be used to derive the OEM Master key on the host side. This key identifier will be identified in this example by ELE_base_id The public key of this key will be identified in this example by ELE_peer_key.

```
1 int res = -1;
2 enum smw_status_code status = SMW_STATUS_OK;
3 struct smw_generate_key_args args = { 0 };
4 struct smw_key_descriptor key = { 0 };
5 struct smw_key_attributes *attributes = &key.attributes;
6 struct smw_keypair_buffer buffer = { 0 };
7
8 /* Allocate the public key buffer of the key */
9 buffer.gen.public_length = 64;
10 buffer.gen.public_data = malloc(buffer.gen.public_length);
11 if (!buffer.gen.public_data)
12     return res;
13
14     args.subsystem_name = SMW_SUBSYSTEM_NAME_ELE;
15     args.key_descriptor = &key;
16
17 key.type_name = SMW_KEY_TYPE_NAME_SECP_R1;
18 key.security_size = 256;
19 key.buffer = &buffer;
20
21 attributes.permitted_algo = SMW_ATTR_ALGO_KEY_AGREEMENT(SMW_ATTR_
↵ALGO_ECDH,
22
23
↵ALGO_HKDF,
24
↵HASH_SHA256);
attributes.usage_flags = SMW_ATTR_USAGE_DERIVE;
```

(continues on next page)

(continued from previous page)

```
25 attributes.attributes = SMW_ATTR_SET_PERSISTENCE(attributes.  
    ↪ attributes,  
26                                     SMW_ATTR_  
    ↪ PERSISTENCE_TRANSIENT);  
27  
28 status = smw_generate_key(&args);  
29 if (status != SMW_STATUS_OK) {  
30     free(buffer.gen.public_data);  
31     return -1;  
32 }  
33  
34 /*  
35  * At this stage:  
36  * - ``ELE_peer_key`` is the buffer.gen.public_data  
37  * - ``ELE_base_id`` is the identifier returned in key.id  
38  */  
39  
40 ...
```

1. On host, generates and exports an ECDSA NIST 256 bits keypair

The public key is exported if the operation success. The public key is the host peer key that will be used to derive the OEM Master key on the device side.

In this example, the Host peer key is named Host_peer_key.

```
1 int res = -1;  
2 mbedtls_pk_context key;  
3 mbedtls_entropy_context entropy;  
4 mbedtls_ctr_drbg_context ctr_drbg;  
5 mbedtls_ecp_group grp;  
6 mbedtls_ecp_point Host_base_key;  
7 unsigned char Host_peer_key[MBEDTLS_ECP_MAX_PT_LEN];  
8 mbedtls_pk_type_t pk_alg = MBEDTLS_PK_ECKEY;  
9 size_t len = 0;  
10  
11 mbedtls_pk_init(&key);  
12 mbedtls_entropy_init(&entropy);  
13 mbedtls_ctr_drbg_init(&ctr_drbg);  
14  
15  
16 res = mbedtls_pk_setup(&key, mbedtls_pk_info_from_type(pk_alg));  
17 if (res)  
18     /* Failure */  
19  
20 res = mbedtls_ctr_drbg_seed(&ctr_drbg, mbedtls_entropy_func,  
21                             &entropy,  
22                             (const unsigned char *) "ecdsa",  
23                             strlen(pers));  
24 if (res)  
25     /* Failure */
```

(continues on next page)

(continued from previous page)

```
26
27 res = mbedtls_ecp_gen_key(MBEDTLS_ECP_DP_SECP256R1,
28                           mbedtls_pk_ec(key),
29                           mbedtls_ctr_drbg_random,
30                           &ctr_drbg);
31 if (res)
32     /* Failure */
33
34 mbedtls_ecp_group_init(&grp);
35 mbedtls_ecp_point_init(&pt);
36
37 res = mbedtls_ecp_export(ecp, &grp, NULL, &Host_base_key);
38 if (res)
39     /* Failure */
40
41 res = mbedtls_ecp_point_write_binary(&grp, &Host_base_key,
42                                     MBEDTLS_ECP_PF_UNCOMPRESSED,
43                                     &len, Host_peer_key,
44                                     sizeof(Host_peer_key));
45 if (res)
46     /* Failure */
47
48 /*
49  * At this stage the ``Host_peer_key`` is the public key value
50  * starting with
51  * the uncompress tag 0x4.
52  */
53 ...
```

1. On device, derive the OEM Master key

- Host_peer_key is the host peer public key buffer.
- ELE_base_id is the ELE base key identifier.

Fixed info is optional. If set, its digest must be present in the signed content payload.

Signed content payload must be configured as explained in *OEM Master key* and where the peer public key digest must be the result of SHA256(Host_peer_key). This signed content payload key will be identified in this example by OEM_Payload.

In this example, the OEM Master key is a persistent key with a key identifier named ELE_OEM_MK_ID. It's assumed that the signed payload is valid with slat flags set to 0 (no salt used to derived OEM Master key, wrapping and signing keys). In this example, the fixed info is not set.

```
1 int res = -1;
2 enum smw_status_code status = SMW_STATUS_OK;
3 struct smw_derive_key_args args = { 0 };
4 struct smw_kdf_oem_master_key_args oem_mk_args = { 0,
5     ↪};
```

(continues on next page)

(continued from previous page)

```
5 struct smw_key_descriptor key_base = { 0 };
6 struct smw_derived_key_descriptor key_derived = { 0 }
  ↳;
7
8 args.subsystem_name = SMW_SUBSYSTEM_NAME_ELE;
9 args.key_descriptor_base = &key_base;
10 args.kdf_name = SMW_KDF_NAME_OEM_MASTER_KEY;
11 args.kdf_arguments = &oem_mk_args;
12 args.store_derived_key = true; /* Request to store
  ↳the key in Secure Storage */
13 args.key_descriptor_base = &key_base;
14 args.key_descriptor_derived = &key_derived;
15
16 oem_mk_args.op = SMW_OEM_MK_OP_NAME_DERIVE;
17 oem_mk_args.peer_public_buffer = Host_peer_key;
18 oem_mk_args.peer_public_buffer_length = sizeof(Host_
  ↳peer_key);
19 oem_mk_args.payload = OEM_Payload;
20 oem_mk_args.payload_length = sizeof(OEM_Payload);
21
22 key_base.id = ELE_base_id;
23
24 key_derived.type_name = SMW_KEY_TYPE_NAME_DERIVE;
25 key_derived.security_size = 256;
26 key_derived.id = ELE_OEM_MK_ID;
27 key_derived.attributes.permitted_algo = SMW_ATTR_
  ↳ALGO_KEY_DERIVATION_HKDF(SHA256);
28 key_derived.attributes.usage_flags = SMW_ATTR_USAGE_
  ↳DERIVE;
29 key_derived.attributes.attributes = SMW_ATTR_
  ↳PERSISTENCE_PERSISTENT;
30
31
32 status = smw_derive_key(&args);
33 if (status != SMW_STATUS_OK)
34     return -1;
35
36 /*
37  * At this stage, the OEM Master key is created in
  ↳the EdgeLock Enclave,
38  * as persistent key identifier `ELE_OEM_MK_ID`
39  */
```

Note

The wrapping and signing keys of the key import blob are not derived at this step but each time the Key Import API is executed.

1. On host, generates same OEM Master key

This operation is done in two steps: key agreement and key derivation.

In this example, the host OEM Master key identifier will be named `Host_OEM_MK_ID`.

1. Key agreement:

This step generates a shared secret that will be used as input of key derivation step.

The algorithm used is ECDH.

In this example, the base key is the key generated in previous step and named `Host_base_key`. The peer key is the device public key generated in previous step and named `ELE_peer_key`.

Here's a sample of code that can be used to generate the shared secret:

```
1  int res = -1;
2  uint32_t peer_key_size = (256 / 8) * 2;
3  uint32_t mbed_peer_key_size = peer_key_size + 1;
4  uint8_t peer_key[mbed_peer_key_size];
5  mbedtls_ecp_group grp;
6  mbedtls_ecp_point peer_pt;
7  mbedtls_mpi ecdh;
8
9  /* Prepare public key */
10 memset(&grp, 0, sizeof(grp));
11 res = mbedtls_ecp_group_load(&grp, MBEDTLS_ECP_DP_SECP256R1);
12 if (res != 0)
13     return failure;
14
15 /* Prepare public key, a 0x04 leading byte must be added */
16 peer_key[0] = 0x04;
17 memcpy(peer_key_key + 1, ELE_peer_key, peer_key_size);
18
19 /* Load public key from buffer */
20 mbedtls_ecp_point_init(&peer_pt);
21 res = mbedtls_ecp_point_read_binary(&grp, &peer_key, mbed_
    ↳ peer_key,
22                                     mbed_peer_key_size);
23 if (res != 0)
24     /* Failure */
25
26 /* Compute ECDH */
27 mbedtls_mpi_init(&ecdh);
28
29 /*
30  * The ``Host_base_key`` is the result of the key generated in
31  * previous step.
32  */
33 res = mbedtls_ecdh_compute_shared(&grp, &ecdh, &peer_pt,
34                                     &Host_base_key.private_d,
35                                     my_mbedtls_rand, NULL);
36 if (res != 0)
37     /* Failure */
```

(continues on next page)

(continued from previous page)

```
38
39  /*
40  * At this point, the shared secret is the buffer located at
41  * ecdh.private_p. Its size in bytes is 8 * ecdh.private_n
42  * (ecdh.private_n is the size in limbs, i.e number of 64_
43  ↪ bits blocks).
44  *
45  * CAUTION: Switch your environment configuration, MbedTLS_
46  ↪ may store
47  * the shared secret ecdh.private_p in little endian BUT we_
48  ↪ need it
49  * in big endian for the next step (key derivation).
50  * Then a little endian to big endian conversion is needed.
51  */
52
53 ...
```

Note

my_mbedtls_rand must be a user function with the following definition: `int my_mbedtls_rand(void *rng_state, unsigned char *output, size_t len)`. The function fills output buffer with a random value of len bytes. It must return 0 on success. Implementation depends on your environment.

2. Key derivation:

In this step, the shared secret buffer generated in previous step is used as input. Let's identify it as `ecdh` and `ecdh_size` (size in bytes).

Note

ECDH buffer must be in big endian.

Here's a sample of code that can be used to derive the host OEM Master key named `Host_OEM_MK` key:

```
1  int res = -1;
2  const mbedtls_md_info_t *md = NULL;
3  uint32_t salt_size = 0;
4  uint8_t *salt = NULL;
5  uint32_t info_size = 0;
6  uint8_t *info = NULL;
7  uint32_t Host_OEM_MK_size = 256 / 8;
8  uint8_t *Host_OEM_MK[Host_OEM_MK_size];
9
10 /* Set md info */
11 md = mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);
12 if (md == NULL)
```

(continues on next page)

(continued from previous page)

```
13  /* Failure */
14
15  /*
16   * In this example:
17   * Salt is NULL and salt_size is 0: a string of zeros of SHA256 bytes
18   * length will be used.
19   *
20   * Info is NULL and info_size is 0: no fixed info are used in the
21   * derivation process.
22   *
23   * Both Salt and Fixed info could be enabled/set but previous steps
24   * must defined the same.
25   */
26  res = mbedtls_hkdf(md, salt, salt_size, ecdh, ecdh_size, info,
27                   info_size, Host_OEM_MK, Host_OEM_MK_size);
28  if (res != 0)
29      /* Failure */
30
31  ...
```

2. On host, derives the wrapping key

Use third party crypto service (MbedTLS library).

In this step, the OEM Master key is the Host_OEM_MK key buffer generated in previous step is used as input.

Here's a sample of code that can be used to derive the wrapping key identified by Host_OEM_WRAP_key key:

```
1  int res = -1;
2  const mbedtls_md_info_t *md = NULL;
3  uint32_t salt_size = 0;
4  uint8_t *salt = NULL;
5  uint32_t info_size = 24;
6  uint8_t *info = "oemelefwkeyimportwrap256";
7  uint32_t Host_OEM_WRAP_key_size = 256 / 8;
8  uint8_t *Host_OEM_WRAP_key[Host_OEM_WRAP_key_size];
9
10 /* Set md info */
11 md = mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);
12 if (md == NULL)
13     /* Failure */
14
15 /*
16  * In this example:
17  * Salt is NULL and salt_size is 0: a string of zeros of SHA256 bytes
18  * length will be used.
19  *
20  * Salt could be enabled/set but previous steps must defined the_
  ↪ same.
```

(continues on next page)

(continued from previous page)

```
21  *
22  * Info is imposed by ELE FW.
23  */
24  res = mbedtls_hkdf(md, salt, salt_size, Host_OEM_MK, Host_OEM_MK_
    ↪size,
25                      info, info_size, Host_OEM_WRAP_key,
26                      Host_OEM_WRAP_key_size);
27  if (res != 0)
28      /* Failure */
29
30  ...
```

1. On host, derives the signing key

Use third party crypto service (MbedTLS library).

In this step, the OEM Master key is the Host_OEM_MK key buffer generated in previous step is used as input.

Here's a sample of code that can be used to derive the wrapping key identified by Host_OEM_CMACE_key key:

```
1  int res = -1;
2  const mbedtls_md_info_t *md = NULL;
3  uint32_t salt_size = 0;
4  uint8_t *salt = NULL;
5  uint32_t info_size = 24;
6  uint8_t *info = "oemelefwkeyimportcmac256";
7  uint32_t Host_OEM_CMACE_key_size = 256 / 8;
8  uint8_t *Host_OEM_CMACE_key[Host_OEM_CMACE_key_size];
9
10 /* Set md info */
11 md = mbedtls_md_info_from_type(MBEDTLS_MD_SHA256);
12 if (md == NULL)
13     /* Failure */
14
15 /*
16  * In this example:
17  * Salt is NULL and salt_size is 0: a string of zeros of SHA256 bytes
18  * length will be used.
19  *
20  * Salt could be enabled/set but previous steps must defined the_
    ↪same.
21  *
22  * Info is imposed by ELE FW.
23  */
24  res = mbedtls_hkdf(md, salt, salt_size, Host_OEM_MK, Host_OEM_MK_
    ↪size,
25                      info, info_size, Host_OEM_CMACE_key,
26                      Host_OEM_CMACE_key_size);
27  if (res != 0)
```

(continues on next page)

```

28  /* Failure */
29
30  ...

```

1. On host, build the EdgeLock Enclave blob to import the key

The blob must be built by yourself and encoded as described in Table 4.20.

Third party crypto service (MbedTLS library) is used to wrap the imported key and signed the blob.

Here's a sample of code that can be used to wrap the imported key with RFC3394 algorithm:

```

1  uint32_t imported_key_size = 256 / 8;
2  uint8_t imported_key[imported_key_size];
3  uint32_t wrap_key_size = imported_key_size + 8; // RFC3394 algorithm
   ↳ adds 8 bytes
4  uint8_t wrap_key[wrap_key_size];
5  uint32_t out_len = 0;
6  mbedtls_nist_kw_context kw_ctx;
7
8  /* MbedTLS KW context init and setup */
9  mbedtls_nist_kw_init(&kw_ctx);
10
11 res = mbedtls_nist_kw_setkey(&kw_ctx, MBEDTLS_CIPHER_ID_AES,
12                               Host_OEM_WRAP_key, 256, 1);
13 if (res != 0)
14     /* Failure */
15
16 /* Execute key wrapping operation */
17 res = mbedtls_nist_kw_wrap(&kw_ctx, MBEDTLS_KW_MODE_KW, imported_key,
18                             imported_key_size, wrap_key, &out_len,
19                             wrap_key_size);
20 if (res != 0)
21     /* Failure */
22
23 ...

```

Here's a sample of code that can be used to sign the blob with CMAC algorithm:

```

1  int res = -1;
2  const mbedtls_cipher_info_t *cipher_info = NULL;
3  uint32_t signature_size = 16; // Default CMAC output size
4  uint32_t message_size = tlv_size - signature_size; // TLV size is the
   ↳ size in bytes of the full TLV buffer
5  uint8_t tlv[tlv_size]; // The TLV buffer you built previously
6  uint8_t *signature_ptr = tlv + message_size;
7
8  cipher_info = mbedtls_cipher_info_from_type(MBEDTLS_CIPHER_AES_256_
   ↳ ECB);
9  if (cipher_info == NULL)

```

(continues on next page)

(continued from previous page)

```
10     return failure;
11
12     res = mbedtls_cipher_cmac(cipher_info, Host_OEM_CMAC_key, 256, tlv,
13                               message_size, signature_ptr);
14     if (res != 0)
15         /* Failure */
16
17     ...
```

Here's an example of EdgeLock Enclave blob to import an AES 256 bits key:

- identifier = 0x66
- permitted algorithm is defined to permit all ciphers mode
- usage encrypt/decrypt
- key is persistent
- lifecycle is current device lifecycle
- OEM Master key identifier = 0x2

```
1  uint8_t aes_key_ex[] = {
2      0x40, 0x15, 0x65, 0x64, 0x67, 0x65, 0x6c, 0x6f,
3      0x63, 0x6b, 0x65, 0x6e, 0x63, 0x6c, 0x61, 0x76,
4      0x65, 0x69, 0x6d, 0x70, 0x6f, 0x72, 0x74, 0x41,
5      0x04, 0x00, 0x00, 0x00, 0x66, 0x42, 0x04, 0x84,
6      0xc0, 0xff, 0x00, 0x43, 0x04, 0x00, 0x00, 0x03,
7      0x00, 0x44, 0x02, 0x24, 0x00, 0x45, 0x04, 0x00,
8      0x00, 0x01, 0x00, 0x46, 0x04, 0xc0, 0x02, 0x00,
9      0x01, 0x47, 0x04, 0x00, 0x00, 0x00, 0x00, 0x50,
10     0x04, 0x00, 0x00, 0x00, 0x02, 0x51, 0x04, 0x00,
11     0x00, 0x00, 0x01, 0x54, 0x04, 0x00, 0x00, 0x00,
12     0x01, 0x55, 0x28, 0x23, 0xb4, 0xa9, 0xf5, 0x9a,
13     0x91, 0x9c, 0xd6, 0xfc, 0x6d, 0x27, 0xa7, 0xa6,
14     0x27, 0x4e, 0xed, 0xf5, 0x6e, 0x92, 0x9f, 0x04,
15     0xeb, 0xed, 0xef, 0xf0, 0x31, 0xe4, 0xc9, 0x51,
16     0x9d, 0x12, 0x4d, 0x42, 0x50, 0xe8, 0x98, 0x82,
17     0x95, 0x42, 0x96, 0x5e, 0x10, 0x77, 0x17, 0xd2,
18     0x6c, 0x47, 0x55, 0x67, 0xf4, 0x0b, 0xeb, 0x7f,
19     0x39, 0x31, 0xea, 0x8b, 0x86
20 };
```

1. On device, import the key.

The SMW API `smw_import_key` is used to import the key.

In the following example, the key imported is the blob example of the AES 256 bits key above.

```
1  int res = -1;
2  enum smw_status_code status = SMW_STATUS_OK;
3  struct smw_import_key_args args = { 0 };
```

(continues on next page)

(continued from previous page)

```
4 struct smw_key_descriptor key = { 0 };
5 struct smw_key_attributes *attributes = &key.attributes;
6 struct smw_keypair_buffer buffer = { 0 };
7
8 /* Set the private key buffer to use the blob example */
9 buffer.gen.public_length = sizeof(aes_key_ex);
10 buffer.gen.public_data = aes_key_ex;
11
12     args.subsystem_name = SMW_SUBSYSTEM_NAME_ELE;
13     args.key_descriptor = &key;
14
15 key.id = 0x66;
16 key.type_name = SMW_KEY_TYPE_NAME_AES;
17 key.security_size = 256;
18 key.buffer = &buffer;
19
20 attributes.permitted_algo = SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION(SMW_
    ↪ ATTR_ALGO_AES,
21
    SMW_
    ↪ ATTR_MODE_ANY);
22 attributes.usage_flags = SMW_ATTR_USAGE_ENCRYPT | SMW_ATTR_USAGE_
    ↪ DECRYPT;
23 attributes.attributes = SMW_ATTR_SET_PERSISTENCE(attributes.
    ↪ attributes,
24
    SMW_ATTR_
    ↪ PERSISTENCE_PERSISTENT);
25
26
27 status = smw_import_key(&args);
28
29 ...
```

4.2 SECO capabilities

4.2.1 Key manager

Table 4.21: SECO Key type

Key type	Key security size(s)
AES	128 / 192 / 256
Secp R1	256 / 384
Brainpool R1	256 / 384
HMAC	224 / 256 / 384 / 512

Operations supported:

- Generate

-
- Export (only public key in HEX or Base64 format)
 - Delete
 - Derive¹
 - Get key attributes
 - Get key buffers' length
 - Get key security size
 - Get key type name
 - Commit key storage (do nothing)

Key group: The SMW Library is managing the SECO key group automatically. The library is selecting a key group depending if a key is persistent/permanent or transient.

- Persistent/Permanent keys are in key groups from 0 to 511.
- Transient keys are in key groups from 512 to 1023.

4.2.1.1 Key policy

The SECO subsystem doesn't support key policy attribute. Defining the key attribute will be ignored and if attribute is defined the API returns the warning *SMW_STATUS_KEY_POLICY_WARNING_IGNORED*.

4.2.2 Hash

Table 4.22: SECO Hash

Hash Algorithm
SHA1 ²
SHA224
SHA256
SHA384
SHA512

Operations supported:

- One shot and multi-part²

¹ Only TLS12_KEY_EXCHANGE when hardware supports it

² Operation is performed by the SMW library

4.2.3 Signature

Table 4.23: SECO Signature

Signature Type	Key type	Key security size(s)	Hash algorithm
ECDSA	Secp R1	256	SHA256
		384	SHA384
	Brainpool R1	256	SHA256
		384	SHA384

Operations supported:

- Sign One-shot and Multi-part
- Verify One-shot and Multi-part

Note

Signature attribute TLS_MAC_FINISH available only when hardware supports it.

Note

Message to sign/verify is full or hashed depending on the algorithm 64-bits word definition additional parameters (bits[39:32]).

Caution

Signature Multi-part is supported only if the hash operation is enabled.

4.2.4 Random

Length: 1 to UINT32_MAX

4.2.5 MAC

Table 4.24: SECO MAC

MAC Type	Key type	Key security size(s)	Hash
CMAC	AES	128 / 192 / 256	N/A
HMAC	HMAC	224	SHA224
		256	SHA256
		384	SHA384
		512	SHA512

HMAC Key generation and HMAC generation is not working on all SECO Firmware and may return SMW_STATUS_SUBSYSTEM_FAILURE.

Operations supported:

- Compute MAC
- Verify MAC

4.2.6 Cipher

Table 4.25: SECO Cipher

Cipher Mode	Key type	Key security size(s)
ECB No Padding	AES	128 / 192 / 256
CBC No Padding		

One-shot operations supported:

- Encrypt
- Decrypt

4.2.7 Operation context

Operations supported:

- Allocate
- Cancel

4.2.8 Data Storage manager

Data Storage manager allows to store and retrieve data. The data ID is a 32-bits value.

The subsystem allows to:

- store and retrieve user data.

The subsystem doesn't allow to:

- encrypt and sign data before storing it.
- delete a data.

4.2.9 AEAD

Table 4.26: SECO AEAD

AEAD Mode	Key type	IV length (bytes)	Tag length (bytes)
CCM	AES	12	16
GCM		Encryption: • 0 (subsystem generates full IV) • 4 (subsystem generates 8 bytes of IV) Decryption: • 12 (user supplied full IV)	16

One-shot operations supported:

- AEAD Encryption
- AEAD Decryption

4.2.10 Key Derivation

- TLS 1.2 (TLS1-PRF)⁶

SECO does not support multiple TLS1.2 operations, so the implementation of the TLS1.2 API uses the context supplied as a parameter to store intermediate data when attempting to generate the master secret. When calling the TLS1.2 API to compute the key expansion, the input of this operation is merged with the context and the actual operation is being executed. Thus, the master secret key id returned from the first operation is not valid, and only be used after the subsequent key expansion operation.

Only ECDH(E) key exchange is supported, and the following ciphersuites:

Table 4.27: SECO supported ciphersuites

SMW encryption name	OpenSSL equivalent
SMW_TLS12_ENC_NAME_AES_128_CBC	ECDHE-ECDSA-AES128-SHA256
SMW_TLS12_ENC_NAME_AES_128_GCM	ECDHE-ECDSA-AES128-GCM-SHA256
SMW_TLS12_ENC_NAME_AES_256_CBC	ECDHE-ECDSA-AES256-SHA384
SMW_TLS12_ENC_NAME_AES_256_GCM	ECDHE-ECDSA-AES256-GCM-SHA384

⁶ Only when supported by the hardware

4.3 TEE capabilities

4.3.1 Key manager

Table 4.28: TEE Key type

Key type	Key security size(s)
AES	128 / 192 / 256
DES	56
DES3	112 / 168
Secp R1	192 / 224 / 256 / 384 / 521
Ed25519	255
RSA	256 to 4096 ¹
HMAC	64 to 1024 bits ²
SM4	128
HKDF IKM ³	8 to 4096 bits ²

Operations supported:

- Generate
- Import
- Export (only public key in HEX or Base64 format)
- Delete
- Get key attributes
- Get key buffers' length
- Get key security size
- Get key type name
- Commit key storage (do nothing)
- *Key Derivation*

4.3.1.1 Key policy

When creating a new key, the key policy must be specified through the operation key attributes.

The following Table 4.29 lists all key usages applicable in TEE subsystem. A key policy defines one or more key usage.

¹ multiple of 2 bits

² multiple of 8 bits

³ Only key import is supported. HKDF IKM keys cannot be generated. This key can be used as IKM for key derivation using HKDF.

Table 4.29: TEE Key usages

USAGE	Description
ENCRYPT	Permission to encrypt a message
DECRYPT	Permission to decrypt a message
SIGN_MESSAGE	Permission to sign a message
SIGN_HASH	Permission to sign a message hashed
VERIFY_MESSAGE	Permission to verify the signature of a message
VERIFY_HASH	Permission to verify the signature of message hashed
DERIVE	Permission to derive other keys from this key
EXPORT	Permission to export the public key only

The TEE subsystem has limited algorithm restrictions per key usage:

- For key generation and import intended for asymmetric encryption operations, the permitted algorithm must be specified. If not specified, the operation will fail with the error status *SMW_STATUS_PERMITTED_ALGO_INVALID*.
- For all other operations, defining permitted algorithm(s) will not be taken into account and the operation will return the warning status *SMW_STATUS_KEY_POLICY_WARNING_IGNORED*.

4.3.2 Hash

Table 4.30: TEE Hash

Hash Algorithm
MD5
SHA1
SHA224
SHA256
SHA384
SHA512
SHA3_224
SHA3_256
SHA3_384
SHA3_512
SM3
SHAKE256

Operations supported:

- One shot and multi-part

Note

The SHAKE256 is an extendable-output function (XOF) where digest length can be any length as detailed in the *FIPS 202* <<https://doi.org/10.6028/NIST.FIPS.202>>.

4.3.3 Signature

Table 4.31: TEE Signature

Signature Type	Key type	Key security size(s)	Hash algorithm
ECDSA	Secp R1	192 / 224 / 256 / 384 / 521	• SHA1 • SHA224 • SHA256 • SHA384 • SHA512 • SHA3_224 • SHA3_256 • SHA3_384 • SHA3_512 • None (Message hashed)
RSA_PKCS1V15	RSA	256 to 4096 ⁴	• MD5 • SHA1 • SHA224 • SHA256 • SHA384 • SHA512 • SHA3_224 • SHA3_256 • SHA3_384 • SHA3_512
RSA_PSS	RSA	256 to 4096 ⁴	• SHA1 • SHA224 • SHA256 • SHA384 • SHA512 • SHA3_224 • SHA3_256 • SHA3_384 • SHA3_512
PURE_EDDSA	ED25519	255	N/A
EDDSA_PH	ED25519	255	None (pre-hashed)
EDDSA_CTX	ED25519	255	N/A

Operations supported:

- Sign One-shot and Multi-part

⁴ multiple of 2 bits

- Verify One-shot and Multi-part

Note

Message to sign/verify is full or hashed depending on the algorithm 64-bits word definition additional parameters (bits[39:32]).

Caution

Signature Multi-part is supported only if the hash operation is enabled.

4.3.4 MAC

Table 4.32: TEE MAC

MAC Type	Key type	Key security size(s)	Hash
CMAC	AES	128 / 192 / 256	N/A
HMAC	HMAC	64 to 512 bits ⁵	MD5
		80 to 512 bits ⁵	SHA1
		112 to 512 bits ⁵	SHA224
		192 to 1024 bits ⁵	SHA256
		256 to 1024 bits ⁵	SHA256
		256 to 1024 bits ⁵	SHA384
		80 to 1024 bits ⁵	SHA512
			SM3

Operations supported:

- Compute MAC
- Verify MAC

4.3.5 Random

Length: 1 to SIZE_MAX

⁵ multiple of 8 bits

4.3.6 Cipher

Table 4.33: TEE Cipher

Cipher Mode	Key type	Key security size(s)
ECB No Padding	AES	128 / 192 / 256
CBC No Padding		
CTS		
CTR		
XTS		
ECB No Padding	DES	56
CBC No Padding		
ECB No Padding	DES3	112 / 168
CBC No Padding		
ECB No Padding	SM4	128
CBC No Padding		
CTR		

Operations supported:

- Encrypt⁶
- Decrypt⁶

4.3.7 Operation context

Operations supported:

- Allocate
- Cancel
- Copy

4.3.8 AEAD

Table 4.34: TEE AEAD

AEAD Mode	Key type	IV length (bytes)	Tag length (bytes)
CCM	AES	Up to 13	4 / 6 / 8 / 10 / 12 / 14 / 16
GCM		Up to 16	12 / 13 / 14 / 15 / 16

⁶ one shot and multi-part

Note

User has the capability to request the TA to generate part of full operation IV in the limit of the maximum value depending of the operation mode as detailed in the [Table 4.34](#).

Operations supported:

- Encryption one shot and multi-part
- Decryption one shot and multi-part

4.3.9 Asymmetric encryption and decryption

Table 4.35: TEE Asymmetric encryption and decryption

Encryption mode	Key type	Key security size(s)	Hash algorithm
OAEP	RSA	256 to 4096 ⁷	• SHA1 • SHA224 • SHA256 • SHA384 • SHA512
PKCS1V15	RSA	256 to 4096 ⁷	N/A
No Padding	RSA	256 to 4096 ⁷	N/A

Operations supported:

- Encryption
- Decryption

4.3.10 Data Storage manager

Data Storage manager allows to store and retrieve data. The data ID is a 32-bits value.

The subsystem allows to:

- store and retrieve user data.
- delete a data.

The subsystem doesn't allow to:

- encrypt and sign data before storing it.

4.3.11 Key Derivation

Supported Key Derivation Functions

- HMAC-based Key Derivation Function (HKDF)

⁷ multiple of 2 bits

-
- ECDH Key Derivation Function

The subsystem supports deriving a key from an existing HKDF IKM or EC key as well as from a plaintext buffer. Subsystem allows to store the derived key upon user request and also allows exporting the derived key if the derived key buffer is set.

In order to work on any Operating System (OS), the SMW library requires a module called OSAL. This OSAL must be implemented by the SMW library integrator to work on a specific OS.

The OSAL is the entry point of the SMW library. It's in charge of the library initialization, load, unload. In addition, the OSAL must implement a key database manager to convert a subsystem key identifier to a library identifier that might be PSA compatible or not.

The SMW source package contains an example of OSAL in the folder *osal/Linux* running on Linux and used to validate the library. This code example can be modified by the library integrator.

5.1 SMW interface

5.1.1 Introduction

The OSAL module must refer to the following structures and functions to be linked with the SMW core library. The content and prototype of the structures and functions can't be changed otherwise the core library may not build or work correctly.

5.1.2 struct smw_osal_object

struct **smw_osal_object**
OSAL object descriptor

5.1.2.1 Definition

```
struct smw_osal_object {  
    struct smw_object_descriptor *obj_desc;  
    unsigned int obj_id_subsystem;  
}
```

5.1.2.2 Members

obj_desc
Object descriptor (user information)

obj_id_subsystem
Object identifier in subsystem

5.1.2.3 Description

The user API identifier is transported by the `obj_desc->id` while the object identifier in the subsystem is not known or accessible by the user except when defined by the user at object creation. In this case the object identifier in the subsystem is same as user.

5.1.3 struct smw_ops

struct **smw_ops**

SMW OSAL operations

5.1.3.1 Definition

```
struct smw_ops {
    void (*critical_section_start)(void);
    void (*critical_section_stop)(void);
    int (*mutex_init)(void **mutex);
    int (*mutex_destroy)(void **mutex);
    int (*mutex_lock)(void *mutex);
    int (*mutex_unlock)(void *mutex);
    int (*thread_create)(unsigned long *thread, void *(*start_routine)(void_,
↪*), void *arg);
    int (*thread_cancel)(unsigned long thread);
    void (*vprint)(unsigned int level, const char *format, va_list arg);
    void (*hex_dump)(unsigned int level, const unsigned char *addr, unsigned_
↪int size, unsigned int align);
    void (*register_active_subsystem)(smw_subsystem_t subsystem_name);
    int (*get_subsystem_info)(smw_subsystem_t subsystem_name, void *info);
    bool (*is_lib_initialized)(void);
    int (*get_obj_info)(struct smw_osal_object *descriptor);
    int (*add_obj_info)(struct smw_osal_object *descriptor);
    int (*update_obj_info)(struct smw_osal_object *descriptor);
    int (*delete_obj_info)(struct smw_osal_object *descriptor);
    int (*find_obj_init)(void **find_ctx, struct smw_osal_object *descriptor);
    int (*find_obj_next)(void *find_ctx, struct smw_osal_object *descriptor);
    int (*find_obj_final)(void *find_ctx);
}
```

5.1.3.2 Members

critical_section_start

[optional] Start critical section

critical_section_stop

[optional] Stop critical section

mutex_init

[mandatory] Initialize a mutex

mutex_destroy

[mandatory] Destroy a mutex

mutex_lock

[mandatory] Lock a mutex

mutex_unlock
[mandatory] Unlock a mutex

thread_create
[mandatory] Create a thread

thread_cancel
[mandatory] Cancel a thread

vprint
[optional] Print debug trace

hex_dump
[optional] Print buffer content

register_active_subsystem
[optional] Register the active Secure Subsystem

get_subsystem_info
[mandatory] Get Subsystem configuration info

is_lib_initialized
[mandatory] Check if the library was successfully initialized by OSAL

get_obj_info
[mandatory] Get an object information from database

add_obj_info
[mandatory] Add an object information into database

update_obj_info
[mandatory] Update an object information into database

delete_obj_info
[mandatory] Delete an object information from database

find_obj_init
[mandatory] Initialize the find object query

find_obj_next
[mandatory] Find the next object

find_obj_final
[mandatory] Close the find object query

5.1.3.3 Description

This structure defines the SMW OSAL. Functions pointers marked as [mandatory] must be assigned. Functions pointers marked as [optional] may not be assigned. `mutex_*` functions pointers are optional together. `critical_*` functions pointers are optional together.

5.1.4 smw_init

enum *smw_status_code* **smw_init**(const struct *smw_ops* *ops)

Initialize the SMW library.

Parameters

- **ops** (const struct *smw_ops**) – pointer to the structure describing the OSAL.

5.1.4.1 Description

This function initializes the Security Middleware. It verifies that ops is valid and then initializes SMW modules.

5.1.4.2 Return

See `enum smw_status_code`

- SMW_STATUS_OK - Initialization is successful
- SMW_STATUS_OPS_INVALID - ops is invalid
- SMW_STATUS_MUTEX_INIT_FAILURE - Mutex initialization has failed

5.1.5 smw_deinit

`enum smw_status_code smw_deinit(void)`

Deinitialize the SMW library.

Parameters

- **void** – no arguments

5.1.5.1 Description

This function deinitializes the Security Middleware. It frees all memory dynamically allocated by SMW.

5.1.5.2 Return

See `enum smw_status_code`

- SMW_STATUS_OK - Deinitialization is successful
- SMW_STATUS_INVALID_LIBRARY_CONTEXT - Library context is not valid
- SMW_STATUS_MUTEX_DESTROY_FAILURE - Mutex destruction has failed

5.2 Linux example

5.2.1 Introduction

The OSAL interface is the library API specific to the Operating System. It's under the charge of the library integrator to adapt the OSAL library part to the OS targeted.

Below is a C code example configuring and loading the SMW library with the given OSAL example.

```
#define DEFAULT_OBJ_DB "/var/tmp/obj_db_smw_test.dat"

static const struct tee_info tee_default_info = {
    { "11b5c4aa-6d20-11ea-bc55-0242ac130003" }
};

static const struct se_info se_default_info = { 0x534d5754, 0x444546,
                                                1000 }; // SMWT, DEF

int main(int argc, char *argv[])
```

(continues on next page)

```

{
    int res = ERR_CODE(FAILED);

    // Configure the TEE Subsystem: TA UUID (and so key storage)
    res = smw_osal_set_subsystem_info(SMW_SUBSYSTEM_NAME_TEE, &tee_default_
↪info,
                                   sizeof(tee_default_info));
    if (res != SMW_STATUS_OK)
        goto exit;

    // Configure the SECO Subsystem: Key storage identifier and replay
    res = smw_osal_set_subsystem_info(SMW_SUBSYSTEM_NAME_SECO, &se_default_
↪info,
                                   sizeof(se_default_info));
    if (res != SMW_STATUS_OK)
        goto exit;

    // Open/Create the application object database
    res = smw_osal_open_obj_db(DEFAULT_OBJ_DB, strlen(DEFAULT_OBJ_DB) + 1);
    if (res != SMW_STATUS_OK)
        goto exit;

    // Load and initialize the library. OSAL is loading the application
    // SMW configuration file defined by the system environment variable
    // 'SMW_CONFIG_FILE'
    res = smw_osal_lib_init();
    if (res != SMW_STATUS_OK)
        goto exit;

    // Execute the application
    ...

    exit:

    return res;
}

```

5.2.2 struct tee_info

struct **tee_info**

TEE Subsystem information

5.2.2.1 Definition

```

struct tee_info {
    char ta_uuid[TEE_TA_UUID_SIZE_MAX];
}

```

5.2.2.2 Members

ta_uuid
TA UUID

5.2.3 struct se_info

struct **se_info**
Secure Enclave information

5.2.3.1 Definition

```
struct se_info {  
    unsigned int storage_id;  
    unsigned int storage_nonce;  
    unsigned short storage_replay;  
}
```

5.2.3.2 Members

storage_id
Key storage identifier

storage_nonce
Key storage nonce

storage_replay
Replay attack counter (Not used on ELE)

5.2.4 union subsystem_info

union **subsystem_info**
Union of all subsystem information

5.2.4.1 Definition

```
union subsystem_info {  
    struct tee_info tee;  
    struct se_info se;  
}
```

5.2.4.2 Members

tee
TEE Subsystem information

se
Secure Enclave information

5.2.5 smw_osal_latest_subsystem_name

smw_subsystem_t **smw_osal_latest_subsystem_name**(void)

Return the latest Secure Subsystem name

Parameters

- **void** – no arguments

5.2.5.1 Description

In DEBUG mode only, function returns the name of the latest Secure Subsystem invoked by SMW. This Secure Subsystem have been either explicitly requested by the caller or selected by SMW given the operation arguments and the configuration file. In other modes, function always returns NULL.

5.2.5.2 Return

In DEBUG mode only, the name of the Secure Subsystem.

5.2.6 smw_osal_lib_init

enum *smw_status_code* **smw_osal_lib_init**(void)

Initialize the SMW library

Parameters

- **void** – no arguments

5.2.6.1 Description

This function must be the first function called by the application opening a library instance. It loads the subsystem configuration set in the linux environment variable SMW_CONFIG_FILE.

5.2.6.2 Return

SMW_STATUS_OK - Library initialization success
SMW_STATUS_LIBRARY_ALREADY_INIT - Library already initialized otherwise any of the smw status

5.2.7 smw_osal_set_subsystem_info

enum *smw_status_code* **smw_osal_set_subsystem_info**(*smw_subsystem_t* subsystem, void *info, size_t info_size)

Set the Subsystem configuration information

Parameters

- **subsystem** (*smw_subsystem_t*) – Subsystem name
- **info** (void*) – Subsystem information
- **info_size** (size_t) – Size in bytes of info parameter

5.2.7.1 Description

This function must be called before a subsystem is loaded.

5.2.7.2 Return

See [enum smw_status_code](#)

- SMW_STATUS_OK - Success
- SMW_STATUS_SUBSYSTEM_LOADED - Subsystem is already loaded
- SMW_STATUS_INVALID_PARAM - Function parameter error
- SMW_STATUS_ALLOC_FAILURE - Allocation failure
- SMW_STATUS_UNKNOWN_SUBSYSTEM_NAME - Unknown subsystem name

5.2.8 smw_osal_open_obj_db

enum [smw_status_code](#) **smw_osal_open_obj_db**(const char *file, size_t len)

Setup the object database file to open

Parameters

- **file** (const char*) – Fullname of the object database
- **len** (size_t) – Length of the file string

5.2.8.1 Description

If the library is already initialized, database may be already in use by another thread. Hence it can be reconfigured. This function must be called before [smw_osal_lib_init\(\)](#).

5.2.8.2 Return

See [enum smw_status_code](#)

- SMW_STATUS_OK - Success
- SMW_STATUS_LIBRARY_ALREADY_INIT - Library already initialized
- SMW_STATUS_CONFIGURATION_FAILURE - Error of configuration
- SMW_STATUS_ALLOC_FAILURE - Out of memory

P

- psa_aead_abort (C function), 242
- psa_aead_decrypt (C function), 243
- PSA_AEAD_DECRYPT_OUTPUT_MAX_SIZE (C macro), 212
- PSA_AEAD_DECRYPT_OUTPUT_SIZE (C macro), 212
- psa_aead_decrypt_setup (C function), 245
- psa_aead_encrypt (C function), 246
- PSA_AEAD_ENCRYPT_OUTPUT_MAX_SIZE (C macro), 212
- PSA_AEAD_ENCRYPT_OUTPUT_SIZE (C macro), 213
- psa_aead_encrypt_setup (C function), 248
- psa_aead_finish (C function), 249
- PSA_AEAD_FINISH_OUTPUT_SIZE (C macro), 213
- psa_aead_generate_nonce (C function), 251
- PSA_AEAD_NONCE_LENGTH (C macro), 214
- psa_aead_operation_init (C function), 252
- psa_aead_operation_t (C type), 237
- psa_aead_set_lengths (C function), 253
- psa_aead_set_nonce (C function), 254
- PSA_AEAD_TAG_LENGTH (C macro), 214
- psa_aead_update (C function), 255
- psa_aead_update_ad (C function), 256
- PSA_AEAD_UPDATE_OUTPUT_MAX_SIZE (C macro), 215
- PSA_AEAD_UPDATE_OUTPUT_SIZE (C macro), 215
- psa_aead_verify (C function), 257
- PSA_AEAD_VERIFY_OUTPUT_SIZE (C macro), 216
- PSA_ALG_AEAD_TAG_LENGTH (C macro), 164
- PSA_ALG_AEAD_WITH_AT_LEAST_THIS_LENGTH_TAG (C macro), 164
- PSA_ALG_AEAD_WITH_DEFAULT_LENGTH_TAG (C macro), 163
- PSA_ALG_AEAD_WITH_SHORTENED_TAG (C macro), 164
- PSA_ALG_AT_LEAST_THIS_LENGTH_MAC (C macro), 167
- PSA_ALG_DETERMINISTIC_ECDSA (C macro), 165
- PSA_ALG_ECDSA (C macro), 166
- PSA_ALG_FULL_LENGTH_MAC (C macro), 167
- PSA_ALG_GET_HASH (C macro), 168
- PSA_ALG_HKDF (C macro), 168
- PSA_ALG_HKDF_EXPAND (C macro), 170
- PSA_ALG_HKDF_EXTRACT (C macro), 169
- PSA_ALG_HMAC (C macro), 171
- PSA_ALG_IS_AEAD (C macro), 172
- PSA_ALG_IS_AEAD_ON_BLOCK_CIPHER (C macro), 172
- PSA_ALG_IS_ASYMMETRIC_ENCRYPTION (C macro), 172
- PSA_ALG_IS_BLOCK_CIPHER_MAC (C macro), 173
- PSA_ALG_IS_CIPHER (C macro), 173
- PSA_ALG_IS_DETERMINISTIC_ECDSA (C macro), 173
- PSA_ALG_IS_ECDH (C macro), 174
- PSA_ALG_IS_ECDSA (C macro), 174
- PSA_ALG_IS_FFDH (C macro), 174
- PSA_ALG_IS_HASH (C macro), 175
- PSA_ALG_IS_HASH_AND_SIGN (C macro), 175
- PSA_ALG_IS_HASH_EDDSA (C macro), 175
- PSA_ALG_IS_HKDF (C macro), 176
- PSA_ALG_IS_HKDF_EXPAND (C macro), 176
- PSA_ALG_IS_HKDF_EXTRACT (C macro), 176
- PSA_ALG_IS_HMAC (C macro), 177
- PSA_ALG_IS_KEY_AGREEMENT (C macro), 177
- PSA_ALG_IS_KEY_DERIVATION (C macro), 177
- PSA_ALG_IS_KEY_DERIVATION_STRETCHING (C macro), 178
- PSA_ALG_IS_MAC (C macro), 178
- PSA_ALG_IS_MAC_TRUNCATED (C macro), 178
- PSA_ALG_IS_PBKDF2_HMAC (C macro), 178
- PSA_ALG_IS_RANDOMIZED_ECDSA (C macro), 179
- PSA_ALG_IS_RAW_KEY_AGREEMENT (C macro), 179

PSA_ALG_IS_RSA_OAEP (C macro), 180
 PSA_ALG_IS_RSA_PKCS1V15_SIGN (C macro), 180
 PSA_ALG_IS_RSA_PSS (C macro), 180
 PSA_ALG_IS_RSA_PSS_ANY_SALT (C macro), 180
 PSA_ALG_IS_RSA_PSS_STANDARD_SALT (C macro), 181
 PSA_ALG_IS_SIGN (C macro), 181
 PSA_ALG_IS_SIGN_HASH (C macro), 181
 PSA_ALG_IS_SIGN_MESSAGE (C macro), 182
 PSA_ALG_IS_SP800_108_COUNTER_HMAC (C macro), 182
 PSA_ALG_IS_STANDALONE_KEY_AGREEMENT (C macro), 179
 PSA_ALG_IS_STREAM_CIPHER (C macro), 182
 PSA_ALG_IS_TLS12_PRF (C macro), 183
 PSA_ALG_IS_TLS12_PSK_TO_MS (C macro), 183
 PSA_ALG_IS_VENDOR_TLS13 (C macro), 176
 PSA_ALG_IS_WILDCARD (C macro), 183
 PSA_ALG_KEY_AGREEMENT (C macro), 184
 PSA_ALG_KEY_AGREEMENT_GET_BASE (C macro), 184
 PSA_ALG_KEY_AGREEMENT_GET_KDF (C macro), 185
 PSA_ALG_PBKDF2_HMAC (C macro), 185
 PSA_ALG_RSA_OAEP (C macro), 186
 PSA_ALG_RSA_PKCS1V15_SIGN (C macro), 186
 PSA_ALG_RSA_PSS (C macro), 187
 PSA_ALG_RSA_PSS_ANY_SALT (C macro), 187
 PSA_ALG_SP800_108_COUNTER_HMAC (C macro), 188
 PSA_ALG_TLS12_PRF (C macro), 189
 PSA_ALG_TLS12_PSK_TO_MS (C macro), 190
 PSA_ALG_TRUNCATED_MAC (C macro), 191
 PSA_ALG_VENDOR_TLS13 (C macro), 170
 psa_algorithm_t (C type), 230
 psa_asymmetric_decrypt (C function), 259
 PSA_ASYMMETRIC_DECRYPT_OUTPUT_SIZE (C macro), 217
 psa_asymmetric_encrypt (C function), 261
 PSA_ASYMMETRIC_ENCRYPT_OUTPUT_SIZE (C macro), 217
 psa_attest_key (C function), 341
 psa_attest_key_get_size (C function), 342
 PSA_BLOCK_CIPHER_BLOCK_LENGTH (C macro), 191
 psa_cipher_abort (C function), 262
 psa_cipher_decrypt (C function), 263
 PSA_CIPHER_DECRYPT_OUTPUT_MAX_SIZE (C macro), 218
 PSA_CIPHER_DECRYPT_OUTPUT_SIZE (C macro), 218
 psa_cipher_decrypt_setup (C function), 264
 psa_cipher_encrypt (C function), 266
 PSA_CIPHER_ENCRYPT_OUTPUT_MAX_SIZE (C macro), 219
 PSA_CIPHER_ENCRYPT_OUTPUT_SIZE (C macro), 219
 psa_cipher_encrypt_setup (C function), 267
 psa_cipher_finish (C function), 269
 PSA_CIPHER_FINISH_OUTPUT_SIZE (C macro), 220
 psa_cipher_generate_iv (C function), 270
 PSA_CIPHER_IV_LENGTH (C macro), 221
 psa_cipher_operation_init (C function), 271
 psa_cipher_operation_t (C type), 238
 psa_cipher_set_iv (C function), 272
 psa_cipher_update (C function), 273
 PSA_CIPHER_UPDATE_OUTPUT_MAX_SIZE (C macro), 221
 PSA_CIPHER_UPDATE_OUTPUT_SIZE (C macro), 222
 psa_copy_key (C function), 274
 psa_crypto_init (C function), 276
 psa_destroy_key (C function), 277
 psa_dh_family_t (C type), 232
 psa_ecc_family_t (C type), 232
 psa_export_key (C function), 278
 PSA_EXPORT_KEY_OUTPUT_SIZE (C macro), 222
 psa_export_public_key (C function), 280
 PSA_EXPORT_PUBLIC_KEY_OUTPUT_SIZE (C macro), 223
 psa_generate_key (C function), 282
 psa_generate_random (C function), 284
 psa_get_key_algorithm (C function), 284
 psa_get_key_attributes (C function), 285
 psa_get_key_bits (C function), 286
 psa_get_key_id (C function), 286
 psa_get_key_lifetime (C function), 287
 psa_get_key_type (C function), 287
 psa_get_key_usage_flags (C function), 288
 psa_hash_abort (C function), 288
 PSA_HASH_BLOCK_LENGTH (C macro), 224
 psa_hash_clone (C function), 289
 psa_hash_compare (C function), 290
 psa_hash_compute (C function), 290
 psa_hash_finish (C function), 291
 PSA_HASH_LENGTH (C macro), 225
 psa_hash_operation_init (C function), 292
 psa_hash_operation_t (C type), 238
 psa_hash_resume (C function), 293

[psa_hash_setup \(C function\), 294](#)
[psa_hash_suspend \(C function\), 295](#)
[PSA_HASH_SUSPEND_HASH_STATE_FIELD_LENGTH \(C macro\), 225](#)
[PSA_HASH_SUSPEND_INPUT_LENGTH_FIELD_LENGTH \(C macro\), 226](#)
[PSA_HASH_SUSPEND_OUTPUT_SIZE \(C macro\), 226](#)
[psa_hash_update \(C function\), 296](#)
[psa_hash_verify \(C function\), 297](#)
[PSA_HMAC_LENGTH \(C macro\), 227](#)
[psa_import_key \(C function\), 298](#)
[psa_initial_attest_get_token \(C function\), 340](#)
[psa_initial_attest_get_token_size \(C function\), 340](#)
[psa_its_get \(C function\), 345](#)
[psa_its_get_info \(C function\), 346](#)
[psa_its_remove \(C function\), 347](#)
[psa_its_set \(C function\), 345](#)
[psa_key_agreement \(C function\), 326](#)
[psa_key_attributes_init \(C function\), 300](#)
[psa_key_attributes_t \(C type\), 239](#)
[psa_key_derivation_abort \(C function\), 300](#)
[psa_key_derivation_get_capacity \(C function\), 301](#)
[psa_key_derivation_input_bytes \(C function\), 301](#)
[psa_key_derivation_input_integer \(C function\), 303](#)
[psa_key_derivation_input_key \(C function\), 304](#)
[psa_key_derivation_key_agreement \(C function\), 305](#)
[psa_key_derivation_operation_init \(C function\), 306](#)
[psa_key_derivation_operation_t \(C type\), 241](#)
[psa_key_derivation_output_bytes \(C function\), 307](#)
[psa_key_derivation_output_key \(C function\), 308](#)
[psa_key_derivation_set_capacity \(C function\), 311](#)
[psa_key_derivation_setup \(C function\), 312](#)
[psa_key_derivation_step_t \(C type\), 233](#)
[psa_key_derivation_verify_bytes \(C function\), 313](#)
[psa_key_derivation_verify_key \(C function\), 314](#)
[psa_key_id_t \(C type\), 233](#)

[PSA_KEY_LIFETIME_FROM_PERSISTENCE_AND_LOCATION \(C macro\), 197](#)
[PSA_KEY_LIFETIME_GET_LOCATION \(C macro\), 197](#)
[PSA_KEY_LIFETIME_GET_PERSISTENCE \(C macro\), 198](#)
[PSA_KEY_LIFETIME_IS_VOLATILE \(C macro\), 198](#)
[psa_key_lifetime_t \(C type\), 233](#)
[psa_key_location_t \(C type\), 234](#)
[psa_key_persistence_t \(C type\), 234](#)
[PSA_KEY_TYPE_DH_GET_FAMILY \(C macro\), 202](#)
[PSA_KEY_TYPE_DH_KEY_PAIR \(C macro\), 202](#)
[PSA_KEY_TYPE_DH_PUBLIC_KEY \(C macro\), 203](#)
[PSA_KEY_TYPE_ECC_GET_FAMILY \(C macro\), 203](#)
[PSA_KEY_TYPE_ECC_KEY_PAIR \(C macro\), 203](#)
[PSA_KEY_TYPE_ECC_PUBLIC_KEY \(C macro\), 203](#)
[PSA_KEY_TYPE_IS_ASYMMETRIC \(C macro\), 204](#)
[PSA_KEY_TYPE_IS_DH \(C macro\), 204](#)
[PSA_KEY_TYPE_IS_DH_KEY_PAIR \(C macro\), 204](#)
[PSA_KEY_TYPE_IS_DH_PUBLIC_KEY \(C macro\), 204](#)
[PSA_KEY_TYPE_IS_ECC \(C macro\), 204](#)
[PSA_KEY_TYPE_IS_ECC_KEY_PAIR \(C macro\), 205](#)
[PSA_KEY_TYPE_IS_ECC_PUBLIC_KEY \(C macro\), 205](#)
[PSA_KEY_TYPE_IS_KEY_PAIR \(C macro\), 205](#)
[PSA_KEY_TYPE_IS_PUBLIC_KEY \(C macro\), 205](#)
[PSA_KEY_TYPE_IS_RSA \(C macro\), 205](#)
[PSA_KEY_TYPE_IS_RSA_KEY_PAIR \(C macro\), 205](#)
[PSA_KEY_TYPE_IS_RSA_PUBLIC_KEY \(C macro\), 206](#)
[PSA_KEY_TYPE_IS_UNSTRUCTURED \(C macro\), 206](#)
[PSA_KEY_TYPE_KEY_PAIR_OF_PUBLIC_KEY \(C macro\), 206](#)
[PSA_KEY_TYPE_PUBLIC_KEY_OF_KEY_PAIR \(C macro\), 207](#)
[psa_key_type_t \(C type\), 236](#)
[psa_key_usage_t \(C type\), 236](#)
[psa_mac_abort \(C function\), 316](#)
[psa_mac_compute \(C function\), 316](#)
[PSA_MAC_LENGTH \(C macro\), 227](#)
[psa_mac_operation_init \(C function\), 318](#)
[psa_mac_operation_t \(C type\), 241](#)
[psa_mac_sign_finish \(C function\), 318](#)

psa_mac_sign_setup (C function), 319
 PSA_MAC_TRUNCATED_LENGTH (C macro), 227
 psa_mac_update (C function), 321
 psa_mac_verify (C function), 322
 psa_mac_verify_finish (C function), 323
 psa_mac_verify_setup (C function), 324
 psa_ps_create (C function), 351
 psa_ps_get (C function), 349
 psa_ps_get_info (C function), 350
 psa_ps_get_support (C function), 353
 psa_ps_remove (C function), 350
 psa_ps_set (C function), 348
 psa_ps_set_extended (C function), 352
 psa_purge_key (C function), 325
 psa_raw_key_agreement (C function), 328
 PSA_RAW_KEY_AGREEMENT_OUTPUT_SIZE (C macro), 228
 psa_reset_key_attributes (C function), 330
 psa_set_key_algorithm (C function), 330
 psa_set_key_bits (C function), 331
 psa_set_key_id (C function), 331
 psa_set_key_lifetime (C function), 332
 psa_set_key_type (C function), 333
 psa_set_key_usage_flags (C function), 333
 psa_sign_hash (C function), 334
 psa_sign_message (C function), 335
 PSA_SIGN_OUTPUT_SIZE (C macro), 229
 psa_status_t (C type), 354
 psa_storage_create_flags_t (C type), 343
 psa_storage_info_t (C struct), 344
 psa_storage_uid_t (C type), 343
 psa_verify_hash (C function), 337
 psa_verify_message (C function), 338

S

se_info (C struct), 407
 smw_aead (C function), 47
 smw_aead_aad_args (C struct), 45
 smw_aead_args (C struct), 47
 smw_aead_data_args (C struct), 45
 smw_aead_final (C function), 50
 smw_aead_final_args (C struct), 46
 smw_aead_info (C struct), 15
 smw_aead_init (C function), 49
 smw_aead_init_args (C struct), 43
 smw_aead_mode_t (C type), 150
 smw_aead_op_type_t (C type), 151
 smw_aead_update (C function), 50
 smw_aead_update_aad (C function), 49
 smw_allocate_context (C function), 42
 smw_asymmetric_decrypt (C function), 53

smw_asymmetric_encrypt (C function), 53
 smw_asymmetric_encrypt_info (C struct), 19
 smw_asymmetric_encryption_algo_t (C type), 155
 smw_asymmetric_encryption_args (C struct), 52
 smw_asymmetric_encryption_mode_t (C type), 155
 SMW_ATTR_ALGO_AEAD (C macro), 126
 SMW_ATTR_ALGO_ASYMMETRIC_ENCRYPTION_RSA (C macro), 130
 SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_DSA (C macro), 124
 SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_ECDSA (C macro), 123
 SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_EDDSA (C macro), 123
 SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_RSA (C macro), 124
 SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_CLIENT (C macro), 125
 SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_NO_LABEL (C macro), 125
 SMW_ATTR_ALGO_ASYMMETRIC_SIGNATURE_TLS_1_2_SERVER (C macro), 125
 SMW_ATTR_ALGO_DIGEST (C macro), 122
 SMW_ATTR_ALGO_KEY_AGREEMENT (C macro), 129
 SMW_ATTR_ALGO_KEY_ATTESTATION_ECDSA (C macro), 129
 SMW_ATTR_ALGO_KEY_ATTESTATION_MAC (C macro), 129
 SMW_ATTR_ALGO_KEY_DERIVATION_DH (C macro), 127
 SMW_ATTR_ALGO_KEY_DERIVATION_ECDH (C macro), 127
 SMW_ATTR_ALGO_KEY_DERIVATION_EDDSA (C macro), 128
 SMW_ATTR_ALGO_KEY_DERIVATION_HKDF (C macro), 127
 SMW_ATTR_ALGO_KEY_DERIVATION_TLS12 (C macro), 128
 SMW_ATTR_ALGO_KEY_DERIVATION_TLS13 (C macro), 128
 SMW_ATTR_ALGO_MAC (C macro), 126
 SMW_ATTR_ALGO_MAC_HMAC (C macro), 126
 SMW_ATTR_ALGO_SYMMETRIC_ENCRYPTION (C macro), 123
 smw_attr_algo_t (C type), 106
 smw_attr_attributes_t (C type), 108
 SMW_ATTR_GET_ALGO (C macro), 130

SMW_ATTR_GET_CLASS (*C macro*), 132
 SMW_ATTR_GET_CURVE (*C macro*), 131
 SMW_ATTR_GET_HASH (*C macro*), 131
 SMW_ATTR_GET_KDF (*C macro*), 131
 SMW_ATTR_GET_LENGTH (*C macro*), 115
 SMW_ATTR_GET_MAC_LENGTH (*C macro*), 119
 SMW_ATTR_GET_MODE (*C macro*), 130
 SMW_ATTR_GET_PERSISTENCE (*C macro*), 140
 SMW_ATTR_GET_SALT_LENGTH (*C macro*), 119
 SMW_ATTR_GET_SIGN_PARAM (*C macro*), 120
 SMW_ATTR_GET_TAG_LENGTH (*C macro*), 119
 SMW_ATTR_IS_LC_CLOSED (*C macro*), 144
 SMW_ATTR_IS_LC_CLOSED_LOCKED (*C macro*), 144
 SMW_ATTR_IS_LC_CURRENT (*C macro*), 143
 SMW_ATTR_IS_LC_OPEN (*C macro*), 144
 SMW_ATTR_IS_MIN_LENGTH (*C macro*), 115
 SMW_ATTR_IS_MIN_MAC_LENGTH (*C macro*), 118
 SMW_ATTR_IS_MIN_SALT_LENGTH (*C macro*), 118
 SMW_ATTR_IS_MIN_TAG_LENGTH (*C macro*), 118
 SMW_ATTR_IS_MSG_HASHED (*C macro*), 120
 SMW_ATTR_IS_PERMANENT (*C macro*), 142
 SMW_ATTR_IS_PERSISTENT (*C macro*), 141
 SMW_ATTR_IS_READ_ONCE (*C macro*), 146
 SMW_ATTR_IS_READ_ONLY (*C macro*), 145
 SMW_ATTR_IS_SIGN_EDDSA_CONTEXT (*C macro*), 122
 SMW_ATTR_IS_SIGN_EDDSA_PREHASHED (*C macro*), 121
 SMW_ATTR_IS_TRANSIENT (*C macro*), 141
 SMW_ATTR_SET_HASH (*C macro*), 132
 SMW_ATTR_SET_LC_CLOSED (*C macro*), 143
 SMW_ATTR_SET_LC_CLOSED_LOCKED (*C macro*), 143
 SMW_ATTR_SET_LC_CURRENT (*C macro*), 142
 SMW_ATTR_SET_LC_OPEN (*C macro*), 142
 SMW_ATTR_SET_LENGTH (*C macro*), 114
 SMW_ATTR_SET_MAC_LENGTH (*C macro*), 116
 SMW_ATTR_SET_MIN_LENGTH (*C macro*), 115
 SMW_ATTR_SET_MIN_MAC_LENGTH (*C macro*), 117
 SMW_ATTR_SET_MIN_SALT_LENGTH (*C macro*), 117
 SMW_ATTR_SET_MIN_TAG_LENGTH (*C macro*), 117
 SMW_ATTR_SET_MODE (*C macro*), 132
 SMW_ATTR_SET_MSG_HASHED (*C macro*), 120
 SMW_ATTR_SET_PERMANENT (*C macro*), 141
 SMW_ATTR_SET_PERSISTENCE (*C macro*), 139
 SMW_ATTR_SET_PERSISTENT (*C macro*), 140
 SMW_ATTR_SET_READ_ONCE (*C macro*), 145
 SMW_ATTR_SET_READ_ONLY (*C macro*), 145
 SMW_ATTR_SET_SALT_LENGTH (*C macro*), 116
 SMW_ATTR_SET_SIGN_EDDSA_CONTEXT (*C macro*), 122
 SMW_ATTR_SET_SIGN_EDDSA_PREHASHED (*C macro*), 121
 SMW_ATTR_SET_SIGN_PARAM (*C macro*), 121
 SMW_ATTR_SET_TAG_LENGTH (*C macro*), 116
 SMW_ATTR_SET_TRANSIENT (*C macro*), 140
 smw_attr_storage_id_t (*C type*), 109
 SMW_ATTR_USAGE_IS_CACHE (*C macro*), 136
 SMW_ATTR_USAGE_IS_COPY (*C macro*), 136
 SMW_ATTR_USAGE_IS_DECRYPT (*C macro*), 137
 SMW_ATTR_USAGE_IS_DERIVE (*C macro*), 139
 SMW_ATTR_USAGE_IS_ENCRYPT (*C macro*), 137
 SMW_ATTR_USAGE_IS_EXPORT (*C macro*), 137
 SMW_ATTR_USAGE_IS_SIGN_HASH (*C macro*), 138
 SMW_ATTR_USAGE_IS_SIGN_MESSAGE (*C macro*), 138
 SMW_ATTR_USAGE_IS_VERIFY_HASH (*C macro*), 139
 SMW_ATTR_USAGE_IS_VERIFY_MESSAGE (*C macro*), 138
 SMW_ATTR_USAGE_SET_CACHE (*C macro*), 133
 SMW_ATTR_USAGE_SET_COPY (*C macro*), 133
 SMW_ATTR_USAGE_SET_DECRYPT (*C macro*), 134
 SMW_ATTR_USAGE_SET_DERIVE (*C macro*), 136
 SMW_ATTR_USAGE_SET_ENCRYPT (*C macro*), 134
 SMW_ATTR_USAGE_SET_EXPORT (*C macro*), 133
 SMW_ATTR_USAGE_SET_SIGN_HASH (*C macro*), 135
 SMW_ATTR_USAGE_SET_SIGN_MESSAGE (*C macro*), 134
 SMW_ATTR_USAGE_SET_VERIFY_HASH (*C macro*), 135
 SMW_ATTR_USAGE_SET_VERIFY_MESSAGE (*C macro*), 135
 smw_attr_usage_t (*C type*), 108
 smw_cancel_operation (*C function*), 42
 smw_cipher (*C function*), 37
 smw_cipher_args (*C struct*), 31
 smw_cipher_data_args (*C struct*), 30
 smw_cipher_final (*C function*), 39
 smw_cipher_info (*C struct*), 14
 smw_cipher_init (*C function*), 38
 smw_cipher_init_args (*C struct*), 29
 smw_cipher_mode_t (*C type*), 150
 smw_cipher_op_type_t (*C type*), 151
 smw_cipher_update (*C function*), 39

smw_commit_key_storage (C function), 78
 smw_commit_key_storage_args (C struct), 73
 smw_config_check_aead (C function), 16
 smw_config_check_asymmetric_decrypt (C function), 20
 smw_config_check_asymmetric_encrypt (C function), 19
 smw_config_check_cipher (C function), 14
 smw_config_check_derive_key (C function), 11
 smw_config_check_digest (C function), 9
 smw_config_check_generate_key (C function), 11
 smw_config_check_mac (C function), 17
 smw_config_check_sign (C function), 12
 smw_config_check_verify (C function), 13
 smw_config_load (C function), 18
 smw_config_subsystem_loaded (C function), 9
 smw_config_subsystem_present (C function), 9
 smw_config_unload (C function), 18
 smw_context_args (C struct), 41
 smw_copy_context (C function), 43
 smw_copy_context_args (C struct), 42
 smw_data_attributes (C struct), 86
 smw_data_descriptor (C struct), 87
 smw_data_info_args (C struct), 90
 smw_deinit (C function), 405
 smw_delete_data (C function), 92
 smw_delete_data_args (C struct), 90
 smw_delete_key (C function), 76
 smw_delete_key_args (C struct), 72
 smw_derive_key (C function), 74
 smw_derive_key_args (C struct), 59
 smw_derived_key_descriptor (C struct), 57
 smw_device_attestation (C function), 83
 smw_device_attestation_args (C struct), 80
 smw_device_get_lifecycle (C function), 85
 smw_device_get_uuid (C function), 84
 smw_device_lifecycle_args (C struct), 82
 smw_device_reprovision (C function), 85
 smw_device_reprovision_args (C struct), 82
 smw_device_reprovision_prepare (C function), 85
 smw_device_set_lifecycle (C function), 84
 smw_device_uuid_args (C struct), 81
 smw_eddsa_params (C struct), 23
 smw_encryption_args (C struct), 87
 smw_export_key (C function), 75
 smw_export_key_args (C struct), 71
 smw_find_object_db (C function), 95
 smw_find_object_db_args (C struct), 94
 smw_find_object_db_final (C function), 96
 smw_find_object_db_init (C function), 95
 smw_find_object_db_next (C function), 96
 smw_generate_key (C function), 74
 smw_generate_key_args (C struct), 58
 smw_get_data_info (C function), 92
 smw_get_key_attributes (C function), 77
 smw_get_key_attributes_args (C struct), 72
 smw_get_key_buffers_lengths (C function), 76
 smw_get_key_type_name (C function), 76
 smw_get_security_size (C function), 77
 smw_get_version (C function), 8
 smw_hash (C function), 31
 smw_hash_algo_t (C type), 149
 smw_hash_args (C struct), 21
 smw_hash_final (C function), 32
 smw_hash_final_args (C struct), 23
 smw_hash_init (C function), 32
 smw_hash_init_args (C struct), 21
 smw_hash_update (C function), 32
 smw_hash_update_args (C struct), 22
 smw_hkdf_args (C struct), 68
 smw_hkdf_expand_args (C struct), 67
 smw_hkdf_extract_args (C struct), 67
 smw_import_key (C function), 75
 smw_import_key_args (C struct), 70
 smw_init (C function), 404
 smw_kdf_ecdh_args (C struct), 70
 smw_kdf_hkdf_args (C struct), 68
 smw_kdf_oem_master_key_args (C struct), 78
 smw_kdf_t (C type), 154
 smw_kdf_tls12_args (C struct), 60
 smw_kdf_tls12_key_expansion_args (C struct), 64
 smw_kdf_tls12_master_secret_args (C struct), 63
 smw_kdf_tls12_op_args (C struct), 65
 smw_kdf_tls12_random_data (C struct), 62
 smw_kdf_tls12_session_hash (C struct), 62
 smw_kdf_tls13_args (C struct), 66
 smw_key_attestation (C function), 78
 smw_key_attestation_args (C struct), 73
 smw_key_attributes (C struct), 56
 smw_key_descriptor (C struct), 57
 smw_key_format_t (C type), 151
 smw_key_info (C struct), 10
 smw_key_privacy_t (C type), 149
 smw_key_type_t (C type), 148
 smw_keypair_buffer (C struct), 55

smw_keypair_gen (*C struct*), 54
 smw_keypair_rsa (*C struct*), 54
 smw_lifecycle_t (*C type*), 154
 smw_mac (*C function*), 40
 smw_mac_algo_t (*C type*), 150
 smw_mac_args (*C struct*), 27
 smw_mac_info (*C struct*), 16
 smw_mac_verify (*C function*), 40
 smw_object_descriptor (*C struct*), 93
 smw_object_type_t (*C type*), 148
 smw_oem_master_key_op_t (*C type*), 154
 smw_operation_t (*C type*), 146
 smw_ops (*C struct*), 403
 smw_osal_latest_subsystem_name (*C function*), 408
 smw_osal_lib_init (*C function*), 408
 smw_osal_object (*C struct*), 402
 smw_osal_open_obj_db (*C function*), 409
 smw_osal_set_subsystem_info (*C function*), 408
 smw_retrieve_data (*C function*), 92
 smw_retrieve_data_args (*C struct*), 89
 smw_rng (*C function*), 37
 smw_rng_args (*C struct*), 28
 smw_sign (*C function*), 33
 smw_sign_args (*C struct*), 88
 smw_sign_final (*C function*), 35
 smw_sign_init (*C function*), 34
 smw_sign_update (*C function*), 34
 smw_sign_verify_args (*C struct*), 24
 smw_sign_verify_final_args (*C struct*), 26
 smw_sign_verify_init_args (*C struct*), 25
 smw_sign_verify_update_args (*C struct*), 26
 smw_signature_algo_t (*C type*), 151
 smw_signature_info (*C struct*), 12
 smw_signature_type_t (*C type*), 152
 smw_status_code (*C enum*), 98
 smw_store_data (*C function*), 91
 smw_store_data_args (*C struct*), 89
 smw_subsystem_t (*C type*), 146
 smw_tls12_enc_t (*C type*), 153
 smw_tls12_kek_t (*C type*), 152
 smw_tls12_op_t (*C type*), 153
 smw_tls13_expand_label (*C function*), 98
 smw_tls13_expand_label_args (*C struct*), 96
 SMW_TLS13_EXPANDED_LABEL_LENGTH (*C macro*), 97
 smw_update_object_db (*C function*), 94
 smw_verify (*C function*), 35
 smw_verify_final (*C function*), 37
 smw_verify_init (*C function*), 36
 smw_verify_update (*C function*), 36
 subsystem_info (*C union*), 407

T

tee_info (*C struct*), 406